



AUTORA: DRA. ANAISA HERNÁNDEZ GONZÁLEZ
FACULTAD DE INGENIERÍA INDUSTRIAL
INSTITUTO SUPERIOR POLITÉCNICO JOSÉ ANTONIO
ECHEVERRÍA

Mayo 2004

Indice

Introducción

Capítulo 1 Rational Unified Process

- 1.1 Proceso de Desarrollo de Software.
- 1.2 El Lenguaje Unificado de Modelación.
- 1.3 El Proceso Unificado de Desarrollo de Software.
- 1.4 Productos de Rational.
- 1.5 Conclusiones.
- 1.6 Referencias bibliográficas

Capítulo 2 Modelo del negocio.

- 2.1 Modelamiento del negocio.
- 2.2 Modelo de casos de uso del negocio.
 - 2.2.1 Diagrama de casos de uso del negocio.
 - 2.2.2 Estructuración de los casos de uso del negocio.
- 2.3 Realización de los casos de uso del negocio.
 - 2.3.1 Descripción textual.
 - 2.3.2 Diagrama de actividades.
 - 2.3.3 Diagrama de clases.
- 2.4 Reglas de negocio.
- 2.5 Conclusiones.
- 2.6 Referencias bibliográficas.

Capítulo 3 Requisitos

- 3.1 Flujo de trabajo de trabajo de requerimientos.
 - 3.1.1 Requerimientos.
 - 3.1.2 Modelo de casos de uso del sistema.
- 3.2 Organización en paquetes.
- 3.3 Conclusiones
- 3.4 Referencias bibliográficas

Capítulo 4 Modelo de análisis

- 4.1 Flujo de trabajo de análisis.
- 4.2 Realización de los casos de uso en el análisis.
 - 4.2.1 Descripción de los casos de uso.
 - 4.2.2 Modelo del análisis.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

4.2.2.1 Identificación de clases, atributos y relaciones.

4.2.2.2 Diagrama de clases del análisis.

4.3 Organización en paquetes.

4.4 Conclusiones.

4.5 Referencias bibliográficas.

Capítulo 5 Diseño

5.1 Flujo de trabajo de diseño

5.2 Realización de los casos de uso en el diseño.

5.2.1 Refinamiento de la descripción de los casos de uso en el diseño.

5.2.2 Diagramas de interacción.

5.2.3 Diagrama de clases del diseño.

5.3 Estándares en el diseño de la interfaz y los reportes del diseño.

5.4 Diseño de la Base de Datos.

5.5 Conclusiones.

5.6 Referencias bibliográficas.

Capítulo 6 Sistemas en Tiempo Real

6.1 Características de los Sistemas en Tiempo Real.

6.2 Proceso de Desarrollo de Software.

6.3 Caso de estudio: Semáforo.

6.4 Conclusiones.

6.5 Referencias bibliográficas.

Capítulo 7 Modelo de implementación

7.1 Flujo de trabajo de trabajo de diseño.

7.1.1 Implementación de aplicaciones de tres capas.

7.1.2 Diagrama de despliegue.

7.1.3 Mapeo del diseño a la implementación.

7.2 Flujo de trabajo de implementación

7.2.1 Diagrama de componentes.

7.2.2 Construcción del software.

7.3 Conclusiones.

7.4 Referencias bibliográficas

Capítulo 8 Modelo de prueba

9.1 Flujo de trabajo de trabajo de prueba.

9.2 Casos de prueba.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

8.3 Procedimientos de prueba.

8.4 Conclusiones.

8.5 Referencias bibliográficas.

Capítulo 9 Caso de estudio: Galería de arte

9.1 Caso de estudio.

9.2 Modelo del negocio.

9.3 Requisitos.

9.4 Análisis.

9.5 Diseño.

9.6 Modelo de implementación.

9.7 Modelo de prueba.

Bibliografía

Introducción

En la actualidad en la Industria de Software hay tendencia al crecimiento del volumen y complejidad de los productos, los proyectos están excesivamente tardes, se exige mayor calidad y productividad en menos tiempo y hay insuficiente personal calificado; por lo que se puede decir que la fallas de los proyectos de software se debe a:

- **Planificación irreal:** Los usuarios piden un sistema para hoy que tenga costo 0 y los ingenieros no son capaces de de enfrentar un plan porque no están entrenados para usar métodos de planificación y, frecuentemente, las estimaciones no se basan en datos reales.
- **Mala calidad del trabajo:** Las prácticas pobres de Ingeniería, la carencia de métricas adecuadas de calidad y las decisiones de los directivos guiadas por una planificación irreal; traen como consecuencia tiempos de pruebas impredecibles, productos con muchos defectos, demoras en la aceptación de los usuarios y una extensa garantía de servicio y reparaciones. Una pobre calidad afecta la planificación y torna ineficiente el proceso de prueba.
- **Personal inadecuado:** En múltiples ocasiones el personal asignado a un proyecto se incorpora tarde, no cubre las necesidades en cuanto a cantidad y calidad y se incorporan a tiempo parcial al proyecto. Como consecuencia el trabajo se demora o descuida, es ineficiente y sufre la moral del equipo. Con independencia del plan, los proyectos deben comenzar en tiempo y con todo el personal.
- **Cambios no controlados:** Es importante recordar que siempre ocurren cambios en los requerimientos, que los planes del proyecto se basan en el alcance del trabajo conocido, que los cambios siempre requieren más trabajo, sin planes detallados los equipos no pueden estimar el efecto o magnitud de los cambios y que si los equipos no controlan cada cambio, se pierde gradualmente el control del plan del proyecto.

Para enfrentar esta situación las empresas requieren desarrollar o adquirir una disciplina en el desarrollo del software y controlar que los ingenieros usen de forma consistente los nuevos métodos. Cualquier camino que siga una empresa de software para obtener buena calidad implica que tiene que mejorar el proceso de desarrollo de software, por lo tanto, se requiere utilizar los métodos y procedimientos de la Ingeniería y Gestión de Software.

La Ingeniería de Software es una tecnología multicapa en la que, según Pressman, se pueden identificar: los métodos (indican cómo construir técnicamente el software), el proceso (es el fundamento de la Ingeniería de Software, es la unión que mantiene juntas las capas de la tecnología) y las herramientas (soporte automático o semiautomático para el proceso y los métodos).

Las 6 mejores prácticas (enfoques probados) que están siendo usadas por organizaciones exitosas en el desarrollo de software son:

1. **Administre requerimientos:** identifique y represente las funcionalidades requeridas y otras restricciones y decisiones en forma de requerimientos que puedan ser rastreados durante el desarrollo de software.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

2. **Use arquitectura de componentes:** defina una arquitectura robusta y flexible que use componentes (módulos que cumplen una función clara) nuevos y existentes ensamblados.
3. **Modele visualmente:** modele el sistema usando elementos visuales que escondan los detalles, pero que brinden una abstracción adecuada para entender en su totalidad el sistema.
4. **Verifique calidad:** compruebe la calidad a partir de analizar cómo se han implementado los requerimientos durante todo el proceso de desarrollo.
5. **Desarrolle iterativamente:** construir la solución a través de refinamientos sucesivos en múltiples iteraciones.
6. **Controle cambios:** a partir de que un cambio es aceptado hay que controlar su cumplimiento, documentar y divulgar al equipo qué cambió y aislar el lugar del cambio mientras se esté cumplimentando.

La aplicación de las mejores logra crear equipos de alto rendimiento que producen proyectos más exitosos porque están en plazo, en presupuesto y satisfacen las necesidades del usuario.

El objetivo de libro es proporcionar un proceso de desarrollo de software en el que se garantice la aplicación de las mejores prácticas, para lo cual en este capítulo nos adentramos en sus principales características.

Capítulo 1 Rational Unified Process

Un proceso define “quién” está haciendo “qué”, “cuándo” y “cómo” par alcanzar un determinado objetivo¹. En este capítulo se definen las principales características del Proceso Unificado de Desarrollo propuesto por Ivar Jacobson, James Rumbaugh y Grady Booch.

1.1- Proceso de Desarrollo de Software.

Un Proceso de Desarrollo de Software es la definición del conjunto de actividades que guían los esfuerzos de las personas implicadas en el proyecto, a modo de plantilla que explica los pasos necesarios para terminar el proyecto². En la figura 1 se indica que este conjunto de actividades, en el proceso de desarrollo de software que proponen Jacobson, Rumbaugh y Booch, tiene la misión de transformar los requerimientos del usuario en un producto de software; de manera que los integrantes del equipo y todo aquel que pueda estar interesado en el producto final, tenga la misma visión y no ocurra cuando no se aplica un proceso de desarrollo (figura 2).



Figura 1 Proceso de Desarrollo de Software.

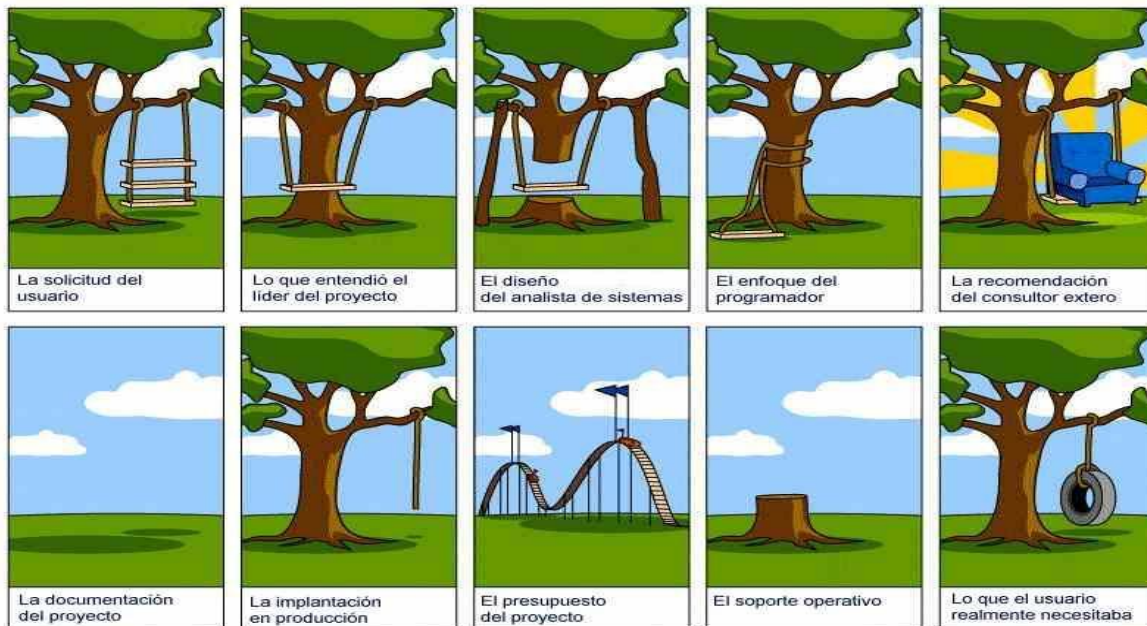


Figura 2 Visiones del producto final.

¹ Jacobson, I.; Booch, G. y Rumbaugh, J.; “El Proceso Unificado de Desarrollo de software”. 2000. Página XVI.

² Jacobson, I.; Booch, G. y Rumbaugh, J.; “El Proceso Unificado de Desarrollo de software”. 2000. Página 13.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Por lo tanto, las piedras angulares del proceso de desarrollo del software son: el proyecto, las personas y el producto; siendo las características del cliente, el entorno de desarrollo y las condiciones del negocio, elementos que influyen en el proceso (figura 3).

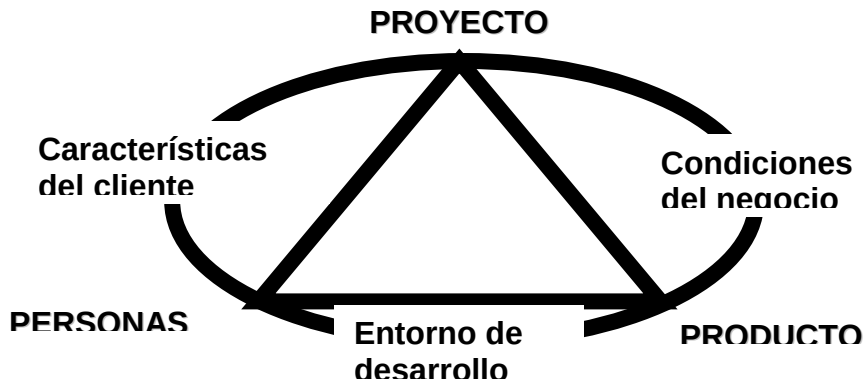


Figura 3 Piedras angulares del Proceso de Desarrollo de Software.

El conocimiento y la experiencia que crea y sostiene la evolución del producto lo tienen las personas. Ellas también financian, se benefician, lo prueban y planifican. Sin un personal competente y experimentado es imposible crear productos competitivos que satisfagan las necesidades de los clientes. Además, el modo en que organiza y gestiona un producto (factibilidad del proyecto, definición del equipo que lo desarrolló, identificación y análisis de riesgos que implican su realización, planificación, nivel de entendimiento de lo que se está haciendo y sensación de que avanza en el cumplimiento) afectan a las personas involucradas en su realización.

Un proyecto es un elemento organizativo a través del cual se gestiona el desarrollo del software.

Un proyecto de desarrollo obtiene una versión de un producto que contiene modelos, código fuente, documentación y un ejecutable. Este producto va evolucionando durante el proceso de desarrollo desde un proyecto inicial o innovador (prototipo inicial) hasta convertirse en un release del proyecto.

En la figura 4 se aprecia como a cada nivel (organización, equipo, individuo) se han ido desarrollando métodos que mejoran el proceso en dependencia de las actividades que se realizan y las personas que participan. En este curso se estudiará un método desarrollado para su aplicación por parte de un equipo de proyecto, en el que se dejan claros los roles que juegan los trabajadores dentro de los flujos de trabajo que los integran.

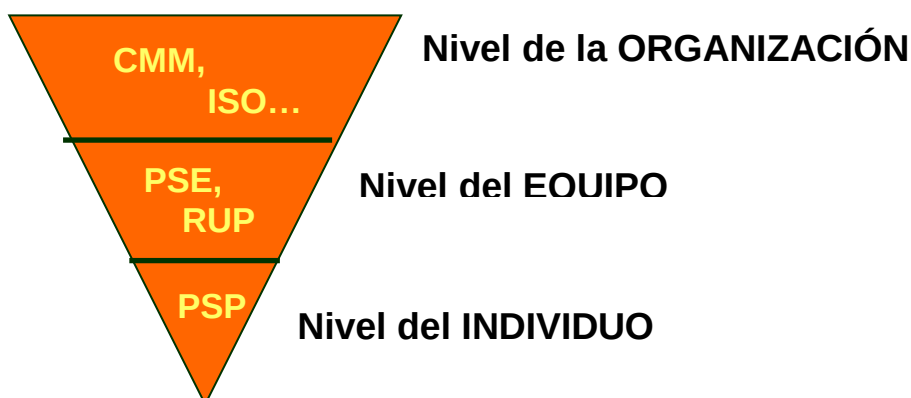


Figura 4 Métodos de mejora del proceso de desarrollo de software en los distintos niveles de implementación.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- CMM (Modelo de Maduración de Capacidades): Modelo que brinda un procedimiento para pasar de un proceso inmaduro que pocos conocen, no se mide, se aplica solo en ocasiones, es percibido como poco eficiente y es interpretado de manera distinta; a un proceso maduro definido, documentado, en el que el personal está entrenado en el proceso, practicado, apoyado, mantenido, controlado, verificado, medido y medible.
- ISO (Organización de Estándares Internacional): Define estándares de calidad, las normas ISO 9001, ISO 9002, ISO 9003 e ISO 9004 contiene elementos asociados con el software.
- PSE (Proceso de software en Equipo): Ayuda a seguir de manera consistente los métodos disciplinados de ingeniería creando una disciplina en la que directivos (crean un ambiente donde el trabajo disciplinado sea notorio y reconocido), miembros (conozcan y estén convencidos del trabajo disciplinado) y el equipo (usen métodos de trabajo en equipo para planificar y establecer metas) trabajan en conjunto.
- RUP (Proceso Unificado de Rational - Proceso Unificado de Desarrollo de Software): Es un proceso que de manera ordenada define las tareas y quién de los miembros del equipo de desarrollo las hará. Es una guía para usar UML.
- PSP (Proceso de Software Personal): Proceso que busca cambiar el comportamiento individual haciendo que las personas comprendan la necesidad de la aplicación de métodos disciplinados y sepa organizar y planificar su trabajo, chequear su ejecución y dirigir la calidad del software.

1.2 - El Lenguaje Unificado de Modelación

UML es un lenguaje para visualizar, especificar, construir y documentar los artefactos de un sistema que involucra una gran cantidad de software³ (figura 5).

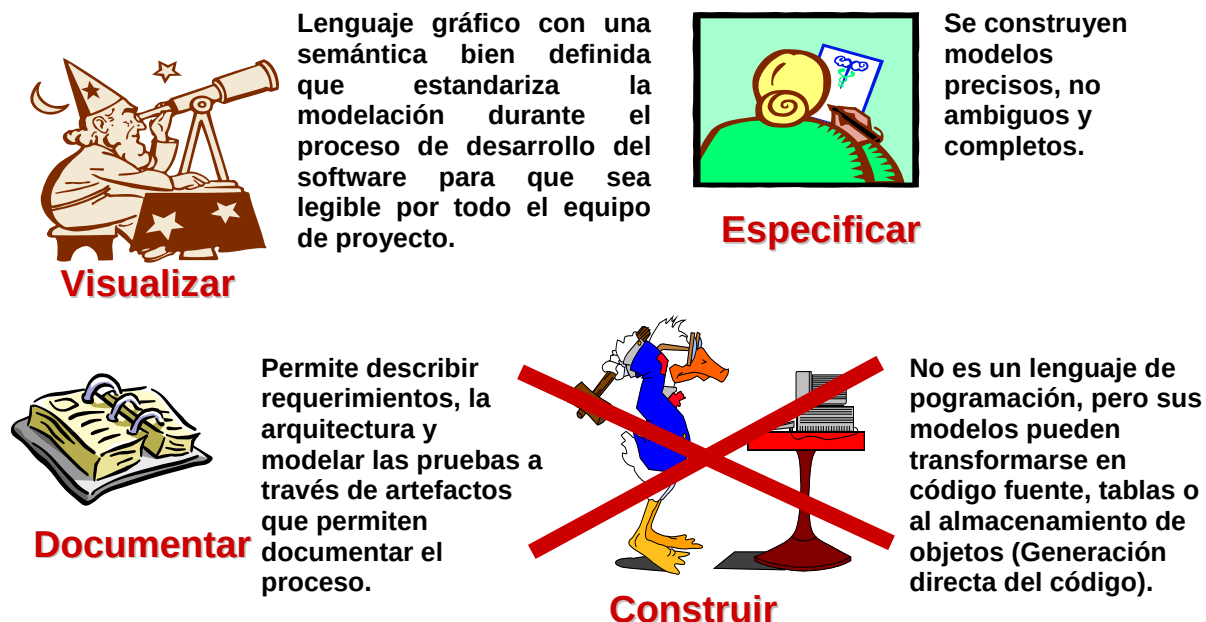


Figura 5 Definición de UML.

³ Booch, G.: Rumbaugh, J. y Jacobson, I.; "El Lenguaje Unificado de Modelado". 2000. Página 11.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

UML es el resultado, en principio, de la unión de los métodos de Booch (Object Oriented Analysis and Design with Application) y Rumbaugh (OMT- Object Modeling Technique) para producir lo que en principio se conoció como el Método Unificado, pero que con la unión de Jacobson (OOSE-Object Oriented software Engineering: A use case driven approach) dio paso al Lenguaje Unificado de Modelación. En Noviembre de 1997 este lenguaje (en su versión 1.1) fue adoptado como el estándar por el OMG (Object Modeling Group).

Es importante recalcar que UML no es una guía para realizar el análisis y diseño orientado a objetos, es decir, no es un proceso. UML es un lenguaje que permite la modelación de sistemas con tecnología orientada a objetos.

UML no es el fruto de estos tres autores, con los años y desde sus surgimiento se ha enriquecido de diferentes trabajos (Figura 6).

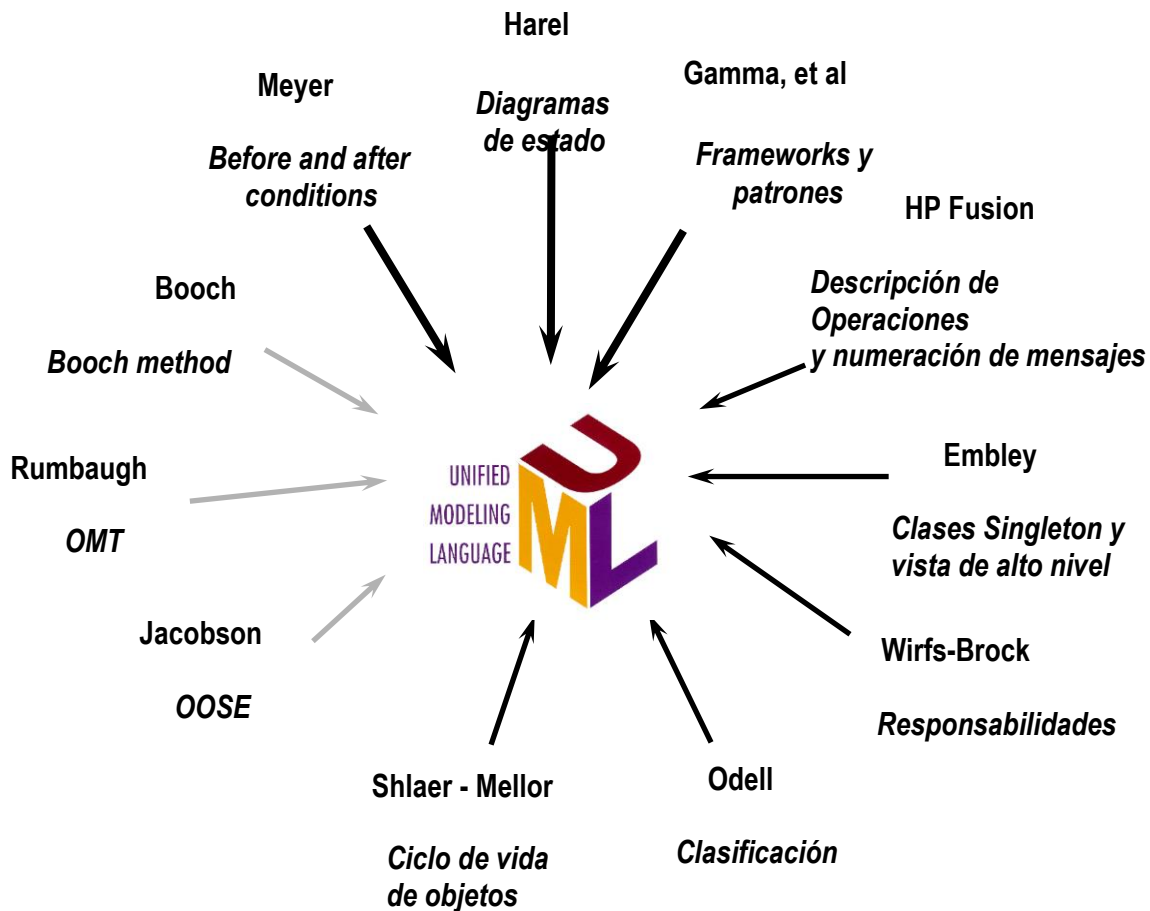


Figura 6 Contribuciones a UML.

El modelo gráfico de UML tiene un vocabulario en el que se identifican:

1. Elementos (abstracciones que constituyen los bloques básicos de construcción).
 - Estructurales: Partes que representan cosas.
 - Clase: Conjunto de objetos que comparten atributos, operaciones, relaciones y semántica.
 - Colaboración: Colección de operaciones que especifican un servicio de una clase o un componente.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Colaboración: Define la interacción entre los elementos que proporcionan un comportamiento cooperativo mayor que la suma de los comportamientos de sus elementos.
 - Caso de uso: Conjunto de secuencia de acciones que un sistema ejecuta y que produce un resultado observable para un actor.
 - Clase activa: Clase cuyos objetos tienen uno o más procesos o hilos de ejecución.
 - Componente: Es una parte física y reemplazable de un sistema que conforman un conjunto de interfaces y proporciona la implementación de dicho conjunto.
 - Nodo: Elemento físico que dispone de memoria y con frecuencia capacidad de almacenamiento.
 - Comportamiento: Partes del modelo que representan el comportamiento en el tiempo y el espacio.
 - Interacción: Conjunto de mensajes intercambiados entre un conjunto de objetos para alcanzar un propósito específico.
 - Máquina de estado: Especifica las secuencias de estados por las que pasa un objeto o una interacción durante su vida.
 - Agrupamiento: Cajas en las cuales puede descomponerse un modelo.
 - Paquete: Mecanismo de propósito general para organizar elementos en grupos.
 - Anotación: Comentarios que se pueden aplicar para describir, clarificar y hacer observaciones sobre cualquier elemento de un modelo.
2. Relaciones: Ligan los elementos.
- Dependencia: relación semántica que indica que un cambio en un elemento afecta a la semántica de otro elemento.
 - Asociación: Relación estructural que describe las conexiones entre objetos.
 - Generalización/Especialización: Relación en la que el hijo comparte la estructura y el comportamiento del padre.
 - Realización: Relación semántica entre clasificadores, en donde un clasificador especifica un contrato que otro clasificador garantiza que cumplirá.
3. Diagramas: Es la representación gráfica de un conjunto de elementos. Visualizan un sistema desde diferentes perspectivas.
- Diagramas de estructura estática:
 - Diagrama de clases: Conjunto de clases, interfaces y colaboraciones; así como sus colaboraciones.
 - Diagrama de objetos: Conjunto de objetos y sus relaciones.
 - Diagramas de comportamiento:
 - Diagramas de interacción (secuencia y colaboración): Objetos y sus relaciones, incluyendo los mensajes que pueden ser enviados entre ellos.
 - Diagrama de estados: Muestra una máquina de estado que consta de estados, transiciones, eventos y actividades.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Diagrama de actividad: Es un tipo especial de diagrama de estados que muestra el flujo de actividades dentro de un sistema.
- Diagrama de casos de uso: Conjunto de casos de uso y actores y sus relaciones.
- Diagramas de implementación:
 - Diagrama de componentes: Organización y las dependencias entre un conjunto de componentes.
 - Diagrama de despliegue: Configuración de nodos de procesamiento en tiempo de ejecución y los componentes que residen en ellos.

1.3 - El Proceso Unificado de Desarrollo de Software

RUP es el resultado de varios años de desarrollo y uso práctico en el que se han unificado técnicas de desarrollo, a través del UML, y trabajo de muchas metodologías utilizadas por los clientes. La versión que se ha estandarizado vio la luz en 1998 y se conoció en sus inicios como Proceso Unificado de Rational 5.0; de ahí las siglas con las que se identifica a este proceso de desarrollo.

Como RUP es un proceso, en su modelación define como sus principales elementos:

Trabajadores ("quién")		Define el comportamiento y responsabilidades (rol) de un individuo, grupo de individuos, sistema automatizado o máquina, que trabajan en conjunto como un equipo. Ellos realizan las actividades y son propietarios de elementos.
Actividades ("cómo")		Es una tarea que tiene un propósito claro, es realizada por un trabajador y manipula elementos.
Artefactos ("qué")		Productos tangibles del proyecto que son producidos, modificados y usados por las actividades. Pueden ser modelos, elementos dentro del modelo, código fuente y ejecutables.
Flujo de actividades ("Cuándo")		Secuencia de actividades realizadas por trabajadores y que produce un resultado de valor observable.

En RUP se han agrupado las actividades en grupos lógicos definiéndose 9 flujos de trabajo principales. Los 6 primeros son conocidos como flujos de ingeniería y los tres últimos como de apoyo. En la figura 7 se representa el proceso en el que se grafican los flujos de trabajo y las fases y muestra la dinámica expresada en iteraciones y puntos de control.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

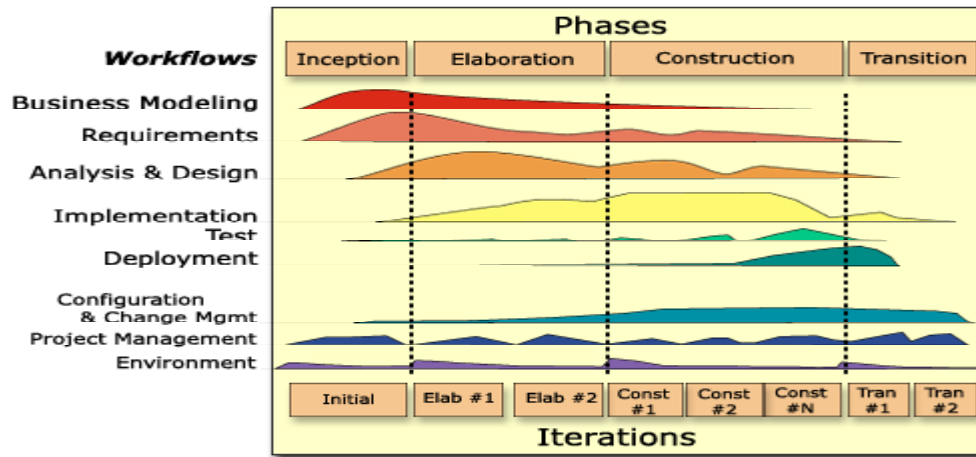


Figura 7 Fases-Flujos de trabajo.

Flujos de trabajo:

- **Modelamiento del negocio:** Describe los procesos de negocio, identificando quiénes participan y las actividades que requieren automatización.
- **Requerimientos:** Define qué es lo que el sistema debe hacer, para lo cual se identifican las funcionalidades requeridas y las restricciones que se imponen.
- **Análisis y diseño:** Describe cómo el sistema será realizado a partir de la funcionalidad prevista y las restricciones impuestas (requerimientos), por lo que indica con precisión lo que se debe programar.
- **Implementación:** Define cómo se organizan las clases y objetos en componentes, cuáles nodos se utilizarán y la ubicación en ellos de los componentes y la estructura de capas de la aplicación.
- **Prueba (Testeo):** Busca los defectos a lo largo del ciclo de vida.
- **Instalación:** Produce release del producto y realiza actividades (empaquete, instalación, asistencia a usuarios, etc.) para entregar el software a los usuarios finales.
- **Administración del proyecto:** Involucra actividades con las que se busca producir un producto que satisfaga las necesidades de los clientes.
- **Administración de configuración y cambios:** Describe cómo controlar los elementos producidos por todos los integrantes del equipo de proyecto en cuanto a: utilización/actualización concurrente de elementos, control de versiones, etc.
- **Ambiente:** Contiene actividades que describen los procesos y herramientas que soportarán el equipo de trabajo del proyecto; así como el procedimiento para implementar el proceso en una organización.

Fases:

- **Conceptualización (Concepción o Inicio):** Se describe el negocio y se delimita el proyecto describiendo sus alcances con la identificación de los casos de uso del sistema.
- **Elaboración:** Se define la arquitectura del sistema y se obtiene una aplicación ejecutable que responde a los casos de uso que la comprometen. A pesar de que se desarrolla a profundidad una parte del sistema, las decisiones sobre la arquitectura se hacen sobre la

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

base de la comprensión del sistema completo y los requerimientos (funcionales y no funcionales) identificados de acuerdo al alcance definido.

- **Construcción:** Se obtiene un producto listo para su utilización que está documentado y tiene un manual de usuario. Se obtiene 1 o varios release del producto que han pasado las pruebas. Se ponen estos release a consideración de un subconjunto de usuarios.
- **Transición:** El release ya está listo para su instalación en las condiciones reales. Puede implicar reparación de errores.

El ciclo de vida de RUP se caracteriza por:

1. **Dirigido por casos de uso:** Los casos de uso reflejan lo que los usuarios futuros necesitan y desean, lo cual se capta cuando se modela el negocio y se representa a través de los requerimientos. A partir de aquí los casos de uso guían el proceso de desarrollo ya que los modelos que se obtienen, como resultado de los diferentes flujos de trabajo, representan la realización de los casos de uso (cómo se llevan a cabo).
2. **Centrado en la arquitectura:** La arquitectura muestra la visión común del sistema completo en la que el equipo de proyecto y los usuarios deben estar de acuerdo, por lo que describe los elementos del modelo que son más importantes para su construcción, los cimientos del sistema que son necesarios como base para comprenderlo, desarrollarlo y producirlo económicamente. RUP se desarrolla mediante iteraciones, comenzando por los CU relevantes desde el punto de vista de la arquitectura. Tal como se aprecia en la figura 8, el modelo de arquitectura se representa a través de en las que se incluyen los diagramas de UML.

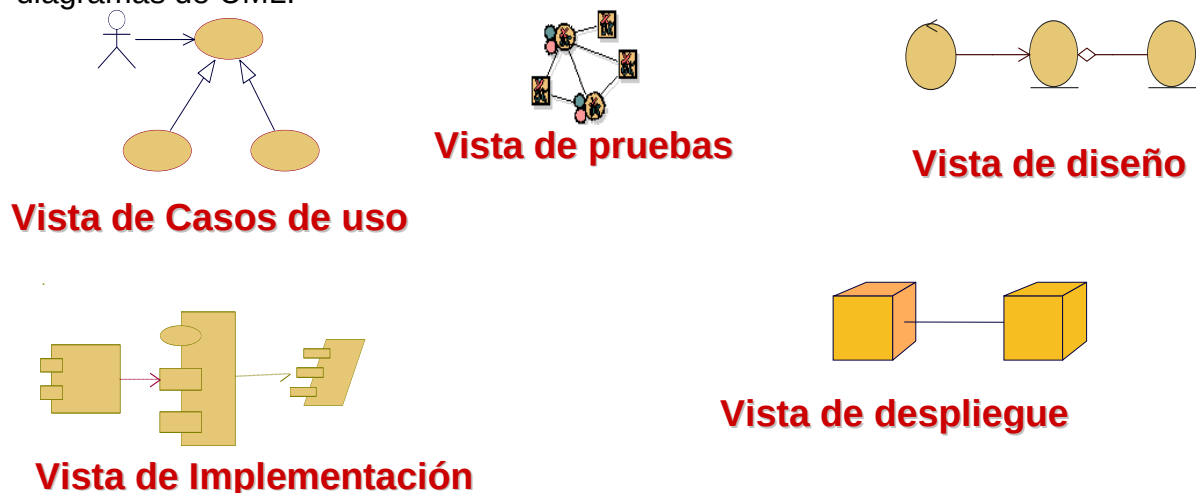


Figura 8 Vista del modelo de arquitectura.

3. **Iterativo e Incremental:** Aunque la figura 7 puede sugerir que los flujos de trabajo se desarrollan en cascada, la "lectura" de este gráfico tiene que ser vertical y horizontal. RUP propone que cada fase se desarrolle en iteraciones. Una iteración involucra actividades de todos los flujos de trabajo, aunque desarrolla fundamentalmente algunos más que otros. Por ejemplo, en una iteración de elaboración centra su atención en el análisis y diseño, aunque refina los requerimientos y obtiene un producto con un determinado nivel, pero que irá creciendo incrementalmente en cada iteración.

Aunque cada iteración tiene que proponerse un incremento en el proceso de desarrollo, todas deben aportar al principal resultado de la fase en la que se desarrolla, por lo que los puntos de control evalúan:

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Conceptualización Objetivos
- Elaboración Arquitectura
- Construcción Funcionalidad operativa
- Transición Release del sistema

Para lograr estos cuatro hitos, hay que construir determinados artefactos definidos dentro de los flujos de trabajo involucrados.

1.4 - Productos de Rational

La suite de Rational ofrece varios productos, destacándose los siguientes:

- **Rational Requisite Pro** – Mantiene a todo el equipo de desarrollo actualizado a través del proceso de desarrollo de aplicaciones haciendo que los requerimientos se puedan escribir, comunicar y cambiar fácilmente. 
- **Rational ClearQuest** - Un producto Windows y basado en Web de administración de solicitudes de cambio que permite a los equipos de proyecto rastrear y administrar todas las actividades de cambio que ocurren durante el desarrollo del ciclo de vida. 
- **Rational Rose** - La herramienta líder en el mundo de modelización visual para el proceso de modelización del negocio, análisis de requerimientos y diseño de arquitectura de componentes. 
- **Rational SoDA** – Automatiza la producción de documentación para todo el proceso de desarrollo de software, reduciendo dramáticamente el tiempo y el costo de documentar el software. 
- **Rational ClearCase** – Herramienta de administración de configuración de software, líder en el mercado, que da a los administradores de proyecto la posibilidad de rastrear la evolución de cada proyecto de desarrollo de software. 

1.5 - Conclusiones

RUP brinda un proceso integrado que utiliza el estándar de notación UML para permitir desarrollar un proceso de forma iterativa e incremental a partir de la identificación e implementación de los casos de uso.

1.6 - Referencias Bibliográficas

- Jacobson, I.; Booch, G. y Rumbaugh, J.; “El Proceso Unificado de Desarrollo de software”. 2000. Addison-Wesley. Prólogo, Capítulos 1-5 y Apéndices A y B. Páginas 3-104, 407-424..
- Booch, G.; Rumbaugh, J. y Jacobson, I.; “El Lenguaje Unificado de Modelado”. 2000. Addison-Wesley.
- Pressman, Roger; Ingeniería de software. Un enfoque práctico. 2002. McGraw.Hill/Interamericana de España. Prólogo y Capítulos 1, 2, 3 y 11. Páginas 3-48 y 181-195.

Capítulo 2 Modelo del negocio

Obtener los requisitos funcionales que se derivarán en un producto de software nuevo o la mejora de uno existente, requiere de un estudio de la organización. Este estudio está contemplado dentro del flujo de trabajo de Modelamiento del negocio, desarrollándose la mayoría de sus actividades dentro de la fase de Concepción (o Inicio).

La fase de Concepción tiene por finalidad definir la visión, los objetivos y alcance del proyecto; obteniéndose como resultado los requerimientos del sistema que satisfacen las necesidades de la organización.

En este capítulo se hará hincapié en el modelamiento del negocio.

2.1 - Modelamiento del negocio

Un sistema, por pequeño que sea, generalmente es complicado. Por eso se necesita dividirlo en piezas si se pretende comprenderlo y gestionar su complejidad. Esas piezas se pueden representar a través de modelos que permitan abstraer sus características esenciales.

De ahí, que en el campo del software también resulte útil la creación de modelos que organicen y presenten los detalles importantes de problemas reales que se vinculan con el sistema informático a construir. Estos modelos deben cumplir una serie de propiedades, entre ellas la de ser coherentes y relacionados. Uno de los modelos útiles previo al desarrollo de un software es el modelo del negocio.

El modelado del negocio es una técnica para comprender los procesos del negocio de la organización. Los propósitos que se persiguen al realizarse el modelado del negocio, son:

- Entender la estructura y la dinámica de la organización.
- Entender los problemas actuales e identificar mejoras potenciales.
- Asegurarse de que los clientes, usuarios finales y desarrolladores tienen una idea común de la organización.
- Derivar los requerimientos del sistema a partir del modelo de negocio que se obtenga.

Para alcanzar estos objetivos, el workflow de la modelación del negocio, describe cómo desarrollar la visión de la nueva organización que se pretende alcanzar, y sobre la base de esta visión, definir los procesos, roles y responsabilidades de esa organización en el modelo de casos de uso del negocio y el modelo de objetos del negocio.

El flujo de trabajo, de acuerdo a RUP, se desarrolla según se indica en la figura 1.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

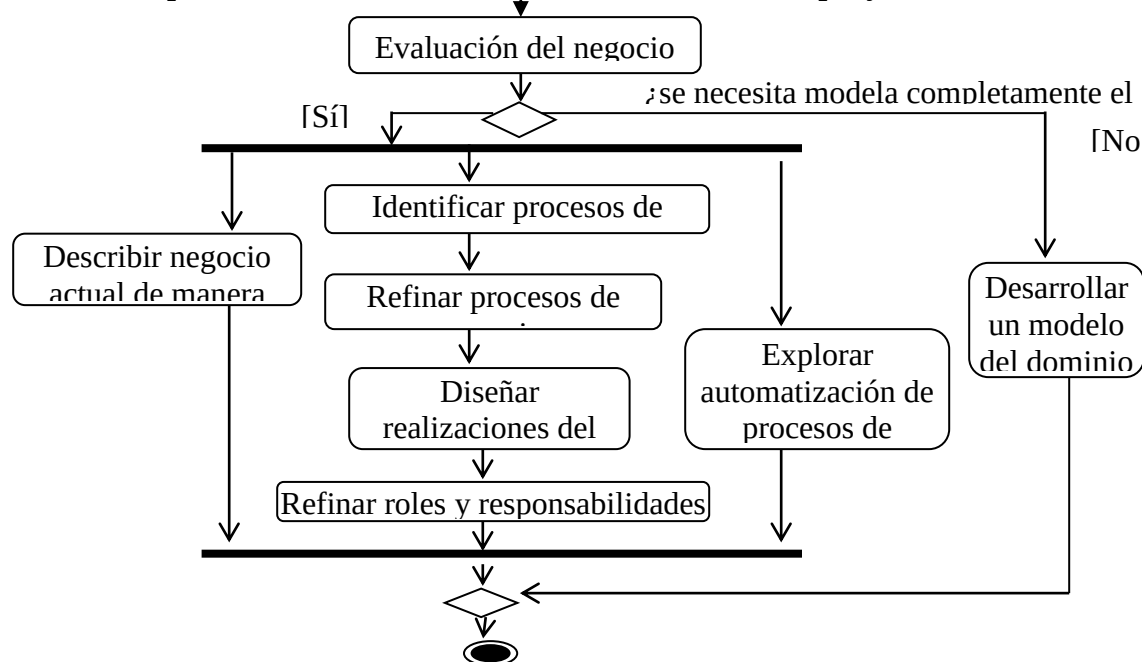


Figura 1 Flujo de trabajo para el modelamiento del negocio según RUP.

Evaluación del estado del Negocio consiste en básicamente en evaluar el estado actual de la organización en la cual el sistema será explotado.

Dependiendo de la situación o escenario que se presente, hay varias alternativas de desarrollar este proceso:

- Si se determina que no es necesario un modelo completo del negocio se realizará lo que se conoce como un *modelamiento del dominio*.

Un modelo del dominio captura los tipos más importantes de objetos que existen o los eventos que suceden en el entorno donde estará el sistema.

El modelo del dominio se considera en RUP un subconjunto del llamado *modelo de objetos del negocio*. Más adelante se hablará de ese modelo. Se puede desarrollar un modelo de objetos del negocio enfocado a la explicación de los productos, entregables o eventos que son importantes en el negocio. Esos modelos, que no incluyen las responsabilidades de las personas que ejecutan las actividades, se refieren a veces como modelo del dominio).

- Si se determina que no habrán cambios importantes en los procesos de negocio, se necesitarán describir esos procesos y derivar los requerimientos del sistema de información. Es decir, si los procesos están claramente definidos y no se van a introducir cambios entonces solo es necesario modelar el negocio propuesto. En este escenario basta con conocer el mapa de la organización y los procesos para comprender mejor los requerimientos de la aplicación a construir. En este caso no se pretende cambiar la organización, aunque en realidad la implantación del sistema siempre incluye algún nivel de mejora del negocio.
- Si se realiza el modelamiento con la intención de lograr una reingeniería del negocio existente, se debería modelar tanto el negocio actual como el nuevo negocio (Por ejemplo, de una subasta pública a una subasta en Internet, de ventas directas en una tienda a una tienda virtual).

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

La descripción del negocio actual consiste en entender los procesos y la estructura de la organización (sin entrar en detalles), identificando claramente los objetivos estratégicos de la organización y los procesos que los soportan, el flujo actual de los procesos involucrados en el campo de acción, el análisis crítico de cómo se ejecutan esos procesos y el análisis comparativo con otras soluciones presentes en el mercado.

La descripción del negocio propuesto en detalle tendrá entre sus actividades principales la identificación de los procesos de negocio, delimitación el *modelo de casos de uso del negocio*, la estructuración y la especificación de *los casos de uso del negocio*, la identificación de *trabajadores y entidades del negocio* que ejecutan las realizaciones de los casos de uso del negocio y detallar la definición de las entidades del negocio y las responsabilidades de los trabajadores del negocio.

La exploración de la automatización del proceso de negocio significa explorar qué partes del negocio pueden y deben ser automatizadas. Esto implicará la *especificación de los requerimientos del software* y la elaboración del *modelo de casos de uso del sistema* en una primera aproximación.

Estos tres grupos de actividades se pueden desarrollar en paralelo. En cualquier caso, hasta que todas las actividades no estén concluidas, no culmina el modelamiento del negocio.

En resumen, el objetivo del *modelo del negocio* es describir los procesos, existentes u observados, con el propósito de comprenderlos. Se especifican aquí qué procesos del negocio soportará el sistema. Además de identificar los objetos del dominio o del negocio implicados, este modelo establece las competencias que se requieren de cada proceso: sus trabajadores, sus responsabilidades y las operaciones que llevan a cabo.

El flujo de trabajo de la modelación del negocio está relacionado con otros flujos de trabajo:

- El workflow de Requerimientos hace uso de los modelos del negocio como una importante entrada para lograr la comprensión de los requerimientos del sistema.
- El workflow de Análisis y Diseño hace uso de las entidades del negocio como una entrada al proceso de identificación de las clases entidades en el modelo de diseño.

Los trabajadores del negocio que intervienen en la realización de estas actividades son:

- Analista de procesos de negocio: Responsable de la arquitectura del negocio por lo que dirige y coordina el proceso de modelamiento del negocio. Decide cuáles son actores y los procesos del negocio y las relaciones entre ellos y cuáles son las reglas de negocio a tener en cuenta.
- Diseñador del negocio: Describe los procesos de negocio y como parte de la realización de estos procesos identifica a las entidades y trabajadores del negocio y sus relaciones. Define cuáles son los requerimientos en la automatización.
- Stakeholders: Personas u organizaciones que están activamente implicadas en el negocio ya sea porque participan en él o porque sus intereses se ven afectados con los resultados del proyecto. Pueden ser los propietarios, la dirección, quienes financian, los clientes, los trabajadores, los proveedores, la competencia, la comunidad local, etc.
- Revisor del modelo de negocio: Revisa formalmente el modelo de casos de uso del negocio y de objetos de negocio obtenido.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Los principales artefactos que se obtienen como resultado del modelamiento del negocio son:

- Modelo de casos de uso del negocio: Describe los procesos de negocio de una empresa en términos de casos de uso y actores del negocio, que se corresponden con los procesos del negocio y los clientes, respectivamente.
- Modelo de objetos del negocio: Es un modelo de objetos que describe cómo colaboran los trabajadores y las entidades del negocio dentro del flujo de trabajo del proceso de negocio.
- Especificaciones complementarias del negocio: Otras descripciones contenidas en documentos u obtenidas por otras vías; que permitan un mayor entendimiento del negocio y que contribuyan a su modelamiento.
- Glosario de términos: Lista de concepto asociados al negocio que son comúnmente usados y que deben ser del dominio del equipo de desarrollo para poder modelar el negocio y dar una solución a la problemática encontrada.

2.2 - Modelamiento de casos de uso del negocio

El modelo del negocio describe el negocio en términos de casos de usos del negocio, que corresponde a lo que generalmente se le llama *procesos*.

El modelo de Casos de Uso del Negocio es un modelo que describe los procesos de un negocio (casos de uso del negocio) y su interacción con elementos externos (actores), tales como socios y clientes, es decir, describe las funciones que el negocio pretende realizar y su objetivo básico es describir cómo el negocio es utilizado por sus clientes y socios (figura 2).



Figura 2 Estereotipos usados en el modelo de casos de uso del negocio.

El modelo de caso de uso del negocio implicará la determinación de los actores y casos de uso del negocio. Con esta actividad se pretende:

- Identificar los procesos en el negocio.
- Definir las fronteras del negocio que van a modelarse.
- Definir quién y qué interactuarán con el negocio.
- Crear diagramas del modelo de casos de uso del negocio.

Actores del negocio:

Un actor del negocio es cualquier individuo, grupo, entidad, organización, máquina o sistema de información externos; con los que el negocio interactúa. Lo que se modela como actor es el rol que se juega cuando se interactúa con el negocio para beneficiarse de sus resultados.

Son actores candidatos del negocio:

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Socios
- Proveedores
- Autoridades (legales, reguladoras, etc.)
- Propietarios sin no están dentro del negocio que se modela.
- Sistemas de información externos al negocio.
- Otras partes del negocio si este es grande y esas partes no están dentro del campo de acción del negocio que se modela.

Para cada actor del negocio que se identifica se debe escribir una breve descripción que incluya sus responsabilidades y por qué interactúa con el negocio.

Casos de uso del negocio:

Un proceso de negocio es un grupo de tareas relacionadas lógicamente que se llevan a cabo en una determinada secuencia y manera y que emplean los recursos de la organización para dar resultados en apoyo a sus objetivos.

Un caso de uso del negocio representa a un proceso de negocio, por lo que se corresponde con una secuencia de acciones que producen un resultado observable para ciertos actores del negocio. Desde la perspectiva de un actor individual, define un flujo de trabajo completo que produce resultados deseables.

Para identificar los procesos de negocio es muy importante tener en cuenta que deben generar un valor para el negocio o mitigar los costos del negocio.

Para encontrar casos de uso del negocio se pueden clasificar los procesos de negocio en tres categorías:

- Núcleo: Considere qué valor reciben los actores del negocio primarios y más importantes: los clientes. Busque los procesos del negocio respondiendo a la pregunta: ¿cuáles son los servicios básicos que un cliente recibe del negocio?.
- Soporte: Contiene las actividades que no benefician al cliente directamente. Para identificarlos se pueden buscar actividades como las siguientes: desarrollo y mantenimiento de personal, de las tecnologías de información y de la oficina, seguridad y actividades legales.
- Gerencial: Estos procesos se encuentran buscando los procesos que tienen que ver con el manejo del negocio en su conjunto. Normalmente se relacionan con el actor propietario. Busque actividades como las siguientes. Desarrollar y proporcionar información sobre el negocio a los dueños e inversionistas, preparar las metas del presupuesto a largo plazo, etc.

Clasificar un proceso en alguna de estas categorías dependen del campo de acción que se esté modelando por que, por ejemplo, si el campo de acción involucra la Gestión de Recursos Humanos, puede que los procesos de desarrollo y mantenimiento del personal sean del núcleo y no de soporte. En definitiva lo importante no es clasificar los procesos sino tener una guía para orientarse.

En la figura 3 se muestra un ejemplo asociado con la prestación de servicios en un restaurante, en el que se han identificado los tres tipos de procesos.



Figura 3 Ejemplo de los tipos de procesos.

Otra manera de encontrar los casos de uso del negocio es que los expertos del dominio describan cada actividad en el negocio existente, y entonces se agrupan estas actividades en procesos de negocio. Esta forma de identificación está asociada con el concepto de función (un grupo funcional que responde a un objetivo de la organización y que puede involucrar a varias áreas).

Para una empresa cualquiera productora, podrán definirse las funciones y procesos que se describen en la tabla 1.

Tabla 1 Ejemplo de funciones y procesos de negocio.

Función	Procesos de negocio
Distribución	• Recepción • Embarque • Transportación • Inventarios
Compras	• Elección de proveedores • Pago a proveedores • Despacho
Personal	• Cubrimiento de plantilla • Capacitación • Beneficios • Pago a trabajadores
Finanzas	• Presupuesto • Costos

Otro punto de partida para definir los procesos de negocio pueden ser los objetivos estratégicos de la organización. Dado que estos pueden ser de mucha abstracción, cada uno suele descomponerse en subobjetivos más concretos. Para cada subobjetivo no descompuesto se pudiera asignar un proceso de negocio que esté asociado a un caso de uso del negocio.

Por ejemplo, una empresa de servicios puede tener como un objetivo estratégico "Satisfacer pedidos de un cliente". Este puede subdividirse, entre otros, en: "Atender pedidos de

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

clientes” y “Solicitar insumos a proveedores”. Estos objetivos pueden servir de base para los procesos de negocio de la figura 4.

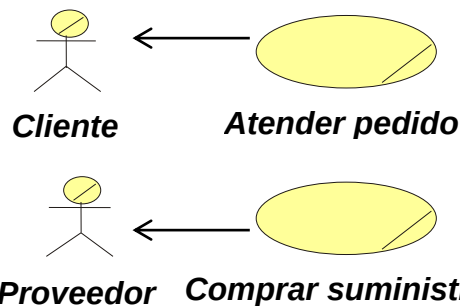


Figura 4 Ejemplo de procesos de negocio derivados de los objetivos estratégicos de la organización.

El nombre de un caso de uso del negocio debe expresar qué sucede cuando la instancia del caso de uso sea ejecutada. Por tanto debe ser nombrado en forma activa, comúnmente en gerundo (Por ejemplo, Chequeo de equipaje, Compra de suministros) o un verbo (Por ejemplo, Chequear equipaje, Comprar suministros).

Algunas consideraciones acerca de actores del negocio

- Todo lo que interacciona con el ambiente del negocio se modela con actores.
- Cada actor humano expresa un rol, no una persona específica.
- Cada actor modela algo fuera del negocio.
- Cada actor se involucra con un caso de uso, al menos como regla.
- Cada actor tiene una descripción y un nombre que explica su rol en relación al negocio.

Algunas consideraciones acerca de los casos del uso del negocio

- Su nombre y descripción breve son claras y fáciles de comprender, incluso para personas externas al equipo que modela el negocio.
- Cada caso de uso del negocio es completo desde la perspectiva de un actor externo. Por ejemplo, el caso de uso Manejar Reclamos en una compañía de seguros comienza cuando el cliente hace un reclamo. El caso de uso del negocio Manejar Reclamos no es completo a menos que incluya acerca de la decisión de la compañía de seguros con respecto al cliente y del pago por compensación, de ser apropiado.
- Cada caso de uso del negocio normalmente se involucra con, al menos, un actor. Los casos de uso del negocio se inician por actores, interactúa con actores para realizar las actividades y envía resultados.
- Es posible que un caso de uso de apoyo no interactúe con ningún actor. Esto es cierto si el caso de uso del negocio se inicia por evento interno y no tiene que interactuar con un actor para realizar las actividades.

2.2.1 - Diagrama de casos de uso del negocio.

Un diagrama de casos de uso del negocio representa gráficamente a los procesos del negocio y su interacción con los actores del negocio. En la figura 5 se muestra cómo quedaría el diagrama para el ejemplo del restaurante visto anteriormente.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

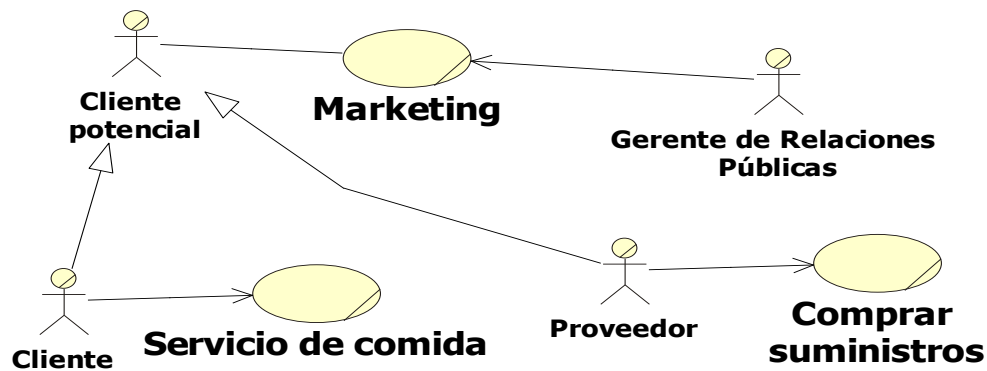


Figura 5 Ejemplo de Diagrama de casos de uso del negocio.

Se pueden agrupar casos de uso del negocio para particionar el diagrama en subdiagramas más pequeños; de manera que se definirían paquetes y estos a su vez podrían relacionarse entre sí. Un paquete (estereotipo: ) es un mecanismo de propósito general para organizar en grupos los elementos.

Los criterios para agrupar podrían ser los siguientes:

- Casos de uso de negocio que se ocupan de la misma información.
- Casos de uso de negocio usados por el mismo grupo de actores.
- Casos de uso de negocio que se ejecutana menudo en una sucesión.
- Los casos de uso de negocio más importantes.
- Un caso de uso de negocio específico y sus relaciones con los actores de negocio y otros casos de uso de negocio.

Subdividir el diagrama en varios subdiagramas debe hacerse si se hace más claro el modelo de casos de uso del negocio.

Algunos convenios que se adoptan en la representación del Diagrama de casos de uso del negocio son:

- Un caso de uso de negocio puede asociarse con uno o varios actores del negocio.
- Un caso de uso de negocio se comunica con al menos un actor, sino hay error en el modelo, excepto cuando se trata de un caso de uso abstracto o un caso de uso en una relación de generalización/especialización si en el padre se describe toda la comunicación.

Los actores del negocio actúan recíprocamente con el negocio. Ambas partes pueden tomar la iniciativa en la interacción. La navegabilidad indica quién inicia la comunicación en la interacción y se muestra con una flecha. Por cada flecha de comunicación se asume un mensaje de retorno.

El sentido de la flecha indica:

- Si apunta al caso de uso del negocio, la comunicación la inicia el actor del negocio (Figura 6a).
- Si apunta al actor del negocio, entonces la comunicación la inicia el caso de uso del negocio (Figura 6b).
- Si la comunicación la puede iniciar cualquiera de los dos, se muestra sin saetas (Figura 6c).

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

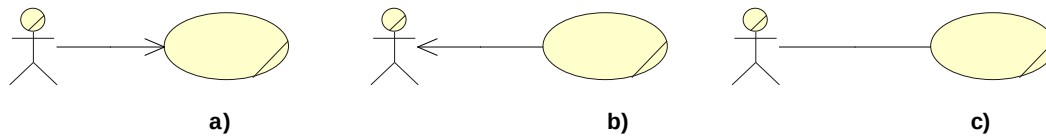


Figura 6 Navegabilidad en las relaciones de comunicación entre actores del negocio y casos de uso del negocio.

No se debe confundir navegabilidad con flujos de datos, la navegabilidad indica relación de iniciación.

Algunos convenios que se usarán en la representación de la navegabilidad son:

- La flecha de iniciación del actor al caso de uso de negocio siempre se muestra, aún si más tarde el caso de uso de negocio inicia la comunicación con el actor que lo mostró. En este último caso solo se pone la flecha del actor al caso de uso del negocio.
- El resto de las flechas pueden ser omitidas e incluirlas para esclarecer el diagrama.

También se puede representar la multiplicidad de la asociación, lo cual indica con cuántas instancias de un caso de uso de negocio una instancia de un actor de negocio puede actuar recíprocamente y, al mismo tiempo, muestra con cuántas instancias de un actor de negocio un caso de uso de negocio puede interactuar. En la figura 7 se muestra la relación del actor Pasajero con el caso de uso del negocio Check-in individual, en la que queda claro que un Pasajero solo chequea una vez y que el chequeo individual se realiza a muchos pasajeros.

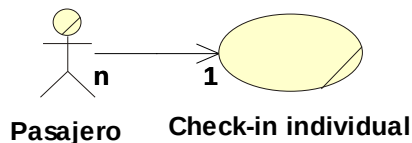


Figura 7 Ejemplo de multiplicidad entre actores del negocio y casos de uso del negocio.

El convenio que usaremos será representar la navegabilidad, pero no la multiplicidad, aún cuando no hay problemas si esta última se representa.

2.2.2 - Estructuración de los casos de uso del negocio.

Para hacer que los casos de uso de negocio sean más fáciles de comprender, reutilizar partes del flujo que se comparte entre varios casos de uso del negocio y facilitar el mantenimiento del modelo de casos de uso del negocio, es que se propone estructurar los casos de uso del negocio.

La actividad consiste en extraer el comportamiento en casos de uso del negocio que necesitan considerarse como *casos de uso abstractos*. El término abstracto se refiere a aquellos casos de uso que existen solamente para que otros casos de uso lo reutilicen. Ejemplos de tal comportamiento lo son: comportamiento común a varios casos de uso y comportamiento opcional en un caso de uso.

La mayoría de los workflows (flujo de trabajo: secuencia de actividades que producen un resultado de valor, que son realizados por trabajadores del negocio y que utilizan o generan objetos) pueden concebirse como varios subflujos que constituyen el flujo total. Algunas

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

veces varios casos de uso del negocio tienen un subflujo común, o el mismo flujo aparece en diferentes puntos de un caso de uso del negocio. Si este comportamiento común tiene un volumen importante y forma una parte independiente y delimitada de manera natural; el modelo puede ser más claro si este comportamiento se extrae a un caso de uso del negocio separado. Este nuevo caso de uso entonces es incluido en el caso de uso original (relación *include*), es una extensión de aquél (relación *extend*) o un caso de uso padre de aquél (relación de generalización).

Por tanto, en la estructuración del modelo se consideran 3 tipos de relaciones entre los casos de uso del negocio.

De manera general el caso de uso del negocio que representa la modificación se le llama *caso de uso de adición* y el caso de uso del negocio que se modifica se le llama *caso de uso base*.

Relación de Inclusión.

Una relación *include* es una relación desde un caso de uso base a un caso de uso de inclusión, que especifica cómo el comportamiento definido para el caso de uso de inclusión se inserta explícitamente dentro del comportamiento definido para el caso de uso base.

Se utiliza para dividir partes de un flujo de trabajo de cuyos resultados, y no del método para obtenerlo, depende el caso de uso base. Se puede hacer esta partición si simplifica la comprensión del caso de uso base o si el comportamiento separado puede reutilizarse en otros casos de uso.

En la figura 8 se muestra un ejemplo de reutilización en el que, independientemente de si el chequeo del equipaje es un interés de un pasajero o un guía de turista que atiende a un grupo de pasajeros, hay un subflujo común que asociada al proceso de manipulación del equipaje.

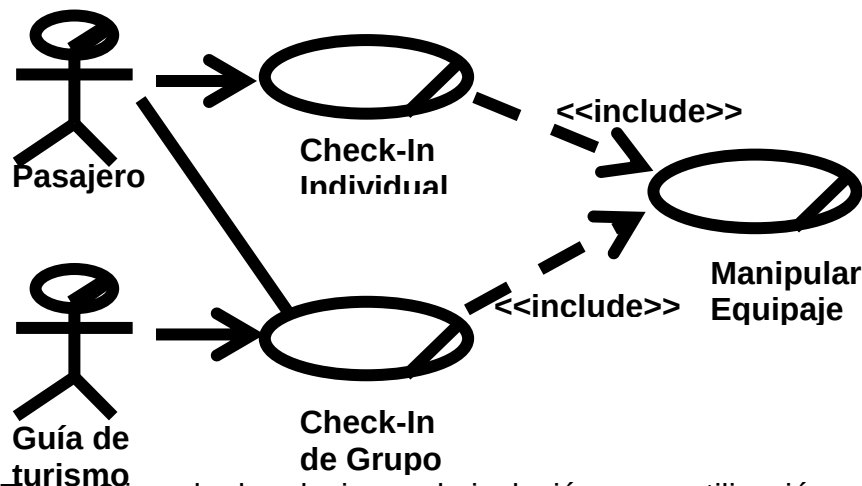


Figura 8 Ejemplo de relaciones de inclusión por reutilización.

En la figura 9 se muestra un ejemplo de particionamiento en el que puede definirse un subflujo completo que involucra a varias actividades y del que se obtiene un resultado. Este es un caso en el que el particionamiento permite una mejor comprensión del modelo.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

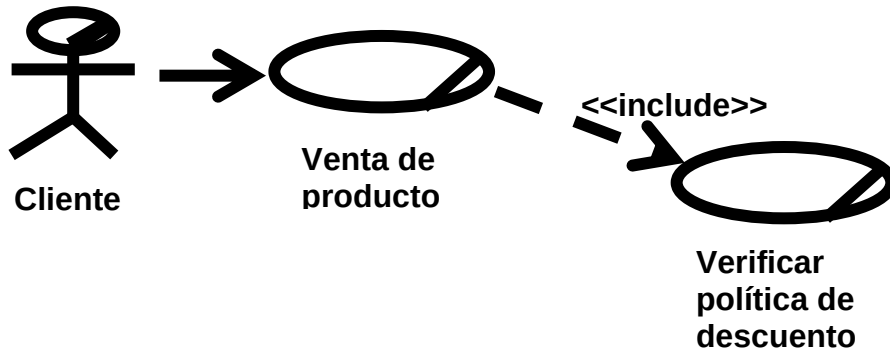


Figura 9 Ejemplo de relaciones de inclusión por particionamiento.

Más de un nivel de relaciones de inclusión dificulta la comprensión del modelo.

Una instancia de un caso de uso de negocio que sigue la descripción de un caso de uso base, también seguirá la descripción del caso de uso incluido. Una inclusión de un caso de uso de negocio siempre es abstracto y no necesita tener relaciones con un actor.

Relación de extensión

Es una relación de un caso de uso de extensión a un caso de uso base, que especifica cómo el comportamiento definido por el caso de uso de extensión puede insertarse dentro del comportamiento definido por el caso de uso base.

Una vez identificado el flujo de un caso de uso del negocio, se puede encontrar un comportamiento que es condicional u opcional. Si esa parte del comportamiento es relevante es probable que se desee describirla por separado. Una forma natural de hacerlo es describirla en una sección separada dentro de la documentación del flujo, pero otra alternativa es describirla como un caso de uso separado que sea una extensión del caso de uso original.

Esta última opción es recomendada si la parte extraída es relevante, delimitada de forma natural y si se desea mantener lo suficientemente simple el caso de uso original, o si esa parte extraída es relevante a varios casos de uso.

Condionalmente agrega un flujo al caso de uso del negocio que ya está completo de por sí.

Por tanto, una relación de este tipo (extend) se emplea para mostrar alguna de las siguientes situaciones:

- Comportamiento opcional
- Comportamiento que es ejecutado solamente bajo ciertas condiciones (Ej: disparo de una alarma)
- Flujos distintos que pueden ejecutarse en base a la selección del actor

Una instancia de un caso de uso de negocio que está opcionalmente extendido por otro caso de uso, primero sigue la descripción del caso de uso base y, entonces, si se dan las condiciones que disparan el caso de uso extendido, se sigue la descripción de ese caso de uso. Cuando se alcanza el fin del caso de uso extendido, se vuelve a seguir la descripción del caso de uso base.

En la figura 10 se retoma el proceso de negocio “Check-in individual”, en el cual se muestra que algunos pasajeros tienen que pasar un proceso adicional al que se llama “Manejo especial de equipaje”.

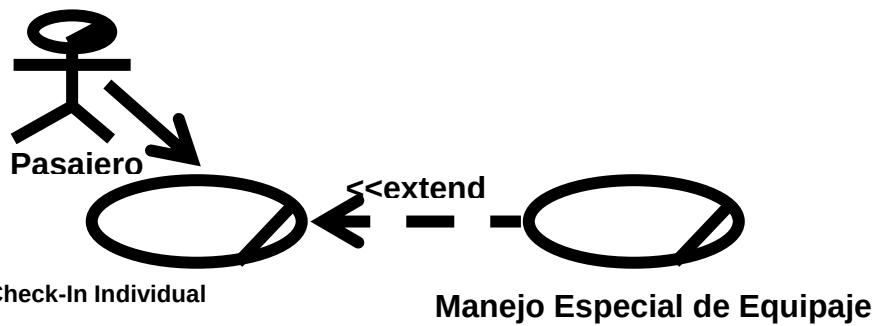


Figura 10 Ejemplo de extensión.

Los casos de uso base que son extendidos tienen que tener significado y ser completos en sí mismos, aun cuando el workflow del caso de uso extendido no se ejecute. La mayoría de los casos de uso de negocio que se extienden no pueden ejecutarse solos.

Generalización/Especialización entre casos de uso de negocio

Un caso de uso generalización es una relación de un caso de uso hijo a un caso de uso padre que especifica cómo el hijo puede especializar todo el comportamiento y características descritas para el padre.

Se utiliza para mostrar que los flujos comparten la estructura, objetivo y comportamiento. Un caso de uso padre puede especializarse en uno o más casos de uso hijos que representan formas más específicas del padre.

Una vez identificado el flujo de cada caso de uso del negocio, se pueden encontrar estructuras y comportamientos que son comunes a varios casos de uso. Para no tener que describir el mismo flujo varias veces, se puede colocar el comportamiento común en un caso de uso del negocio.

Una instancia del caso de uso que ejecuta un caso de uso hijo seguirá el flujo de eventos descrito para el caso de uso padre, pero insertando comportamiento adicional y modificando el comportamiento de acuerdo al flujo de eventos del caso de uso hijo.

En la figura 11 se muestra un ejemplo de un proceso que describe las visitas que un vendedor realiza a los clientes. Este proceso tiene varios puntos en común al inicio y término de las vistas, pero su desarrollo es diferente en dependencia de si la visita es a un cliente nuevo o ha uno que ya ha tenido contactos con la empresa.

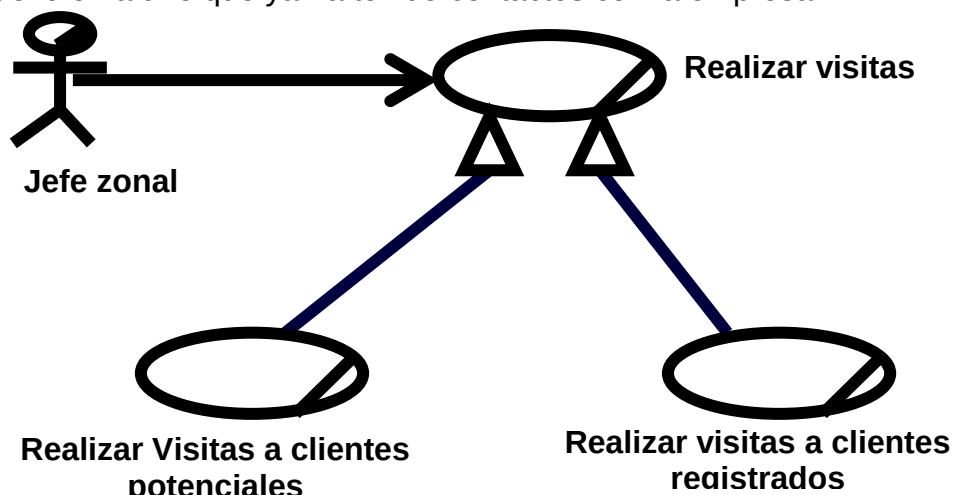


Figura 11 Ejemplo de generalización/especialización entre casos de uso del negocio.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Relación de Generalización/Especialización entre actores

Una relación de generalización de una clase hija de actor del negocio a otra clase padre de actor del negocio indica que el hijo hereda el rol que la clase padre puede jugar respecto a un caso de uso del negocio.

Varios actores del negocio pueden jugar el mismo rol en un caso de uso particular del negocio. Por ejemplo, en la figura 12 se muestran algunos procesos de negocio vinculados en la gestión de un hospital, que en el que se han identificado, entre otros, a los actores del negocio: Administrador de consulta externa y Administrador de hospitalización; que cuando interactúan con el proceso de negocio “Despachar medicamentos en farmacia”, juegan el mismo rol: “*Cliente*”. Para el campo de acción que se está modelando de un hospital, es importante describir los procesos “Asignar citas” y “Asignar camas”, siendo los interesados en estos procesos el Administrador de consulta externa y el Administrador de hospitalización, respectivamente, por lo que se modelan los actores de negocio.

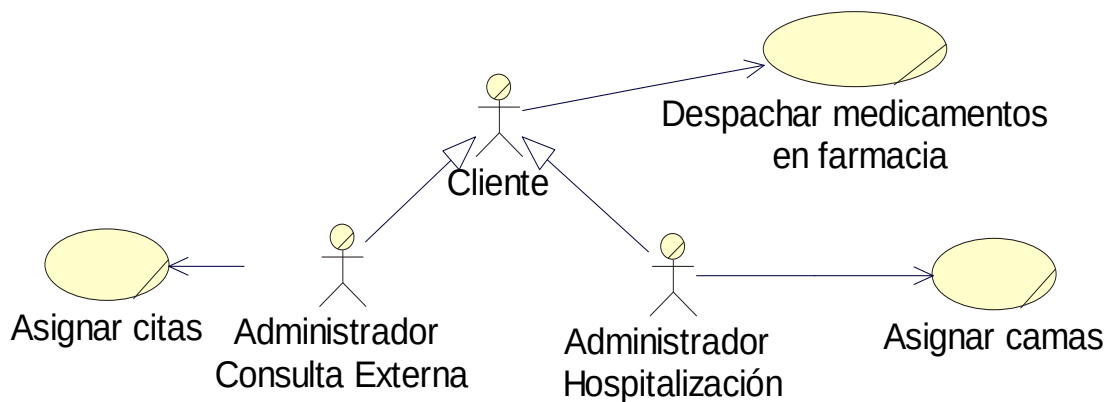


Figura 12 Ejemplo de generalización/especialización entre actores de negocio.

2.3- Realización de los casos de uso del negocio.

La realización de un caso de uso de negocio muestra cómo colaboran los trabajadores y entidades de negocio para ejecutar el proceso. Cada realización se puede documentar y utilizar los diagramas de actividad, secuencia y clases y descripciones textuales. Consideramos que los diagramas de actividad y clases y una descripción textual, es suficiente para describir completamente el proceso de negocio y dan información necesaria para los flujos de trabajo que se ejecutan posteriormente.

No obstante, se puede construir un diagrama de secuencia por cada caso de uso de negocio en el que se muestre gráficamente los detalles de la interacción entre los trabajadores del negocio y de estos los actores del negocio; y cómo se tiene acceso a las entidades de negocio durante la ejecución del caso de uso del negocio.

Un trabajador del negocio es una abstracción de una persona (o grupo de personas), una máquina o un sistema automatizado; que actúa en el negocio realizando una o varias actividades, interactuando con otros trabajadores del negocio y manipulando entidades del negocio. Representa un rol.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Las entidades de negocio representa a los objetos que los trabajadores del negocio toman, inspeccionan, manipulan, producen o utilizan durante la realización de los casos de uso de negocio. Comúnmente representan un documento o una parte esencial de un producto. Algunas veces representa cosas no tangibles como el conocimiento acerca de un mercado o cliente.

2.3.1 - Descripción textual.

La descripción textual de un caso de uso de negocio se formaliza en un documento generalmente llamado “Especificación del caso de uso de negocio”. Este documento puede tener el siguiente formato:

Nombre del caso de uso del negocio:	Nombre
Actores del negocio:	Lista de actores que se relacionan con el caso de uso, indicando quién lo inicia.
Propósito:	Breve descripción del objetivo del proceso.
Resumen: Descripción del proceso completo indicando quién inicia y cómo se inicia, cuál es el flujo de trabajo a grandes rasgos y quién finaliza el proceso y cómo se hace. La descripción debe mencionar a los actores y trabajadores del negocio y a las actividades más importantes que se ejecutan.	
Casos de uso asociados:	Listado de casos de uso incluidos y extendidos de este caso de uso base, indicando el tipo de relación.
Flujo de trabajo	
Acción del actor	Respuesta del negocio
Se indica al actor (o actores) y la interacción que tiene con el negocio.	Se describe el flujo de trabajo indicando todas las actividades del negocio que ocurren en el orden que se suceden, cuál es el trabajador del negocio que las realiza y su relación con las entidades del negocio. Deben quedar claros los puntos intermedios en los que puede finalizar el proceso.
Prioridad:	Indicar cuál es la prioridad de este proceso dentro del negocio que se modela.
Mejoras:	Mejoras que tendrá el proceso cuando algunas de sus actividades sean automatizadas.
Cursos alternos: Comportamiento que no está en el flujo normal y que ocurre bajo ciertas condiciones que pueden darse en el flujo normal.	

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

El flujo de trabajo normal de un caso de uso de negocio puede tener un flujo básico de actividades y flujos alternativos de actividades. El flujo básico cubre lo que siempre ocurre cuando el caso de uso de negocio es ejecutado, y el flujo alternativo describe alternativas que pueden darse cuando se produce el caso de uso, pero que no se dan de forma excepcional como ocurre con los flujos alternativos. Por ejemplo, en la descripción de la producción de una pieza, en dependencia de qué pieza se va a construir, hay un grupo de actividades diferentes porque pasan por diferentes máquinas y la actividad es realizada por trabajadores diferentes.

Para ejemplificar los artefactos que pueden utilizarse para describir la realización de los casos de uso del negocio, se usará como ejemplo el subdiagrama que se presenta en la figura 13.



Figura 13 Ejemplo utilizado para describir los artefactos usados en la realización de los casos de uso de negocio.

La descripción textual de este caso de uso quedaría:

Nombre del caso de uso del negocio:	Atender proyecto nuevo
Actores del negocio:	Proyectista (Inicia)
Propósito:	Registrar nuevos proyectos que sean viables de realizar técnica y económicamente.
Resumen: El caso de uso se inicia cuando el Proyectista presenta al Jefe de la obra un nuevo proyecto para su evaluación. Una vez que el Jefe de obra y el Económico evalúa, técnica y económicamente su viabilidad, se registra el proyecto. El caso de uso finaliza con la notificación al Proyectista sobre la aceptación o rechazo de su proyecto.	
Casos de uso asociados:	-
Flujo de trabajo	
Acción del actor	Respuesta del negocio
1 El Proyectista entrega las características y plano de un nuevo proyecto para su evaluación.	2 El Jefe de la obra recibe el nuevo proyecto y analiza su viabilidad técnica. a) Si no es viable técnicamente, informa al Proyectista. b) En caso contrario, se solicita al Económico que analice la viabilidad económica y se pasa al paso 4.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

3 El proyectista recibe la notificación de que su proyecto ha sido rechazado, finaliza el proceso. 7 El Proyectista recibe la notificación de que el proyecto ha sido aprobado.	4 El Económico evalúa económicamente el proyecto y entrega los resultados al Jefe de obra. 5 El Jefe de obra verifica la evaluación emitida. a) Si no es viable económicamente, informa al proyectista y se pasa al paso 3. b) En caso contrario, registra el proyecto aprobado. 6 El Jefe de la obra notifica al proyectista que su proyecto ha sido aprobado.
Prioridad:	Es el primer paso dentro del proceso de ejecución de un obra
Mejoras:	El registro de la información sobre los proyecto aumentará el control de la obra y facilitará su seguimiento. La automatización de los procesos de evaluación reducirá el tiempo de respuesta y permitirá a estos trabajadores mejorar su gestión.
Cursos alternos: -	

Los caso de uso que tienen relaciones de generalización/especialización tiene comportamiento similar y actividades que los diferencian. Para estos casos de uso se propone segmentar la descripción de los casos de uso en segmentos iguales (con todas las actividades del padre que se hace igual en los hijos, aunque los hijos pueden redefinirlos) y segmentos que los hijos realizan de forma diferente.

Para el subdiagrama de la figura 13, las descripciones de los procesos de negocio quedarán como sigue:

- Caso de uso Padre**

Nombre del caso de uso del negocio:	Realizar Visitas
Actores del negocio:	Jefe Zonal (Inicia)

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Propósito:	Realizar una visita a un cliente para vender algún servicio.
Casos de uso asociados	-
Resumen: El caso de uso se inicia cuando el Jefe de Zona comunica al vendedor el plan de visitas del día. El vendedor realiza todas las visitas en plan. Al finalizar entrega los formularios que se han ido llenando en cada visita.	
Flujo de trabajo	
Acción del actor	Respuesta del negocio
Segmento 1: Iniciar visitas	
1. El Jefe de Zona comunica el plan de visitas del día al Vendedor	2. El Vendedor recoge los formularios necesarios según el plan de visitas del día 3. Una vez en la casa del Cliente el Vendedor llena el formulario con los datos generales del Cliente 4. El Vendedor registra la fecha y hora de comienzo de la visita
Segmento 2: Desarrollo de la visita	
Segmento 3: Terminar visitas	
3. El Jefe Zonal recibe todos los documentos generados durante las visitas.	1. El Vendedor registra hora de fin de visita 2. El vendedor entrega al final del día todos los formularios con los datos de las visitas realizadas al Jefe de Zona.
Prioridad:	-
Mejoras:	El registro de la información mejorará el control que se tiene sobre el trabajo de los vendedores. Se propone que tendrán una terminal portátil de procesamiento de datos
Cursos alternos: -	

- Casos de uso Hijos

Nombre del caso de uso del negocio:	Realizar Visitas a Clientes Potenciales
Actores del negocio:	Jefe Zonal (Inicia)

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Propósito:		Realizar una visita a un cliente potencial para vender algún servicio.
Casos de uso asociados		-
Resumen: El caso de uso se inicia cuando el Jefe de Zona comunica al vendedor el plan de visitas del día. El vendedor realiza todas las visitas a los posibles nuevos clientes comunicándoles los servicios que ofrece la empresa y creando contratos de intención con aquellos clientes potenciales que soliciten algunos de los servicios ofrecidos. Al finalizar entrega los formularios que se han ido llenando en cada visita.		
Flujo de trabajo		
Acción del actor	Respuesta del negocio	
Segmento 1: Iniciar visitas		
Segmento 2: Desarrollo de la visita		
	1. El Vendedor comunica al Cliente los servicios que ofrece la empresa. 2. Si el cliente potencial solicita algún servicio de los ofertados se establecen los términos del contrato y se firma una Intención de Contrato. En caso contrario el vendedor registra la negativa del Cliente y pasa a 4. 3. Si el Cliente ha solicitado algún servicio, el Vendedor pacta la fecha y hora de la próxima visita. 4. El Cliente firma constancia de visita.	
Segmento 3: Terminar visitas		
Prioridad:	-	
Mejoras:	El registro de la información mejorará el control que se tiene sobre el trabajo de los vendedores. Se propone que tendrán una terminal portátil de procesamiento de datos.	
Cursos alternos: -		

Nombre del caso de uso del negocio:	Realizar Visitas a Clientes Registrados
Actores del negocio:	Jefe Zonal (Inicia)
Propósito:	Realizar una visita a un cliente registrado para vender algún servicio.
Casos de uso asociados	
Resumen: El caso de uso se inicia cuando el Jefe de Zona comunica al vendedor el plan de visitas del día. El vendedor realiza todas las visitas a los clientes para cobrar las pólizas de seguro según los términos de sus contratos y comunicarles los nuevos servicios que ofrece la empresa. Al finalizar entrega los formularios que se han ido llenando en cada visita.	

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Flujo de trabajo	
Acción del actor	Respuesta del negocio
Segmento 1: Iniciar visitas	
Segmento 2: Desarrollo de la visita	
	1. El Vendedor cobra póliza de seguro 2. El Vendedor comunica nuevos servicios y/o modalidades de servicios y nuevas promociones. 3. Si el Cliente se acoge a una promoción, nueva modalidad, o a un nuevo servicio se establecen los términos para actualizar el contrato y el Vendedor lo registra en el formulario. 4. El Vendedor pacta fecha y hora de próxima visita y lo registra en el formulario.
Segmento 3: Terminar visitas	
Prioridad:	-
Mejoras:	El registro de la información mejorará el control que se tiene sobre el trabajo de los vendedores. Se propone que tendrán una terminal portátil de procesamiento de datos.
Cursos alternos: -	

Cuando hay casos de uso incluidos o extendidos, en la descripción d ser invocados en el punto de inclusión o extensión indicando los resultados que se tienen de su ejecución y, para el caso de los extendidos, la condición que tiene que cumplirse.

2.3.2 - Diagrama de actividades.

Los casos de uso del negocio consisten de secuencias de actividades que, en conjunto, producen algo para el actor del negocio. El proceso (workflow) consiste de un flujo básico de una o más alternativas de flujos. La estructura del flujo se describe gráficamente con la ayuda de un *diagrama de actividad*.

El diagrama de actividad es un grafo (grafo de actividades) que contiene estados en que puede hallarse una actividad. Un estado de actividad representa la ejecución de una sentencia de un procedimiento, o el funcionamiento de una actividad en un flujo de trabajo. En vez de esperar un evento, como en un estado de espera normal, un estado de actividad espera la terminación de su cómputo. Cuando la actividad termina, entonces la ejecución procede al siguiente estado de actividad dentro del grafo. Una transición de terminación es activada en un diagrama de actividades cuando se completa la actividad precedente.

Un diagrama de actividades puede contener *bifurcaciones*, así como divisiones de control en hilos concurrentes. Los hilos concurrentes representan actividades que se pueden realizar concurrentemente por los diversos objetos o personas en una organización. La concurrencia se presenta con frecuencia a partir de la agregación, en la cual cada objeto tiene su propio hilo concurrente. Las actividades concurrentes se pueden realizar simultáneamente o en

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

cualquier orden. Un grafo de actividades es como un organigrama tradicional, excepto que permite control de concurrencia además de control secuencial: una gran diferencia.

Un diagrama de actividad describe un proceso que explora el orden de las tareas o actividades que logran los objetivos del negocio. Es similar a un diagrama de estados en el cual todos o la mayoría de los estados son *estados de actividad* y en la cual todas o la mayoría de las *transiciones* se disparan al completarse las acciones en los estados fuentes precedentes.

Por tanto, un diagrama o grafo de actividades puede contener:

Estado de actividad: Representa la ejecución de una sentencia de un procedimiento o el funcionamiento de una actividad en un flujo de trabajo (Figura 14c). Casos especiales son: un estado inicial (figura 14a) y uno o varios estados finales (figura 14b), apuntan hacia la primera actividad y salen de actividades que finalizan la ejecución del proceso, respectivamente. Las actividades que serán objeto de automatización deben de estar resaltadas.

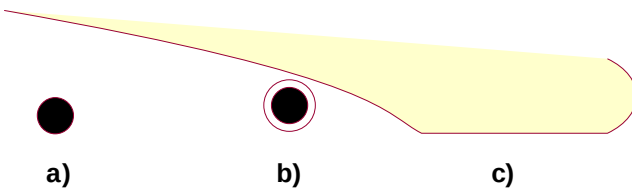


Figura 14 Notación de estados de actividad. a) Estado inicial. b) Estado final. c) Estado de actividad.

Transición: Indica cuál estado de actividad sigue a otro (figura 15).



Figura 15 Notación de transición.

Decisión: Se usa para mostrar cursos alternativos en el flujo de trabajo (Figura 16a) o como punto de unión (figura 16b). En el caso de los flujos alternativos se representa después de una actividad en la que se concluye cuál es el camino alternativo a seguir (pueden ser dos o más caminos alternativos); indicándose para cada camino alternativo el valor de la guarda. Cuando se usa como símbolo de unión es para mostrar dónde los caminos alternativos se vuelven a unir. En la figura 1, cuando se representa el flujo de trabajo del modelamiento del negocio, se observan los dos usos.

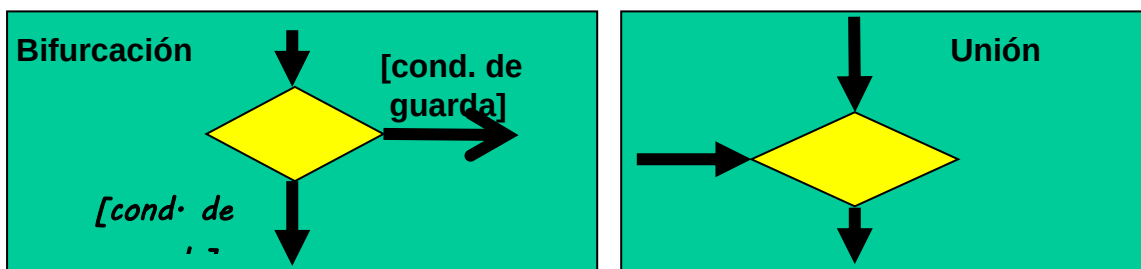


Figura 16 Notación de decisión. a) Caminos de unión. b) Unión.

Barra de sincronización: Puede usarse para mostrar una división (figura 17a) o una unión de control (figura 17b). En el caso de ser una división de control, de la barra parten múltiples flechas, cada una de ellas constituye el inicio de hilos concurrentes, es decir, de actividades que pueden desarrollarse de forma paralela. En el caso de la

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

unión de control, se representa una barra a la que fluyen varios hilo de actividades concurrentes y de la que fluye una actividad, para indicar que todos los hilos concurrentes tiene que haber concluido para que se ejecute la actividad que parte de la barra.



Figura 17 Notación de barras de sincronización. a) División de unión. b) Unión de control.

Calles (Swinlanes): Cada una de las calles representa una responsabilidad llevada a cabo por una parte de la organización (Workers-trabajadores). El orden relativo de las calles no tiene significado semántico. Cada estado de actividad se asigna a una calle y una transición puede cruzar las calles (figura 18).

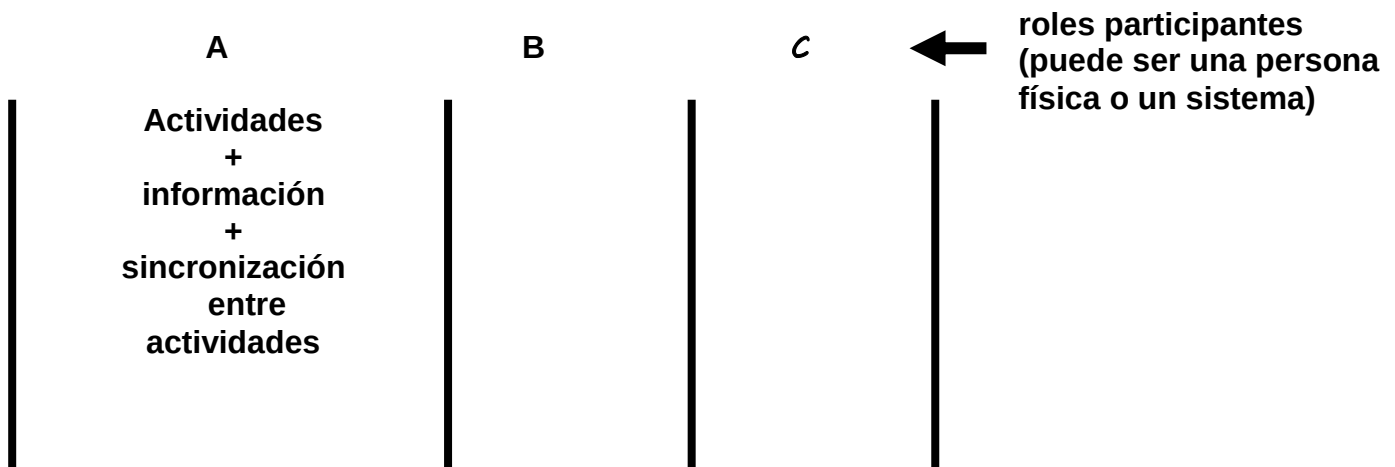
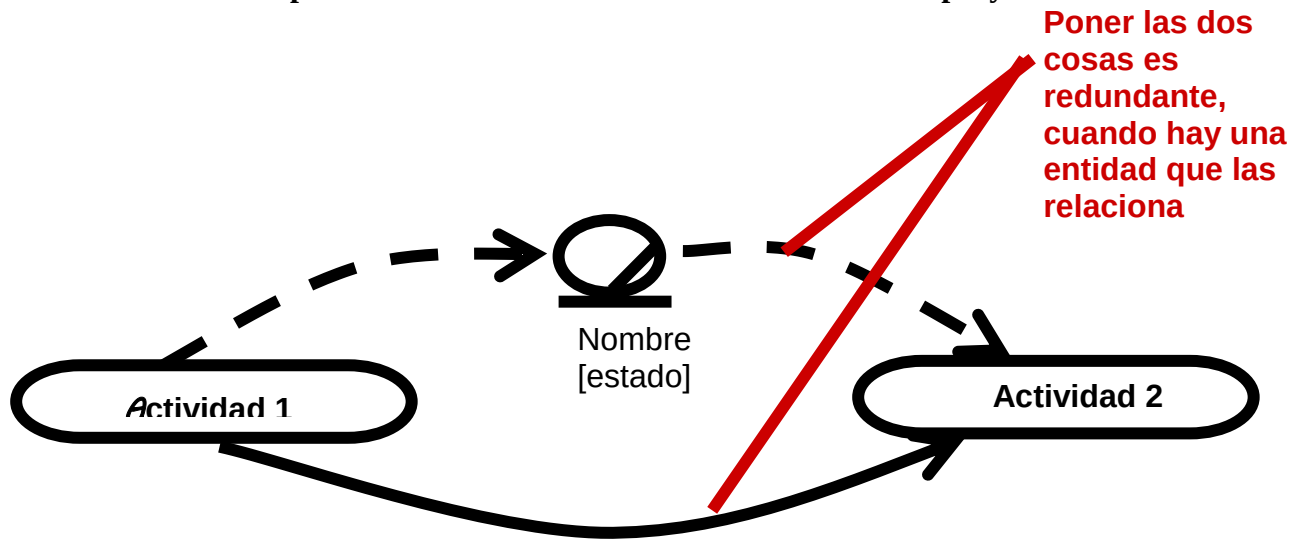


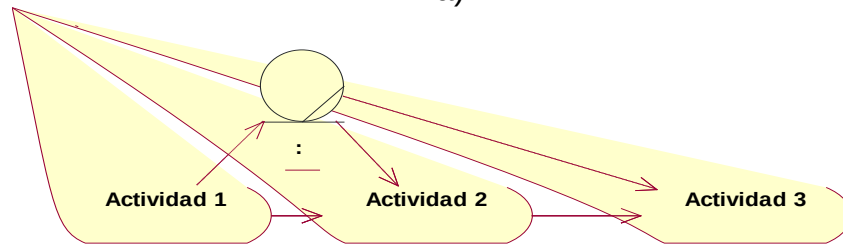
Figura 18 Notación de calles.

Flujo de objetos: Muestran cómo se generan y utilizan los objetos del negocio dentro del flujo de trabajo. En el caso de la figura 19a se muestra cómo la entidad se genera en la actividad predecesora y se usa solamente en la sucesora. En este caso es redundante indicar la transición, en caso de que no sea así hay que indicar todos los flujos (figura 19b). El mismo objeto puede manipularse por una sucesión de actividades que cambian su estado, por lo tanto, el objeto puede aparecer varias veces, pero en un estado diferente. La relación entre la actividad y el objeto se representa con una línea discontinua.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software



a)



b)

Figura 19 Notación de flujo de actividades.

En la figura 20 se muestra el diagrama de actividades que se corresponde con el caso de uso del negocio descrito anteriormente. Las actividades en amarillo serán las que se automatizarán.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

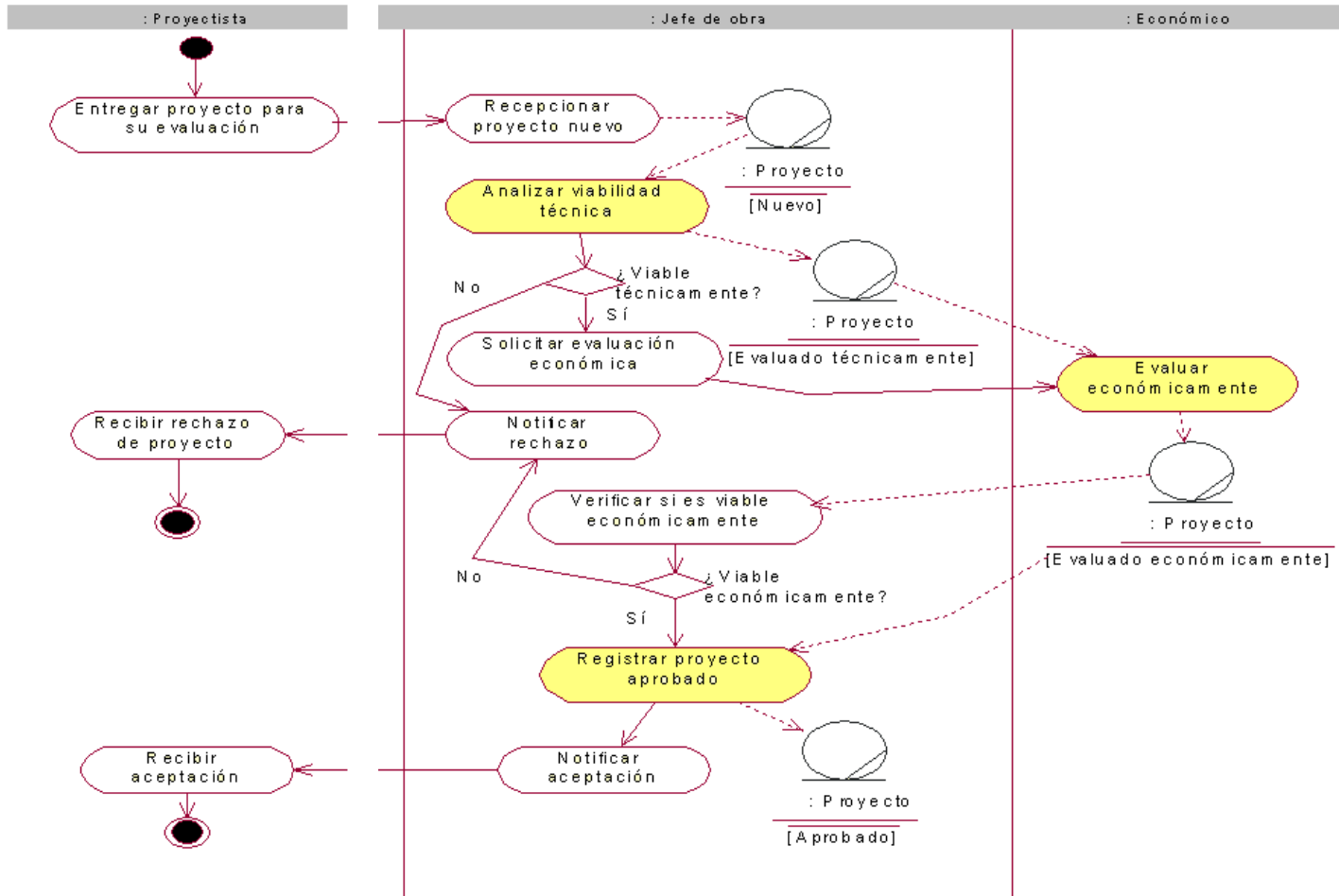


Figura 20 Diagrama de actividades del proceso de negocio “Atender proyecto nuevo”.

2.3.3 - Diagrama de clases.

El diagrama de clases, como artefacto que se construye para describir el modelo de objetos del negocio, muestra la participación de los trabajadores y entidades del negocio y la relación entre ellos. Aunque se puede construir un único diagrama, se recomienda confeccionarlo para cada caso de uso de negocio para una mejor claridad.

Como todo diagrama de clases, se pueden representar, además de la asociación, los distintos tipos de relaciones entre las entidades de negocio (agregación, composición y generalización / especialización), la cardinalidad y navegabilidad de las relaciones, pero para efectos de su utilización posterior es suficiente con mostrar la relación entre los trabajadores y estos con las entidades.

En la figura 21 se muestra el diagrama resultante del caso de uso que se ha descrito en epígrafes anteriores. Fíjese que solo se han representado los trabajadores y entidades de negocio y líneas que unen a los trabajadores que se relacionan y a estos con las entidades que manipulan.

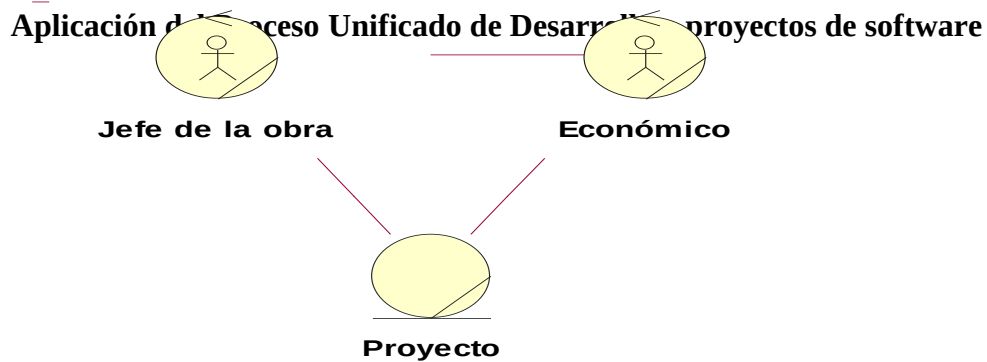


Figura 21 Ejemplo de diagrama de clases del modelo de objetos.

2.4- Reglas de negocio.

Las reglas de negocio describen políticas que deben cumplirse o condiciones que deben satisfacerse, por lo que regulan algún aspecto del negocio.

El proceso de especificación implica que hay que “identificarlas” dentro del negocio, “evaluar” si son relevantes dentro del campo de acción que se está modelando e “implementarlas” en la propuesta de solución.

Son múltiples las clasificaciones que se dan a las reglas de negocio. Sin pretender hacer un tratado sobre el tema, podría asumirse la siguiente clasificación:

1. Reglas de estructura

- Término: Conceptos en el contexto del negocio.
Ejemplo: Estudiante, Libro
- Modelo de datos: Controla que la información básica almacenada para cada atributo o propiedad de un concepto es válida.
Ejemplo: La cantidad de libros de un mismo título es mayor que cero.
- Relación: Controla las relaciones entre los datos.
Ejemplo: El estudiante solicita un libro. A la asociación entre el libro y el estudiante se le denomina Préstamo.

2. Reglas de derivación

- Inferencia: Especifican que un hecho es cierto por inferencia.
Ejemplo: Un estudiante que debe un préstamo de un libro se convierte en un estudiante moroso.
- Cálculo: Controla la obtención de información que se puede calcular a partir de la ya existente.
Ejemplo: La cantidad de libros que un estudiante tiene en préstamo es la suma de los préstamos en los que está involucrado.

3. Reglas de acción

- Flujo: Determinan y limitan cómo fluye la información a través de un sistema.
Ejemplo: Un estudiante solicita un libro en préstamo en una biblioteca, el personal que lo atiende verifica si es un estudiante moroso. En caso negativo, el personal procede a registrar el préstamo y le entrega el libro. Si no es posible, se le informa y recoge el libro para colocarlo en su estante.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Restricciones de operaciones: Especifican condiciones que deben ser ciertas para asegurarse que una operación se ejecute correctamente.
Ejemplo: A un estudiante no se le puede prestar otro libro si está clasificado como moroso.
- Estímulo y respuesta: Restrigen el comportamiento especificando cuándo y qué condiciones deben cumplirse para que una operación de respuesta sea inmediatamente ejecutada.
Ejemplo: Si se ha llegado a la fecha en que un estudiante debe entregar un libro y no lo ha hecho, se procede a enviarle un e-mail.

La descripción que se hace de las reglas de negocio es independiente de su implementación y puede expresarse en español estructurado, diagramas o descripciones textuales.

No es importante que durante el proceso de identificación se clasifiquen siguiendo los criterios anteriores u otros criterios; lo necesario es que queden formuladas en algún lenguaje. Un algoritmo que puede ayudarnos es realizar el análisis, para cada caso de uso de negocio, siguiendo el orden que se muestra en la figura 22.

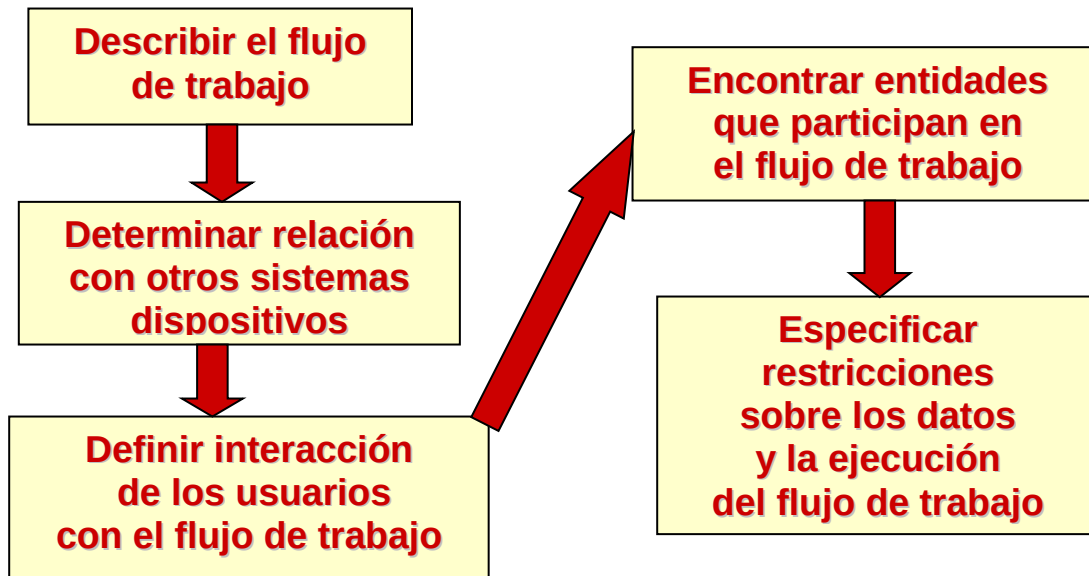


Figura 22 Algoritmo para obtener las reglas de negocio asociadas a un proceso de negocio.

2.5 - Conclusiones

El propósito del modelo de negocio es ayudarlo a usted a lograr una mejor comprensión del problema que su software tiene que resolver. De hecho, los requerimientos para la aplicación pueden ser derivados a partir del modelo de negocio.

El modelo de negocio consiste en el modelo de casos de uso del negocio y el modelo de objeto del negocio. El modelo de casos de uso del negocio describe la forma, en que el negocio es utilizado por los usuarios y los partners. Los workflows de los casos de uso del negocio se describen detalladamente utilizándose los diagramas de actividades. El modelo de objeto del negocio, que contiene las entidades del negocio y los trabajadores del negocio, identifica todos los “roles” y las cosas” en el negocio.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Jacobson, Rumbaugh y Booch no solo nos dicen qué debemos hacer, también han elaborado lista de chequeos que nos permiten revisar y perfeccionar el trabajo realizado.

Modelo de Caso de Uso de Negocio

- Cada Caso de Uso de Negocio debe describir un flujo de trabajo que produzca un resultado de valor para un Actor de Negocio.
- Cada Caso de Uso de Negocio debe estar completo desde la perspectiva de los que están fuera (Actores).
- Los Casos de Uso de Negocio deben ejecutar sólo las actividades que forman parte del negocio.
- Si el Diagrama de Actividad asociado a un Caso de Uso de Negocio sólo consta de una actividad no es un Caso de Uso de Negocio pues no es un proceso.
- No se deben considerar como Casos de Uso los distintos pasos de un mismo proceso.
- Los nombres de los Casos de Uso deben ser claros y reflejar el propósito. Para ello se deben utilizar verbos en infinitivo.
- Las relaciones entre Casos de Uso sólo pueden ser de inclusión (debe aparecer el estereotipo <include>), de extensión (aparecer el estereotipo <extend>) o de generalización-especialización.
- No pueden haber Casos de Uso de Negocio que no estén vinculados a ningún Actor de Negocio. La excepción la constituyen los Casos de Usos extendidos, incluidos y los generalizados.
- No pueden haber Actores de Negocio que no estén vinculados a ningún Caso de Uso de Negocio.
- Cada Actor de Negocio debe corresponderse con un rol y no con una persona física (un mismo rol puede ser jugado por varias personas físicas y una misma persona física puede desempeñar varios roles a la vez).
- Una generalización-especialización entre Actores de Negocio sólo tiene sentido si todos están asociados con al menos un Caso de Uso de Negocio.
- Para un Caso de Uso un Trabajador de Negocio no puede ser considerado como Actor y viceversa.
- En todo Diagrama de Casos de Uso de Negocio se debe indicar quien inicia la comunicación entre Actores y Casos de Uso. Una flecha del Actor al Caso de Uso indica que el Actor inició la comunicación. Una flecha del Caso de Uso al Actor indica que se desea comunicar un resultado o se le solicita una información al Actor.
- Demasiados Casos de Uso dificultan la comprensión.
- Casos de Uso muy extensos dificultan la comprensión.

Diagramas de Actividad

- Cada Diagrama de Actividad tiene que tener uno y sólo un estado inicial y al menos un estado final.
- De toda actividad hay que transitar siempre a: otra actividad, una decisión, una barra de sincronización o un estado final.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Toda actividad debe estar precedida por: otra actividad, una decisión, una barra de sincronización o un estado inicial.
- En el diagrama debe quedar claro el orden en que ocurren las actividades y además debe quedar claro cuando el orden de las actividades no es significativo (pueden ejecutarse en paralelo) utilizando para ello, barras de sincronización.
- La barra de sincronización no se debe usar para unir los hilos (flujos) que salen de una decisión.
- Todas las actividades que están descritas dentro de un Diagrama de Actividad tienen que ser posibles.
- Las calles sólo se deben corresponder con Unidades Organizativas Trabajadores de Negocio o Actores de Negocio.
- Una misma Entidad de Negocio puede ser utilizada como entrada (con un mismo estado) en más de una actividad.
- Siempre que una actividad del diagrama transforme una Entidad de Negocio debe señalarse ésta como entrada y salida pero con dos estados diferentes.
- Varias actividades no pueden hacer igual transformación a la misma Entidad de Negocio.
- El flujo de actividades del diagrama debe ser claro y fácil de comprender
- Para indicar en un Diagrama de Actividad la ejecución de un Caso de Uso extendido o incluido debe colocarse una actividad con el mismo nombre del Caso de Uso. Se sugiere resaltar dicha actividad (ejemplo: cambiando su color).
- Una buena práctica es resaltar dentro del diagrama aquellas actividades que se pretenden automatizar.

Diagrama de Clases del Modelo Objeto de Negocio

- Sólo debe incluir: las Entidades de Negocio, la relación que existe entre ellas y los Trabajadores de Negocio que interactúan con dichas entidades.

Además, debe existir un balance entre los artefactos que se construyen, de manera que:

- Cada Caso de Uso debe tener asociado al menos un Diagrama de Actividad que lo describa.
- Todos las Entidades de Negocio que aparecen en los Diagramas de Actividades deben aparecer en el Diagrama de Clases del Modelo Objeto de Negocio.
- Todos los Trabajadores de Negocio que aparecen en los Diagramas de Actividades manipulando alguna Entidad de Negocio deben aparecer en el Diagrama de Clases del Modelo Objeto de Negocio.

2.6 – Referencias Bibliográficas

- Rational Unified Process.
- Jacobson, I.; Booch, G. y Rumbaugh, J.; “El Proceso Unificado de Desarrollo de software”. 2000. Addison-Wesley. Páginas 115-119.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Booch, G.; Rumbaugh, J. y Jacobson, I.; “El Lenguaje Unificado de Modelado”. 2000. Addison-Wesley. Páginas 197-201, 225-239.
- Rumbaugh, J.; Jacobson, I. y Booch, G.; “El Lenguaje Unificado de Modelado. Manual de referencia. 2000. Addison-Wesley. Páginas 120-121, 157-162, 305-312.
- Von Hallen, B.; “Building a Business Rules”. [http://www.Kpiusa.com/](http://www.Kpiusa.com/ReadingRoom/ReadingRoom.htm) ReadingRoom/ReadingRoom.htm.
- Agualló, P.; “Desarrollo Cliente / servidor: ubicación de las reglas de negocio”. <http://www.ctv.es/USERS/pagullo/arti/esbr/esbr.htm>.
- Falção, H. y Fontes, J.; “¿En quién se pone el foco?. Identificando stakeholders para la formulación de la misión organizacional”. Revista del CLAD Reforma y Democracia”. No. 15 (Octubre 1999). Caracas.
- “Análisis de las partes interesadas”. <Http://wbIn0018.worldbank.org/caribbean/CaribbeanCMUOL.nsf/FILE/Db-an-pi.doc>.
- González, E.; “La empresa ante sus grupos de intereses: Una aproximación desde la literatura del análisis de los stakeholders”. Papeles de Ética y Dirección, No. 4, 1999.

Capítulo 3 Requisitos

Lograr una comunicación efectiva entre los usuarios y el equipo de proyecto y entre estos últimos, con el objetivo de llegar a un entendimiento de lo que hay que hacer, es la clave del éxito en la producción de un software. Durante muchos años muchas aplicaciones han fallado (no se culminaron o no se usaron) porque existieron incongruencias entre lo que el usuario quería, lo que realmente necesitaba, lo que interpretaba cada miembro del equipo de proyecto y lo que realmente se obtiene.

Aquí radica la importancia que en los últimos se ha dado a la identificación de los requerimientos como punto de partida en el proceso de desarrollo del software.

El flujo de trabajo de modelamiento del negocio nos enseña a describir el negocio actual y a modelar el negocio propuesto, da una visión de qué es necesario hacer para dar respuesta a la solicitud del usuario. En esta clase se enseñará cómo se obtienen los requerimientos y cómo estos son tratados en otros flujos de trabajo para convertirlos en un producto de software. El modelamiento del negocio brinda un vía natural para determinar los requerimientos del sistema de información.

3.1- Flujo de trabajo de Requerimientos

Durante el proceso de obtención de los requerimientos juega un papel esencial el cliente, que se convierte en un miembro más del equipo de proyecto por lo que el resultado de la captura de requerimientos debe ser escrito en su lenguaje.

Los trabajadores que participan en este flujo de trabajo son:

- **Analista del sistema:** Define los alcances del sistema e identifica a los actores y casos de uso que permiten modelar completa y consistentemente el sistema.
- **Especificador de casos de uso:** Describe detalladamente cada caso de uso de acuerdo a las funcionalidades que engloba.
- **Arquitecto:** Describe la vista de la arquitectura del modelo de casos de uso, definiendo la prioridad de cada caso de uso para decidir en qué iteración será desarrollado cada uno.

Los artefactos que construyen estos trabajadores son:

- **Modelo de casos de uso:** Es un modelo del sistema que contiene actores, casos de uso y sus relaciones.
- **Actor:** Terceros fuera del sistema que interactúan con él.
- **Caso de uso:** Fragmentos de funcionalidad que el sistema ofrece para aportar un resultado de valor para sus actores.
- **Descripción de la arquitectura (vista del modelo de casos de uso):** Representa los casos de uso significativos para la arquitectura ya que describen alguna funcionalidad importante y crítica o algún requisito que deba priorizarse.
- **Glosario de términos:** Términos comunes que se utilizan para describir el sistema.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- **Prototipo de interfaz usuario:** Presentación de la interfaz del producto que representa la funcionalidad contenida en los casos de uso; de manera que permita que el usuario verifique que el sistema va a satisfacer sus necesidades.

El flujo de trabajo que se realiza en el proceso de captura de requerimientos se describe en la figura 1. Es importante señalar que es necesario valorar la factibilidad de que el sistema contemple cada uno de los requerimientos deseados de acuerdo al alcance del sistema. Para aquellos requerimientos que están en el alcance, se modela el sistema.

Como parte de este modelamiento del sistema las principales actividades que se realizan son:

- Encontrar actores y casos de uso.
- Priorizar casos de uso.
- Detallar casos de uso.
- Prototipar la interfaz de usuario.
- Estructurar el modelo de casos de uso.

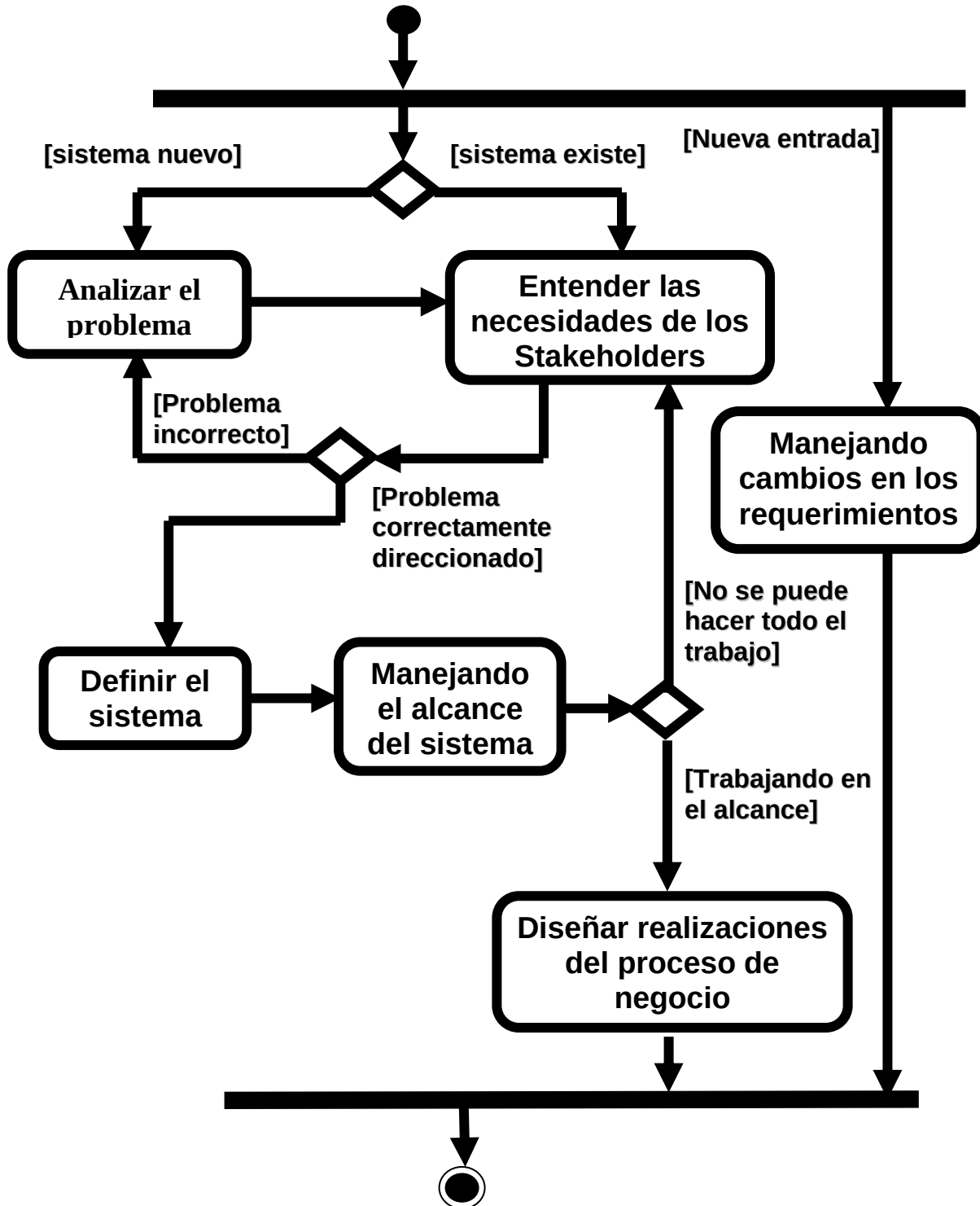


Figura 1 Flujo de trabajo de requerimientos.

3.1.1 Requerimientos

En el capítulo anterior comenzamos a analizar la situación que presentaba la galería de arte “Arte Cubano”. Como resultado de este modelamiento del negocio se llegó a que el problema a resolver es que existen dificultades con el control sobre las obras de arte (Pinturas)

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

aceptadas, que están pendientes de evaluación o rechazadas y las ventas; lo que dificulta las gestiones asociadas con el montaje de exposiciones y la venta de obras. El personal de la galería (Anfitriones, Especialistas de arte, Especialistas en economía y el Gerente General) no puede realizar el trabajo con la eficiencia requerida, lo que conlleva a pérdidas económicas y no se satisface la misión social de la galería como promotor de la cultura nacional. Esto provoca que los artistas pierdan confianza en la galería y sea poco visitada. Se necesita desarrollar una aplicación que controle las solicitudes recibidas, la evaluación de las obras y la venta de obras. Por lo tanto, se necesita desarrollar una aplicación que controle las solicitudes recibidas, la evaluación de las obras y la venta de obras.

En este capítulo se usará esencialmente este caso de estudio para ejemplificar los artefactos que se describan.

Las áreas de esfuerzo del análisis de requerimientos están en (Figura 2):

- Reconocimiento del problema como lo ve el usuario.
- Evaluación del problema y síntesis de la evaluación: Como evaluación se entiende a la definición de los datos observables, la evaluación del flujo y contenido de la información. La definición de las funciones del software, entender el comportamiento del software ante eventos, el establecimiento de las características de la interfaz y el descubrimiento de restricciones adicionales de diseño. Toda esta evaluación se sintetiza en la definición en detalle de los datos, funciones y el comportamiento del sistema.
- Modelado: Creación de modelos que ayuden a entender la entidad a construir.
- Construcción de un prototipo de alto nivel del sistema.
- Revisión por parte del usuario.
- Firma del contrato si las partes están de acuerdo.

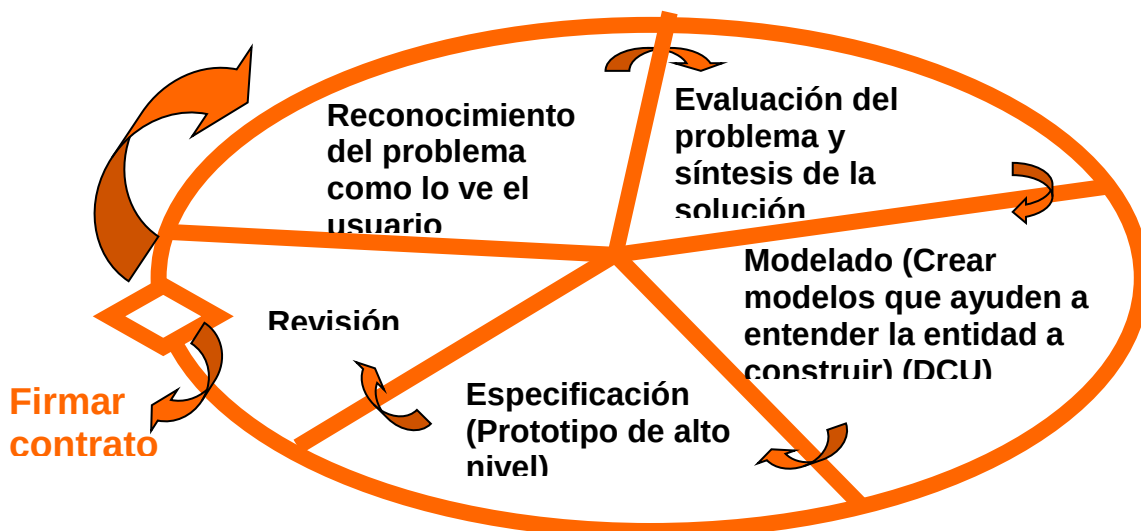


Figura 2 Áreas de esfuerzo del análisis de requerimientos.

Todas las ideas que los clientes, usuarios y miembros del equipo de proyecto tengan acerca de lo que debe hacer el sistema, deben ser analizadas como candidatas a requisitos. Los requisitos se pueden clasificar en: funcionales y no funcionales. Los requerimientos funcionales son capacidades o condiciones que el sistema debe cumplir, en cambio, los no funcionales se refieren a cualidades del sistema.

Los requisitos pueden, además, clasificarse en:

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Nominales (funcionales): Objetivos y metas para un sistema que tienen que estar presentes para que el cliente esté satisfecho.
- Esperados (no funcionales y funcionales): Están implícitos, puede que el usuario no los declare, pero si no se satisfacen el cliente no está conforme.
- Innovadores (funcional y no funcional): Características que van más allá de las expectativas del cliente.

Requerimientos funcionales

En la realización de los casos de uso del negocio, se obtienen las actividades que serán objeto de automatización. Estas actividades no son exactamente los requerimientos funcionales, pero si son el punto de partida para identificar qué debe hacer el sistema.

Los requerimientos funcionales no alteran la funcionalidad del producto, esto quiere decir que los requerimientos funcionales se mantienen invariables sin importarle con que propiedades o cualidades se relacionen. Los requerimientos no funcionales también añaden funcionalidad al producto, pues hacen que un producto sea fácil de usar, seguro, o interactivo demanda cierta cantidad de procesamiento. Sin embargo la razón fundamental de que esta funcionalidad sea parte del producto es brindarle a este las características deseadas.

Por ejemplo, para el caso de la galería de arte, podrían definirse como requerimientos funcionales, entre otros, a los siguientes:

1. Registrar solicitud de exposición.
2. Asignar automáticamente código a una solicitud.
3. Asignar solicitud a un especialista de arte.
4. Enviar correo a especialista de arte sobre solicitud asignada.
5. Registrar información de artista.
6. Consultar en el sistema automatizado de RRHH los especialistas de arte.

Requerimientos no funcionales

Los requerimientos no funcionales son propiedades o cualidades que el producto debe tener. Debe pensarse en estas propiedades como las características que hacen al producto atractivo, usable, rápido o confiable, por ejemplo, pudiera desearse que el sistema responda dentro de un intervalo de tiempo especificado o que obtenga los resultados de los cálculos con un nivel de precisión dado. En muchos casos los requerimientos no funcionales son fundamentales en el éxito del producto. Normalmente están vinculados a requerimientos funcionales, es decir una vez se conozca lo que el sistema debe hacer podemos determinar cómo ha de comportarse, qué cualidades debe tener o cuán rápido o grande debe ser.

Estas propiedades no se requieren por ser actividades fundamentales del producto como pudieran serlo el cálculo, manipulación de datos, etc., sino porque el cliente desea que las actividades fundamentales se realicen de cierta forma o tengan determinadas cualidades.

Los requerimientos no funcionales forman una parte significativa de la especificación. Son importantes para que clientes y usuarios puedan valorar las características no funcionales del producto, pues si se conoce que el mismo cumple con la toda la funcionalidad requerida, las

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

propiedades no funcionales, como cuán usable, seguro, conveniente y agradable, pueden marcar la diferencia entre un producto bien aceptado y uno con poca aceptación.

Los requerimientos no funcionales incluyen:

- Conjunto de facilidades.
- Capacidades.
- Seguridad.

Existen múltiples categorías para clasificar a los requerimientos no funcionales, siendo las siguientes representativas de un conjunto de aspectos que se deben tener en cuenta, aunque no limitan a la definición de otros.

- **Requerimientos de apariencia o interfaz externa**

Este tipo de requerimiento describe la apariencia del producto. Es importante destacar que no se trata del diseño de la interfaz en detalle sino que especifican cómo se pretende que sea la interfaz externa del producto. Atendiendo a este criterio podemos decir, por ejemplo, producto debe ser:

- Muy legible.
- Simple de usar.
- Autoritario, para que los usuarios se sientan confiados.
- Discreto para que los usuarios no lo noten.
- Colorido y atractivo para los niños.
- Interactivo.
- Profesional o tipo ejecutivo.

Los requerimientos de apariencia se vuelven más importantes a medida que los productos de software se mueven hacia áreas más orientadas al consumidor. Productos muy sofisticados como las cámaras digitales, cámaras de video u organizadores personales le dan la impresión a los clientes de ser muy fáciles de operar, sin embargo contienen un gran nivel de funcionalidad.

Si se siente tentado a usar el prototipo para describir los requerimientos de apariencia e interfaz externa del producto, recuerde que el mismo no expresa los requerimientos, sino el resultado de estos.

Requerimientos de apariencia también pueden ser necesidades de cumplir con normas estándares como por ejemplo las interfaces de sistemas tipo Windows/Apple/Motif, o con los estándares de la empresa para la cual se esté desarrollando el software.

- **Requerimientos de Usabilidad**

Estos requerimientos describen los niveles apropiados de usabilidad, dados los usuarios finales del producto, para ello debe revisarse la especificaciones de los perfiles de usuarios y echarle un vistazo a las clasificaciones de sus niveles de experiencia.

Debe conocerse:

- ¿ Qué tipo de personas son ?
- ¿ Qué tipo de producto necesitan para realizar su trabajo ?

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- ¿ Qué tipo de requerimiento de usabilidad haría el producto adecuado para ellos?

Los requerimientos de usabilidad se derivan de una combinación de lo que el cliente está tratando de lograr con el producto y lo que los usuarios finales esperan del mismo, estos elementos deben tenerse claro antes de escribirlos. Estos requerimientos también pueden cubrir otros aspectos como:

- Porcentaje de aceptación por los usuarios.
 - Productividad ganada con la introducción del producto.
 - Proporción de errores (y por consiguiente su reducción).
 - Facilidad de uso por personas que hablen otros idiomas distintos al del país donde el producto fue creado.
 - Accesibilidad para personas discapacitadas.
 - Consistencia en la interfaz de usuario.
 - Documentación de usuario, material de entrenamiento.
 - (Nunca debe olvidarse!) Facilidad de uso por personas sin experiencia previa con las computadoras
- Requerimientos de Rendimiento

Imponen condiciones a los Requerimientos Funcionales. Por ejemplo, para una acción específica pueden definirse parámetros tales como: Velocidad de procesamiento o cálculo, Eficiencia, Disponibilidad, Precisión, Tiempo de respuesta, Tiempo de recuperación y Aprovechamiento de los recursos.

La necesidad de velocidad debe ser genuina, muchas veces se desea que las cosas se realicen rápidamente sin que exista una razón real para esto, por ejemplo para producir un reporte mensual, no hay necesidad de hacerlo rápidamente, por otro lado el éxito del producto puede depender de la velocidad, y, en el caso de los sistemas en tiempo real, muchos de los requerimientos de rendimientos son indispensables para su funcionamiento.

- Requerimientos de Soporte

Abarcan todas las acciones a tomar una vez que se ha terminado el desarrollo del software con motivos de asistir a los clientes de este así como lograr su mejoramiento progresivo y evolución en el tiempo. Pueden incluir: Pruebas, Extensibilidad, Adaptabilidad, Mantenimiento, Compatibilidad, Configuración, Servicios, Instalación, Internacionalización y Requerimientos de Portabilidad

Deben usarse para especificar que el producto de software podrá ser usado en diferentes plataformas, por ejemplo:

El producto podrá ser usado bajo el sistema operativo Linux.

Es importante tener en cuenta que el documento de los requerimientos es un contrato que se establece con el cliente, en este caso se está afirmando que el producto será implementado en algún momento en la nueva plataforma. Podrá especificarse también las características de esta así como el tiempo necesario para realizar la transición.

- Requerimientos de Seguridad

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Este es quizás el tipo de requerimiento más difícil, que provocará los mayores riesgos si no se maneja correctamente. La seguridad puede ser tratada en tres aspectos diferentes:

- **Confidencialidad:** La información manejada por el sistema esta protegida de acceso no autorizado y divulgación.
- **Integridad:** la información manejada por el sistema será objeto de cuidadosa protección contra la corrupción y estados inconsistentes, de la misma forma será considerada igual a la fuente o autoridad de los datos. Pueden incluir también mecanismos de chequeo de integridad y realización de auditorías.
- **Disponibilidad:** Significa que los usuarios autorizados se les garantizará el acceso a la información y que los dispositivos o mecanismos utilizados para lograr la seguridad no ocultarán o retrasarán a los usuarios para obtener los datos deseados en un momento dado.

La seguridad de un sistema no solo tiene en cuenta la seguridad del sistema propiamente dicho sino, además, el ambiente en el que se usará el sistema., por lo que se tiene que contemplar la seguridad física del lugar donde se usa la aplicación, los controles administrativos que se establecen de acceso al sistema y las regulaciones legales que afecta o determinan el uso del sistema y que serán tenidas en cuenta si se incumple (figura 3).



Figura 3 Niveles de seguridad.

La seguridad es un requerimiento no funcional que genera posiblemente requerimiento funcionales. ¿Cuáles requerimientos?. Depende de la propuesta de seguridad que se tenga para el sistema.

Supongamos que la propuesta incluye que a la aplicación solo tienen acceso determinando usuarios a determinadas opciones del sistema. Esto implica que el sistema tiene que permitir:

1. Validar el ingreso al sistema de un usuario.
2. Actualizar usuarios.
3. Actualizar perfiles de usuario.
4. Asignar perfil a usuario.
5. Actualizar opciones del sistema.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

6. Asignar opciones a perfil.
7. Cambiar contraseña.

Si el sistema contemplará realizar copias de seguridad, fueran realizadas automáticamente cada cierto tiempo que decida un administrador del sistema, se añadirían a los requerimientos anteriores:

8. Hacer copia de seguridad.
9. Configurar copia de seguridad.

Si, además, el sistema necesitara llevar el control de quién realiza cada modificación de información, se adicionaría los siguientes requerimientos:

10. Registrar transacción en bitácora.
11. Consultar transacciones realizadas.

En la figura 4 se muestra un ejemplo del diagrama de casos de uso que se obtendría en el que se reflejan los casos de uso asociados con el tema de seguridad y que incluyen las funcionalidades anteriores.

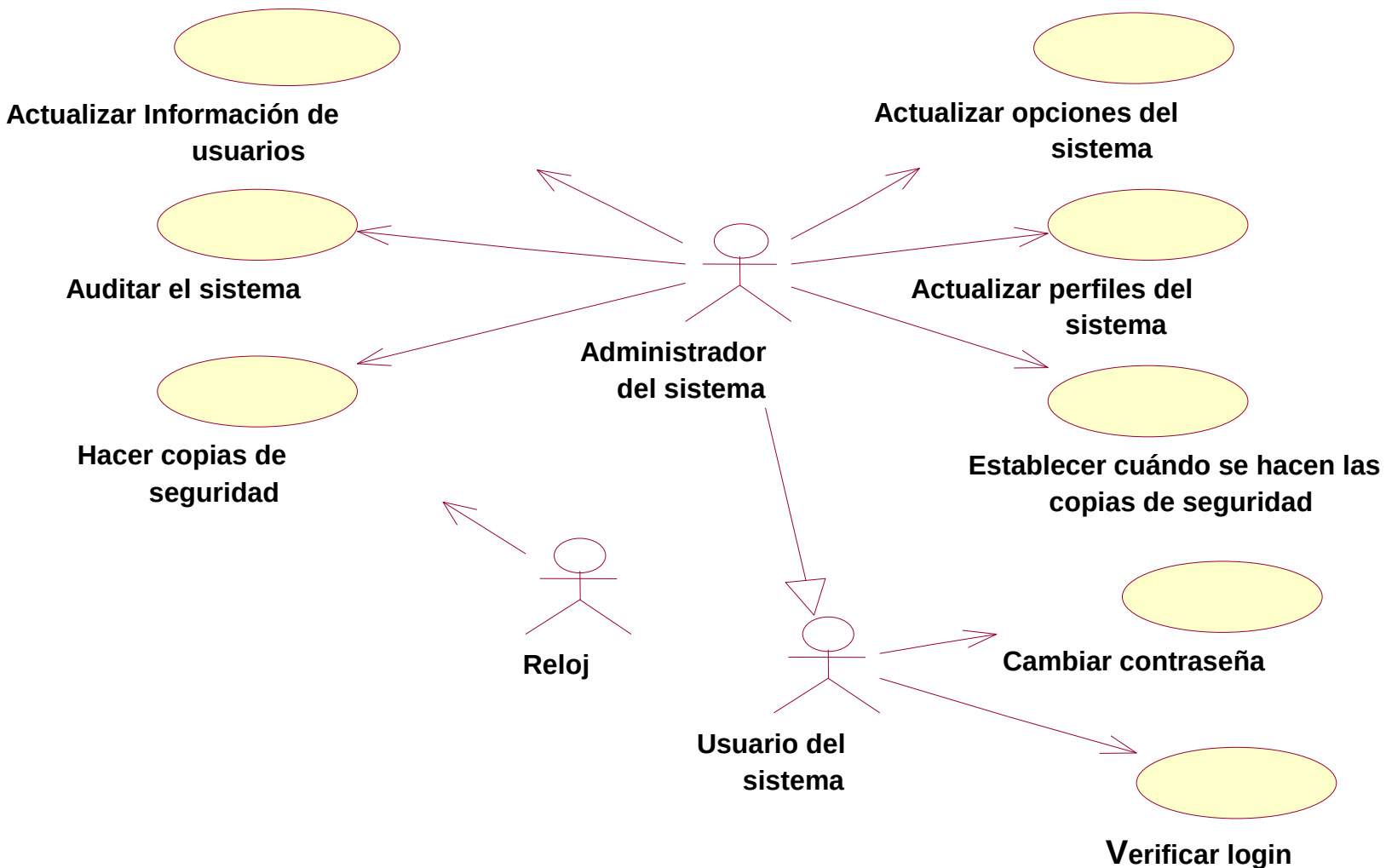


Figura 4 Ejemplo de Diagrama de casos de uso de seguridad.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Requerimientos Políticos y Culturales

Son factores especiales que pudieran hacer el producto no utilizable debido a costumbres humanas, preferencias o prejuicios. La razón fundamental de este tipo de requerimientos aparece cuando se trata de vender el producto en otro país especialmente con una cultura e idioma diferente al país en que se produjo.

También deben ser considerados los requerimientos políticos que muchas veces pueden resultar algo controversiales, o no muy recomendados, sin embargo existen y por tanto son incluidos en esta categoría. Muchas veces no se conoce la justificación de ellos, pues pueden surgir únicamente de la orden de una persona de alto nivel jerárquico dentro de la sociedad o empresa.

Cabe destacar que en este contexto el término político no se aplica para hacer referencias a la política al nivel de país, sino también a escalas menores como es el caso del nivel de empresa, sucursal o departamento, por solo citar algunos.

Ejemplos de este tipo de requerimiento son:

El producto no debe manejar facturas de la empresa A.

El producto no debe contener expresiones que resulten contradictorias a miembros del Grupo de Protección Ambiental.

En otras ocasiones, sucede que un requerimiento no puede ser introducido en una de las categorías regulares, y se toma como solución incluirlo en esta.

- Requerimientos Legales

Son los requerimientos que estipulan las formas en que el software cumple con las leyes vigentes. Incluso para el software que se construye para ser usado dentro de una organización deben observarse las leyes internas de la institución para lograr su cumplimiento por parte del sistema. Muchas veces será necesario acudir a los abogados para la obtención de estos requerimientos.

También será necesario enunciar el cumplimiento con estándares, como por ejemplo, la norma ISO¹ 9000.

- Requerimientos de confiabilidad

Los requerimientos caracterizan la respuesta del sistema ante los fallos o indican cuán robusto de este, posibles factores a ser considerados son:

- Frecuencia y severidad de los fallos.
- Protección contra fallos.
- Recuperación.
- Predicción de fallos.
- Tiempo medio entre fallos.

- Requerimientos de interfaz Interna

Con este tipo de requerimiento se enuncian las diferentes vías de interactuar con el sistema a través de software, pudiera declararse el soporte de modelos de objetos estándares tales como COM, DCOM, CORBA, etc. También se especifican las interfaces a

¹ International Standards Organization.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

componentes comprados o reusados de otras aplicaciones, u otros componentes usados que quedan fuera del alcance del documento de especificación de requerimiento que se este realizando.

- **Requerimientos de Ayudas y Documentación en línea**

Se incluye en caso de existir requerimientos vinculados al sistema de ayuda, documentación en línea, soporte técnico, etc. En esta sección debe incluirse también las instrucciones de instalación, fichero README.TXT en el directorio de los instaladores, estilo de etiquetamiento que serán incluidos en el código como es el caso de los enunciados de los derechos de autor, logos corporativos, iconos y otros elementos de la interfaz de usuario que deban mantenerse consistente con el resto de la documentación.

- **Requerimientos de Software**

Debe mencionarse el software del que se debe disponer, por ejemplo: Sistema Operativo Windows 95 o Superior; Maquina Virtual de Java versión 1.3 o Superior; etc.

- **Requerimientos de Hardware**

Al igual que en la sección anterior enunciar aquí los elementos de hardware de que se disponen, por ejemplo: se requiere disponer de un MODEM estándar o una tarjeta digitalizadora de video, etc.

- **Restricciones en el diseño y la implementación**

Este tipo de requerimiento especifica o restringe la codificación o construcción de un sistema, son restricciones que han sido ordenadas y deben ser cumplidas estrictamente . Ejemplos de ellas son:

- Estándares requeridos.
- Lenguajes de programación a ser usados para la implementación.
- Uso obligatorio de ciertas herramientas de desarrollo.
- Restricciones en la arquitectura o el diseño.

- **Bibliotecas de clases.**

Para el caso de estudio, podría definirse como requerimientos no funcionales, los siguientes:

- La interfaz del sistema debe ser a través de una página Web, personalizada de acuerdo al tipo de usuario que acceda al sistema.
- Para la interacción con el sistema de RRHH está previsto que el sistema tenga acceso de lectura a la tabla de especialistas de arte y técnicas. Esta Base de datos estará en un servidor central, por lo que hay que retroalimentar al usuario si la conexión no puede efectuarse.

3.1.2 – Modelo de casos de uso del sistema

Actores del sistema

Cada trabajador del negocio que tiene actividades a automatizar es un candidato a actor del sistema. Si algún actor del negocio va a interactuar con el sistema, entonces también será un actor del sistema.

Los actores del sistema:

- No son parte de él.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Pueden intercambiar información con él.
- Pueden ser un recipiente pasivo de información.
- Pueden representar el rol que juega una o varias personas, un equipo o un sistema automatizado.

En el ejemplo de la galería de arte, los trabajadores Anfitrión y Sistema Automatizado de Recursos Humanos se convertirán en actores del sistema. En el caso del Anfitrión, se plantea que las actividades: Registrar solicitud y especialista asignado a una solicitud, entre otras, serán automatizadas, por lo que se convierte en un actor que brinda información al sistema. El Sistema Automatizado de Recursos Humanos, no tiene una actividad a automatizar, pero se le pide información que él controla y esta actividad se señala como actividad a automatizar, por lo que este sistema se convierte también en actor.

Casos de uso

Los casos de uso son artefactos narrativos que describen, bajo la forma de acciones y reacciones, el comportamiento del sistema desde el punto de vista del usuario. Por lo tanto, establece un acuerdo entre clientes y desarrolladores sobre las condiciones y posibilidades (requisitos) que debe cumplir el sistema.

Los casos de uso candidatos también se encuentran entre las actividades a automatizar. Esto no significa que una actividad se convierta en un caso de uso porque un caso de uso es un proceso que da un resultado de valor para un actor determinado y una secuencia de actividades a automatizar pueden implicar pasos dentro de un caso de uso.

En el ejemplo de la galería, se pueden agrupar los requerimientos funcionales, descritos anteriormente, en el proceso “Controlar solicitudes de obras de arte”, que se presenta como un caso de uso.

En algunos sistemas se tienen actividades que se ejecutan periódicamente cuando llega cierto instante de tiempo, como por ejemplo, el cálculo de intereses de los clientes en los bancos se realiza todas las noches. Para modelar esto se define un actor ficticio (Reloj) que es el que “dispara” la ejecución del caso de uso (figura 5).

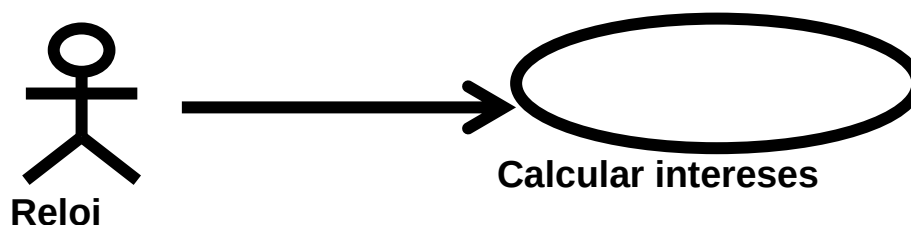


Figura 5 Manejo del tiempo.

La definición de los casos de uso se puede perfeccionar partiendo de que se duplique completamente parte del comportamiento con otros casos de uso o cuando un caso de uso es complejo y largo y su separación facilita que sean manejables y comprensibles. La solución es crear casos de uso independientes definiendo relaciones de inclusión, extensión y generalización / especialización. Esto implica que hay que rescribir el flujo de trabajo de los casos de uso.

- Relación de inclusión

Una relación *include* es una relación desde un caso de uso base a un caso de uso de inclusión, que especifica cómo el comportamiento definido para el caso de uso de

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

inclusión se inserta explícitamente dentro del comportamiento definido para el caso de uso base.

Se utiliza para dividir partes de un flujo de trabajo de cuyos resultados, y no del método para obtenerlo, depende el caso de uso base. Se puede hacer esta partición si simplifica la comprensión del caso de uso base o si el comportamiento separado puede reutilizarse en otros casos de uso.

En la figura 6 se muestra un ejemplo de dos casos de uso que comparten funcionalidades que para ambos casos siempre se realizan.

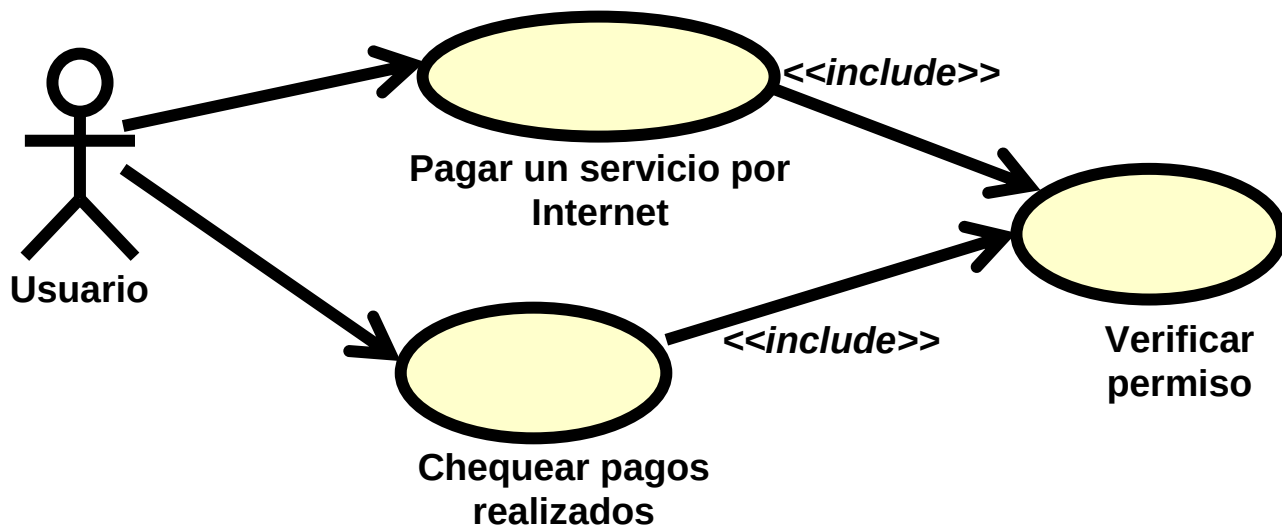


Figura 6 Ejemplo de casos de uso con funcionalidad compartida.

En la figura 7 se muestra un ejemplo de un caso de uso que contiene un subflujo con una relativa independencia que, aunque no se reutiliza, tiene un resultado de valor por lo que puede representarse como un subproceso.

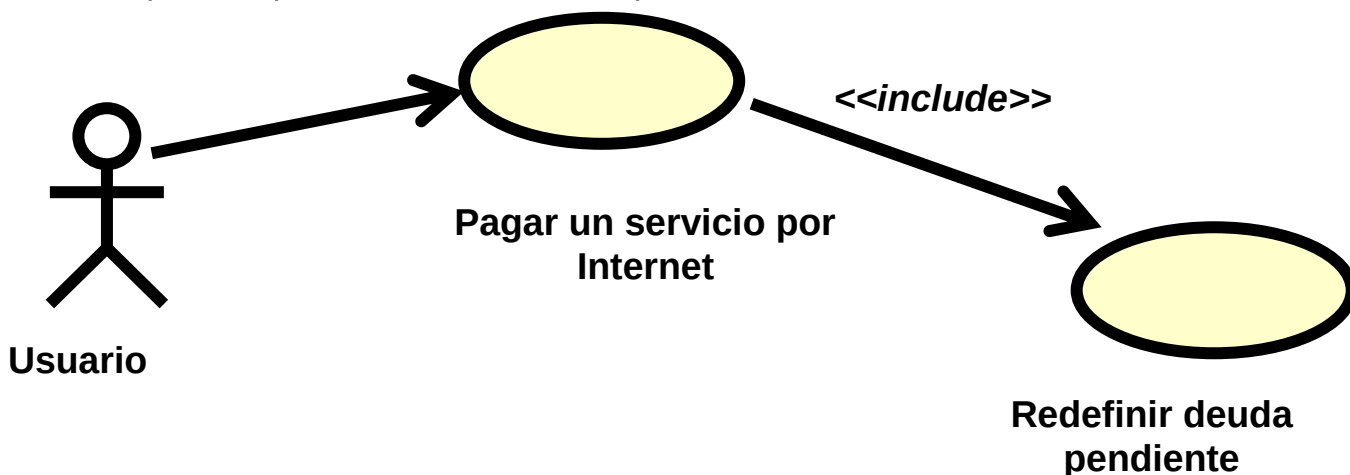


Figura 7 Ejemplo de particionamiento sin reutilización que hace más comprensible el modelo.

- Relaciones de extensión

Es una relación de un caso de uso de extensión a un caso de uso base, que especifica cómo el comportamiento definido por el caso de uso de extensión puede insertarse dentro del comportamiento definido por el caso de uso base.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Una vez identificado el flujo de un caso de uso del negocio, se puede encontrar un comportamiento que es condicional u opcional. Si esa parte del comportamiento es relevante es probable que se desee describirla por separado. Una forma natural de hacerlo es describirla en una sección separada dentro de la documentación del flujo, pero otra alternativa es describirla como un caso de uso separado que sea una extensión del caso de uso original.

Esta última opción es recomendada si la parte extraída es relevante, delimitada de forma natural y si se desea mantener lo suficientemente simple el caso de uso original, o si esa parte extraída es relevante a varios casos de uso.

Condicionamente agrega un flujo al caso de uso del negocio que ya está completo de por sí.

Por tanto, una relación de este tipo (extend) se emplea para mostrar alguna de las siguientes situaciones:

- Comportamiento opcional
- Comportamiento que es ejecutado solamente bajo ciertas condiciones (Ej: disparo de una alarma)
- Flujos distintos que pueden ejecutarse en base a la selección del actor

En la figura 8 se muestra un ejemplo de un caso de uso donde la lógica interna provoca un comportamiento opcional que se da o no, pero en caso de que se dé sigue dos caminos alternativos.

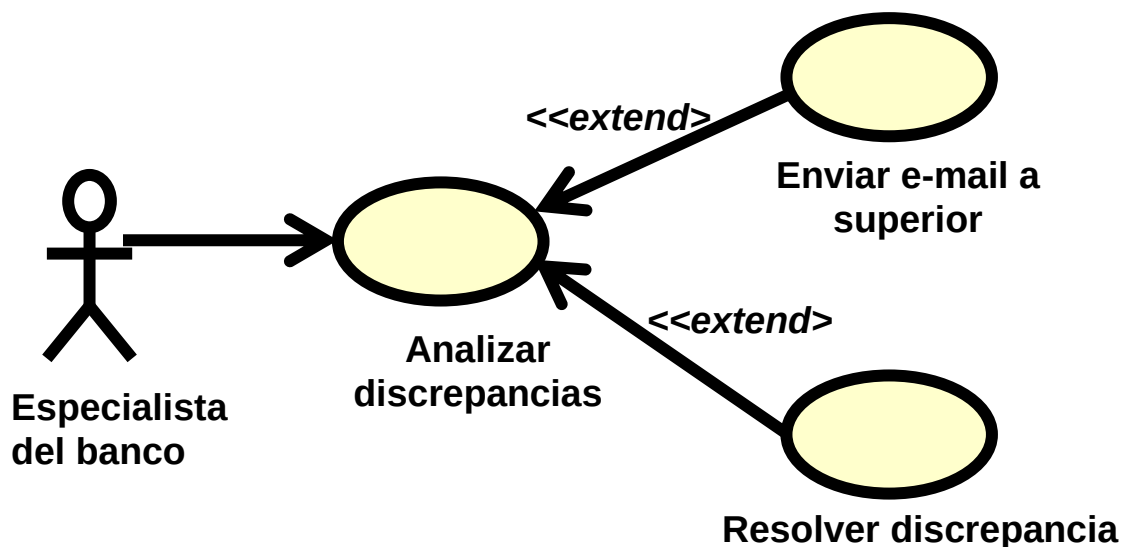


Figura 8 Ejemplo de comportamiento opcional

En la figura 9 se muestra un comportamiento que es ejecutado solo bajo ciertas condiciones, que en este caso están asociadas con el hecho de que no sea posible pagar un servicio en Internet con la cuenta que se dispone y el sistema busque automáticamente si el cliente tiene saldo suficiente en alguna otra cuenta que haya declarado.



Figura 9 Ejemplo de comportamiento que se ejecuta bajo ciertas condiciones.

En la figura 10 se muestra un ejemplo en el que el actor puede seleccionar que se ejecute cierto subflujo ya que , una vez que él encuentra discrepancias, en su decisión reportarlas.

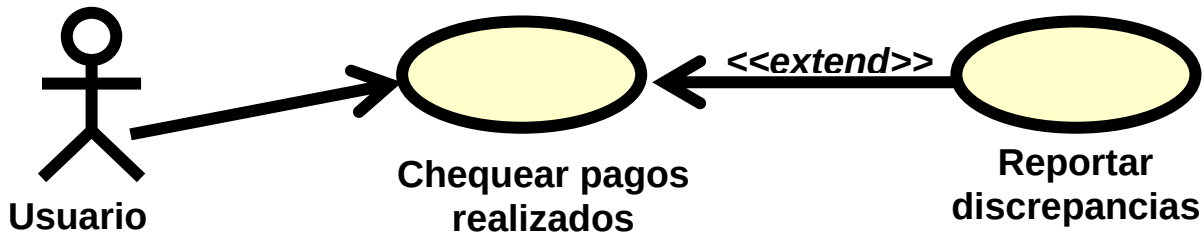
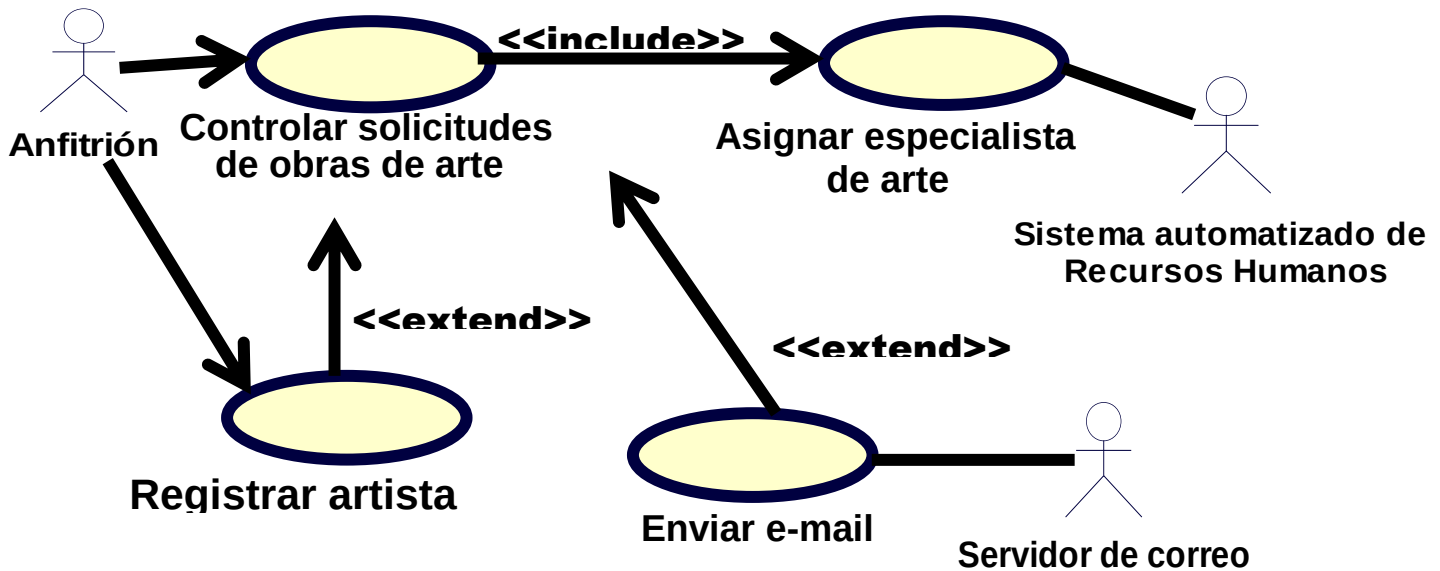


Figura 10 Ejemplo de comportamiento que se ejecuta cuando lo indique el actor.

En el caso del ejemplo de la galería de arte, como se aprecia en la figura 11, se pueden identificar 3 subflujos de trabajo. El subflujo “Enviar correo” es compartido por otros casos de uso, pero en estos casos solo se ejecuta si fue posible asignar especialistas. Para este caso es un extend porque solo se invoca si fue posible asignar especialista de arte a la solicitud. El subflujo “Asignar especialistas de arte” engloba un conjunto de actividades que extraen del sistema automatizado de Recursos Humanos especialistas que se especializan en determinadas técnicas y el Anfitrión selecciona a uno; por lo que aunque no es compartido tiene una relativa independencia. Este subflujo siempre se ejecuta porque el procedimiento de atención a una solicitud específica que siempre se trata de asignar un especialista de arte a una solicitud recibida. El subflujo de “Registrar artista” puede que se ejecute desde este caso de uso cuando el actor lo decida. Este subproceso es un proceso que puede ser invocado por el actor, sin pasas por el caso de uso base.



Ejemplo 11 Ejemplo de relaciones de inclusión / extensión.

Esta descomposición también provoca parte de la funcionalidad asociada a los requerimientos funcionalidades vinculados al caso de uso “Controlar solicitudes de obras de arte”, se realicen en los casos de uso abstractos. Por ejemplo, “Registrar artista” asumiría a los requerimientos:

3. Asignar solicitud a un especialista de arte.
6. Consultar en el sistema automatizado de RRHH los especialistas de arte.

- Generalización / especialización entre casos de uso

Un caso de uso generalización es una relación de un caso de uso hijo a un caso de uso padre que especifica cómo el hijo puede especializar todo el comportamiento y características descritas para el padre.

Se utiliza para mostrar que los flujos comparten la estructura, objetivo y comportamiento. Un caso de uso padre puede especializarse en uno o más casos de uso hijos que representan formas más específicas del padre.

Una vez identificado el flujo de cada caso de uso, se pueden encontrar estructuras y comportamientos que son comunes a varios casos de uso. Para no tener que describir el mismo flujo varias veces, se puede colocar el comportamiento común en un caso de uso del negocio.

Una instancia del caso de uso que ejecuta un caso de uso hijo seguirá el flujo de eventos descrito para el caso de uso padre, pero insertando comportamiento adicional y modificando el comportamiento de acuerdo al flujo de eventos del caso de uso hijo.

En la figura 12 se muestra un caso de uso (Pagar) que describe el comportamiento común entre dos formas de pago que puede realizar un usuario.

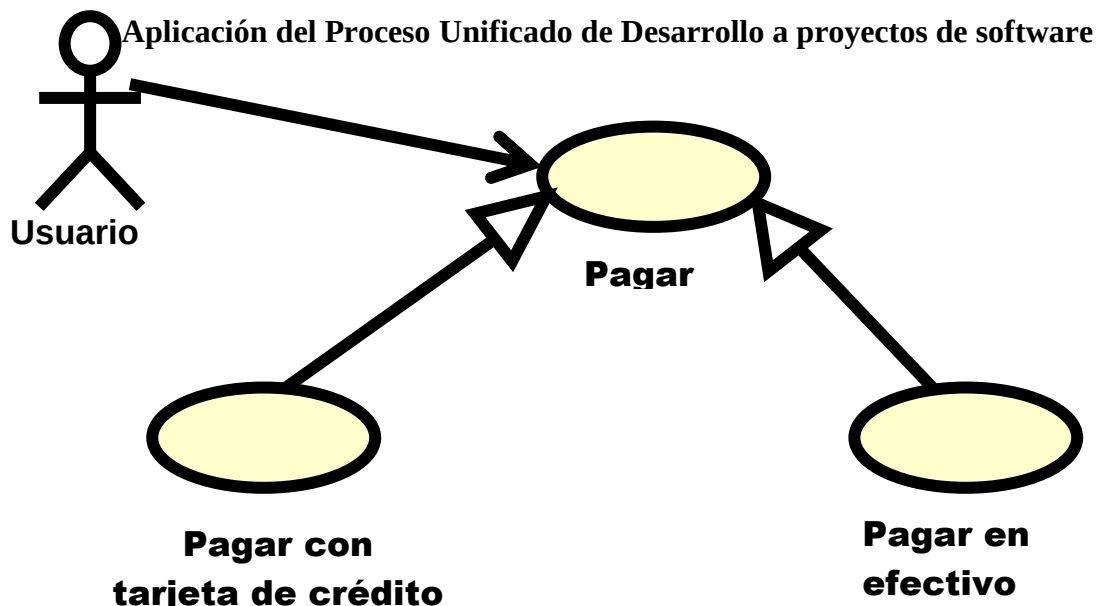


Figura 12 Ejemplo de generalización / especialización entre casos de uso.

- Generalización / especialización entre actores

Una relación de generalización de una clase hija de actor a otra clase padre de actor indica que el hijo hereda el rol que la clase padre puede jugar respecto a un caso de uso.

Varios actores pueden jugar el mismo rol en un caso de uso particular. En la figura 13 se muestra un ejemplo de un nuevo rol que representa a todas las personas que pueden desear chequear el estado de una cuenta bancaria. En este caso los roles de Especialista de banco y usuario se representan ya que interactúan con el sistema a través de otros casos de uso.

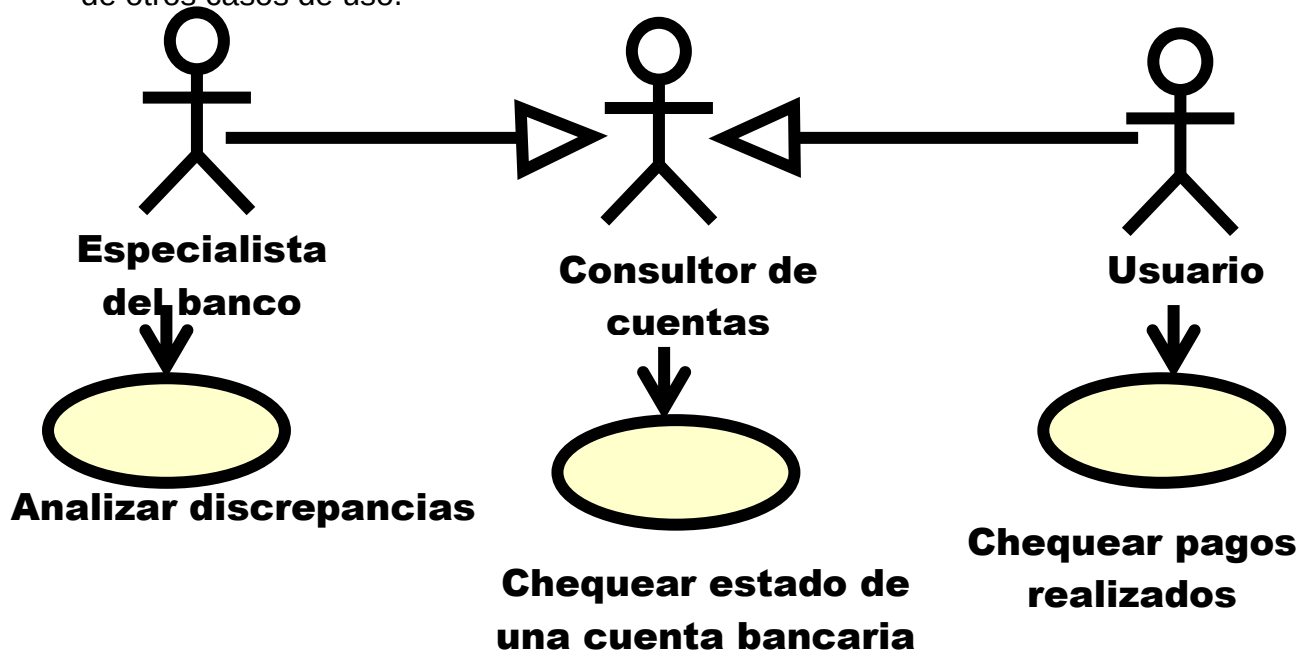


Figura 13 Ejemplo de de Generalización / especialización entre actores.

Descripción textual

Para entender la funcionalidad asociada a cada caso de uso no es suficiente con la representación gráfica del Diagrama de casos de uso. El formato de descripción que se propone no es exactamente el de RUP, pero sí recoge los elementos que clarifican al modelo de casos de uso.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Caso de uso: <Nombre>

Actores: <Nombre de los actores>

Descripción: <Frases que describan las acciones indicando los actores involucrados, debe quedar claro cómo se inicia y termina el proceso y de que forma intervienen los actores>

Referencias: <Listado de requerimientos y casos de uso asociados, indicando tipo de asociación (include o extend)>

Precondiciones: <Cosas que tienen que cumplirse en el sistema para que se ejecute el CU>

Poscondiciones: <Condiciones en las que queda el sistema cuando termina la ejecución del CU>

Requerimientos especiales: <Precisar de qué manera restricciones de tiempo de respuesta, seguridad, velocidad, disponibilidad, exactitud o uso de memoria afectan al caso de uso>

En el caso de la Galería de arte, un ejemplo de descripción de caso de uso es el siguiente:

Caso de uso:	Asignar especialista de arte
Actores:	Anfitrión y Sistema Automatizado de RRHH
Descripción:	El caso de uso se inicia cuando se registra una nueva solicitud, automáticamente el sistema consulta al Sistema Automatizado de RRHH y extrae todos los especialistas de arte que se especializan en las mismas técnicas de las obras que contiene la solicitud. Si no hay especialistas que cumplan el requerimiento, finaliza el caso de uso. En caso contrario, el Anfitrión selecciona de los disponibles a un especialista, culminando la ejecución del caso de uso.
Referencias:	R3 y R6
Precondiciones:	Se registra una nueva solicitud.
Poscondiciones:	Se asocia un especialista de arte a una solicitud en caso de que existan especialistas en las técnicas de las obras de la solicitud.
Requerimientos especiales:	Si no fue posible relacionarse con el Sistema Automatizado de RRHH, hay que informar al usuario para que tome medidas.

El prototipo del sistema que se construye en este punto da una visión de las pantallas diseñadas para cada caso de uso, pero con comportamiento estático, que se presenta al usuario para verificar los requerimientos funcionales. Para el caso de uso descrito anteriormente el prototipo construido se muestra en la figura 14.



Figura 14 Prototipo del caso de uso Asignar especialista de arte.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Priorización de casos de uso

Para poder determinar qué casos de uso se desarrollarán en cada release de construcción, RUP propone clasificarlos de acuerdo al impacto que tienen en la arquitectura en:

- **Críticos:** Más importantes para los usuarios porque cubren las principales tareas o funciones que el sistema ha de realizar. Definen la arquitectura básica.
Ejemplo: Controlar solicitudes de obras
Asignar especialista de arte
- **Secundarios:** Sirven de apoyo a los casos de uso críticos, involucran funciones secundarias y tienen un impacto más modesto sobre la arquitectura, pero deben implementarse pronto porque responden a requerimientos de interés para los usuarios.
Ejemplo: Registrar artista
- **Auxiliares:** No son claves para la arquitectura y completan casos de uso críticos o secundarios.
Ejemplo: Enviar e-mail
- **Opcionales:** Responden a funcionalidades que pueden o no estar en la aplicación, pero que no son imprescindibles en las primeras versiones.

La clasificación de los casos de uso es propia para el sistema. Un mismo caso de uso puede ser auxiliar para un sistema, pero crítico para otros.

Diagrama de casos de uso

Un diagrama de casos de uso representa gráficamente a los procesos y su interacción con los actores.

Cada caso de uso se puede comunicar con uno o más actores.

Cada caso de uso debe comunicarse con al menos un actor, si no aparece ningún actor que se comunique con un caso de uso esto indica error en el modelo de caso de uso o en los requerimientos planteados. Como excepciones a esta última regla podrían considerarse:

- Si el caso de uso es abstracto (no instanciable) no tiene porque incluir relación con actores (aunque tenerlas)
- Un caso de uso hijo en una relación de generalización-especialización no tiene porque tener relación con un actor si el caso de uso padre describe completamente toda la relación con el actor
- Algunos casos de uso se inician acorde a un horario (ejemplo: una vez a la semana o al día) lo que significa que el reloj del sistema es su iniciador. El reloj es interno al sistema, de esta forma el caso de uso no tendría relación de iniciación con ningún actor, pero para esclarecer el modelo debe colocarse un actor ficticio denominado "**Reloj**".

Las relaciones de comunicación entre actores y casos de uso no tienen nombre, sólo se necesita especificar el punto de inicio y el fin de la relación . Esto se hace a través de una línea con saeta que indica inicio y fin de la relación

Cada fin de la relación identifica un rol e incluso una multiplicidad (cuántas instancias del actor o caso de uso pueden asociarse con una instancia de actores o casos de uso)

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Cada rol de una relación de comunicación tiene una propiedad de navegabilidad, indicando quien inicia la comunicación en la interacción. La navegabilidad se muestra mediante una flecha. Si la flecha apunta al caso de uso, esto indica que el actor es quien inicia la relación con el sistema. Si la flecha apunta al actor, el sistema es quien inicia la interacción con el actor. La relación en ambos sentidos se muestra a través de una línea sin saetas (la doble saeta dificulta la comprensión del diagrama). Por cada flecha de comunicación se asume un mensaje de retorno. No se debe confundir navegabilidad con flujo de datos, la navegabilidad sólo debe expresar relación de iniciación. Ejemplo: un cliente requiere que se muestre un dato del sistema, esto se muestra mediante una flecha del actor al caso de uso a caso (representando al sistema), aún cuando la mayor parte de los flujos de datos van del sistema al cliente.

Un actor se comunica con un caso de uso por múltiples razones:

1. Para invocar un caso de uso. Una instancia de un actor siempre invoca una instancia de un caso de uso
2. Para solicitar algún dato almacenado en el sistema, el cual el caso de uso obtiene y presenta al actor.
3. Para cambiar el dato almacenado en el sistema mediante el uso de un diálogo con el sistema.
4. Para reportar que algo especial ha ocurrido alrededor del sistema y que dicho sistema debe cuidar

Un actor inicia un caso de uso. Sin embargo, una vez que este ha comenzado, el caso de uso puede comunicarse con varios actores. Se pueden usar relaciones de comunicación entre casos de uso y actores para mostrar cuáles actores se comunican con ellos. La multiplicidad de las relaciones muestran cuántas instancias del actor se relacionan con una instancia del caso de uso al mismo tiempo.

Los casos de uso de comunican con los actores por muchas razones:

1. Si algo especial ha ocurrido en el sistema, un actor puede necesitar conocerlo.
2. Un caso de uso puede necesitar pedirle a un actor que lo ayude a tomar una decisión si se tienen varias opciones.
3. Es común pero no siempre válido, que el caso de uso espera por una respuesta cuando este le envía una señal al actor. Esto debe quedar explícitamente descrito en el caso de uso.

Estas convenciones pueden esclarecer qué actor inicia el caso de uso y serán las usadas en este curso:

- La flecha de iniciación del actor-caso de uso siempre se muestra, incluso si el caso de uso más tarde inicia comunicación con el actor que lo inicio. Esto debe mostrarse sólo con una flecha actor-caso de uso.
- Una flecha caso de uso-actor puede ser omitida o incluida con el objetivo de esclarecer el diagrama.

3.2 - Organización en paquetes

Los paquetes son un mecanismo de organización de elementos que subdividen el modelo en otros más pequeños que colaboran entre sí. Tienen que ser:

- Cohesivos: sus contenidos deben estar fuertemente relacionados.
- Débilmente acoplados: minimizar las dependencias entre paquetes.

Este particionamiento debe hacerse sobre la base de los requerimientos funcionales y el dominio del problema; y debe ser reconocible por las personas con conocimiento del dominio.

Para ello se propone asignar la mayor parte de un cierto número de casos de uso a un paquete concreto. Se pueden seguir los siguientes criterios de agrupamiento:

- Casos de uso requeridos para dar soporte a un determinado proceso de negocio.
- Casos de uso requeridos para dar soporte a un determinado actor del sistema.
- Casos de uso que están relacionados mediante relaciones de generalización y extensión.
- Casos de uso que se ejecutan en un nodo.

Hay un tipo de paquete que se conoce como paquete de servicio ya que contiene un conjunto de clases relacionadas funcionalmente y un conjunto coherente de acciones relacionadas funcionalmente, que se utilizan en varios casos de uso. Son altamente reutilizables, por lo que no se cumple con ellos el principio de bajo acoplamiento, y pueden emplearse en varias realizaciones de casos de uso diferentes.

En la figura 15 se presenta la notación de los paquetes y la relación de dependencia que se da entre ellos, aunque son válidas los mismos tipos de relaciones que se dan entre clases.

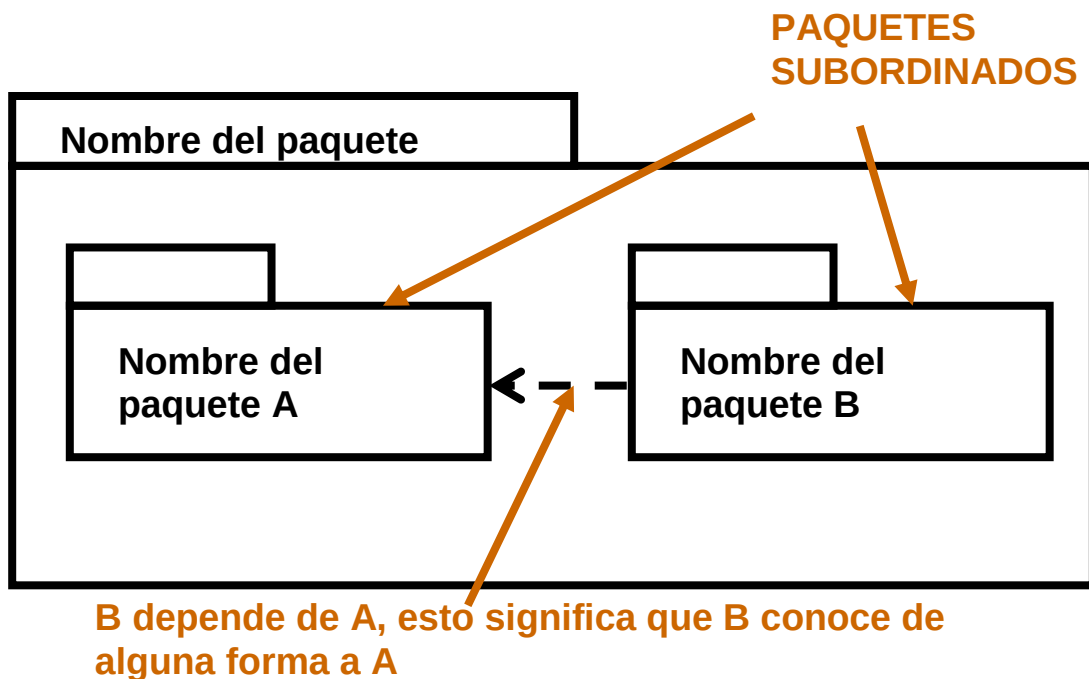


Figura 15 Notación de paquetes.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

En las figuras 16 y 17 se muestra como quedaría el agrupamiento para el caso de estudio de la galería de arte. Se pueden identificar paquetes de servicio (Correo), paquetes que agrupan casos de uso que dan soporte a un determinado actor del sistema (Evaluación de obras)n y paquetes que agrupan a casos de uso relacionados con relaciones de extensión / inclusión (Registro de obras). En particular se describe el paquete de Registro de obras y se aprecia su relación con el paquete de Correo (se incluye el caso de uso que motiva la dependencia indicando su origen – from Correo).

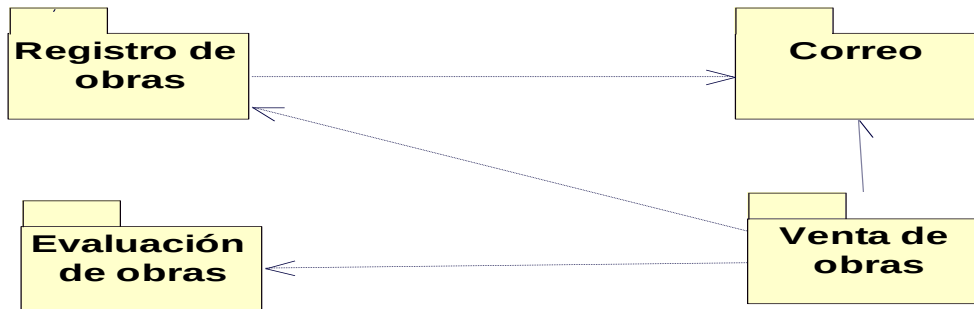


Figura 16 Agrupamiento en paquetes.

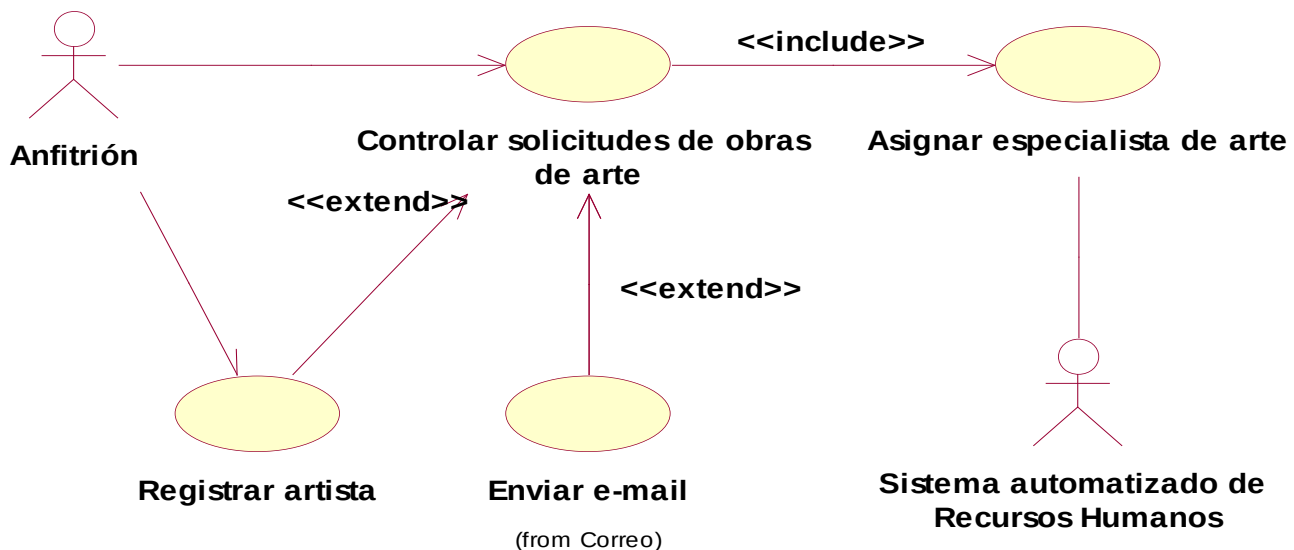


Figura 17 Paquete Registro de obras.

3.3 - Conclusiones

El flujo de trabajo de Requisitos busca establecer un común entendimiento con el cliente sobre los requerimientos del proyecto, siendo los casos de uso un importante artefacto que pone énfasis en la vida del dominio a partir de un proceso.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

La especificación de los requerimientos debe ser precisa, completa y clara. Para verificar lo anterior se presentan los siguientes aspectos cuya valoración objetiva podría ser de gran utilidad en la valoración de la especificación de los requerimientos.

Precisa

- ¿Son los requerimientos consistentes? ¿No se contradicen los unos con los otros?
- ¿Algún requerimientos se encuentra en conflicto con algo que ya se ha declarado o restringido (entorno del negocio, entorno técnico, costo, planificación, y recursos)?
- ¿Soportan los requerimientos los objetivos del negocio, sistema de software y el proyecto?
- ¿Son necesarias todas las actividades y operaciones?
- ¿Algún requerimientos no es requerido o se encuentra fuera del alcance del proyecto?

Completo

- ¿Son las metas y objetivos del sistema de software clara y completamente definidos?
- ¿Se han manejado todos los eventos y condiciones?
- ¿Han sido especificadas todas las operaciones?
- ¿Son las operaciones suficientes para reunir los objetivos del sistema de software?
- ¿Se han especificados todas las definiciones y reglas requeridas?
- ¿Satisface las especificaciones el nivel de detalle requerido el equipo de diseño?

Claros

- ¿Los requerimientos se encuentra limpios de polarización de implementación (no restringidos a una alternativa de diseño específica)?
- ¿Se han declarado en forma precisa y concisa todos los requerimientos?

Las respuestas obtenidas a las interrogantes anteriores nos permiten tener una noción objetiva de lo correcto o no de la especificación de los requerimientos.

En cuanto al modelo de casos de uso recomienda que verifiquemos:

- La sección de introducción de un modelo de casos de uso debe proporcionar un resumen claro y conciso del objetivo y funcionalidad del sistema.
- El modelo debe presentar claramente el comportamiento del sistema; debe resultar fácil de comprender qué hace el sistema revisando el modelo.
 - No deben existir largas cadenas de relaciones include y extend ni para los casos de uso extendido ni para los incluidos, esto dificulta la comprensión del diagrama.
 - Deben existir una mínima dependencia cuando un caso de uso especializado, incluido o extendido necesita conocer sobre la estructura y contenido de otros caso de uso especializado, incluido o extendido.
- Todos los casos de uso tienen que estar identificados; colectivamente los casos de uso cuentan para todo el comportamiento requerido.
- Todos los requerimientos funcionales son mostrados en al menos un caso de uso.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Todos los requerimientos no funcionales que necesitan ser satisfechos por casos de usos específicos tienen que estar mostrados en estos casos de uso.
- El modelo de casos de uso no debe contener comportamiento superfluo, sino que todos los casos de uso deben quedar justificados al trazar los requerimientos funcionales.
- Todas las relaciones entre casos de uso son requeridas (existe justificación para todas las relaciones include, extend y generalización-especialización).
- Cuando el modelo es grande y/o las responsabilidades del modelo son distribuidas por partes resulta apropiado utilizar paquetes.
 - Las referencias cruzadas entre paquetes deben reducirse o eliminarse para prevenir conflictos entre los elementos del modelo
 - El empaquetado es intuitivo y torna al modelo más comprensible

Los paquetes son mecanismos de propósito general para organizar elementos en grupos.

3.4 – Referencias Bibliográficas

- LEFFINGWELL, Dean; “Features, Use Cases, requirements, Oh My!”.2000. Rational Software. [Http://www.rational.com/media/whitepapers/featucreqom.pdf](http://www.rational.com/media/whitepapers/featucreqom.pdf)
- RUMBAUGH, James, JACOBSON, Ivar; BOOCH, Grady, “El lenguaje unificado de modelado”.2000. Addison Wesley. Capítulo 5, 10 y 13 Páginas 55-58, 87-90, 156-162, 120-121, 399-402, 365.
- JACOBSON, Ivar; BOOCH, Grady, RUMBAUGH, James, “El Proceso Unificado de Desarrollo de Software”.2000. Addison Wesley. Capítulos 3, 6 y 7 páginas 31-54, 105-164
- PRESSMAN, Roger “Ingeniería del Software. Un enfoque práctico”.2002. McGraw-Hill/Interamericana de España. Páginas 184-186 “Técnicas para facilitar las especificaciones de la aplicación.
- BOOCH, Grady, RUMBAUGH, James, JACOBSON, Ivar; “El lenguaje unificado de modelado”.2000. Addison Wesley. Capítulos 12, 16, 17 Páginas 147-158, 191-210.

Capítulo 4 Análisis

Como resultado del flujo de trabajo de requisitos se obtiene una vista externa del sistema, que el lenguaje del cliente, describe lo que se espera de él a través del Diagrama de casos de uso. A partir de aquí se debe profundizar en los casos de usos detallándolos de manera que permitan reflejar una vista interna del sistema descrita con el lenguaje de los desarrolladores. En esta vista interna se especifican mejor los casos de uso y se determinan las clases necesarias para llevar a cabo las funcionalidades en ellos contenidos.

Este proceso se desarrolla fundamentalmente dentro de la fase de elaboración y se corresponde principalmente con el flujo de trabajo de análisis y diseño. En esta clase se estudiarán los artefactos, actividades (figura 1) y trabajadores que participan en el análisis.

A pesar de que el modelo del análisis hay un refinamiento de los requisitos, no se toman en cuenta el lenguaje de programación a usar en la construcción, la plataforma en la que se ejecutará la aplicación, los componentes prefabricados o resuables de otras aplicaciones, entre otras características que afectan al sistema, ya que el objetivo del análisis es comprender perfectamente los requisitos del software y no precisar cómo se implementará la solución.

4.1- Flujo de trabajo de análisis

En este flujo se refinan y estructuran los requisitos obtenidos con anterioridad, profundizando el equipo del proyecto en el dominio de la aplicación lo que les permitirá una mayor comprensión del problema para modelar la solución. (Figura 1)

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

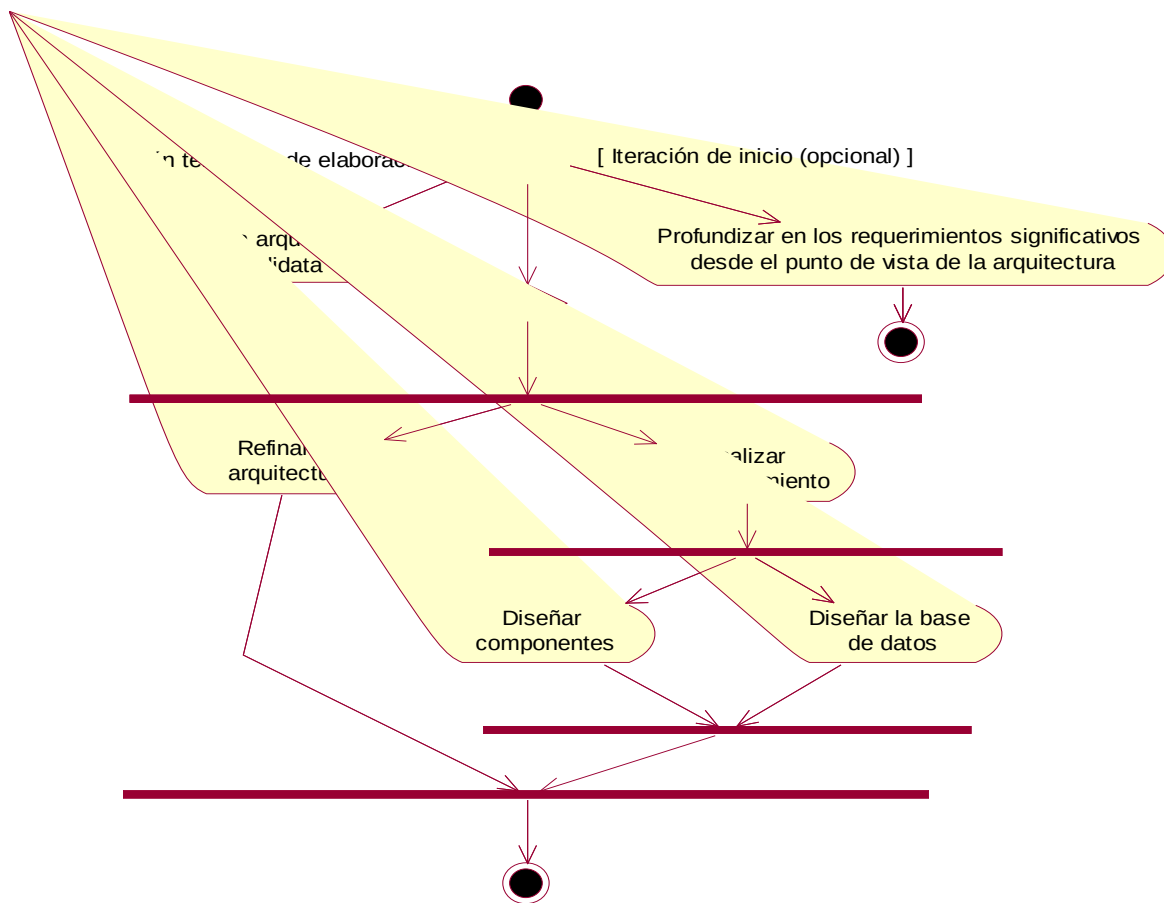


Figura 1 Flujo de trabajo del análisis.

Los trabajadores que participan son:

- Arquitecto: Responde por la integridad del modelo de análisis y por la arquitectura del modelo de análisis.
- Ingeniero de casos de uso: Responde por la integridad de una o más realizaciones de casos de uso (descripción textual del flujo de sucesos, Diagrama de clases que muestra las clases del análisis participantes y los diagramas de interacción que muestran la realización de un flujo particular del caso de uso en términos de objetos del análisis), así como de que respondan los casos de uso a los requisitos que engloba cada uno.
- Ingeniero de componentes: Define y mantiene las clases y las relaciones entre ellas y la integridad de uno o varios paquetes del análisis

Los artefactos que construyen estos trabajadores son:

- Modelo de análisis: Contiene clases del análisis y sus objetos organizados en paquetes que colaboran.
- Clases de análisis: Se centran en los requisitos funcionales y son evidentes en el dominio del problema porque representan conceptos y relaciones del dominio. Tienen atributos y entre ellas se establecen relaciones de asociación, agregación / composición, generalización / especialización y tipos asociativos. RUP propone clasificar a las clases

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

en. Interfaz, de control y entidad. Esta clasificación da robustez al modelo porque los cambios al modelo tienden a afectar a un área en específico.

Entidad: Modelan información que posee larga vida y que es a menudo persistente.

Interfaz: Modelan la interacción entre el sistema y sus actores.

Control: Coordinan la realización de uno o unos pocos casos de uso coordinando las actividades de los objetos que implementan la funcionalidad del caso de uso.

En una aplicación cliente/servidor de tres capas, en la capa de usuario aparecen fundamentalmente clases interfaz ya que allí se ejecutan las aplicaciones del cliente. En la capa intermedia están las clases de control ya que en ella se agrupan los servicios que son compartidos por múltiples aplicaciones. En la servidor capa estarían las clases entidad porque allí se tiene la base de datos.

- Realización de casos de uso del análisis: Describe cómo se lleva a cabo y se ejecuta un caso de uso determinado en término de las clases del análisis y de sus objetos en interacción.
- Paquete de análisis: Organiza los artefactos del análisis en piezas manejables.
- Descripción de la arquitectura (vista del modelo de análisis): Muestra los artefactos significativos para la arquitectura (los anteriores).

Los trabajadores, para construir estos artefactos, realizan las siguientes actividades:

- Análisis de la arquitectura: Identificar paquetes y clases del análisis.
- Analizar un caso de uso: Identificar clases cuyos objetos son necesarios para realizar el caso de uso y distribuir el comportamiento entre los objetos que participan en el caso de uso.
- Analizar una clase: Identificar atributos de las clases y relaciones entre ellas.
- Analizar un paquete: Garantizar alta cohesión y bajo acoplamiento de los paquetes y describir las relaciones entre ellos.

4.2 - Realización de los casos de uso en el análisis

La realización de los casos de uso en el análisis es una colaboración que describe cómo se lleva a cabo y ejecuta un caso de uso determinado en término de las clases del análisis y de sus objetos en interacción (Jacobson, página 177); por lo tanto, se centra en los requerimientos funcionales.

Para ello RUP propone que para cada caso de uso:

- Se construya un diagrama de clases que muestre las clases del análisis participantes.
- Se construya alguno de los tipos de diagrama de interacción para representar el flujo en términos de objetos del análisis.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Una descripción del flujo de sucesos más detallada, pero sin llegar a precisar los atributos, responsabilidades y asociaciones de los objetos participantes.
- Se precisan mejor los requerimientos especiales de acuerdo a las particularidades del flujo de trabajo.

Dado que uno de los diagramas más difíciles de construir de UML son los diagramas de iteración, y que estos deben ser refinados en el diseño de acuerdo a la forma en que será implementada la solución; puede no construirlos en este punto y dedicar los mayores esfuerzos en la identificación de clases del análisis y en la descripción de los casos de uso.

4.2.1- Descripción de los casos de uso

En el análisis, cuando se describen los casos de uso, la descripción es detallada del flujo de trabajo que se produce en la interacción entre actores y casos de uso. En esta descripción pueden identificarse:

- Flujo de trabajo principal: Lo que normalmente pasa cuando se ejecuta el caso de uso. Guía la ejecución del caso de uso.
- Flujo de trabajo principal alternativo: Cubre el comportamiento alternativo que normalmente puede ocurrir cuando se ejecuta el CU (se representa en secciones).
- Flujo alternativo: Comportamiento excepcional en relación con el comportamiento normal (se representa por cursos alternos).

La descripción de un caso de uso en el análisis es una extensión de la descripción que teníamos, en la que se añadiría:

Curso normal de los eventos: (1)	
Acción del actor	Respuesta del sistema
(2)	(3)
Sección <Nombre de la sección>: (4)	
Acción del actor	Respuesta del sistema
(5)	(6)
Cursos alternos	
(7)	

Leyenda

- (1) Flujo de trabajo principal
- (2 y 5) Interacciones del actor con el sistema en el orden en que ocurren
- (4) Flujo de trabajo alternativo.
- (3 y 6) Descripciones numeradas de las respuestas del sistema, indicando los objetos que se afectan. Para el caso de que existan casos de uso asociados con Include o Extend, en el punto de inserción o extensión, expresarlo de la siguiente forma:

Include: Incluir el caso de uso <Nombre del caso de uso><Información que se le suministra ><Información que da como respuesta>

Extend: Extender el caso de uso <Nombre del caso de uso><Información que se le suministra ><Información que da como respuesta>
- (7) Flujo alternativo de eventos: cubre el comportamiento opcional o de carácter excepcional en relación con el comportamiento normal, son alternativas que

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

pueden ocurrir en el número de línea. Descripciones numeradas de las acciones de los actores (en caso de que hubiera) y de las respuestas del sistema.

Veamos el caso de uso Registrar artista del caso de estudio de la Galería de arte. Lo que normalmente ocurre es que un anfitrión desea registrar un nuevo artista, modificar las características de un artista, eliminar un artista o finalizar el registro de artistas. La creación, eliminación y modificación implican cada una un conjunto de actividades, por lo que se recomienda describir cada operación por separado en secciones. En la realización de este caso de uso hay dos excepciones:

- No se puede registrar dos veces a un mismo artista.
- No se permite eliminar un artista que tenga obras en la galería.

Caso de uso:	Registrar artista
Actores:	Anfitrión (Inicia)
Descripción:	El caso de uso se inicia cuando al intentar registrar una nueva solicitud, se detecta que el artista que la presenta no están almacenados sus datos o cuando a la galería le interesa registrar información sobre artistas que desea expongan en la galería. En estos casos, el Anfitrión registra la información de un nuevo artista o los cambios que se produzcan en sus datos. El caso de uso finaliza con el registro de artistas actualizado. Si se va a eliminar a un artista, se verifica si está asociado a una obra, en cuyo caso no se realiza la eliminación; en cualquier otro caso se procede a la eliminación física.
Referencias:	R5
Precondiciones:	El sistema automatizado de RRHH ha replicado las técnicas. Si no se ha replicado, el sistema presenta un mensaje al Anfitrión.
Poscondiciones:	Se actualiza el registro de artistas almacenando los datos del nuevo artista, la modificación de los datos registrados o la eliminación física de una artista si es posible.
Requerimientos especiales	-
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
1 El Anfitrión necesita registrar un nuevo artista, modificar las características de un artista o eliminar un artista.	2 El sistema ejecuta alguna de las siguientes acciones: a) Si decide registrar un nuevo artista, ir a la sección "Nuevo". b) Si decide modificar las características de un artista, ir a la sección "Modificar". c) Si decide eliminar un artista, ir a la sección "Eliminar".
3 El Anfitrión termina el registro de artistas.	4 El sistema finaliza la ejecución del caso de uso.
Sección "Nuevo"	
Acción del actor	Respuesta del sistema
1 El Anfitrión especifica las características de un nuevo artista.	2 El sistema verifica que el artista no esté registrado. En caso de que no exista, se registra el artista.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Sección "Modificar "	
Acción del actor	Respuesta del sistema
1 El Anfitrión selecciona al artista que desea modificar sus características.	2 El sistema presenta las características registradas del artista.
3 El Anfitrión especifica las características que cambian del artista.	4 El sistema incluye los cambios al artista.
Sección "Eliminar "	
Acción del actor	Respuesta del sistema
1 El Anfitrión selecciona el artista que desea eliminar.	2 El sistema verifica si el artista está asociado a una obra. En caso de que no esté asociado, el sistema presenta las características registradas del artista.
3 El Anfitrión confirma la eliminación del artista	4 El sistema elimina el artista.
Cursos alternos	
Sección "Nuevo" : Línea 2 Si el artista ya está registrado, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección. Sección "Eliminar" : Línea 2 Si el artista seleccionado tiene obras registradas, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección.	

4.2.2 - Modelo del análisis

En la construcción del modelo de análisis se tienen que identificar las clases que describen la realización de los casos de uso, los atributos y las relaciones entre ellas. Con esta información se construye el Diagrama de clases del análisis, que por lo general se descompone para agrupar las clases en paquetes. Esta descomposición tiene impacto por lo general en el diseño e implementación de la solución.

4.2.2.1- Identificación de clases, Atributos y Relaciones

Las clases que se identifican están asociadas con el contexto del dominio del problema por lo que representan conceptos y relaciones.

Clases Interfaz

Al modelar la interacción entre el sistema y sus actores, pueden identificarse a partir de estos últimos:

- Al menos una clase que modele la interacción del actor usuario con el sistema, es decir, una clase para cada interacción actor-caso de uso sin preocuparse de que en la solución puede que se presente más de una pantalla dentro de un caso de uso para un mismo usuario.
- Una clase para cada sistema externo que será el responsable de la relación del sistema con cada uno de ellos.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Una clase para cada actor que represente un dispositivo sobre el cual el sistema actúa o recibe información.

Estas dos últimas recomendaciones reponen a un principio del paradigma de objetos en el cual se propone que en la definición de clases existe una alta cohesión interna de manera que si cambia algo solo se afecta(n) aquella(s) clase(s) que está(n) vinculada(s) con esa funcionalidad. En este caso, por ejemplo, sei se modifica la estructura de la tabla que contiene todas las técnicas con las que trabaja la galería (tabla perteneciente a la base de datos del Sistema de Recursos Humanos) o, para el caso de un sistema de pronóstico del tiempo, cambia el dispositivo utilizar para medir la presión atmosférica, solo se afectarán las clases de interfaz porque estos cambios no afectan a la lógica de la aplicación.

En la figura 3 se muestran las clases que se obtendrían a partir del fragmento del Diagrama de casos de uso que se muestra en la figura 2, correspondiente al caso de estudio de la Galería de arte.

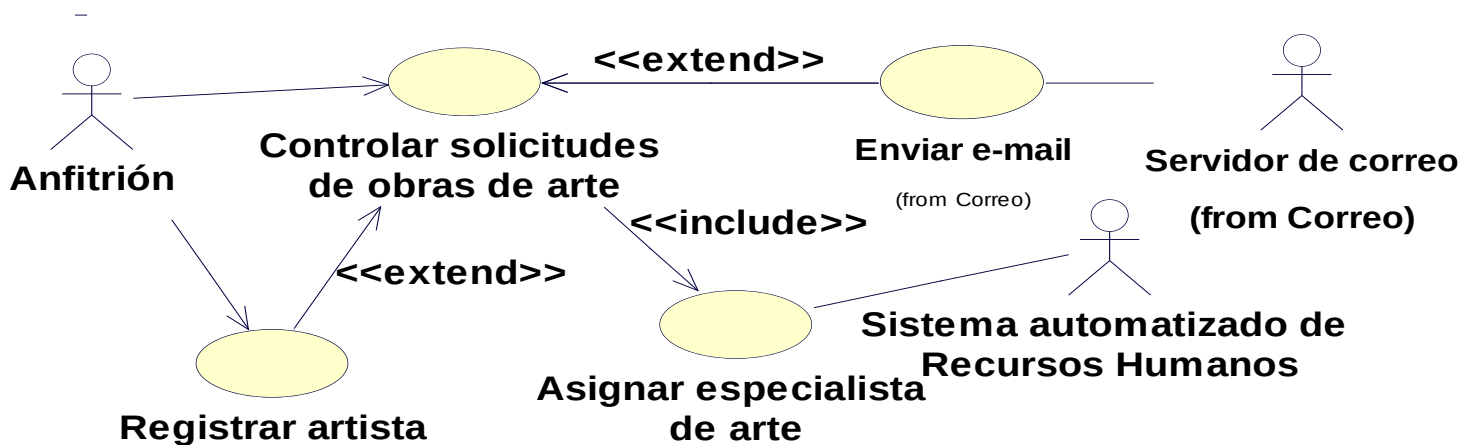


Figura 2 Subconjunto del Diagrama de casos de uso del caso de estudio Galería de arte.

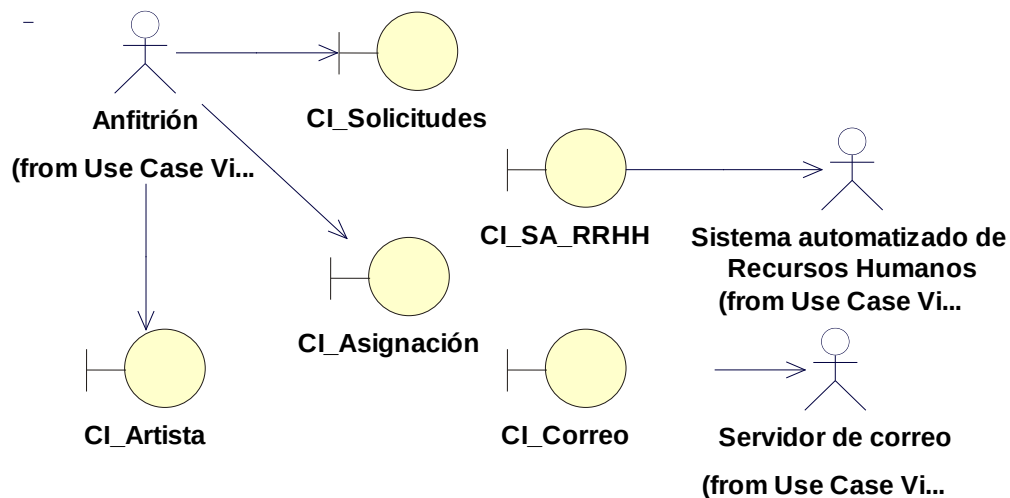


Figura 3 Clases Interfaz correspondientes al registro de obras.

Clases entidad

Estas clases modelan información que posee una larga vida y que amenudo es persistente y fenómenos, conceptos y sucesos que ocurren en el mundo real. La fuente principal de obtención son las clases entidades del negocio y el glosario de términos que se ha ido elaborando. Algunos autores proponen un estudio del texto, a partir de las frases nominales, de manera que los ssutenativos representan objetos y clases.

En la figyura 4 se presentan las clases que se obtienen a partor de los casos de uso incluidos en la figura2.

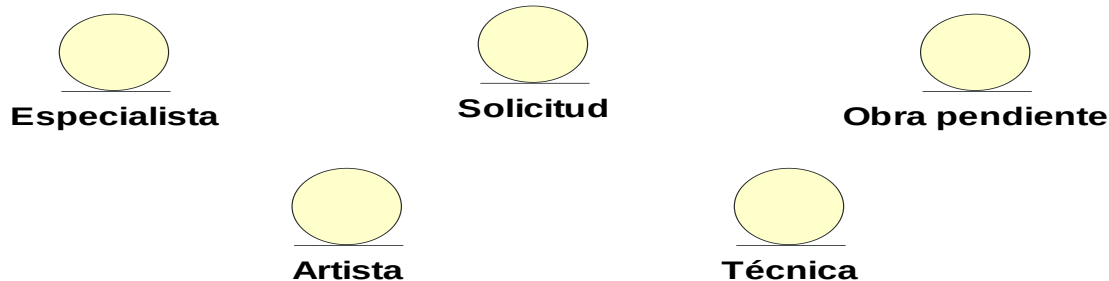


Figura 4 Clases Entidad correspondientes al registro de obras.

Clases de control

Las clases de control coordinan el trabajo de uno o unos pocos casos de uso, coordinando las actividades de los objetos que implementan la funcionalidad del caso de uso, por lo que definen el flujo de control y las transacciones dentro de un caso de uso delegando el trabajo a otros objetos.

En principio se define una clase de control para cada caso de uso, aunque:

- Si el flujo de eventos (diferentes tareas a ser ejecutadas) de un CU, puede ser manejado por las clases de entidad e interfaz creadas, entonces no se justifica una clase de control, siendo la clase de interfaz la que coordina el CU.
- Si el flujo de eventos es complejo o el comportamiento dinámico que describe el CU, puede cambiar independientemente de la interfaz o de la información que almacena el sistema, entonces se debe definir una clase de control.
- Si el flujo de eventos o un subconjunto de un flujo de eventos puede reusarse dentro de la realización de varios CU, entonces se debe definir una clase de control para ese flujo de eventos.
- Si un CU tiene más de un actor, puede crear una clase de control para cada actor si se asocian a funcionalidades independientes y se puede asegurar que el comportamiento de los actores cuando se relacionan con la clase de control, cambia.
- Si varios casos de uso tienen funcionalidades relacionadas (por ejemplo, por la herencia) o trabajan con los mismos objetos, podría definirse una clase de control para todos ellos

En la figura 5 se muestran las clases de control que se obtienen a partir de los casos de uso presentados en la figura 2.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

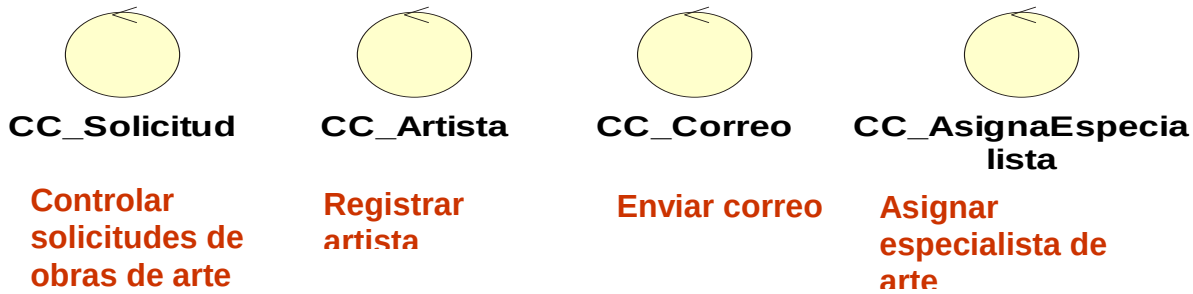


Figura 5 Clases de Control correspondientes al registro de obras.

En la tabla 1 se presentan las relaciones válidas entre los diferentes tipos de clases.

Tabla 1 Relación entre las clases.

	INTERFAZ	CONTROL	ENTIDAD
INTERFAZ	Describe cómo una ventana específica se relaciona con otra ventana	Cuando se dispara un comportamiento en particular	Cuando juega el papel de control
CONTROL	Comunicar al medio el resultado de la ejecución de un comportamiento	Organización jerárquica cuando el comportamiento es muy complejo	Por su papel controlador
ENTIDAD	-	-	Relación entre la información

El ciclo de vida de los objetos dentro de la realización de un caso de uso sería:

Objeto de interfaz: Pueden aparecer en más de un CU si, por ejemplo, una ventana en particular aparece en más de un CU, sin embargo, por lo general se crean y finalizan dentro de la realización de un CU.

Objeto entidad: Normalmente no son particulares de una realización de un CU.

Objeto de control: Suelen encapsular el control asociado con un CU en concreto (por lo tanto, se crean cuando comienza la realización y se destruyen cuando termina), pero hay excepciones cuando:

- un objeto participa en más de una realización de CU diferentes.
- hay varios objetos de control que participan en una misma realización de un CU.
- una realización no requiere objetos de control.

Atributos

Un atributo es una característica o propiedad de todas las instancias de la clase.

Reglas para definirlos:

- Incluir aquellos atributos que se requieren según los requerimientos de información de los casos de uso en los que está involucrada la clase.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- Conservar atributos simples, los complejos sugieren relaciones entre clases.
- Ningún atributo como llave foránea.

Fíjese que si un atributo de una clase hace referencia a otra clase, entonces esto sugiere que existe entre estas clases una relación y no se pone el atributo.

Relaciones

Entre las clases se pueden dar distintos tipos de relaciones.

- Asociaciones
 - Generalización / Especialización
 - Agregación / Composición
- Asociaciones

Son las relaciones estructurales entre las instancias que especifican que los objetos de una clase están conectados con los objetos de otras clase (figura 6).

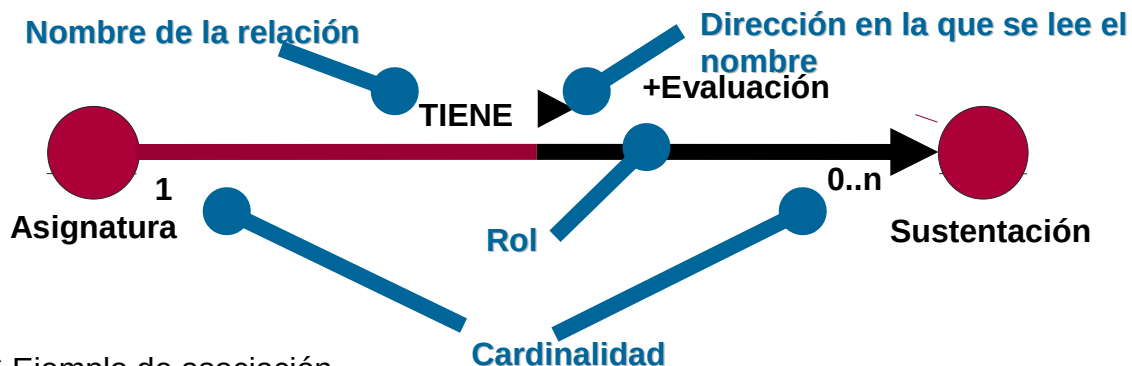


Figura 6 Ejemplo de asociación

En las asociaciones pueden identificarse:

Nombre: Describe la naturaleza de la relación.

Rol: Cara que la clase de un extremo de la asociación presenta a la clase del otro extremo

Multiplicidad: Cuántos objetos pueden conectarse a través de una instancia de una asociación.

Navegación: Es una afirmación sobre la existencia de un recorrido en una relación, a la que se asocia un rol (es opcional). Se puede representar de forma explícita la dirección de la navegación como una flecha que apunte en la dirección del recorrido. La ausencia de flechas indica que la navegación es bidireccional.

- Tipos asociativos

Cuando entre dos objetos existe una relación en la que se pueden identificar atributos relacionados con esta asociación, hay que colocar estos atributos. En el caso de que la multiplicidad fuese de 1:1 ó 1:m, estos atributos se pondrían en cualquiera de las clases o en la clase del extremo de multiplicidad m., respectivamente. Para asociaciones con cardinalidad m:n, se crea una nueva clase (tipo asocactivo) que se nombra como la relación y que tiene como atributos los de la relación (figura 7).

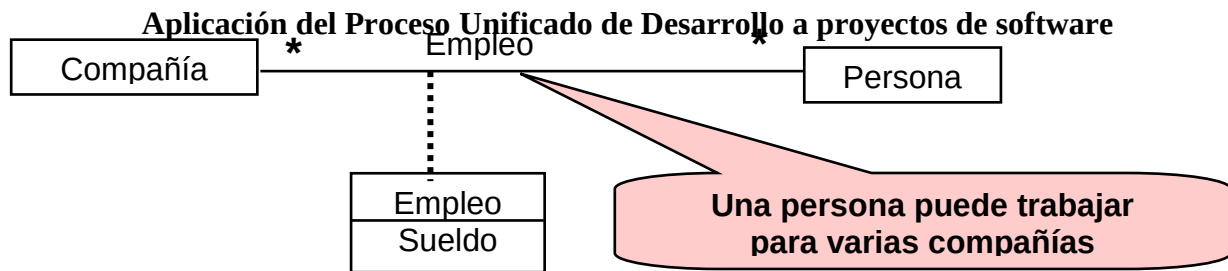


Figura 7 Ejemplo de tipo asociativo.

- Agregación

La agregación es una relación estructural entre instancias en la cual una clase representa el todo y consta de otras que son las partes. Pueden identificarse dos tipos de agregaciones:

- Agregación compositiva o composición: Es una forma de agregación con una fuerte relación de pertenencia y vidas coincidentes de la parte con el todo. Un objeto puede formar parte de solo una parte compuesta a la vez. La parte compuesta es responsable de la disposición de las partes, es decir, debe gestionar la creación y destrucción de las partes. La multiplicidad en el extremo del todo es a los uno 1. En la figura 8 se muestra un ejemplo de este tipo de relación.

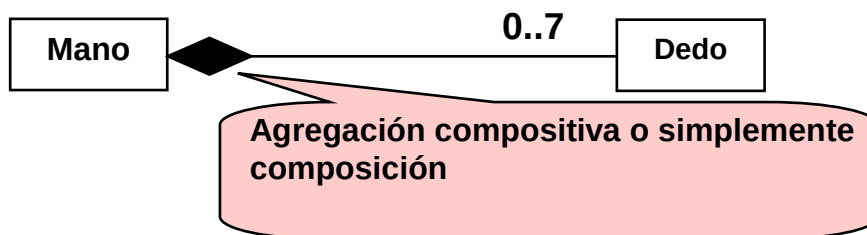


Figura 8 Ejemplo de composición

- Agregación compartida o simplemente agregación. Es aquella en la que las partes pueden estar en muchas instancias compuestas porque la relación entre el todo y sus partes no liga las vidas del todo y las partes. En la figura 9 se muestra un ejemplo de este tipo de relación.

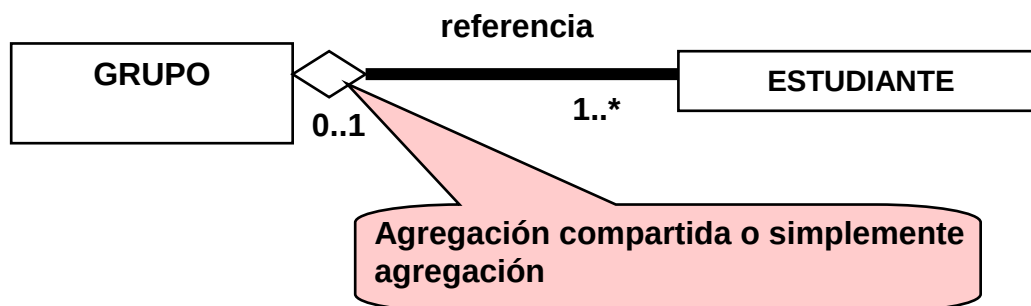


Figura 9 Ejemplo de agregación

La esencia de este tipo de relaciones es determinar relaciones de todo-parte entre las clases. Posteriormente se clasifican en dependencia de la fortaleza de la relación.

- Generalización / especialización

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

La idea consiste en identificar aspectos comunes entre las clases identificadas para abstraer lo común a una superclase y lo particular dejarlo en las subclases. En la figura 10 se muestra un ejemplo de este tipo de relación, fíjense que en el caso del supertipo se estableció que era una clase abstracta lo que Rational Rose representa inclinando el texto del nombre de la clase o poniendo el estereotipo <<abstract>>.

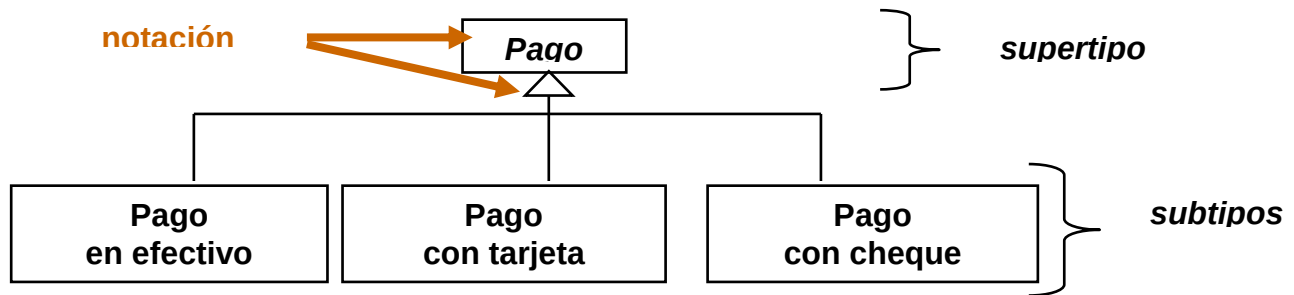


Figura 10 Ejemplo de Generalización / Especialización. Notación.

Además, UML permite especificar dos tipos de restricciones que caracterizan a la herencia:

- Disjoint / Overlapping: Un objeto de un tipo de clase padre no puede o puede, respectivamente, hacer referencia a más de un objeto de los tipos de clase hija.
- Complete / Incomplete: Se han contemplado o no, respectivamente, todos los hijos posibles.

En la figura 11 se muestra un ejemplo en el que se ha definido que un paciente, al que está apto para que se le realice un trasplante de riñón, puede que ya se haya transplantado, pero que siga requiriendo diálisis (overlapping). En este caso se considera incompleta porque, aunque hasta ahora no se ha contemplado en el sistema, hay otros estados que pueden darse a partir de una paciente apto (por ejemplo, asociado a otro tipo de tratamiento que no sea la diálisis).



Figura 11 Ejemplo de Generalización / Especialización. Restricciones.

4.2.2.2 - Diagrama de clases del análisis

Un Diagrama de clases del análisis es un artefacto en el que se representan los conceptos en un dominio del problema. Representa las cosas del mundo real, no de la implementación automatizada de estas cosas.

Pasos para construir el Diagrama de clases del análisis

1. Crear una lista de conceptos significativos e interesantes del dominio

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- a) Identificar como conceptos las cosas sobre las que se desea conservar su información asociadas con:
- Objetos físicos (Estudiante, Asesor).
 - Lugares
 - Transacciones (Evaluación que se le da a un estudiante en una sustentación).
 - Papel de las personas (Estudiante y Asesor, ambos son personas, pero interesan cosas diferentes sobre ellas asociadas a papel que juegan en el sistema).
 - Eventos (Sustentación de un proyecto al que se le da una evaluación).
 - Contenedoras de otras cosas (Tribunal)
 - Cosas dentro de un contenedor (Asesor y Proyecto dentro de un Tribunal).
 - Reglas y políticas, catálogos, manuales o libros que sirvan de referencia para desarrollar algún proceso.
- b) Si se utiliza más de una palabra para un mismo concepto, se selecciona la que sea más importante en términos del resto del sistema (Estudiante=Alumno).
- c) Ser cuidadoso con el uso de los adjetivos, si el uso del adjetivo señala que el comportamiento de la clase es diferente, entonces se define una nueva clase.
- d) Ser cuidadoso con las oraciones en voz pasiva porque siempre implican un sujeto, aunque esté omitido.
- e) No todos los sujetos son conceptos del dominio que es necesario identificar, porque puede que estén fuera del sistema.
- f) Si en el mundo real no se considera a una propiedad X como un tipo de dato simple, probablemente sea un concepto y no un atributo (nombre, dirección y teléfono del Lugar en el que labora el Estudiante).

En resumen:

- **Utilice los nombres existentes en el dominio.**
- **Excluya las características irrelevantes.**
- **No agregue cosas que no existan**

2. Identificar las asociaciones.

Asociación: Relación entre dos conceptos que indican alguna conexión significativa e interesante entre ellos.

Las asociaciones que vale la pena describir son aquellas que incluyen el conocimiento de una relación que ha de preservarse durante algún tiempo.

Recomendaciones:

- a) Preguntarse si entre los conceptos existe una relación relevante para el sistema que responda a alguna de las siguientes preguntas:

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- ¿A es parte de B? .
 - ¿A está contenida en B? (Estudiante-Proyecto).
 - ¿A es una descripción de B?.
 - ¿A es miembro de B?).
 - ¿A es propiedad de B?.
 - ¿A se relaciona con alguna transacción B? (Sustentación-Evaluación, Proyecto-Evaluación).
 - ¿A se comunica con B? (Estudiante-Lugar).
- b) No incluir las relaciones redundantes ni las derivadas.

Es más importante identificar los conceptos que las asociaciones por lo que hay que dedicar más tiempo a lo primero

Para una asociación se define:

- Nombre: Frase nominal que describe la asociación.
- Multiplicidad: Define cuántas instancias de un tipo A pueden asociarse, como mínimo y como máximo, a una instancia del tipo B.
- Navegabilidad: Es una flecha que indica, la dirección en que se navega, significa visibilidad desde la clase fuente hasta la clase destino. Cuando se programa usando un lenguaje de programación orientado a objetos, significa que la clase fuente tiene un atributo que es del tipo de la clase destino.

3. Identificar atributos.

Incluir solo los atributos que, según los requerimientos, son necesarios recordar para que se ejecute el caso de uso.

Recomendaciones:

- a) Definir como atributos las propiedades que tengan valores simples asociados.
- b) No usar atributos que sean llaves foráneas, si se necesita en la definición de una clase una llave foránea, esto indica que se requiere de una asociación entre las clases.
- c) No usar atributos complejos, si se necesita en la definición de una clase un objeto como atributo, esto indica que se requiere de una asociación entre las clases.

En la figura 12 se muestra como quedaría el diagrama de clases a partir de los casos de uso de la figura 2.

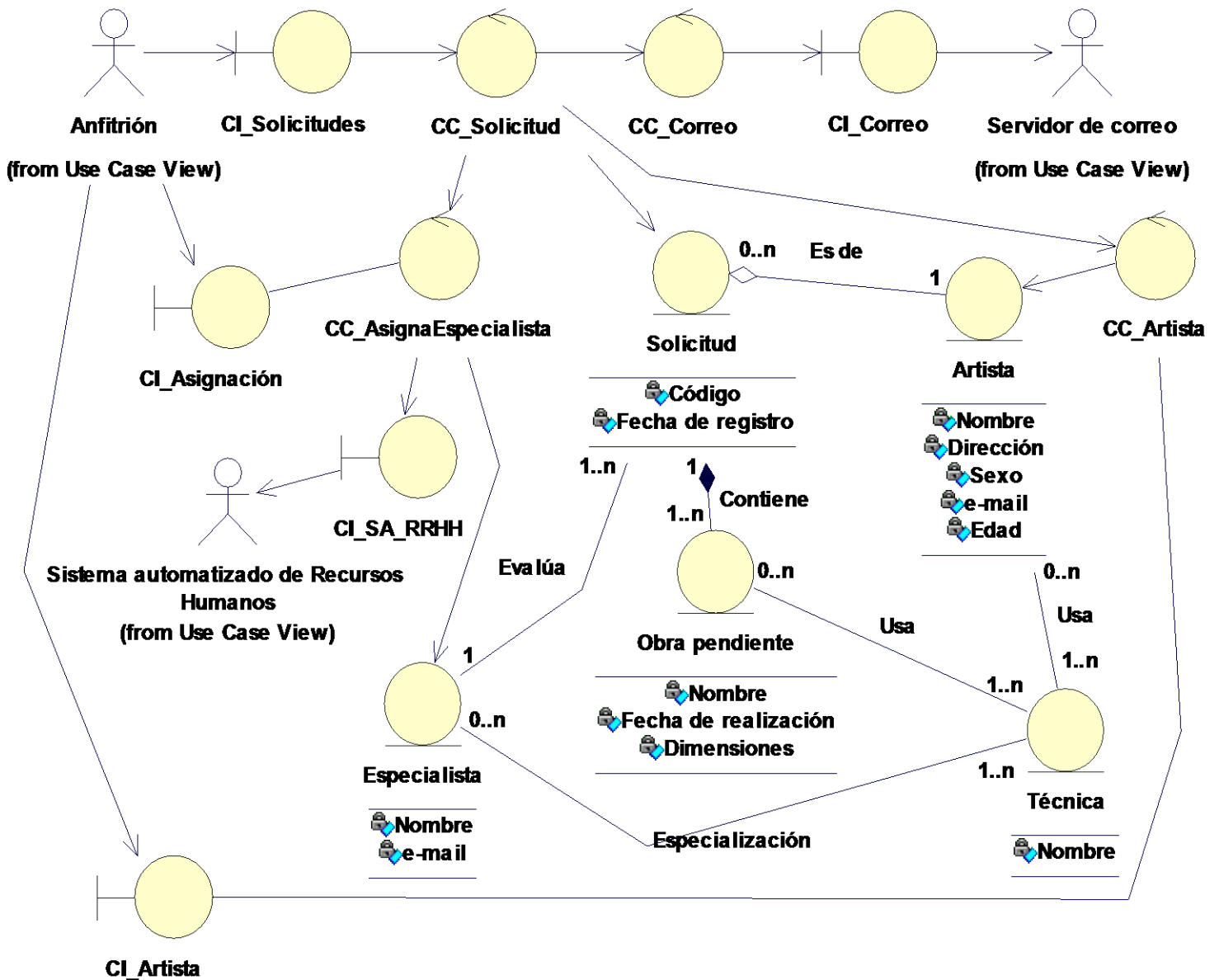


Figura 12 Diagrama de clases correspondiente al Registro de obras.

4.3 - Organización en paquetes

Siguiendo el agrupamiento en paquetes que se hizo en el capítulo anterior, en las figuras 13 y 14 se muestra el contenido del paquete evaluación de obras de acuerdo a los casos de uso que agrupa y las clases que se necesitan para realizarlos. Aunque existe una coincidencia, se pueden usar criterios diferentes de agrupamiento en paquetes, pero tomando en cuenta que los paquetes son la fuente para definir subsistemas de diseño; sería conveniente agrupar siguiendo el mismo criterio de manera que se vaya perfilando cómo quedaría el sistema.

4.5 – Referencias Bibliográficas

- JACOBSON, Ivar; RUMBAUGH, James; BOOCH, Grady, “El proceso unificado de desarrollo”.2000. Addison Wesley. capítulos 8 Páginas 165-181, 185-204
- RUMBAUGH, James, JACOBSON, Ivar; BOOCH, Grady, “El lenguaje unificado de modelado. Manual de referencia”.2000. Addison Wesley. Capítulos 4 y 13 Páginas 42-48, 122-125, 131-143, 156, 162-170, 191-197, 211, 299-303, 367-373, 399-402, 427, 434-435, 446-447.
- LARMAN, Craig “UML y patrones” 1999, Prentice Hall Iberoamericana. Capítulos 9,10 y 11 Páginas 85-103 y 105-119.
- “Rational Unified Process”. Guidelines-Analysis class y Guidelines-Boundary class.
- BOOCH, Grady; RUMBAUGH, James, JACOBSON; Ivar; “El lenguaje unificado de modelado. Manual de referencia”.2000. Addison Wesley. Capítulos 4-5, 8, 12 Páginas 41-64, 93-102, 147-158

Capítulo 5 Diseño

Las actividades contempladas en el análisis comenzaron a adentrarnos en el problema a resolver por lo que a través de ellas representamos una vista intena del sistema en la que, usando el lenguaje de los desarrolladores se refinan los requisitos y se estructuran en base a clases y paquetes. Este proceso continúa en el diseño hasta obtener los objetos que interactúan para cumplir los rqueisitos funcionales y no funcionales obtenidos.

El flujo de trabajo de diseño tiene el propósito de formular los modelos que se centran en los requisitos no funcionales y en el dominio de la solución y que prepara para la implementación y prueba del sistema. Pretende crear un plano del modelo de implementación, por lo que el grueso del esfuerzo está en las últimas iteraciones de elaboración y las primeras de construcción.

En este capítulo se profundiza en los artefactos que se propone se elaboren en el diseño,dejando el modelo de despliegue para el próximo capítulo por su fuerte relación con el flujo de trabajo de implementación.

5.1- Flujo de trabajo de diseño

RUP define el análisis y el diseño como un único flujo de trabajo en el que hay actividades que se realizan desde la fase de Inicip. Durante esta fase se analiza si es posible dar una solución que satisfaga a los requerimientos significadtivos de la arquitectura. En la fase de Elaboración se comienza con un análisis de los elementso significativos de la arquitectura como parte de la 1ra iteración de leaboración y en las siguientes iteraciones se refina la arquitectura hsata diseñar todos sus elementos. En las restantes fases se hace algo de análisis y diseño vinculado con nuevos requerimientos que sean definidos y se decida implementar y con los defectos que haya que corregir como consecuencia de las pruebas realizadas (figura 1).

Los artefactos que se construyen en el diseño son:

- **Modelo de diseño:** Modelo de objetos que describe la realización física de los casos de uso en cómo los requisitos funcionales, no funcionales y otras restricciones del entorno de implementavión tienen impacto en el diseño. Está compúesto por subistemas de diseño, clases de diseño, realizaciones de casos de uso e Interfaz.
- **Clases de diseño:** Es una abstracción de una clase o construcciones similares de implementación.
- **Realización de casos de uso en el diseño:** Diagrama de clases + Diagrama de interacción + Descripción de los casos de uso + Requisitos de implementación.
- **Subsistemas de diseño:** Forma de organizar los artefactos de diseño de manera más manejable. Al ser mecanismos de agrupamiento tiene que cumplir las mismas características de los paquetes.
- **Interfaz:** Especifican las operaciones que proporcionan las clase y los subsistemas de diseño.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

- **Diseño de la Base de Datos:** Estructuran de la información que se desea persistea a partir del medio de almacenamiento que se elija.
- **Descripción de la arquitectura (vista del modelo de diseño):** Incluye todos los anteriores.
- **Modelo de despliegue:** Modelo de objetos que describe la distribución física del sistema en términos de cómo se distribuye la funcionalidad entre los nodos de cómputo.
- **Descripción de la arquitectura (vista del modelo de despliegue)** Incluye el modelo de despliegue.

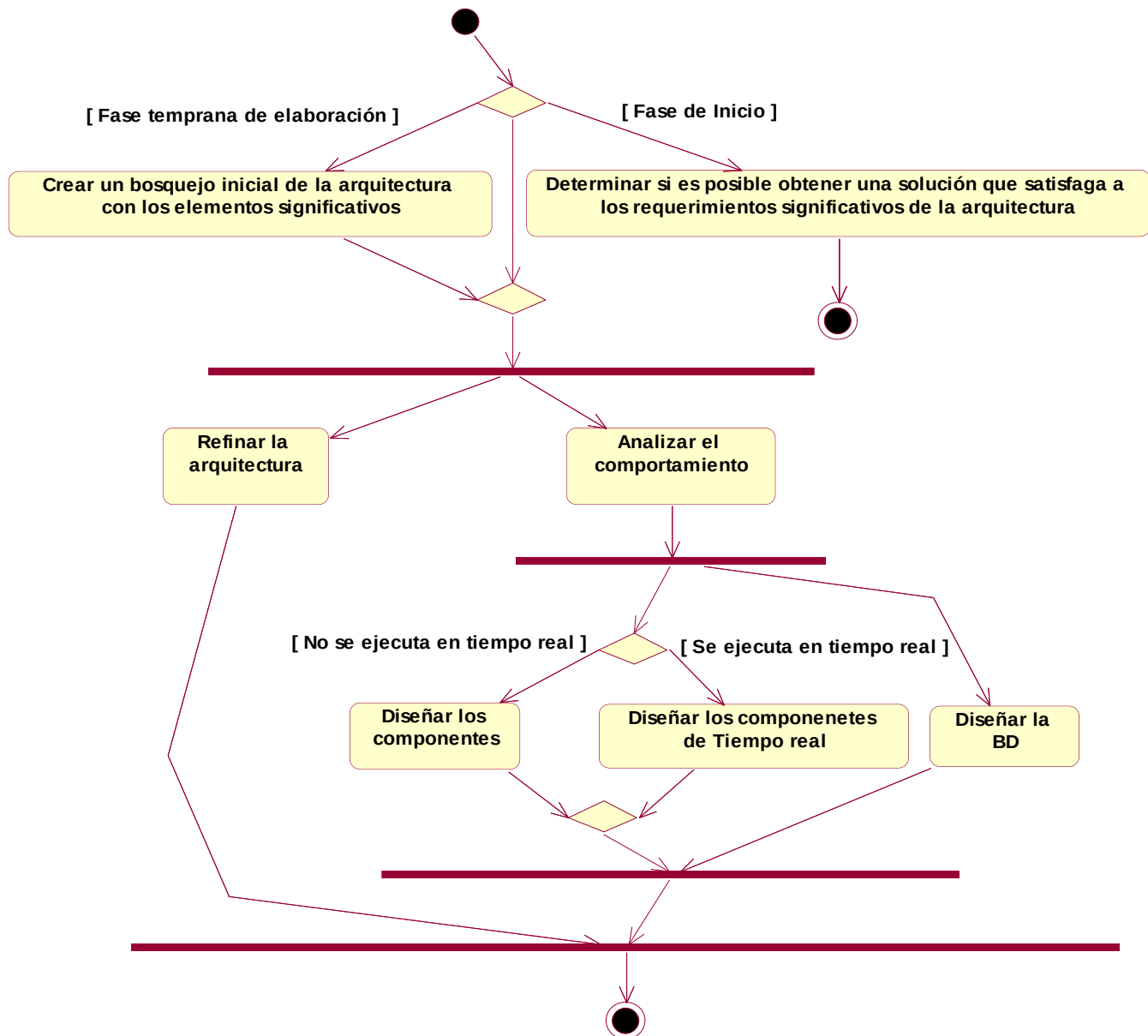


Figura 1 Flujo de trabajo de análisis y diseño.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Los trabajadores que elaboran estos artefactos son:

- **Arquitecto:** Responsable de la integridad de los modelos de diseño y despliegue al garantizar que realicen exclusivamente la funcionalidad descrita en los modelos de caso de uso y de análisis y en los requisitos adicionales.
- **Ingeniero de casos de uso.** Responsable de la integridad de las realizaciones de los casos de uso a él asociados y que estén en correspondencia con los requisitos a ellos asociados.
- **Ingeniero de componentes:** Define y mantiene las operaciones, métodos, atributos, relaciones y requisitos de implementación de las clase que para que se cumplan los requisitos que se espera de ellas de acuerdo a las realizaciones de los casos de uso.

Para construir los artefactos los trabajadores realizan las siguientes actividades:

- **Diseño de la arquitectura:** Identificar nodos y configuraciones de red, subsistemas y sus interfaces, clases, persistencia, distribución, rendimiento, etc.
- **Diseñar un caso de uso:** Identificar los objetos que son necesarios para realizar el caso de uso, distribuir el comportamiento entre los objetos y capturar los requisitos de implementación del caso de uso.
- **Diseñar las clases:** Identificar las clases de diseño a través de las cuales se realicen los casos de uso especificando sus atributos, operaciones, métodos y relaciones entre ellas.
- **Diseñar subsistemas:** Identificar subsistemas cohesivos y con bajo acoplamiento.

5.2- Realización de los casos de uso en el diseño

Una realización de un caso de uso en el diseño describe cómo se realiza ese caso de uso en términos de clases de diseño y la interacción entre sus objetos; para lo cual se requiere:

- describir el flujo de trabajo tomando en cuenta las particularidades del diseño,
- construir un diagrama de clases con las clases necesarias para dar solución al problema planteado y
- construir un diagrama de interacción que muestre un flujo concreto de un caso de uso en términos de interacción entre objetos de clases de diseño.

Una definición importante con respecto a la realización de los casos de uso en el análisis es que toma en cuenta los requerimientos no funcionales que afectan a todo el sistema y los requisitos especiales asociados a cada caso de uso.

5.2.1 – Refinamiento de la descripción de los casos de uso en el diseño

Se describe el diseño concreto del caso de uso a partir de una lógica en particular de entrada y salida y de su impacto en los objetos por lo que en la redacción hay que tener en cuenta:

- Se incluyen los objetos que interactúan para llevar a cabo el caso de uso sin entrar en detalles de nombres de operaciones.
- Se hace referencia a los atributos de los objetos involucrados en la realización.
- No se hace referencia a detalles de interfaz que afectan a la descripción del caso de uso.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Si analizamos la forma de describir los casos de uso que propone Craig Larman en el libro UML y patrones, encontramos que no toma en cuenta la última recomendación. Aunque involucrar los detalles de interfaz puede ser una forma válida de describir los casos de uso, consideramos que esta manera no es recomendable cuando en el equipo no es la misma persona la que juega los roles de Ingeniero de casos de uso, Diseñador de interfaz y Programador ya que cualquier cambio en la interfaz afectaría a la descripción y esta descripción solo debe cambiar si cambian los requerimientos funcionales y no funcionales que se realizan a través del caso de uso.

El formato de descripción de los casos de uso es igual al definido en el análisis. Tomemos como referencia el caso de uso "Registrar artista" y veamos (en negrita) las diferencias de la descripción textual de un caso de uso en el análisis con respecto a la descripción en el diseño. Fíjense que se definen nuevos cursos alternos porque se descubren nuevas excepciones que hay que tener en cuenta. Cualquier cambio en la descripción, precondiciones, postcondiciones u otra cosa (que no sean los propios de este flujo) no es porque se está en diseño sino porque se adquirió mayor claridad sobre las funcionalidades asociadas al caso de uso y la forma en que se llevan a cabo.

ANÁLISIS

Caso de uso:	Registrar artista
Actores:	Anfitrión (Inicia)
Descripción:	El caso de uso se inicia cuando al intentar registrar una nueva solicitud, se detecta que del artista que la presenta no están almacenados sus datos o cuando a la galería le interesa registrar información sobre artistas que desea expongan en la galería. En estos casos, el Anfitrión registra la información de un nuevo artista o los cambios que se produzcan en sus datos. El caso de uso finaliza con el registro de artistas actualizado. Si se va a eliminar a un artista, se verifica si está asociado a una obra, en cuyo caso no se realiza la eliminación; en cualquier otro caso se procede a la eliminación física.
Referencias:	R5
Precondiciones:	El sistema automatizado de RRHH ha replicado las técnicas. Si no se ha replicado, el sistema presenta un mensaje al Anfitrión.
Poscondiciones:	Se actualiza el registro de artistas almacenando los datos del nuevo artista, la modificación de los datos registrados o la eliminación física de un artista si es posible.
Requerimientos especiales	-
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
1 El Anfitrión necesita registrar un nuevo artista, modificar las características de un artista o eliminar un artista.	2 El sistema ejecuta alguna de las siguientes acciones: d) Si decide registrar un nuevo artista, ir a la sección "Nuevo". e) Si decide modificar las características de un artista, ir a la sección "Modificar". f) Si decide eliminar un artista, ir a la sección "Eliminar".
3 El Anfitrión termina el registro de artistas.	4 El sistema finaliza la ejecución del caso de uso.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Sección "Nuevo"	
Acción del actor	Respuesta del sistema
1 El Anfitrión especifica las características de un nuevo artista.	2 El sistema verifica que el artista no esté registrado. En caso de que no exista, se registra el artista.
Sección "Modificar "	
Acción del actor	Respuesta del sistema
1 El Anfitrión selecciona al artista que desea modificar sus características.	2 El sistema presenta las características registradas del artista.
3 El Anfitrión especifica las características que cambian del artista.	4 El sistema incluye los cambios al artista.
Sección "Eliminar "	
Acción del actor	Respuesta del sistema
1 El Anfitrión selecciona el artista que desea eliminar.	2 El sistema verifica si el artista está asociado a una obra. En caso de que no esté asociado, el sistema presenta las características registradas del artista.
3 El Anfitrión confirma la eliminación del artista	4 El sistema elimina el artista.
Cursos alternos	
Sección "Nuevo" : Línea 2 Si el artista ya está registrado, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección.	
Sección "Eliminar" : Línea 2 Si el artista seleccionado tiene obras registradas, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección.	

DISEÑO

Caso de uso:	Registrar artista
Actores:	Anfitrión (Inicia)
Descripción:	El caso de uso se inicia cuando al intentar registrar una nueva solicitud, se detecta que del artista que la presenta no están almacenados sus datos o cuando a la galería le interesa registrar información sobre artistas que desea expongan en la galería. En estos casos, el Anfitrión registra la información de un nuevo artista o los cambios que se produzcan en sus datos. El caso de uso finaliza con el registro de artistas actualizado. Si se va a eliminar a un artista, se verifica si está asociado a una obra, en cuyo caso no se realiza la eliminación; en cualquier otro caso se procede a la eliminación física.
Referencias:	R5
Precondiciones:	El sistema automatizado de RRHH ha replicado las técnicas. Si no se ha replicado, el sistema presenta un mensaje al Anfitrión.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Poscondiciones:	Se actualiza el registro de artistas almacenando los datos del nuevo artista, la modificación de los datos registrados o la eliminación física de una artista si es posible.		
Requerimientos especiales	-		
Curso normal de los eventos			
Acción del actor		Respuesta del sistema	
		1 El sistema presenta de todos los artista registrados su nombre, edad, sexo, dirección y e-mail.	
2 El Anfitrión necesita registrar un nuevo artista, modificar las características de un artista o eliminar un artista. En el caso de la eliminación o modificación, selecciona al artista antes.	3 El sistema ejecuta alguna de las siguientes acciones: g) Si decide registrar un nuevo artista, ir a la sección "Nuevo". h) Si decide modificar las características de un artista, ir a la sección "Modificar". i) Si decide eliminar un artista, ir a la sección "Eliminar".		
El Anfitrión puede repetir el paso 2.			
4 El Anfitrión termina el registro de artistas.	5 El sistema finaliza la ejecución del caso de uso.		
Sección "Nuevo"			
Acción del actor		Respuesta del sistema	
		1 El sistema obtiene las posibles técnicas que puede usar un artista y las presenta para que se puede especificar el nombre, edad, sexo, dirección, e-mail y técnicas que usa el nuevo artista para confeccionar sus obras.	
2 El Anfitrión especifica las características de un nuevo artista (nombre, edad, sexo, dirección, e-mail y técnicas que usa) o decide no registrar al artista.	3 Si decide registrar al artista , el sistema verifica que el artista no esté registrado. En caso de que no exista, se registra el artista.		
Sección "Modificar "			
Acción del actor		Respuesta del sistema	
		1 El sistema presenta las características registradas del artista (nombre, edad, sexo, dirección, e-mail y técnicas que usa) y las técnicas que puede usar y que no estaba usando. Solo se puede modificar la edad, dirección, e-mail y agregar nuevas técnicas.	
2 El Anfitrión especifica las características que cambian del artista (edad, dirección, e-mail o las nuevas técnicas) o	3 Si decide registrar los cambios , el sistema incluye los cambios al artista.		

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

decide no registrar los cambios.	
Sección "Eliminar "	
Acción del actor	Respuesta del sistema
	1 El sistema solicita la confirmación de la eliminación.
2 El Anfitrión confirma la eliminación del artista o decide que no va a eliminar al artista.	3 Si confirma la eliminación, el sistema verifica si el artista está asociado a una obra. En caso de que no esté asociado, elimina el artista.
Cursos alternos	
<i>Sección "Nuevo" : Línea 2</i> Si el artista ya está registrado, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección. <i>Sección "Nuevo" : Línea 3</i> Si el Anfitrión decide no registrar al artista, finaliza la ejecución de la sección y regresa a la línea 2 del curso normal principal. <i>Sección "Modificar" : Línea 3</i> Si el Anfitrión decide no registrar los cambios al artista, finaliza la ejecución de la sección y regresa a la línea 2 del curso normal principal. <i>Sección "Eliminar" : Línea 3</i> Si el artista seleccionado tiene obras registradas, el sistema presenta un mensaje al Anfitrión y regresa a la línea 2 del curso normal principal. <i>Sección "Eliminar" : Línea 3</i> Si el Anfitrión decide no eliminar al artista, finaliza la ejecución de la sección y regresa a la línea 2 del curso normal principal.	

5.2.2 – Diagramas de interacción

Estos diagramas explican gráficamente las interacciones existentes entre las instancias de las clases, habitualmente asociadas con un caso de uso. UML define dos tipos de diagramas de interacción: de colaboración y de secuencia. Cuando se construyen en el diseño no se deben realizar si antes no se construye el diagrama de clases del análisis y se describen los casos de uso en el diseño.

Diagrama de colaboración vs diagrama de secuencia

Los diagramas de colaboración muestran las interacciones entre los objetos en formato de red o grafo (figura 2); en cambio en los diagramas de secuencia el formato es de cerca o muro y se enfatiza en la secuencia.

Ambos diagramas representan lo mismo por lo que CASEs como el Rational Rose generan automáticamente uno a partir del otro.

En el caso de los diagramas de colaboración es indispensable numerar los mensajes para saber en qué orden ocurren. Un diagrama de secuencia representa gráficamente el paso del tiempo si se lee de arriba hacia abajo de izquierda a derecha.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Aunque usualmente la literatura explica los diagramas de interacción usando los diagramas de colaboración como referencia, en esta clase usaremos el de secuencia por tener, a nuestro juicio, mayor legibilidad.

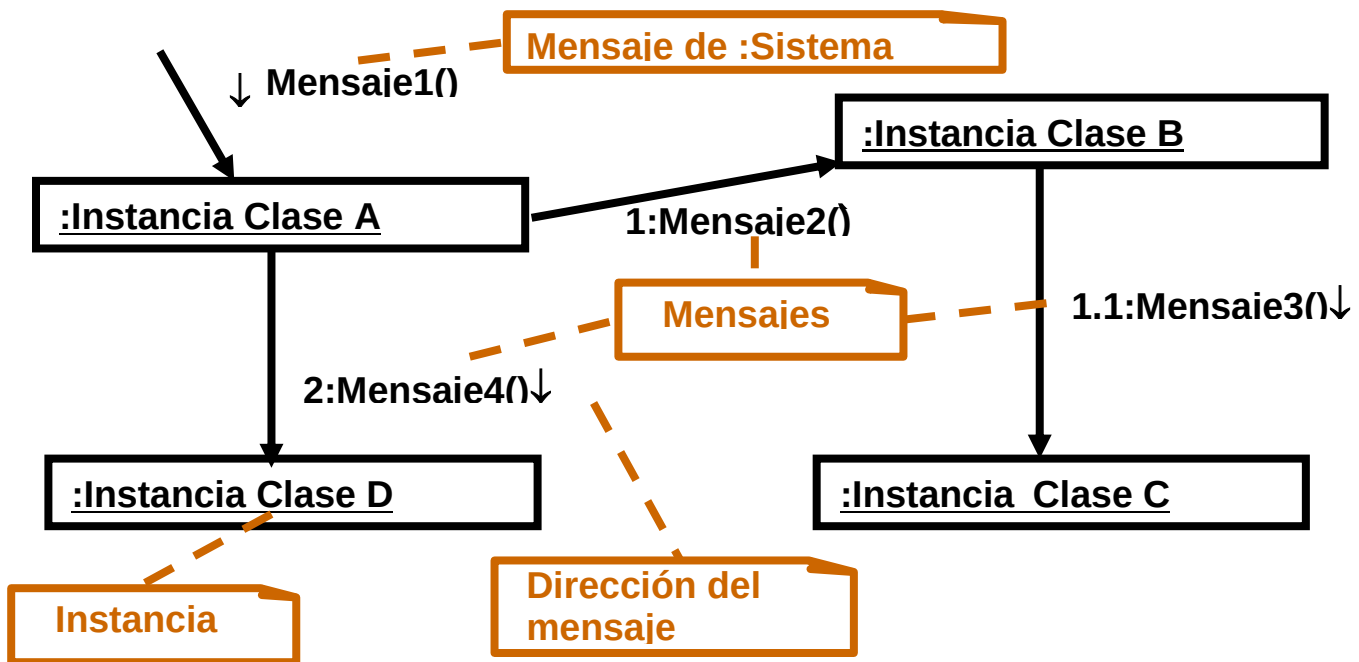


Figura 2 Diagrama de colaboración

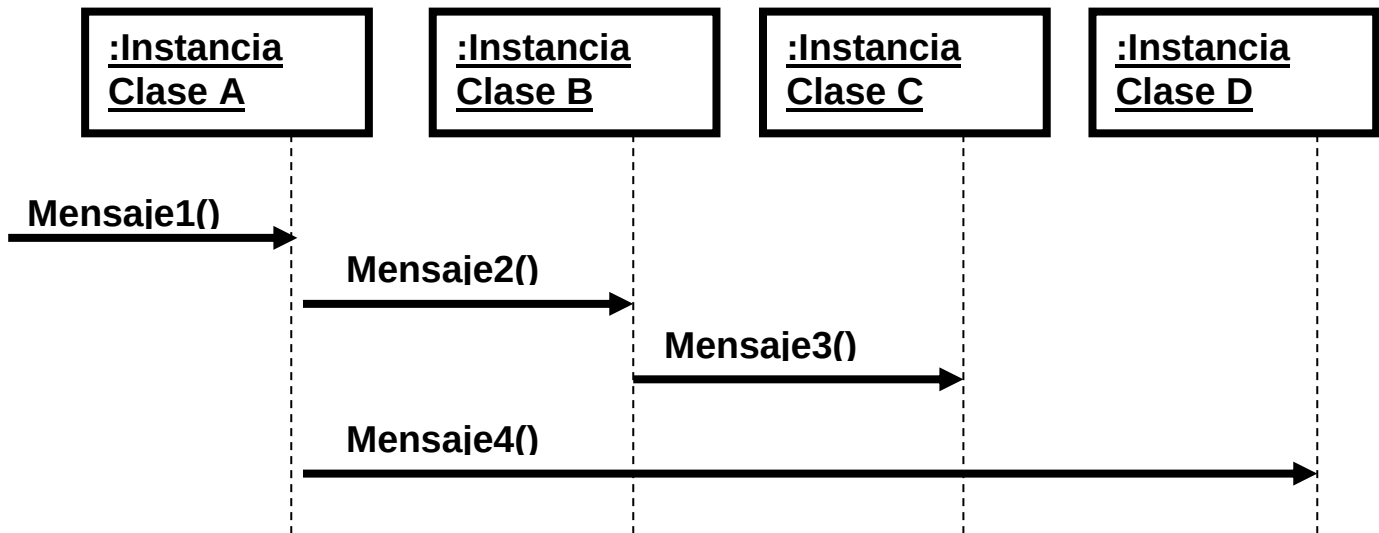


Figura 3 Diagrama de secuencia

Diagrama de secuencia

En el diagrama se representan los actores y todos los objetos necesarios para realizar el caso de uso siguiente la filosofía cliente servidor (figura 4). Seguir esta filosofía implica que cuando un objeto servidor recibe un mensaje se pregunta si por si solo es capaz de dar respuesta; si no puede entonces se comporta como un objeto cliente y envía mensaje(s) al(a los) objeto(s) que lo ayuda(n) a dar respuesta.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

De los objetos parten líneas discontinuas que representan su líneas de vida, una X al final indica la destrucción del objeto. Los mensajes se representan por líneas que enlazan las línea de vida de un objeto cliente con la línea de vida de un objeto servidor. La saeta indica la dirección del mensaje.

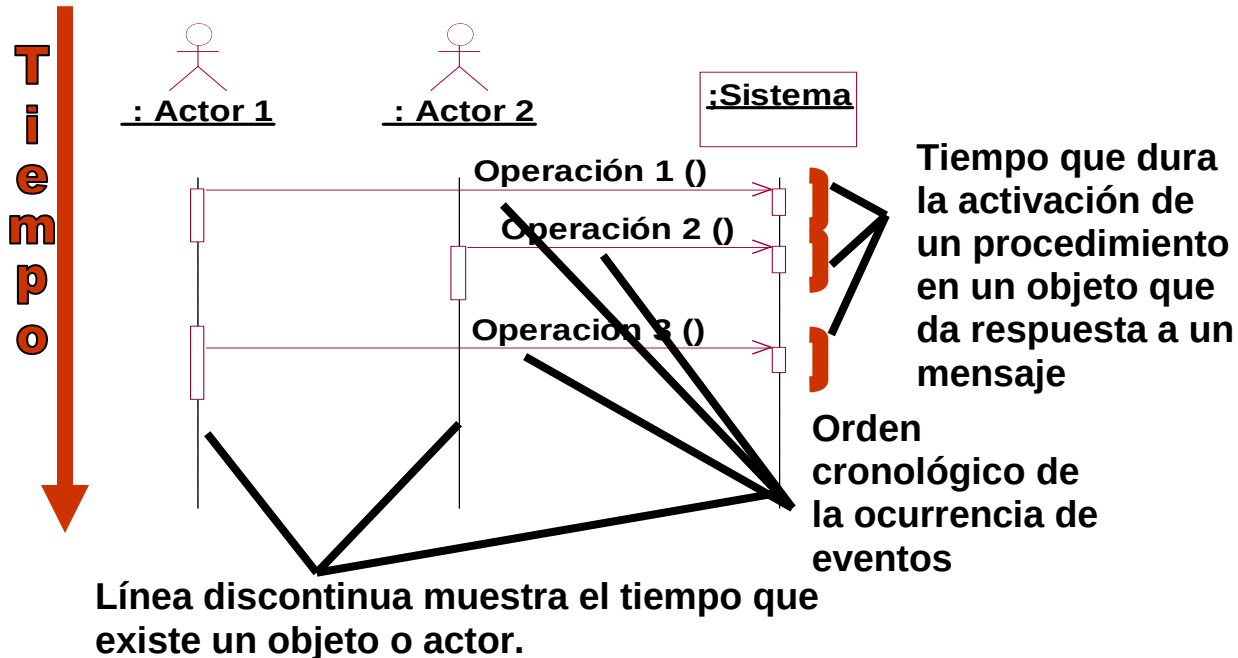


Figura 4 Particularidades del diagrama de secuencia.

El fomato de los mensajes es:

Variable de retorno:=Mensaje (Parámetro:Tipo del parámetro,):Tipo de retorno

Si el mensaje se repite varias veces hay que indicarlo:

- Poniendo antes la cantidad exacta de veces que se repite.
- Encerrando el mensaje entre corchetes: **Valor mínimo [Mensaje] Valor máximo**
- Poniendo antes “*” si se puede repetir varias veces, pero se desconoce cuántas.

Si el mensaje solo se envía si se cumplen ciertas codiciones, entonces se indica antes del mensaje la guarda: **[Condición] Mensaje**

En el diagrama de secuencia se representan objetos de los tipos de clases:

- **Vistas (interfaz o frontera):** Encargados de presentar la información de una clase en un dispositivo de salida con el objetivo de independizar el comportamiento de la presentación del de guardar los datos. Si en el análisis solo se identificaba una clase de este tipo por cada interacción acto-caso de uso, ahora se identifican todas las necesarias de manera que aparecen nuevas clases relacionadas con, por ejemplo, formularios.



- **Interfaz:** Las clases de interfaz con los dispositivos y bases de datos y sistemas automatizados externos que se representaron en el análisis.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software



- **Controladoras:** Dirigen la realización de una o más responsabilidades de alto nivel, para lo cual interactúan con otras clases de menor nivel jerárquico. Pueden ser vistas como mecanismo de control para el sistema. En principio es suficiente con las identificadas en el análisis ya que las nuevas que aparecen buscan minimizar la complejidad de las clases controladoras de casos de uso con mucha complejidad.



- **Entidad:** Modelan la información y el comportamiento asociado de algún fenómeno o concepto, como una persona, un objeto del mundo real o un suceso. Se parte de las definidas en el análisis aunque pueden aparecer nuevas.



- **Multiobjeto:** Representan a un grupo de instancias guardadas en un contenedor u objeto colección, pero no necesariamente se implementa así, representa tan solo un conjunto lógico de instancias. No fueron identificadas en el análisis y aparecen cuando se desea enviar un mensaje a un conjunto de objetos de una misma clase base.



En el libro de Bruegge se brindan un conjunto de heurísticas que ayudan a construir los diagramas de secuencia. Ellas son:

1. Los objetos de control son creados por objetos de interfaz que inician CU.
2. Los objetos de interfaz son creados por objetos de control.
3. Los objetos de entidad son accedidos por objetos de control y de interfaz.
4. Los objetos de entidad nunca tienen acceso a los objetos de interfaz o control y esto hace que sea más fácil compartir objetos de entidad entre CU.
5. Casi todos los casos de uso responden a la siguiente estructura de descripción (figura 5). La excepción son los casos de uso abstractos (incluidos y extendidos) que comienzan con la clase controladora ya que desde el caso de uso que lo incluye o extiende su controladora le envía un mensaje a la otra controladora le envía un mensaje a la otra controladora.

El 1er objeto asociado a una clase interfaz con la que se relaciona el actor es usualmente el que representa al menú a través de cual se el sistema dará acceso al caso de uso. Automáticamente se pasa el control al objeto de la clase controladora del caso de uso. A partir de ahí se representan todos los objetos necesarios para realizar el caso de uso.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

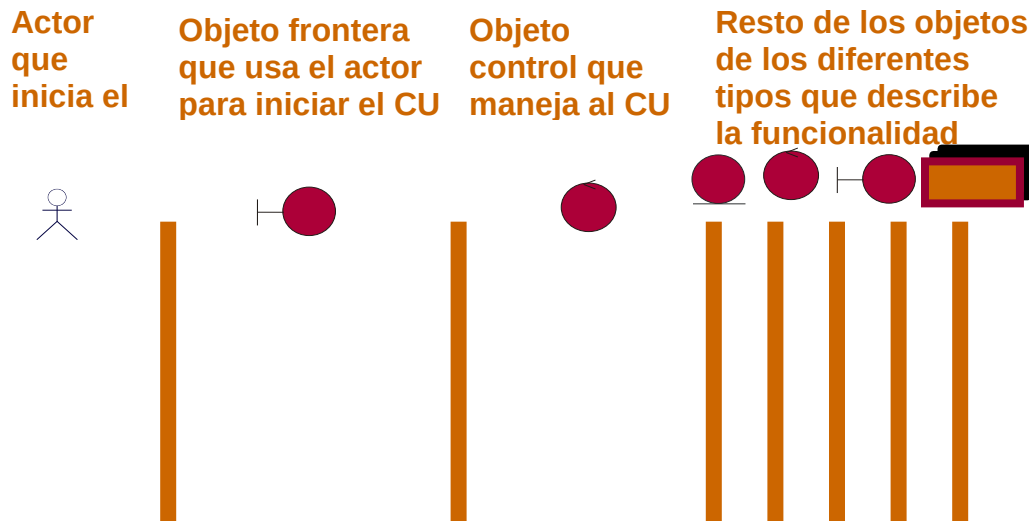
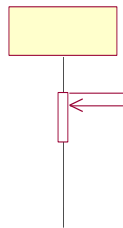


Figura 5 Estructura de un diagrama de secuencia.

6. Ley de Demeter: Como respuesta a un mensaje m , un objeto O debería enviar mensajes solo a los siguientes objetos:

a) A sí mismo



b) A los objetos enviados como argumentos del mensaje m .

c) A los objetos que O crea como parte de su relación ante m .

d) A los objetos que son accesibles directamente desde O , es decir, utilizando los valores de los atributos de O .

Proponemos construir el diagrama de secuencia a partir del flujo de control definido de forma que se leabore un diagrama para el curso normal principal y uno para cada una de las secciones de cada caso de uso sin considerar la representación de las opciones de cancelar si solo implican cerrar vistas, terminar la ejecución del caso de uso o poner un mensaje de advertencia.

En el diagrama hay que tener en cuenta la validación de las condiciones que provocan las excepciones que llevan a los cursos alternos, aún cuando no se pongan las interacciones asociadas a la actuación del sistema si solo implican mensajes de error o terminar la ejecución del caso de uso.

La filosofía de construcción está basada en el control manejado por eventos de manera que el ciclo principal (curso normal principal) esoera un evento externo que posiblemente desemboque en la ejecución de secciones. Cada vez que se tiene disponible un evento se despacha al objeto adecuado en base a la información asociada al evento.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

En la figura 6 se muestra el diagrama de secuencia del curso normal principal del caso de uso “Registrar artista”. Fíjese que lo primero que hace la controladora es verificar si se cumplen las precondiciones y, en caso de que se cumplan, es que continúa la ejecución del caso de uso. Para dar respuesta a lo primero que ocurre (una respuesta del sistema cuando se cumplen las precondiciones) hay que interactuar con otras clases para poder visualizar la información que se indica en la descripción del caso de uso.

En la figura 7 se muestra el diagrama correspondiente a la sección Agregar, fíjense que cuando se construye hay que garantizar que se cumplan las postcondiciones definidas.

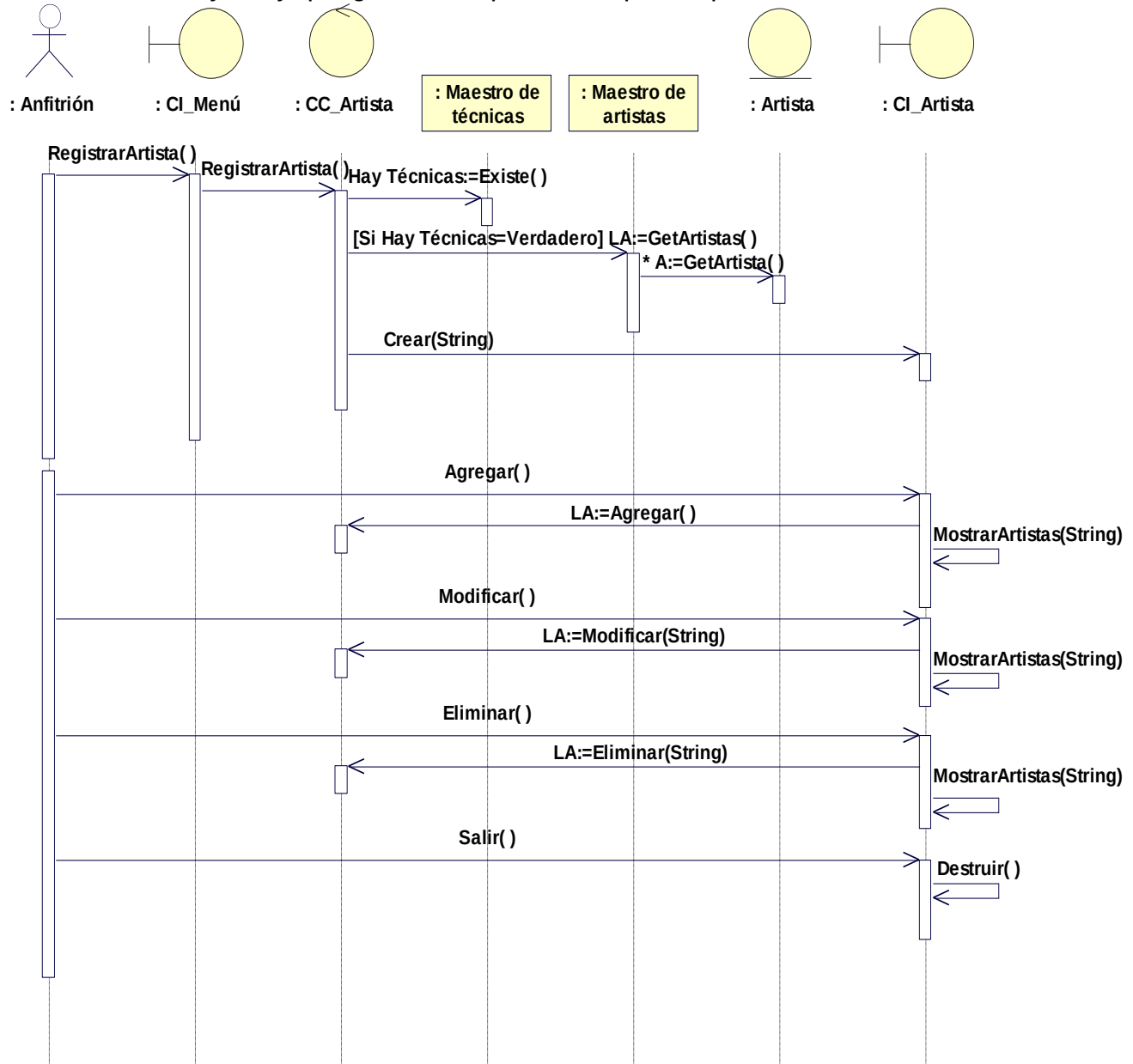


Figura 6 Diagrama de secuencia del caso de uso: Registrar artista- Curso normal principal.

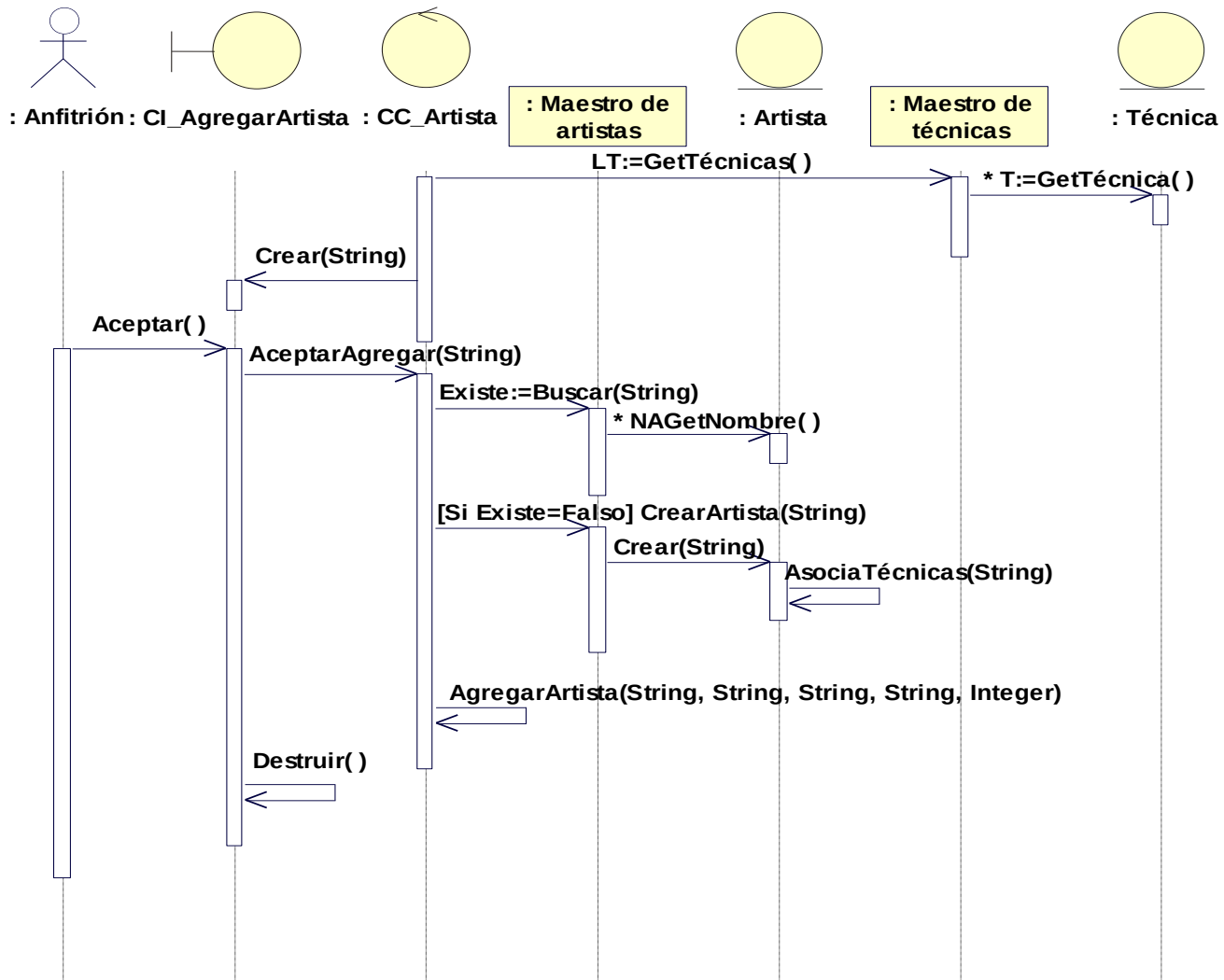


Figura 7 Diagrama de secuencia del caso de uso Registrar artista : Sección Agregar artista.

Patrones de diseño

Para construir eficazmente los diagramas de secuencia se han definido una serie de patrones de diseño que surgen a partir de la posibilidad de odificar algunas situaciones comunes (como la adición de elementos a un conjunto) que tienen una forma común de solucionarse. Por lo tanto, los patrones de diseño son directrices y pincipios estructurados que describen un problema xomún y entregan una buena solución ya aprobada a la que le dan un nombre.

Los patrones de diseño ayudan a diseñar correctamente en menos tiempo, a construir problemas reutilizables y facilitan la documentación. Existen innumerables patrones referenciados en la literatura, en la clase solo haremos referencia a los más utilizados. Para describirlos se usará el formato que propone Craig Larman que consiste en:

Nombre del patrón: <Nombre>

Problema que resuelve: <¿Cuál es el principio fundamental en virtud del cual asignamos las responsabilidades a los objetos?>

Solución: <Asignar una responsabilidad a la clase que tiene la información necesaria para cumplirla>

- **Nombre del patrón:** Polimorfismo (figura 8)

Problema que resuelve: Hay varias implementaciones de un mismo algoritmo dentro de sistema.

Solución: Una clase abstracta con la interfaz genérica que deben tener todas las implementaciones y tantas clases concretas como implementaciones existan.

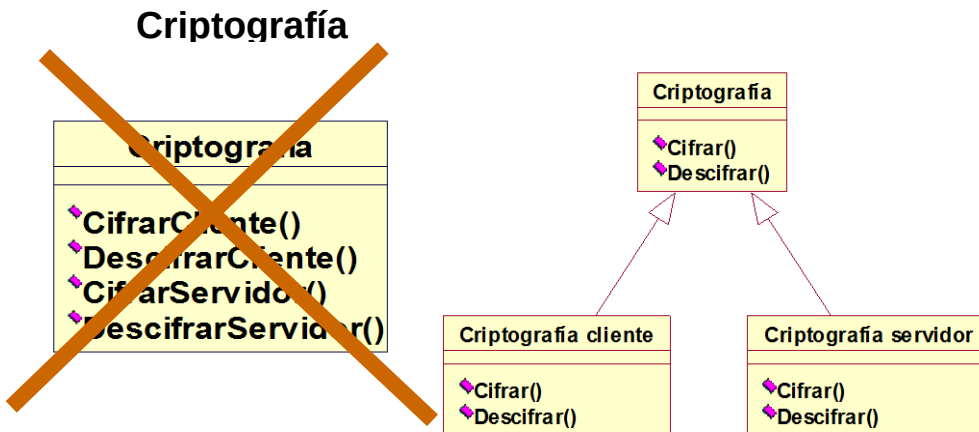


Figura 8 Patrón Polimorfismo

- **Nombre del patrón:** Experto

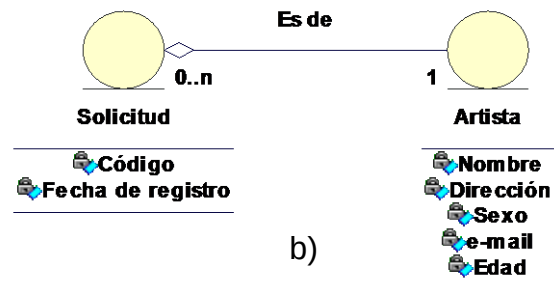
Problema que resuelve: Determinar cuál es la clase que debe asumir una responsabilidad a partir de la información que posee cada una.

Solución: Asignar una responsabilidad al experto en información (clase que cuenta con la información necesaria para cumplir la responsabilidad).

En la figura 9 se muestra un ejemplo asociado al caso de uso “Controlar solicitudes de obras”, en este caso se está modelando el segmento en el cual es necesario mostrar la información de todas las solicitudes registradas (figura 9a) para lo cual hay que pedirle a las clases expertas (figura 9b) la información; de manera que las iteraciones necesarias, desde que se inicia el caso de uso hasta que muestra la información, es el contenido de la figura 9c.

Código	Fecha de registro	Artista	¿Obras evaluadas?
35439	20/05/2003	Roberto González	☐
28748	05/12/1999	René Raclift	☒

a)



b)

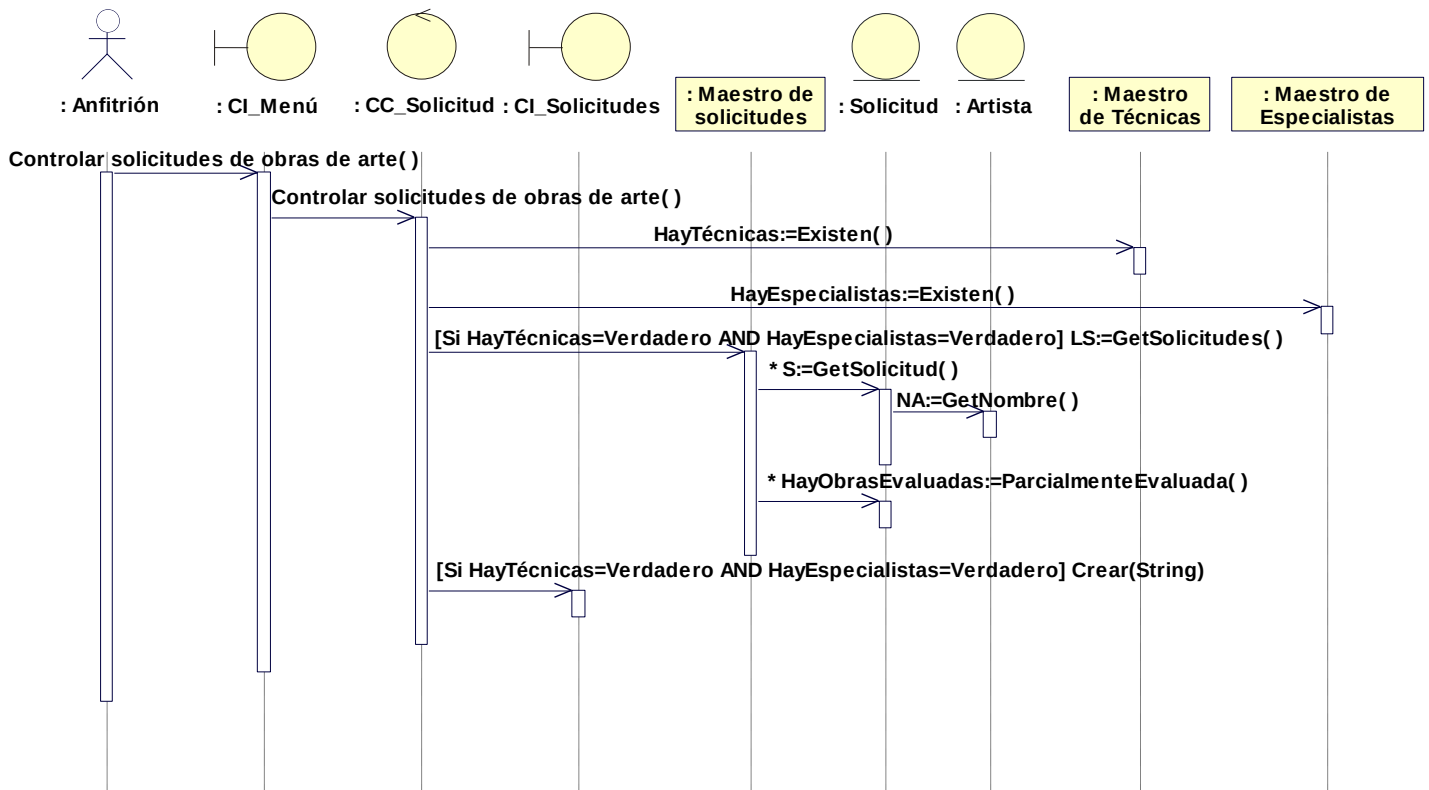


Figura 9 Patrón experto. a) Información a mostrar. b) Clases expertas. c) Diagrama de secuencia.

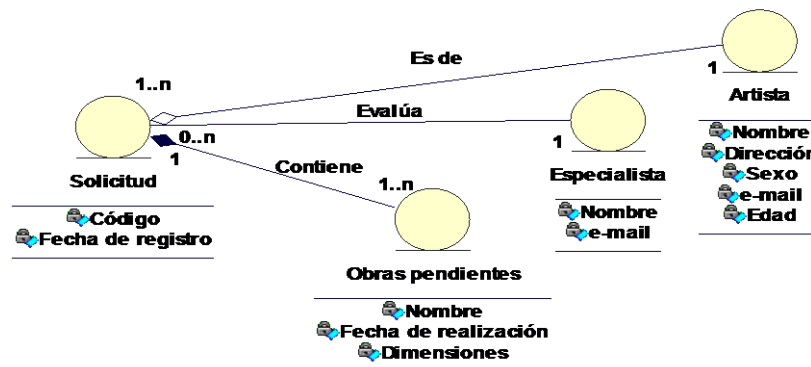
- **Nombre del patrón:** Creador

Problema que resuelve: ¿Quién es el responsable de crear alguna nueva instancia de alguna clase?

Solución: Asignarle a la clase B la responsabilidad de crear las instancias de la clase A en uno de los siguientes casos:

- a) B agrega los objetos de A.
- b) B contiene a los objetos de A.
- c) B registra las instancias de los objetos de A.
- d) B utiliza específicamente los objetos de A.
- e) B tiene los datos de inicialización que serán transmitidos a A cuando este objeto sea creado.

En la figura 19 se muestra un ejemplo de que B contiene a los objetos de A. Fíjense que los objetos correspondientes a las clases artista y especialista ya existen por lo que solo hay que establecer las asociaciones, no sucede así con los objetos obras que hay que crearlos.



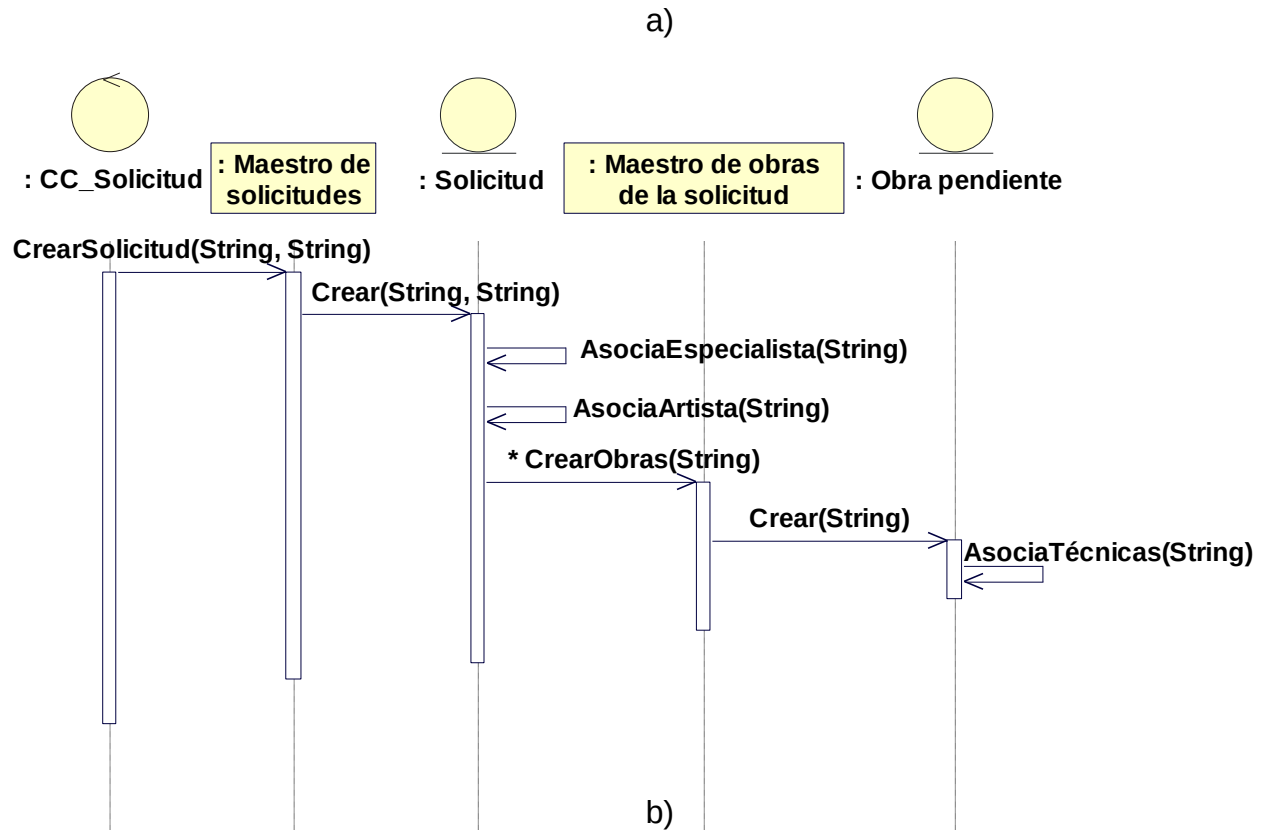


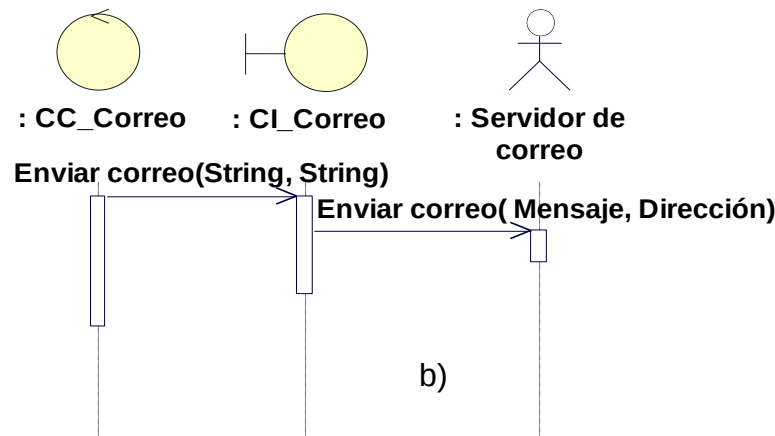
Figura 10 Patrón creador. a) Estructura de las clases. b) Diagrama de secuencia.

- Nombre del patrón:** Controlador (Comando o Fachada)
Problema que resuelve: ¿Quién debería encargarse de atender un evento del sistema?
Solución: Asignar la responsabilidad del manejo del mensaje de los eventos del sistema, a una clase que represente una de las siguientes opciones:
 - de fachada: el sistema global, la empresa u organización.
 - de tarea: algo en el mundo real que es activo y que controla la realización de una tarea.
 - de casos de uso: manejador artificial de todos los eventos del sistema de un caso de uso.

Para el caso de uso “Controlar solicitudes de obras de arte” se había definido como clase controladora a CC-Solicitudes. Este caso de uso tiene relación de inclusión y extensión con otros casos de uso por lo que esta relación se representan enviando mensajes desde esta controladora hasta las controladoras de los casos de uso asociados tal como se indica en la figura 11a, fíjense que para las extensiones hay que poner como guardas las condiciones que tiene que cumplirse para la extensión. En la figura 11b se muestra cómo quedaría el diagrama de secuencia del caso de uso “Enviar correo” tomando en cuenta las heurísticas vistas anteriormente.



a)



b)

Figura 11 Patrón Controlador. a) Relación entre controladoras de casos de uso con relaciones de extensión e inclusión. b) Diagrama de secuencia del caso de uso “Enviar e-mail”.

- **Nombre del patrón:** Indirección

Problema que resuelve: Acoplamientos directos a veces provocan un alto acoplamiento y limitan la posibilidad de reutilización.

Solución: Una clase que sirva de interfaz, creando una indirección.

En la figura 12 se muestra la relación de la clase controladora del caso de uso “Asignar especialistas” con la clase CI_SA-EEHH que crea una indirección con la base de datos de un manipula un software externo para poder obtener a los especialistas de arte especializados en un conjunto de técnicas dadas.

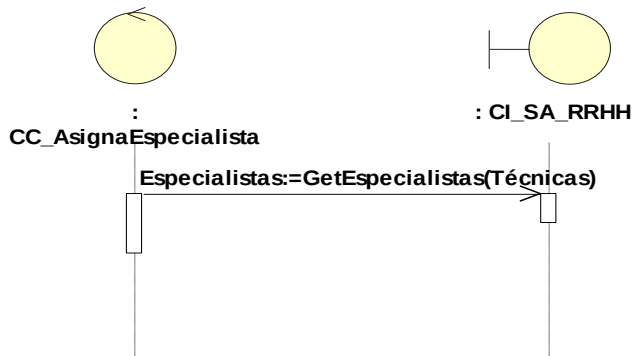


Figura 12 Patrón Indirección.

- **Nombre del patrón:** Bajo acoplamiento

Problema que resuelve: ¿Cómo dar soporte a una dependencia escasa y a un aumento de la reutilización?

Solución: Asignar una responsabilidad para mantener bajo acoplamiento

El acoplamiento es la medida de fuerza con que una clase está conectada a otras clases a las que conoce y recurre a ellas; de manera que un alto acoplamiento implica que se recurre a muchas otras clases y un bajo acoplamiento que no se depende de muchas otras. La meta del diseñador es alcanzar un bajo acoplamiento ya que se reduce el impacto de los cambios y se hace un diseño con clases menos dependientes. En la figura 13 se muestra un ejemplo en el que el acoplamiento es alto con respecto a la clase Solicitud ya que para otras responsabilidades (como por ejemplo la recuperación de información) quién se relaciona con ella es la clase Maestro de solicitudes por lo que la solución correcta sería la de la figura 14.

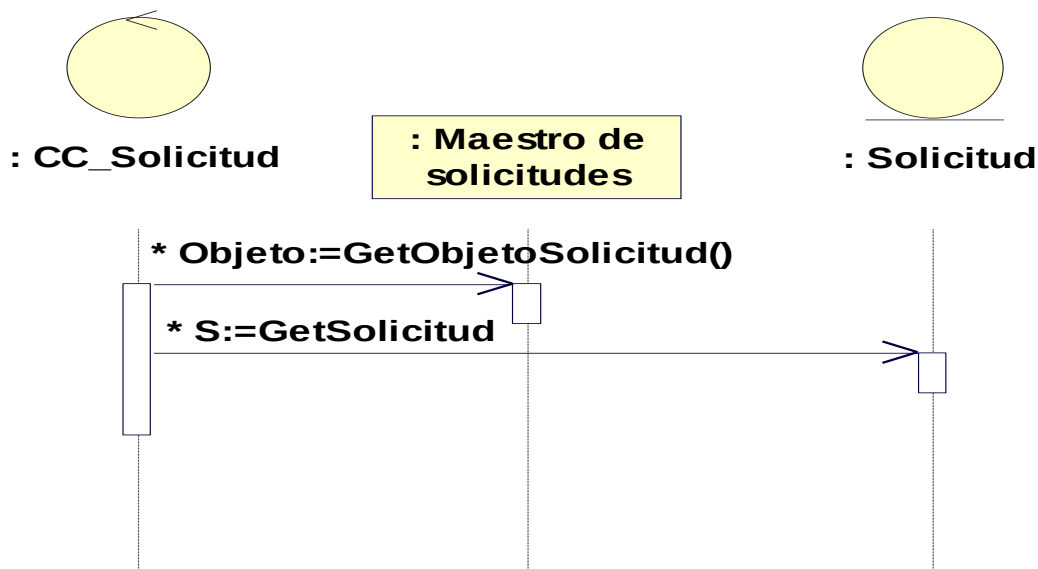


Figura 13 Ejemplo de alto acoplamiento.

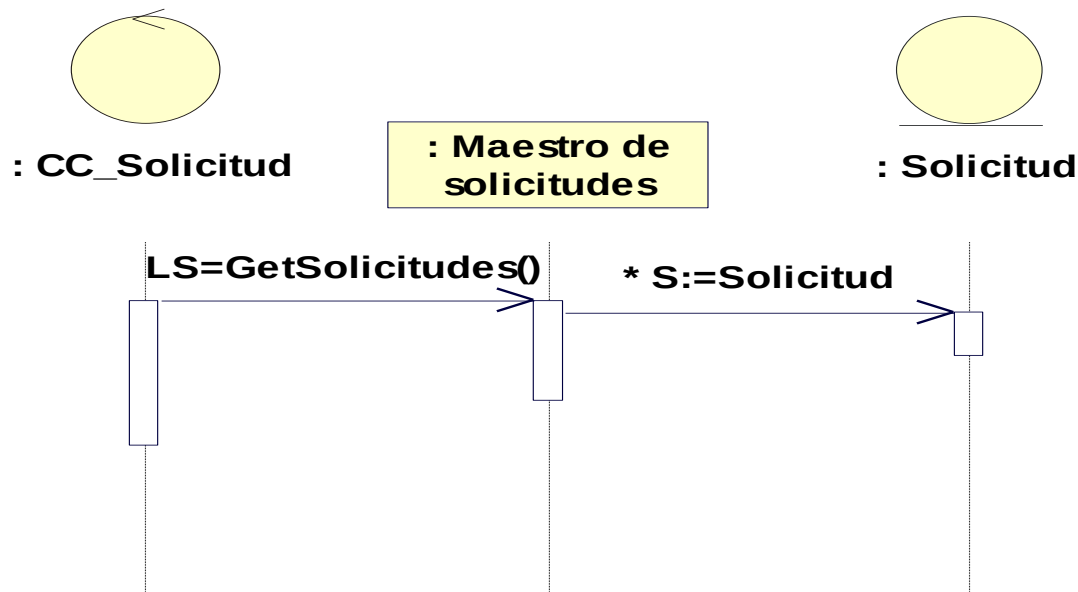


Figura 14 Ejemplo de bajo acoplamiento.

- **Nombre del patrón:** Alta cohesión

Problema que resuelve: ¿Cómo manejar la complejidad dentro de límites manejables?

Solución: Asignar una responsabilidad de modo que la cohesión siga siendo alta.

La cohesión es la medida de la fuerza que une a las responsabilidades de una clase. Una clase con baja cohesión hace muchas cosas no afines o muchas tareas lo cual no es bueno porqu son difíciles de entender, reutilizar y conservar ya que son delicadas al afectarlas constantemente los cambios.

Las clases con alta cohesión mejoran la calidad y la facilidad de uso, se simplifica el mantenimiento,a menudo generan un bajo acoplamiento y son más fáciles de reutiliza. La idea es que cada clase tenga asociadas las responsabilidades que le corresponden de acuerdo a su comportamiento. Es por eso que a la meta de bajo acoplamiento se adiciona la alta cohesión.

- **Nombre del patrón:** Estado

Problema que resuelve: El comportamiento de un objeto depende de su estado (o el tipo). La lógica condicional no es recomendable por su complejidad y su poca capacidad de extensión. **Solución:** Crear una clase para cada estado (o tipo) (No una clase estado o tipo) y aplicar el patrón polimorfismo si corresponde.

En la figura 15 hay un ejemplo de los estado de un objeto Obra en un sistema de planificación y control de avances de obra.

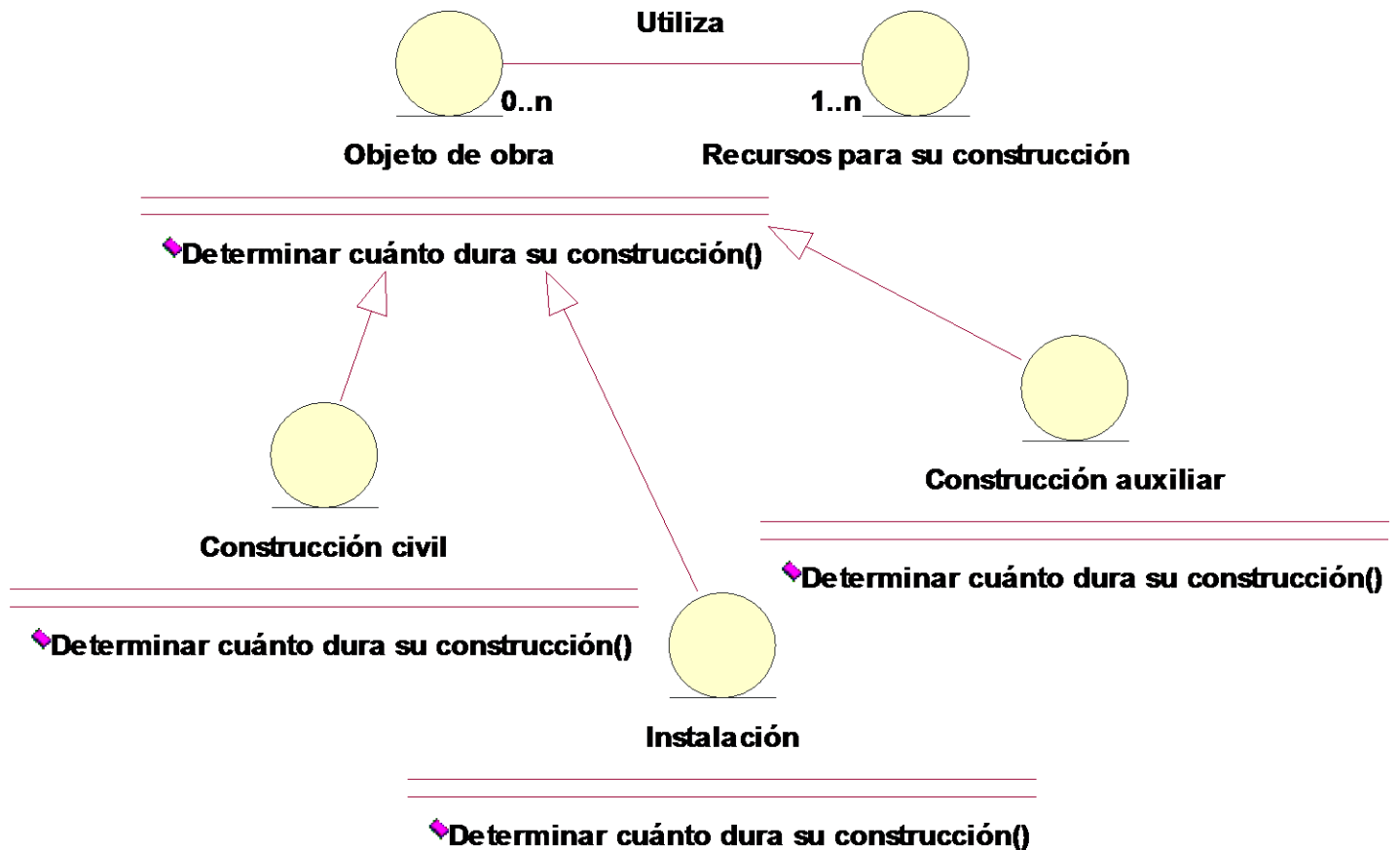


Figura 15 Patrón estado.

Directrices en la creación del diagrama de secuencia

1. Representar los actores y la clase controladora del sistema que recibe las acciones del usuario (Menú).
2. Seleccionar la clase controladora que se encargue del mensaje de las operaciones del sistema.
3. Aplicar el principio de separación de modelo-vista: "No compete a los objetos del dominio comunicarse con los objetos de la interfaz, lo hacen las controladoras."
4. Revisar las postcondiciones que se describieron para ese caso de uso, de manera que se garanticen.

5.2.3 – Diagrama de clases del diseño

El diagrama de clases del análisis contiene:

- Los conceptos que modelan el comportamiento del sistema (entidad, control e interfaz).
- Asociaciones entre conceptos especificando: navegabilidad, multiplicidad, tipos asociativos, generalización/especialización y agregación/composición.
- Los atributos.

Las clases del diseño tienen, además, operaciones, parámetros, atributos, tipo, etc.; necesarios para su implementación en el lenguaje de programación elegido. También está especificada la visibilidad de atributos y operaciones (Figura 16).

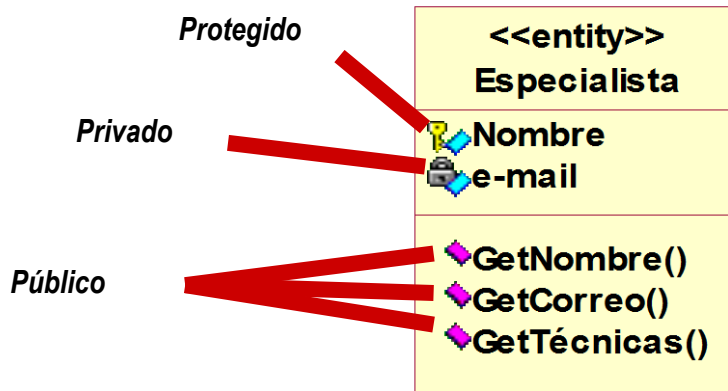


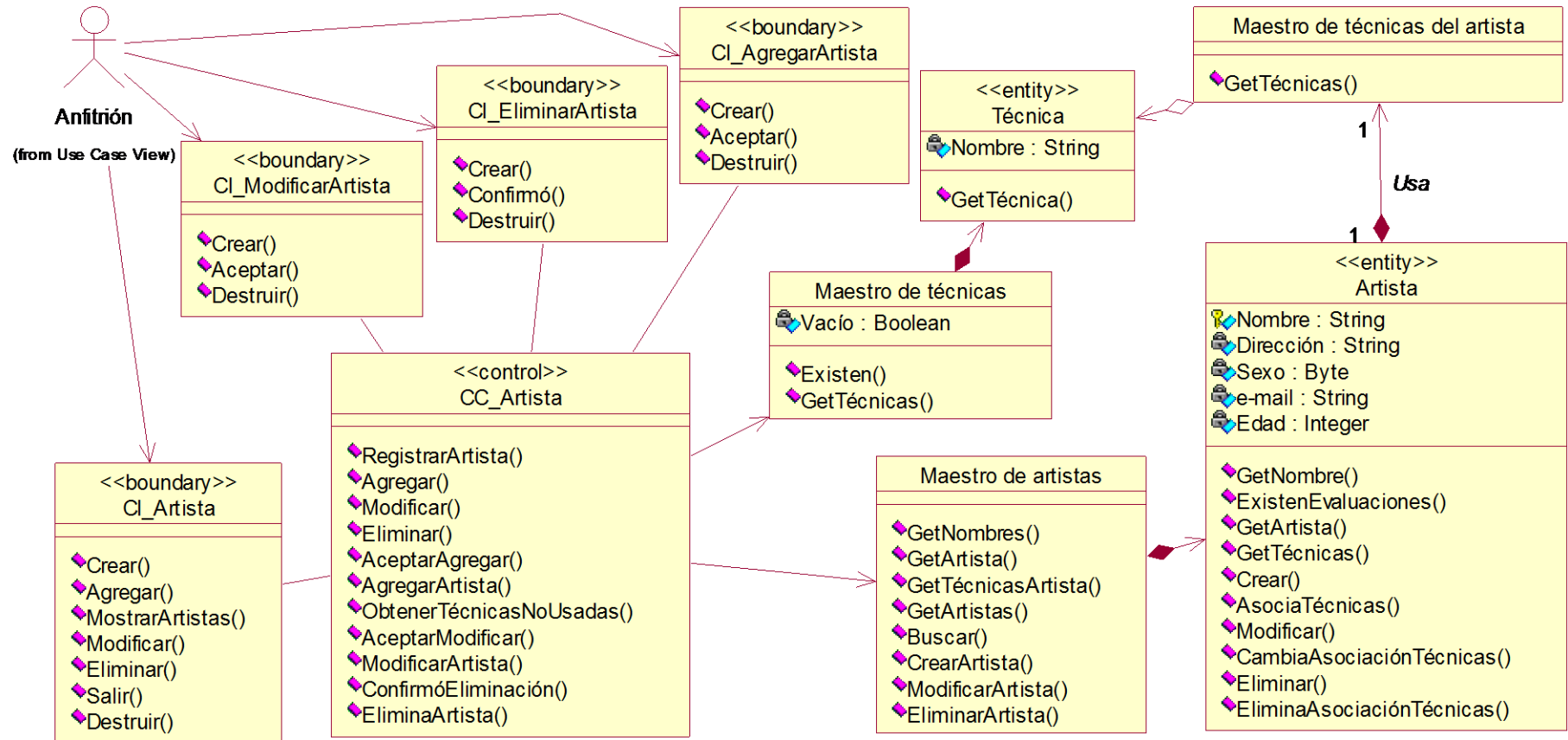
Figura 16 Especificación de una clase en el diseño.

Para crear el diagrama de clases del diseño se deben seguir los siguientes pasos:

1. Revisa las definiciones del DC del análisis actualizando el DC del diseño con las clases y relaciones nuevas o que se eliminan.
Las clases que se quedan tienen que estar en los diagramas de interacción.
2. Agregue los métodos a las clases.
Los métodos se obtienen de la interacción cliente-servidor del diagrama de secuencia, definiendo para el servidor la operación como método.
3. Representar la visibilidad entre dos clases que se relacionan, pero entre las que no existe una asociación.
La dirección de la flecha depende de quién inicie la comunicación. Recuerde que si puede ser iniciada por cualquiera de las dos, no se pone flecha.

En la figura 17 se muestra el diagrama de clases del diseño resultante de los diagramas de secuencia del caso de uso “Registrar artista”.

Figura 17 Diagrama de clases del diseño que se deriva del caso de uso “Registrar artista”.



5.3 – Estándares en el diseño de la interfaz y los reportes del sistema

“Las interfaces hombre-máquina no son más que las interfaces básicas de usuario que incluyen cosas como: menús, ventanas, teclado, ratón, los “beeps” y algunos otros sonidos que la computadora hace, en general, todos aquellos canales por los cuáles se permite la comunicación entre el hombre y la máquina.”

Lewis y Rieman, 1993

Cuando se diseña la interfaz de la aplicación y el formato de los reportes de salida hay que garantizar que sea consistente, garantice la retroalimentación con los usuarios y minimice las equivocaciones que estos pueden cometer. Para ello el diseñador debe buscar respuestas a preguntas como las siguientes:

1. ¿Quién usará el sistema y para qué?.
2. Requerimientos funcionales vs. Opciones del sistema. Paquetes vs. Menú
3. Estándares existentes.
4. Crear sus propios estándares.

Para el caso de estudio pudiera definirse algo como lo siguiente:

Todos los botones tendrán asociado un texto corto y un icono que indican la acción genérica que permiten ejecutar, definiéndose para toda la aplicación los iconos de la figura 18.:



Figura 18 Iconos utilizados en las pantallas del sistemas.

Todos los reportes estarán encabezados por texto de la figura 19.



Figura 19 Formato de encabezamiento de los reportes.

A continuación irá el nombre del reporte.

4 – Diseño de la base de datos

“Una buena aplicación es aquella que al tirarla de un décimo piso sigue funcionando en el suelo”

José A. Ramalho

Para producir software de calidad es necesario el desarrollo de un proceso disciplinado. Esta calidad no se logra con el esfuerzo puntual en alguna o algunas de las fases o etapas del ciclo de vida del software, es un trabajo consciente en la aplicación de técnicas ingenieriles, junto al establecimiento de métricas de calidad desde el inicio de su construcción. En la producción de software el esfuerzo fundamental está en el diseño, no en la codificación y la prueba, que solo ocupan el 15% del tiempo total de desarrollo.

La calidad es un tema complejo por la cantidad y diversidad de aristas que involucra, por lo que este trabajo se propone abordar un punto dentro de todos ellos, crítico en la mayoría de las aplicaciones que se desarrollan en la actualidad, *el diseño de la información almacenada*. La representación de los objetos tiene que ser completa, consistente, correcta, íntegra, fácilmente mantenible, flexible, fácil de usar y fiable.

En el diseño de la base de datos es importante la forma en que los objetos se guardan en los medios de almacenamiento. El diseño de la base de datos (BD) implica negociaciones entre usuarios y diseñadores de acuerdo a las necesidades de la aplicación y las capacidades del gestor.

El paradigma de la orientación a objetos (OO) ha provocado una revolución en los conceptos de la Informática, siendo el cambio más importante desde que surgieron los métodos estructurados, impactando en mayor escala en los lenguajes de programación de alto nivel y en las metodologías de análisis y diseño de sistemas informáticos. En el campo de las BD, en la actualidad se comercializan productos que soportan el modelo relacional, gestores de objetos y sistemas de gestión relacionales que extienden sus capacidades hacia el modelo orientado a objetos (MOO).

Esta diversidad hace difícil el diseño de la BD cuando se modela un sistema siguiendo el enfoque OO.

En el modelo relacional se hace difícil, y en ocasiones imposible, capturar la complejidad semántica del modelo real. Además, se hace compleja la modificación, es decir, la evolución del esquema. También tiene problemas en hacer persistente el comportamiento. Las principales soluciones que se han adoptado son:

- Ampliar el modelo relacional de forma apropiada: en este sentido se han adicionado características como los procedimientos almacenados, disparadores, el tipo de dato de una columna puede ser otra columna.
- Desechar por completo, sustituir por algo nuevo: aparecen nuevos modelos de datos que se incluyen dentro de la 3ra generación (1ra- jerárquico y reticular, 2da-relacional).

Los modelos de 3ra generación deben cumplir las siguientes características definidas por el Comité para Fundación para las Bases de Datos de Avanzadas en 1991:

1. Más allá de los servicios tradicionales de gestión de datos, los SGBD de 3ra generación proporcionarán apoyo a estructuras de objeto y reglas más ricas.

1.1 Deben tener un sistema rico de tipos, siendo deseable:

- Un sistema de tipos de datos abstractos para construir nuevos tipos básicos.
- Un constructor de tipos de matrices.
- Un constructor de tipos de secuencias.
- Un constructor de tipos de registros.

- Un constructor de tipos de conjuntos.
- Funciones como tipos.
- Un constructor de tipos de unión.
- Composición recursiva de los constructores anteriores.

1.2 Permitir que los tipos se organicen en una jerarquía de herencia (simple y múltiple).

1.3 Inclusión de funciones (métodos de las clases) en la BD y permitir que los diseñadores de aplicación tengan a su alcance los beneficios de la encapsulación.

1.4 Asignación automática de identificadores solo cuando no se disponga de una clave primaria.

1.5 Inclusión de reglas (activaciones, restricciones) en la BD no asociadas con los métodos.

2. Los SGBD de 3ra generación deben subsistir a los SGBD de 2da generación.

2.1 Todo el acceso a una BD debe hacerse a través de un lenguaje de consulta no procedimental y de alto nivel.

2.2 Permitir realizar cambios sin afectar la aplicación a través del uso de vistas actualizables o colecciones virtuales que en los SGBD de 2da generación garantizan mejor que en los de 1ra encapsular el acceso a la BD, pero se debe trabajar más en esto.

2.3 Debe quedar oculta la gestión interna que haga el sistema de gestión (los detalles físicos) a los programadores de aplicaciones. No se puede renunciar a la independencia de los datos.

2.4 Para especificar colecciones (tales como matrices, conjuntos, secuencias, etc.) se debe usar alguna de las siguientes vías:

- Extensional : la representación es una colección de apuntadores en el que la condición de miembro de cada conjunto se determina manualmente por el programador de la aplicación.

Ej.: Alumnos (nombre,edad,promedio)

Grupos (número, integrantes)

→ **Apuntadores a los alumnos que pertenecen al grupo**

- Intencional: la representación se especifica mediante expresiones SQL, de manera que el establecimiento de la condición de miembros se mantiene de forma automática.

Ej.: Supongamos que los alumnos se clasifican teniendo en cuenta la edad.

Grupos (número, edad mínima, edad máxima, integrantes)

3. Deben estar abiertos a otros subsistemas.

3.1 Ser accesibles desde múltiples lenguajes de alto nivel.

3.2 Lenguaje común de definición y manipulación de datos.

3.3 SQL es la forma universal de expresar consultas por lo que es un candidato razonable como lenguaje de consulta.

3.4 La relación entre el servidor y una estación de trabajo remota debe ser usando expresiones de consultas.

La situación actual es la siguiente:

- Hay muchas de las características de los SGBD de 3ra generación que cumplen los SGBDO, pero en otras hay profundos desacuerdos.
- MR sólidos fundamentos teóricos, por lo que resulta casi imposible desecharlo por completo, pero requiere cambios.
- Actualmente en el mercado: potentes SGBDR, más de 20 SGBDOO y SGBDR que incluyen capacidades OO.
- Mayor aceptación como estándar del modelo de ODMG, ya se implementa por los SGBDOO.
- Fuerte actividad experimental, sobre todo en los inicios.

Los Sistemas de Gestión de Base de Datos de Objetos (SGBDO) surgen de la convergencia entre la tecnología de las BD y el paradigma de la OO, con el fin de atender a requisitos para los cuales los productos relacionales no dan una respuesta satisfactoria. Un SGBDO integra las capacidades de una BD (persistencia, transacciones, control de concurrencia, recuperación, mecanismos de consulta, integridad, seguridad, ...) con las capacidades de los lenguajes de programación OO (LPOO) (tipos abstractos de datos, herencia, identidad de objetos, comunicación a través de mensajes, encapsulamiento, ...).

A partir de las limitaciones de ambos modelos surgió un nuevo “modelo” que Michael Stronebraker nombra “objeto/relacional, que toma del modelo relacional su seguridad, integridad y fiabilidad) y del modelo de objetos su capacidad para almacenar datos complejos.

Dentro de los SGBDO se destacan los siguientes productos:

1ra generación: Vbase (1er producto, es de 1986), GemStone

2da generación: Poet, IDB Objectivity

3ra generación: Orion, Poet 3.0

Aunque en sus inicios proliferaban los productos, en la actualidad aparecen nuevas versiones no nuevos productos fundamentalmente.

Los gestores objeto/relacional más utilizados son:

Illustra	Agosto 1993
UNISQL	1992
Universal Informix + Illustra	1996
Oracle a partir de la versión 8.8	1999

A excepción de UNISQL que parte de un gestor de objetos (ORION), el resto parte de una idea relacional. El producto que mejor cumple las características de este modelo es el Oracle, que aunque almacena en una estructura de tablas, da una visión de objetos y permite la herencia y manejar métodos desde el lenguaje de consultas.

Diseñar la BD a partir de un MOO no puede hacerse aplicando los métodos tradicionales porque no se trabaja con los mismos conceptos, por lo tanto, se requieren nuevos métodos y herramientas de desarrollo. Además, los métodos y herramientas utilizados para el MOR presentan limitaciones ya que se refieren a los datos y sus relacionales, sobre los que no tienen en cuentas las restricciones a ellos asociados.

La coexistencia entre los modelos relacional y de objetos en los gestores de BD presentes en el mercado hacen que el diseño de la BD esté condicionado en parte por el medio de almacenamiento que se seleccione.

Conceptos básicos

La persistencia es la capacidad de un objeto para existir fuera de un programa, proceso, función o hilo de control; de manera que se conserva su estado y su comportamiento. Para garantizar la persistencia los SGBDR disponen de cuatro operaciones: creación, recuperación, actualización (se refiere a la modificación) y eliminación (CRAE). Un modelo es un conjunto de reglas, conceptos y convenciones que permiten describir “algo”. Cuando se le añade el término de “persistente”, se refiere a los procesos definidos para modelar los aspectos persistentes de una aplicación orientada a objetos.

Según Jesse Liberty, todo modelo o mecanismo de persistencia debe cumplir las siguientes reglas:

- Cada elemento de un objeto (atributo, asociación, jerarquía) tiene que ser proyectado en un elemento de almacenamiento persistente.
- El código fuente de una aplicación incluye las operaciones CRAE. Este código tiene que ser escrito y probado para un mecanismo persistente en particular.
- El formato de almacenamiento persistente tiene que ser mantenido a la par con la definición de los objetos que es necesario almacenar. Durante el desarrollo de un proceso, los objetos pueden cambiar (sus atributos, relaciones, etcétera.) y la definición persistente tiene que corresponderse con estos cambios.

Un diseño de la BD a partir de un análisis OO implicará tomar la decisión de la forma en que se almacenarán los datos, lo que implica en estos momentos decidir por la persistencia a un fichero, a un SGBDO o en BDR con o sin capacidades de la orientación a objetos. La transformación de los objetos tiene que tomar en cuenta que un objeto no solo engloba propiedades sino que también incluye comportamiento.

El análisis de cualquier problema requiere de un estudio global en el que se identifique no solo la información a manipular, sino también restricciones sobre esa información, relaciones que se establecen y procedimientos a ejecutarse que actúan sobre ella.

En el MOO cuando se habla de vista o perspectiva estática se refiere a los conceptos de clase, atributo, relaciones, restricciones de integridad estática, herencia y agregación; por lo que es más rica que el MR. Como vista o perspectiva dinámica se incluyen las definiciones de mensaje, evento, estado, transición, petición o envío de mensaje y restricciones de integridad.

Los objetos vistos desde una perspectiva estática muestran el conjunto de propiedades (atributos) que describen estructuralmente al objeto, de manera que los valores asociados a cada propiedad del objeto caracterizan el estado en un instante.

Las restricciones estáticas buscan hacer imposible la inserción inapropiada de elementos. Por ejemplo, para el caso de que sea un atributo de tipo numérico se puede asociar una regla de negocio de acuerdo su significado (el presupuesto de una empresa no puede ser mayor que la suma del presupuesto de las áreas y sólo puede tomar valores en el intervalo $[0, 1000000]$). Es decir, estas restricciones están fuertemente relacionadas con el dominio de los atributos de los objetos.

Cooling plantea que los objetos en la modelación dinámica responden a cuatro preguntas: ¿qué interacciones tienen lugar entre los objetos?, ¿por qué ocurren?, ¿cuándo ocurren? y ¿qué efectos generan en los objetos?.

A cada clase se le pueden asociar fórmulas de la lógica dinámica:

- Evaluaciones: Caracteriza explícitamente parte del estado de un objeto antes y después de la ocurrencia de una determinada acción.

$\phi \rightarrow [a]\phi$ Produce cambios de estado.

- Derivaciones: Son fórmulas de la forma $\phi \rightarrow \phi'$ que permiten definir atributos derivados (ϕ) en términos de una condición de derivación declarada en ϕ .

$[a] (\phi \rightarrow \phi')$ Expresa como se obtiene el valor de un atributo derivado. Se debe satisfacer en cada estado del objeto. No produce cambio de estado.

- Precondiciones: Prohibiciones para la ocurrencia de acciones.

$\neg \phi \rightarrow [a] \text{false}$ Si $\neg \phi$ se satisface, entonces la ocurrencia de a está prohibida, es decir, ϕ es una fórmula que tiene que ser válida para que pueda ejecutarse la acción.

- Disparo: Se disparan automáticamente, no como consecuencia directa de acciones de los usuarios.

$\phi [\neg \phi] \text{false}$ La acción se activa cuando la condición que plantea la fórmula ϕ se satisface.

- Restricciones de integridad: Son fórmulas que deben ser satisfechas por lo que se evalúan en el modelo cuando se produce un evento que provoca un cambio de estado. Se clasifican en estáticas y dinámicas. Las estáticas siempre son aplicables, en cambio las dinámicas dependen del estado actual.

Donde ϕ , ϕ' y ϕ son fórmulas bien formadas de la lógica de predicado de primer orden y $a \in A$, A es el conjunto de acciones.

El uso de disparadores es muy útil en el mantenimiento de restricciones de integridad (cuando no es posible con las restricciones declarativas definidas en el momento de crear un elemento), en la auditoría de la información (registrando los cambios realizados y la identidad de que los llevó a cabo), y en el aviso automático de se debe ejecutar una acción como consecuencia de un cambio en un elemento.

El término restricciones dinámicas se refiere a la exigencia de la unicidad en el valor que toma(n) el (los) atributo(s) que se defina(n) como llave, en el caso de existir este concepto, o a las implicaciones que puede tener la inserción o eliminación de un elemento cuando hay jerarquía de clases o se trabaja con objetos complejos.

Las restricciones dinámicas también se refieren a las condiciones que deben cumplirse para disparar una operación o que deben asegurarse antes o después de una operación para que ocurra correctamente (precondiciones y postcondiciones).

Para captar esta semántica, las metodologías OO incluyen varios diagramas. En el caso de la perspectiva estática por lo general tiene nombres cercanos a los de “diagrama de clases” o “modelo de objeto”. Para la perspectiva dinámica son usados los diagramas de transición de estado y otros que recogen la interacción entre los objetos, de ellos se podrían obtener varias de las fórmulas (por ejemplo, los disparadores y precondiciones del diagrama de transición de estado) pero no se explotan suficientemente en el proceso de diseño de la BD.

En cuanto al tratamiento de la persistencia en las metodologías de análisis y diseño desde que apareció la 1ra metodología (1989 Coad Yourdon) se han seguido fundamentalmente tres alternativas:

- Eludir el tema, algunos incluso reconocen que no lo abordan (método Fusión).
- La definición de las BD se concreta en decidir cuáles clases se desean sean persistentes y utilizar los que brinda el lenguaje para salvar y recuperar valores.
- Reglas para convertir clases a tablas.

Tomando en cuenta que:

- el diseño de la BD en la tecnología objeto, no fue una actividad en la que se hicieron grandes esfuerzos en los inicios del desarrollo de metodologías,
- a pesar de que la situación no ha cambiado mucho, han ido en incremento las aplicaciones que manipulan información compleja, por lo tanto, hay que definir cómo diseñar para los productos presentes en el mercado,
- Tendencia más explotada es conversión al MR,
- la perspectiva dinámica casi nunca es diseñada, y que
- detrás del desarrollo de los SGBD, irán apareciendo instrumentos y técnicas que permitan diseñar explotando sus potencialidades.

, para enfrentar la situación actual, si queremos analizar y diseñar pensando en objetos, pero existe esta situación en el mercado hay que definir un modelo de persistencia que lo soporte.

Pasos para el diseño de la BD

1. Definir las clases persistentes.
2. Clasificar atributos y relaciones.
3. Realizar el diagrama de clases persistentes.
4. Obtener las restricciones estáticas y las fórmulas dinámicas.
5. Convertir las clases al medio de almacenamiento.

1- Definir las clases persistentes

Todas las clases identificadas en el dominio del análisis no son persistentes. La persistencia es la capacidad de un objeto de mantener su valor en el espacio y en el tiempo. Las clases temporales son manejadas y almacenadas por el sistema en tiempo de ejecución por lo que dejan de existir cuando termina el programa. Es responsabilidad del

diseñador definir cuáles clases son las que deben ser persistentes. En este proceso se recomienda aplicar algunas reglas, ellas son:

1. Cuando una clase que está formada por otras clases es persistente, automáticamente las clases componentes también son persistentes. Lo contrario no se cumple necesariamente.
2. Cuando una clase hija en una jerarquía de clases es persistente, automáticamente son persistentes sus ancestros en el árbol de jerarquía. Lo contrario no se cumple necesariamente.
3. Cuando se define como persistente a una clase que agrupa a objetos de un mismo tipo de clase base (se refiere a las clases listas, colecciones, registros), entonces automáticamente son persistentes todas las clases hijas a partir de la clase base, incluyendo a la clase base, la clase que agrupa no lo es.
4. Cuando hay herencia múltiple, esta debe ser resuelta antes si el medio de almacenamiento a utilizar no soporta este concepto. Una solución es que la clase hija herede de la clase de la que redefine sus métodos y añada un atributo pasivo del tipo de la(s) otra(s) clase(s) de la que heredaba. Si se redefinía comportamiento de más de una clase padre, hay que escoger de cual se quedaría heredando y añadir un atributo pasivo por cada una de las clases padres de las que no heredará. Los métodos que se redefinen, que ya no se reciben por herencia, en su implementación incluirán las relaciones con las clases padres.

Las clases que coordinan el trabajo de varias clases, por lo que el comportamiento que de ellas interesa es el asociado a la función controladora. Estas clases normalmente no son persistentes porque conceptualmente coordinan procesos no contienen información persistente.

Para el caso de estudio, en particular el paquete Registro de solicitudes, se pueden definir como clases persistentes:

- Solicitud
- Obra pendiente
- Artista
- Especialista
- Técnica

No son persistentes:

- CI_Menú
- CC-Solicitud
- CC_Correo
- CC-Artista
- CC_AsignaEspecialista
- CI_AgregarSolicitud
- arObra
- CI_ModificarSolicitud
- CI-EliminarObra
- CI-Asignación
- CI_EliminarSolicitud
- CI_AgregarObra
- CI_Modific

2 – Clasificar atributos y relaciones.

Los atributos se pueden clasificar en:

- Estático: No cambia de valor en el tiempo por lo tanto no se puede ser utilizado. El único evento que lo afecta es el que provoca la creación de la clase que como consecuencia le da valor.
- Dinámico: Son afectados por otros eventos que son los que hacen que cambie de valor.
- Derivado: Cambian cuando se modifican otros atributos. Estos otros atributos integran la fórmula de derivación y pueden pertenecer o no a la clase a la que pertenece el atributo derivado.

Para el caso de la clase Artista podrían definirse como estáticos el nombre y el sexo y como dinámicos la dirección, edad y e_mail.

Dentro de los atributos dinámicos se pueden identificar tres categorías:

- Tipo de dato numérico que incrementa o decrementa su valor en una cantidad o partir de un valor valor actual. Ejemplo: Edad.
- Tipo que puede tomar determinados valores predefinidos y que pasa de una valor a otro ante determinado evento o cuando se cumplen determinadas condiciones. Ejemplo: Estado civil.

Valor actual	Evento	Nuevo valor
-	Nacer	Soltero
Casado	Divorciar	Divorciado
Casado	Enviudar	Viudo
Not Casado	Casar	Casado

- Atributos cuyo nuevo valor no guarda relación con el anterior, por lo que hay que tener claro en algoritmo que le da nuevo valor o si este es definido por el usuario a través de una interfaz con el sistema. Ejemplo: e_mail

Las relaciones de agregación entre las clases pueden clasificarse en:

- Estática: Cuando se crea un objeto de la clase compuesta, el objeto de la clase componente se mantiene sin cambiar su valor, independientemente de los eventos que afectan la compuesta. Ejemplo: Relación de la clase Solicitud con la clase Artista.
- Dinámica: Producto de eventos que afectan a la compuesta, puede cambiar el objeto componente asociado a la compuesta. Ejemplo: Relación de la clase Solicitud con la clase Obra pendiente.

3- Realizar el diagrama de clases persistentes.

Es un diagrama de clases en el que solo aparecen las clases persistentes, de las que hay que expandir detalles estructurales en cuanto a los atributos especificando tipo de dato, valor inicial, rango de valores, si tienen un valor común para todas las instancias de la clase en el momento de creación y otras restricciones de dominio.

Además, hay que buscar patrones comunes que complique el diseño físico para darles solución. Estamos haciendo referencia a las asociaciones cíclicas, relaciones n-arias, de m:n y de 1:1 y la herencia múltiple.

Aunque el concepto de llave no existe en el MOO porque allí se trabaja con el concepto de identificador de objeto que no está ligado a los atributos; como la variante de medio de almacenamiento más utilizada es la conversión hacia una BD relacional u objeto/relacional, se debe definir para las clases entidad el conjunto mínimo de atributos que la identifican, es decir, las llaves primarias (Primary Key).

Rational Rose no obliga a especificar la llave, si no se hace la generación automática adicionando un atributo numérico a la tabla en el momento de la generación de la BD.

Para especificar que atributo es la llave usando el Rational Rose hay que poner el foco en ese atributo en el browse, dar clic derecho y selecciona DataModeler/Part of Object Identity.

Para las clases persistentes definidas con anterioridad se obtendría un adagrama como el de la figura 20.

4- Obtener restricciones estáticas y fórmulas dinámicas.

Existen otras condiciones que deben satisfacer los atributos, vinculadas al dominio, más restrictivas que las ya identificadas, por lo que hay que especificar reglas que indiquen qué valores pueden tomar, o lo que es igual, qué condiciones deben cumplir los atributos. Un formato para expresarlas podría ser: <atributo> = <condición>.

En la condición hay que especificar las clases involucradas y se pueden usar NOT, EXIST, funciones de agregación, AND, OR, IN, BETWEEN. Usando la simbología de notas de UML se pueden incluir en el diagrama de clases.

En la figura 21 se muestra una nota que restringe el valor del atributo número de pasaporte de un turista indicando que pueden existir dos números de pasaportes iguales, pero provenientes de diferentes países.

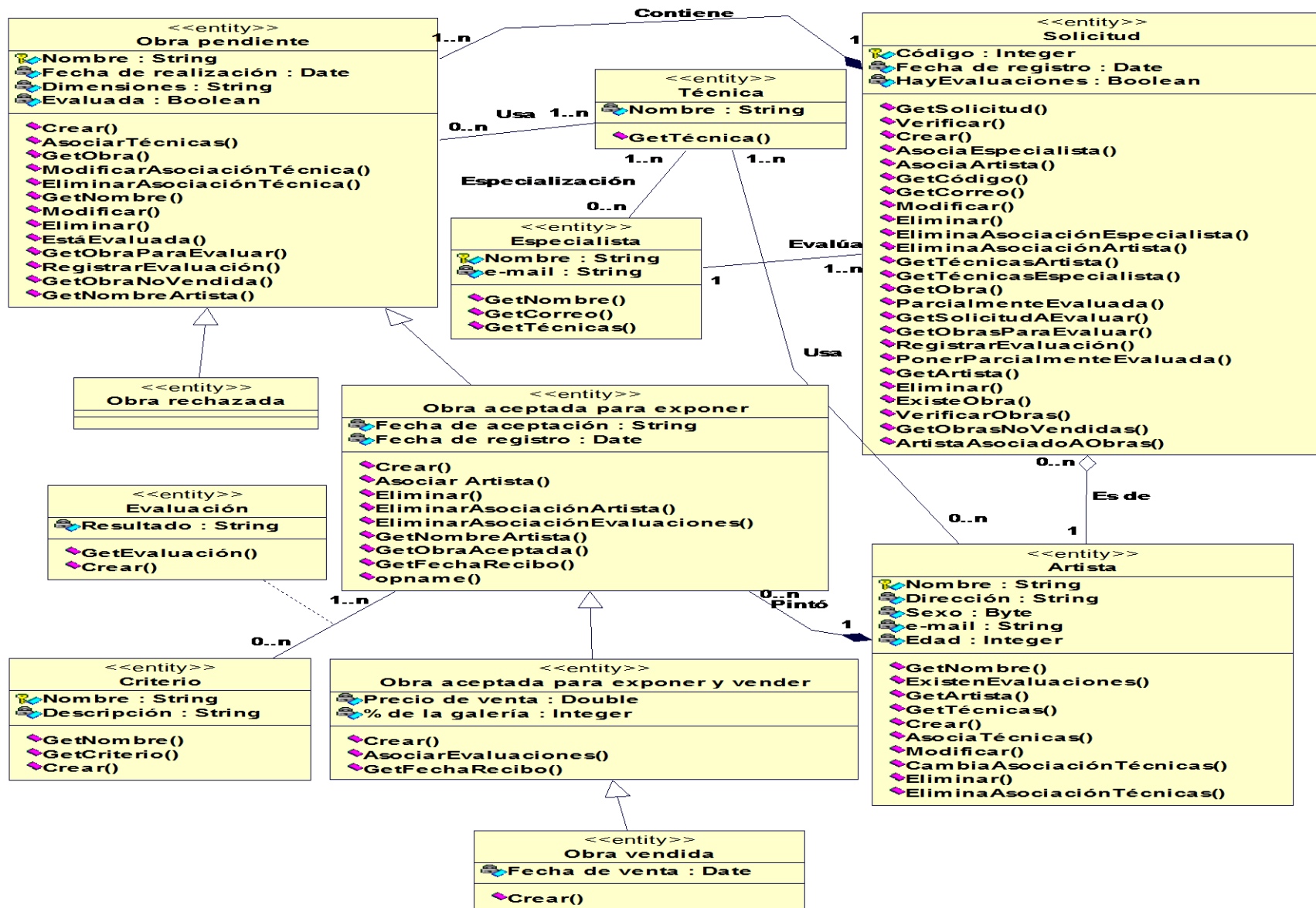
No_Pasaporte = NOT EXIST In Turista
(Turista.No_Pasaporte=NuevoTurista.No_Pasaporte) AND
(Turista.País=NuevoTurista.País)

Figura 21 Ejemplo de restricción de dominio.

Otra característica de un atributo que puede ser implementada de esta forma es cuando el atributo es de tipo enumerativo. Este tipo de dato aparece por 1ra vez en el estándar SQL3 ó SQL:1999, por lo que aún algunos gestores no lo incluyen. Por ejemplo, Estado civil = "Soltero" OR "Casado" OR "Viudo" OR "Divorciado".

Las restricciones se validan para cualquier atributo independientemente de su clasificación, cada vez que se intente cambiar su valor. Su implementación depende de las potencialidades del gestor relacionado (puede ser, por ejemplo, a través de disparadores y procedimientos almacenados o definiéndolas como métodos de la clase).

Figura 20 Diagrama de clases persistentes vinculadas al paquete Registro de solicitudes.



En el caso de las fórmulas dinámicas:

- **Precondiciones:** Para los atributos dinámicos hay que especificar el evento que ocurre y las condiciones que deben evaluarse para provocar el cambio de estado.

Ejemplo: Atributo: Edad
 Precondición: Es el día de nacimiento
 Condición: -

- **Evaluación:** Definir el algoritmo para dar un nuevo valor a el o los atributos dinámicos.

Ejemplo: Atributo: Estado civil
 Fórmula de evaluación:
 CASE
 Nacer: Estado civil:= "Soltero"
 Divorciar: SI Estado civil="Casado"
 ENTONCES Estado civil:= "Divorciado"
 Enviudar: SI Estado civil = "Casado"
 ENTONCES Estado civil:= "Viudo"
 Casar: Si NOT Estado civil="Casado"
 ENTONCES Estado civil:= "Casado"
 ENDCASE

- **Disparadores:** Un disparador es una sentencia que el sistema ejecuta automáticamente como efecto secundario de una modificación a la BD. Un mecanismo de disparo entra a funcionar cuando se adicionan, modifican o eliminan elementos o cuando un atributo dado toma valor determinado o simplemente se modifica.

Para las especificación de los mecanismos de disparo se puede seguir el siguiente formato:

<causa provoca>	que	lo	<clase>:	[condición /<acción>]	ANTES/DESPUES}
↓		↓		↓	
CREAR ELIMINAR MODIFICAR		Nombre de la clase Nombre de la clase Nombre de la clase, atributo y valor		Acciones que se ejecutan como efecto del disparo.	Momento en que se producen las acciones con relación al cambio que provoca el disparo.

En el caso de estudio cuando todas las obras de una solicitud han sido evaluadas, la solicitud se elimina. Supongamos que se definen dos nuevos atributos: Cantidad total de obras y Cantidad de obras ya evaluadas. Cada vez que se evalúa una nueva obra, se incrementa en uno el último contador por lo que cuando ambos valores se igualen, se tiene que eliminar la solicitud.; quedando la especificación:

MODIFICAR Solicitud.Cantidad total de obras = Solicitud.Cantidad de obras ya evaluadas :/ Solicitud. Eliminar {DESPUÉS}

Para especificar los métodos de las clases se pueden usar otras construcciones como las decisiones, selección y repetición, precondiciones/postcondiciones y funciones de agregación (<función de agregación>(<clase o clase.atributo>)[[Condición]]).

Las cardinalidades mínimas representan restricciones de integridad que han de tenerse en cuenta cuando se programan y que generan actualizaciones en cascada. Por ejemplo, cuando se elimina una solitud automáticamente se deben eliminar las Obras pendientes que contiene, pero no el Artista que la hizo. Cuando se crea una solicitud tienen que tener un Artista y una Obra pendiente asociada.

5- Convertir las clases al medio de almacenamiento

Tomando en cuenta las posibilidades de los productos existentes en el mercado para dar solución al almacenamiento de la información conllevan a que los diseñadores tienen tres variantes de solución:

- Si se tiene un SGBDO, hay que garantizar que las definiciones obtenidas se correspondan con el modelo ODMG. La conversión es directa [30].
- Si lo que se tiene es un LPOO que permite el trabajo con una base de datos relacional (BDR) o un SGBDOR, la solución es llevar las clases definidas a tablas.
- Si lo que se tiene es un LPOO que no permite el trabajo con base de datos relacional, pero que permite salvar registros, entonces se deben utilizar las clases que brinda para almacenar los atributos de las clases persistentes en ficheros. Puede que el lenguaje permita la relación con BDR, pero el diseñador escoge como forma de almacenamiento la organización en ficheros.

La selección depende de las aplicaciones y los datos que maneja.

- Implementación de la persistencia para el almacenamiento en ficheros:

Cuando se utiliza esta forma de almacenamiento, se debe garantizar para las clases persistentes que: se añadan métodos para salvar y recuperar información y se defina el código de estos métodos, de acuerdo al tipo de los atributos.

Almacenar los objetos de esta forma implica que se crea un fichero cuya estructura no es fija pues se almacenan objetos de diferentes clases, se leen y escriben bytes.

La forma en que los lenguajes de programación implementan la persistencia es similar a la forma en que la garantizaban los productos de la 1ra generación de SGBDO. La diferencia principal es que los lenguajes almacenan los atributos de los objetos y los gestores guardan también parte del comportamiento asociado a las clases.

Lo anterior implica que si se desea que sea persistente el comportamiento de los objetos, se puede almacenar su implementación y la definición de la clase, pero es responsabilidad del diseñador definir todo lo necesario para recuperar y salvar esta información. La posibilidad de almacenar la estructura completa de una clase está sustentada en el concepto de metaclasses. Una metaclasses define la estructura y comportamiento de una clase, por lo que en tiempo de ejecución se crean clases. Esto puede hacerse siempre y cuando el lenguaje de programación lo permita.

Supongamos que Ud. Selecciona como lenguaje de programación Object Pascal, en este caso la implementación para la clase Artista quedaría:

```
Type .....
{ Clase: Artista}
Artista = class (TPersistent)
protected
    nombre : string;
```

```

    edad : integer;
    sexo : string;
    ....
public
    ....
    procedure ReadData (Reader:TReader);virtual;
    procedure WriteData (Writer:TWriter);virtual;
end;
    ....
Implementation .....
{-----Artista-----}
    ....
    procedure Artista.ReadData
        (Reader:TReader);virtual;
    begin
        inherited ReadData(Reader);
        with Reader to
        begin
            nombre:=ReadString;
            edad:=ReadInteger;
            sexo:=ReadString;
        end;
    end;

    procedure Artista.WriteData (Writer:TWriter);virtual;
    begin
        inherited WriteData(Writer);
        with Reader to
        begin
            WriteString(nombre);
            WriteInteger(edad);
            WriteString(sexo);
        end;
    end;
Initialization
    ....
    classes.Registerclass(Artista);
    ....
end.

```

- Implementación de la persistencia en una BDO.

El grupo ODMG (Object Database Management Group) surgió 1991 con el objetivo de dotar a los clientes de SGBDO de un conjunto de normas que les permitieran escribir aplicaciones portables, lo que ha dado un impulso en su desarrollo. En 1993 se divulgó ODMG-93 como estándar industrial para el almacenamiento persistente de objetos. En 1997 aparece la versión 2.0 de ODMG que es usada como referencia por gestores como Objectivity/DB 5.2. En enero del 2000 se publicó la versión 3.0.

ODMG está compuesto por un modelo de objetos, un lenguaje de definición de objetos (ODL - Object Definition Language), un lenguaje de consulta (OQL - Object Query

Language y un lenguaje de manipulación de objetos (OML – Object Manipulation Language) para los lenguajes Smaltalk, Java y C++.

El modelo de objetos es la base del estándar de ODMG. Define que un objeto está formado por propiedades y comportamiento; de manera que el estado de un objeto está definido por las operaciones (pueden tener parámetros de entrada y salida y retornar un valor). Establece las relaciones que se pueden dar entre los objetos de herencia, asociación y composición, cómo los objetos pueden ser nombrados e identificados y que tienen un identificador único no asociado al valor de sus atributos. Como conceptos importantes incluye, además, las listas, colecciones, conjuntos y herencia múltiple (esto implica que nos necesario darle solución). El modelo de objetos se define como un esquema de la BD por lo que la conversión de las clases es directa sin necesidad de hacer conversiones.

Para profundizar en las características de ODMG se puede acceder a su sitio Web: <http://www.odmg.org>.

- Implementación de la persistencia a una BDR o BDOR

Cuando se selecciona esta variante de implementación hay que convertir las clases a tablas e implementar los métodos de acuerdo a las potencialidades del gestor. Los atributos de las clases se transforman a una columna de la tabla con el mismo nombre.

Rational Rose permite obtener el diseño de la BD, pero exige que:

- Se cree un nuevo paquete en el que se construye el diagrama de clases persistentes.
- Hay que especificar para cada clase que es persistente (Persistent) modificando esta propiedad porque por defecto cuando se crea son transiente (Transient).
- Las clases están localizadas en el lugar que se crean (se pone entre paréntesis “from...” cuando se referencian en otro lugar), pero se exige que se relocalicen en este paquete para lo cual hay que poner el foco en la clase y seleccionar la opción del menú Edit/Relocate o presionar Control+L.

Se puede definir más de un paquete y en cada uno colocar diferentes clases persistentes, pero hay que generar esquemas diferentes para evitar solapamientos porque Rational Rose no garantiza la calidad de la generación si varios paquetes generan hacía un mismo esquema. Se sugiere definir más de un paquete cuando se van a crear bases de datos diferentes.

Un esquema contiene la descripción del modelo de datos a ser empleado para la recuperación y almacenamiento de datos (Figura 22).



<<Schema>> Galería de arte

a)

<<Schema>>

Galería de arte

b)

Figura 22 Notación de esquema. a) Representación en el browse. b) Representación en el modelo de datos.

En la figura 23 se muestra la representación de una tabla en la que quedan especificados los atributos que representan las llaves primarias y extranjeras, antecedidas por las letras **PK** y **FK**; respectivamente. La combinación **P_FK** se pone delante de atributos llaves provenientes de otras tablas como el caso de la tabla que se genera de las relaciones de m:n que tiene como llave primaria las llaves primarias de las tablas que relaciona. En este ejemplo, se definió que el atributo código era la llave primaria por lo que no se generó automáticamente un atributo llave.

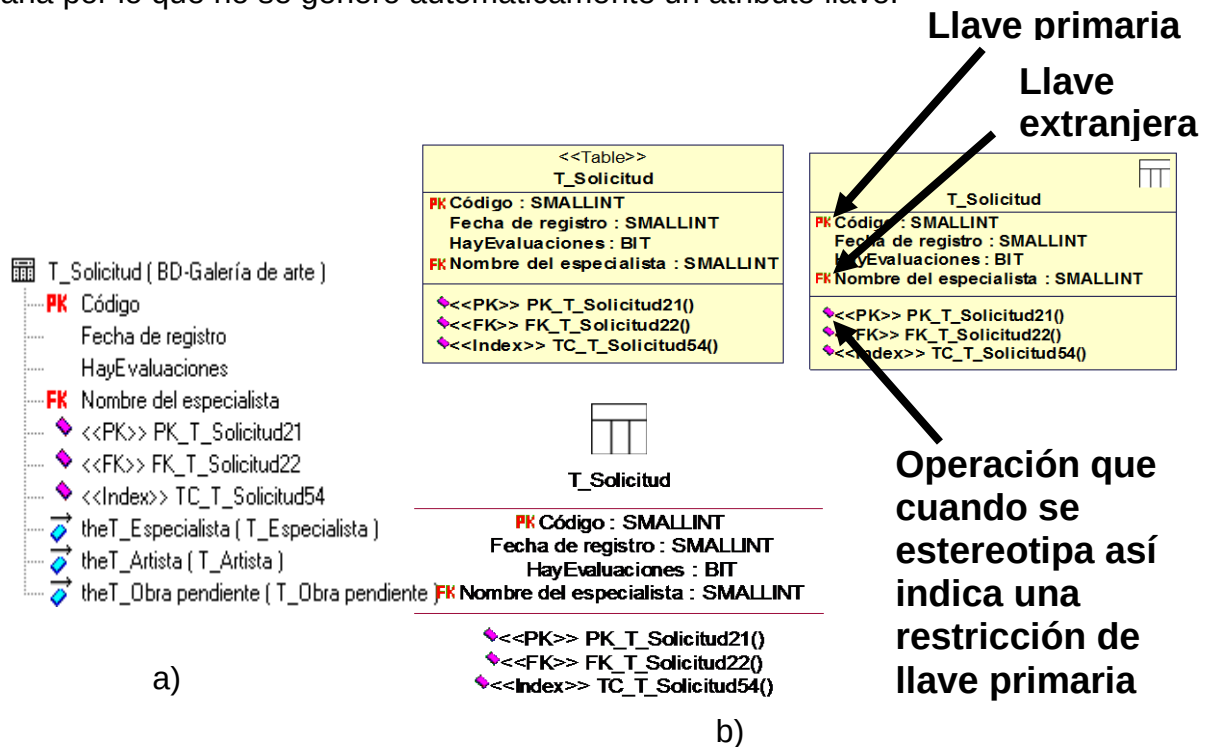
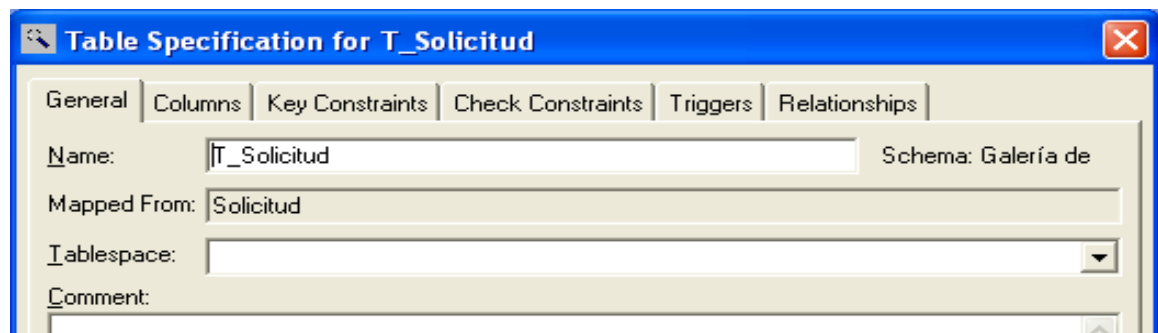


Figura 23 Notación de una tabla. a) Representación en el browse. b) Representación en el modelo de datos.

Una restricción es una regla que se aplica a una columna o toda la tabla. Rational automáticamente genera una restricción para cada atributo llave primaria (PK_T_Solicitud21) y para cada atributo llave extranjera (FK_T_Solicitud22) y otra para que representa un índice asociado a la llave primaria (TC_T_Solicitud54). Además, se puede definir otras restricciones asociadas a, por ejemplo, que un atributo no puede tomar valor único, disparadores y que un atributo toma un valor único; para estos casos se brinda una interfaz que permite realizar estas especificaciones (figura 24).



General Columns Key Constraints Check Constraints Triggers Relationships

Name	PK	Domain	Type	Not Null	Unique	Default
Código	PK		INT	☒	☐	
Fecha de regi...			DATETIME	☒	☐	
HayEvaluacio...			BIT	☒	☐	
Nombre del es...	FK		VARCHAR	☒	☐	

General Columns Key Constraints Check Constraints Triggers Relationships

Name: FK_T_Solicitud38 New... Rename... Delete

Type: Foreign Key Constraint

☐ Unique ☐ Clustered Fill Factor:

Columns: Add > < Remove

Key Columns: Nombre del especialista

Comment:

General Columns Key Constraints Check Constraints Triggers Relationships

☒ Entire Table

Name: New... Rename... Delete

Expression:

Comment:

General Columns Key Constraints Check Constraints Triggers Relationships

Name: New... Rename... Delete

Trigger Event: ☐ Insert ☐ Delete ☐ Update

Action Body:

Comment:

Figura 24 Especificación de restricciones en Rational Rose.

Entre las tablas se identifican relaciones de cualquier tipo entre una tabla padre (extremo 1) y una tabla hija (extremo m). Las relaciones se estereotipan en: Non_Identifying que representa una relación entre dos tablas independientes (figura 25a) e Identifying que representan una relación entre dos tablas dependientes, donde la tabla hija no puede existir sin la tabla padre (figura 25b). Rational estereotipa las relaciones de esta forma cuando genera el esquema asociando las relaciones de herencia, composición y relaciones de m:n como Identifying. Las cardinalidades a las que está haciendo referencia son las máximas.

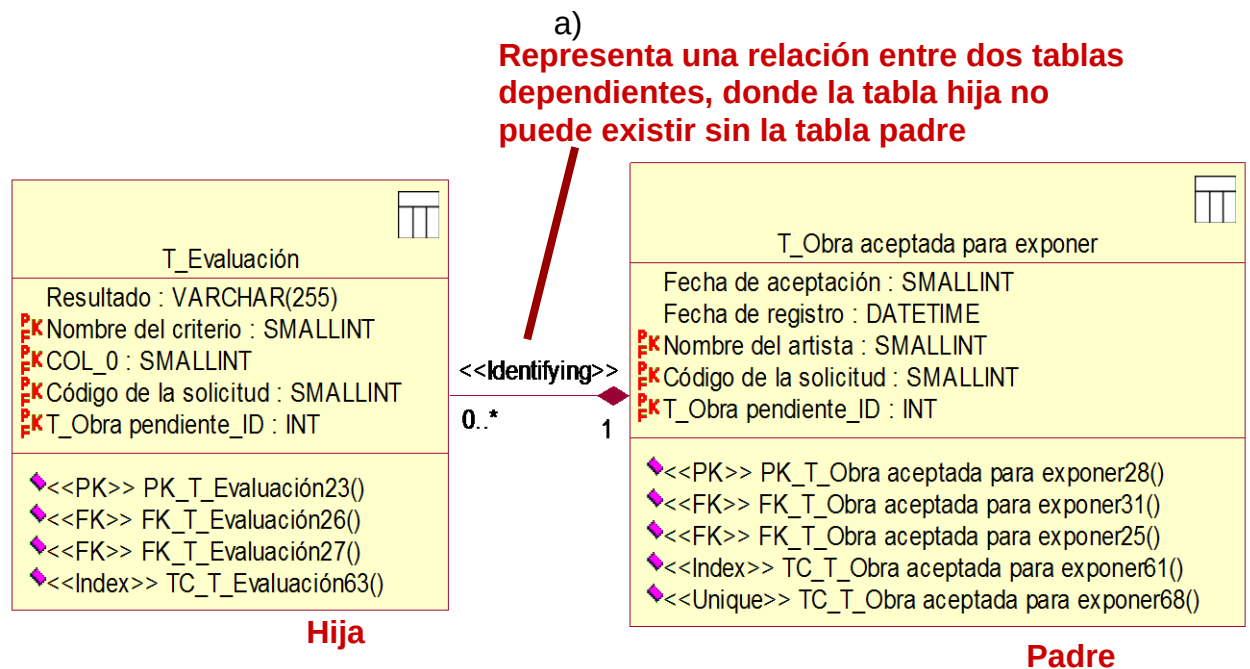
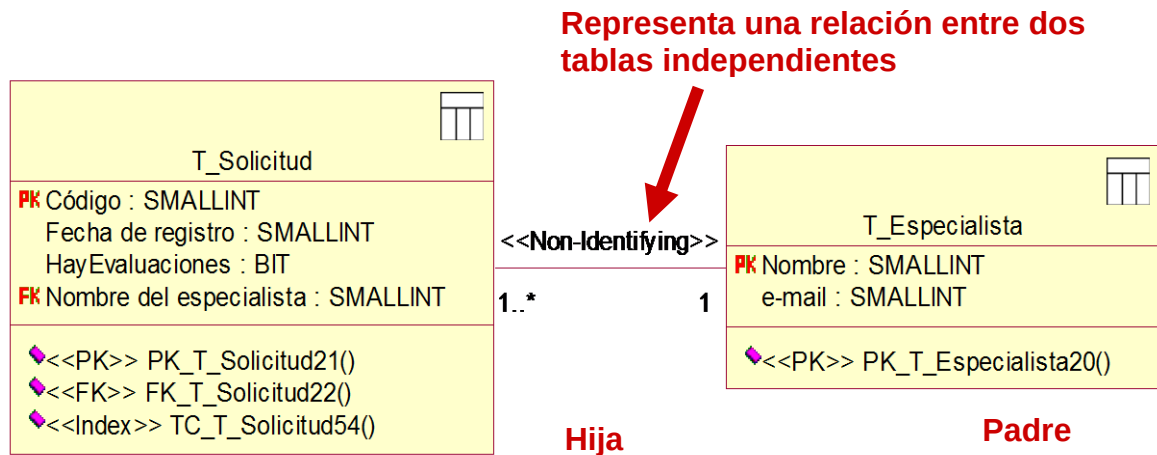


Figura 25 Notación de relaciones. a) Non_Identifying. b) Identifying.

Los procedimientos almacenados son subrutinas que pueden ser invocadas por el trigger o una aplicación. Se pueden modelar solo en el modelo de datos y para crearlas hay que ubicarse en el esquema en el browse y al dar clic derecho seleccionar Data Modeler/New/Stored Procedure.

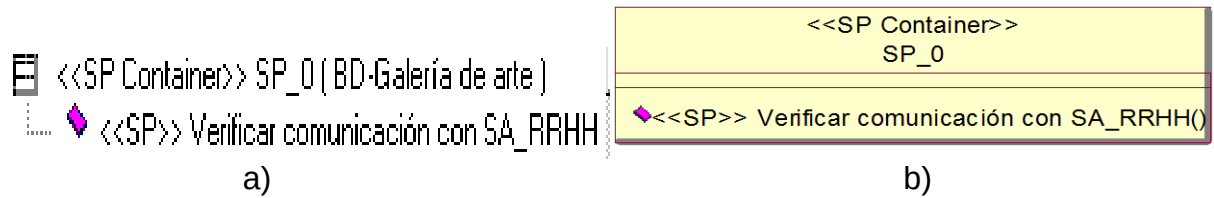


Figura 26 Notación de procedimientos almacenados. a) Representación en el browse. b) Representación en el modelo de datos.

Para convertir las clases a tablas se siguen las siguientes reglas:

a) Cada clase se convierte en una tabla (figura 27).

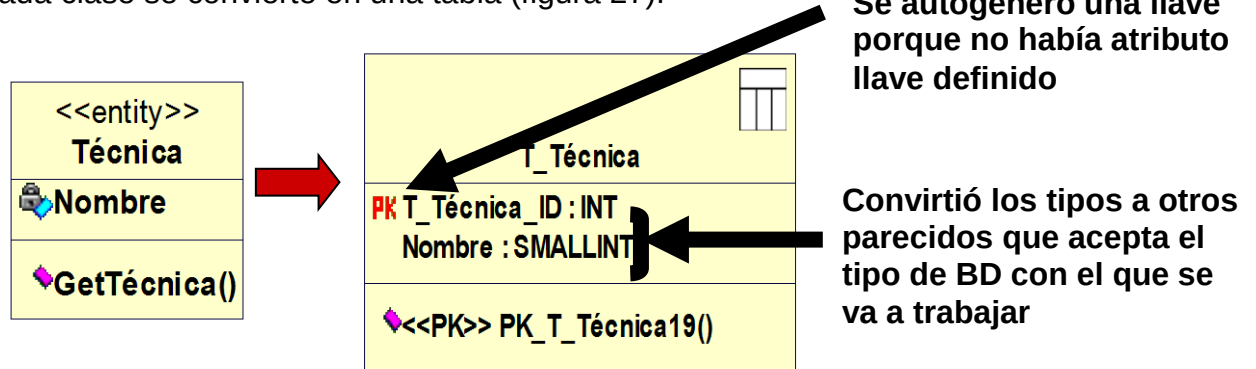


Figura 27 Conversión de clase a tabla.

b) La conversión de la herencia parte de que antes hay que definir cómo conviene implementarla de acuerdo a la necesidad de un rápido acceso a la información vs la garantía de que se chequeen las restricciones de integridad. En Rational cada subclase se transforma en una tabla y se crea una relación de 1:m con la tabla que se generó a partir de la superclase (figura 28).

Las variantes que se pueden contemplar son:

- Una tabla con todos los atributos del padre y de los hijos para lo cual hay que linealizar la jerarquía de clases transformando la jerarquía a una única clase con todos los atributos y métodos y se adiciona un nuevo atributo para identificar ese objeto a qué clase se corresponde.
- Una tabla para cada clase hija que tenga cada una los atributos del padre, para lo cual se eliminan las superclases y se ponen sus atributos y métodos en las subclases. Esta variante es más conveniente cuando las superclases son abstractas.
- Una tabla para cada hija y para cada clase padre por lo que se parte de la jerarquía de clases obtenida.

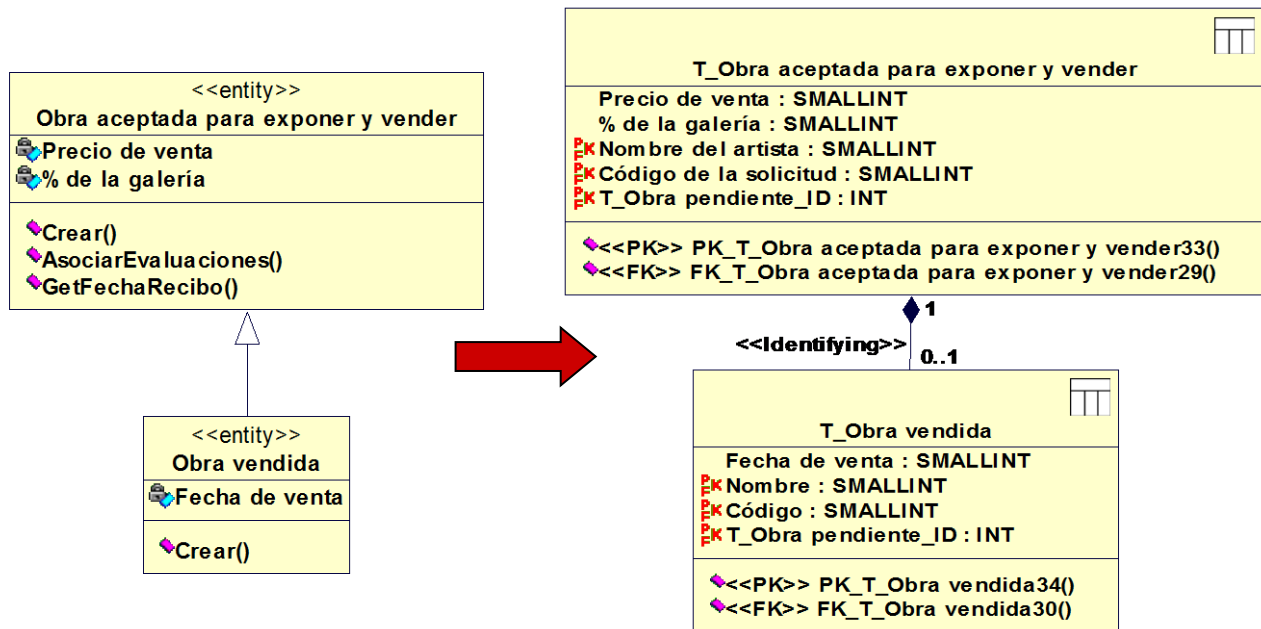


Figura 28 Conversión de la herencia.

c) Convertir la asociación entre dos clases de acuerdo a la cardinalidad.

1:1 Llave de una como atributo de la otra (figura 29). No hay casos en el ejemplo.

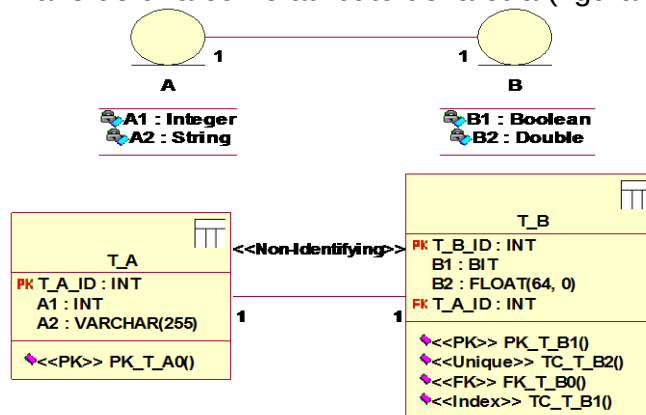


Figura 29 Conversión de la asociación de 1:1.

1:m Llave del extremo 1 como atributo del extremo m (figura 30). En el ejemplo se aplica a la relación entre las clases Especialista y Solicitud.

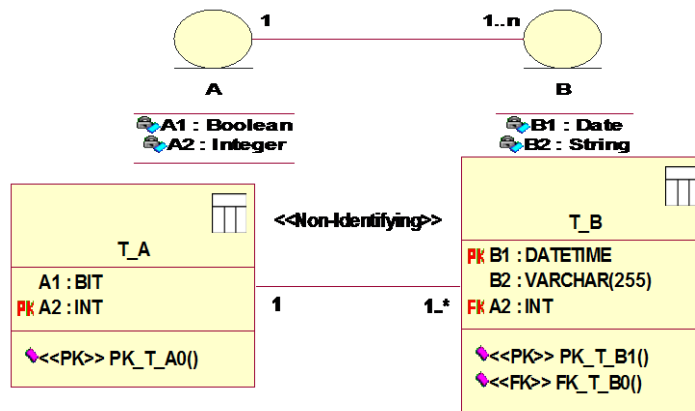


Figura 30 Conversión de la asociación de 1:m.

m:n Tabla nueva que contiene como atributos a las llaves de las tablas que relaciona (figura 31). En el ejemplo se aplica a las relaciones de la clase Técnica con las clases Especialista, Artista y Obra pendiente. En este caso no se tiene un tipo asociativo relacionado con la asociación por lo que la tabla de relación se crea con un nombre autogenerated. Si existiera un tipo asociativo, la tabla toma el nombre del tipo asociativo y, además de las llaves, tendría los atributos de la relación (figura 32). En el ejemplo se aplica a la relación entre las clases Criterio y Obras aceptadas que tiene como tipo asociativo a la Evaluación.

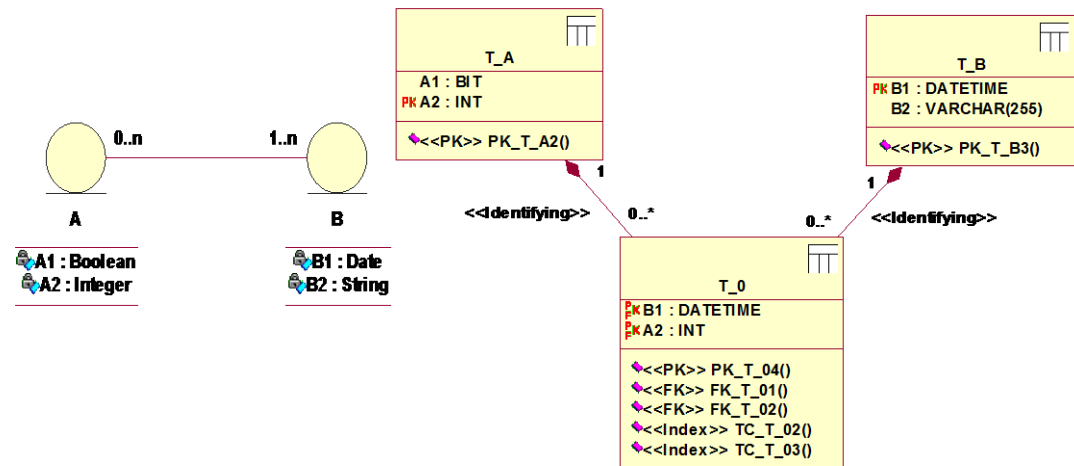


Figura 31 Conversión de la asociación de m:n cuando no hay tipo asociativo.

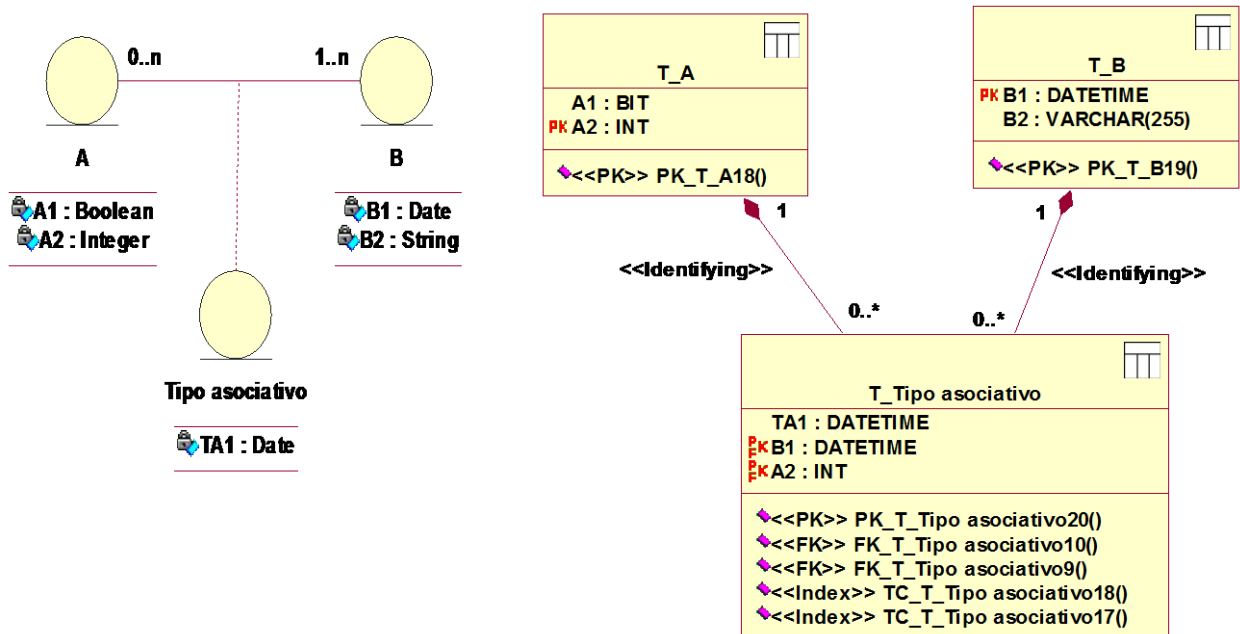


Figura 32 Conversión de la asociación de m:n cuando hay tipo asociativo.

d) Conversión de las agregaciones.

Se define una tabla para cada clase y para el caso de las relaciones de 1:1 y de 1:m, se pone en el extremo de la parte un atributo que es la llave del todo como

llave extranjera (figura 33). Si la relación es una agregación, se etiqueta como Non_Identifying y en el caso de la composición, como Identifying.

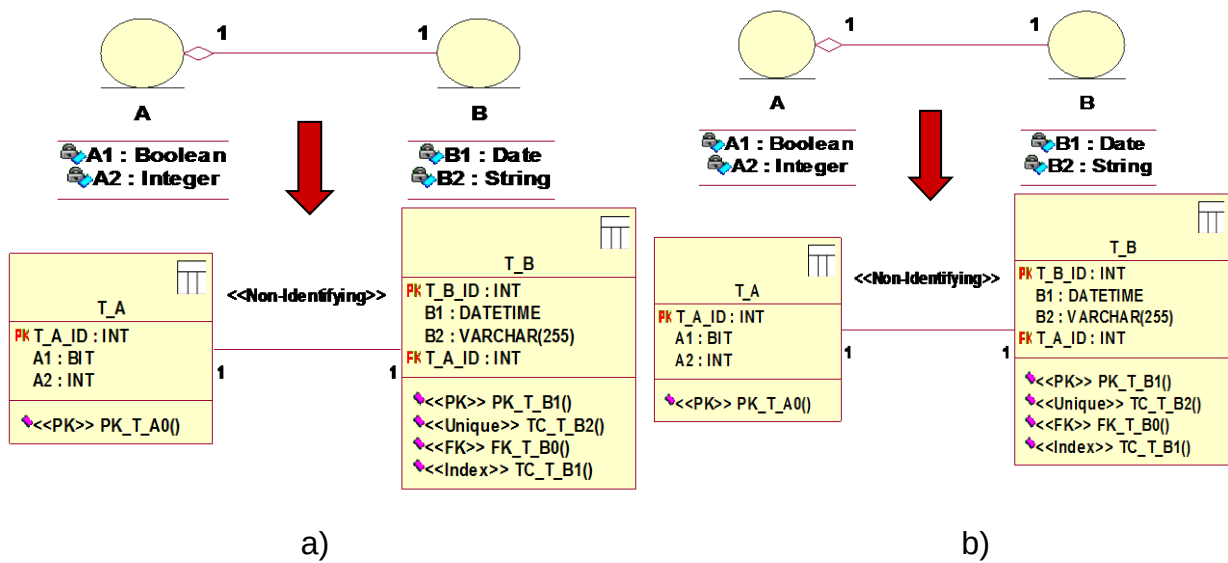


Figura 33 Conversión de las agregaciones con relaciones de 1:1 y 1.m. a) Agregación. b) Composición.

Para el caso de las relaciones de m:n entre el todo y las partes, se transforma igual que las asociaciones de m:n, independientemente de si es una agregación o composición y siempre se etiquetan como Identifying. (figura 34)

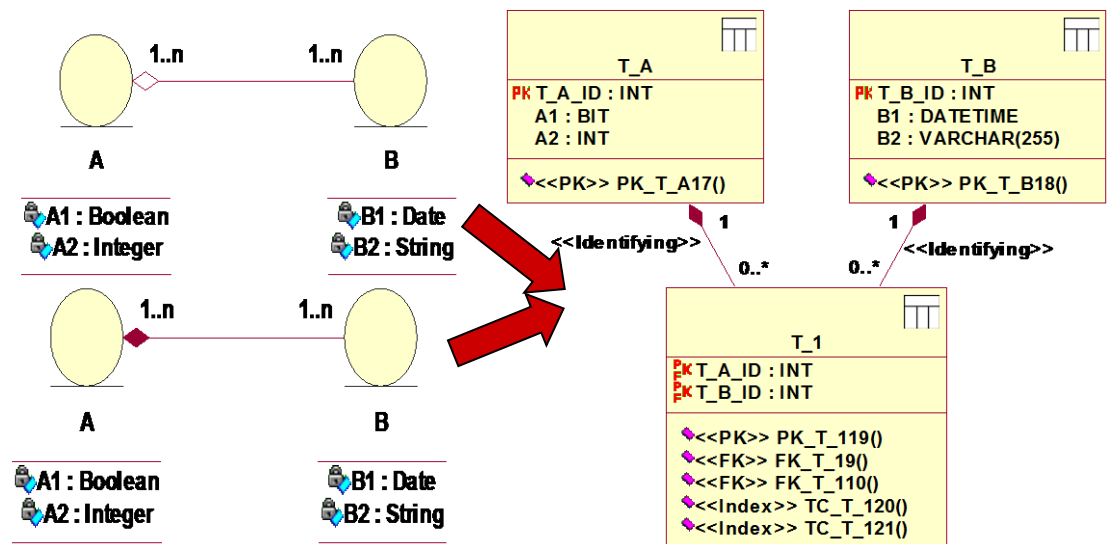


Figura 34 Conversión de agregaciones con relaciones de m:n.

En la figura 35 se presenta el esquema de la base de datos que se genera a partir del diagrama de clases persistentes de la figura 20. Para generarlo hay que poner el foco en el paquete que lo contiene en el browse y al dar clic derecho seleccionar la opción Data Modeler/Transform to datamodel.

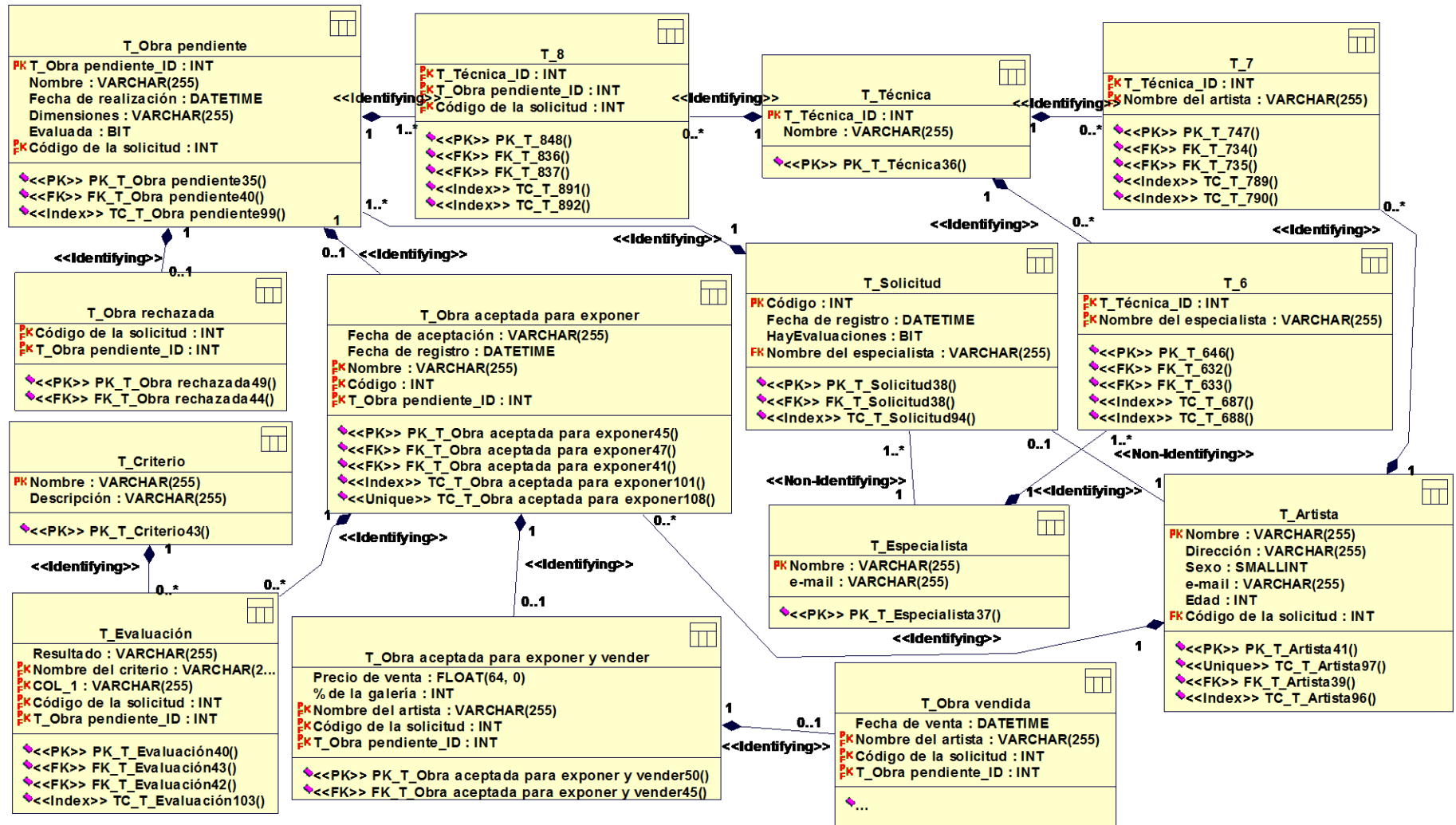
Rational Rose para generar solicita la Base de datos con la cual se va a trabajar, por lo que antes hay que crearla ubicando el foco en el Component View y al dar clic derecho seleccionar Data Modeler/New/Database. Posteriormente se puede

especificar con que gestor se quiere trabajar porque por defecto define para el estándar ANSI SQL – 92, pero se pueden seleccionar gestores como DB2, Oracle, Informix, etc.

Para implementar los métodos hay que tener en cuenta lo siguiente:

- Asociar a las tablas solo los métodos relativos al acceso e integridad de los datos (operaciones de Creación, eliminación, Actualización y Modificación) ya que se implementan con las construcciones del lenguaje.
- Reglas del negocio:
 - Se tiene un LPOO: Crear las clases, pero solo con los métodos asociados a las reglas del negocio.
 - No se dispone de un LPOO: Disparadores, Procedimientos almacenados.

Figura 35 Modelo de datos.



5.5- Conclusiones

Existe un fuerte acoplamiento entre la descripción de los casos de uso y los diagramas de interacción y entre estos últimos y el diagrama de clases obtenido en este punto. Como resultado se obtiene el modelo de diseño como punto de partida de la implementación.

El diseño de la BD a partir de un MOO está condicionado por el medio de implementación. La calidad del diseño de la BD depende de la calidad de las clases definidas. En la obtención de la estructura estática estamos obteniendo algo más que lo que tradicionalmente se obtendría del proceso de normalización. Siempre que se posea un LPOO se debe implementar una capa persistente de clases.

En el caso del diseño de la BD se aplican las palabras de Chris Stone (pertence a OMG):

“Es parecido a estar sentado al borde del agua con la marea baja, rozando para no mojarnos. Podemos, o bien saltar ahora, o bien correr en dirección opuesta, o bien esperar a que la marea nos rodee.”

5.6- Referencias bibliográficas

- JACOBSON, Ivar; RUMBAUGH, James; BOOCH, Grady, “El proceso unificado de desarrollo”.2000. Addison Wesley. capítulos 9 Páginas 205-254.
- RUMBAUGH, James, JACOBSON, Ivar; BOOCH, Grady, “El lenguaje unificado de modelado. Manual de referencia”.2000. Addison Wesley. Capítulos 8 y 13 Páginas 75-80, 214, 216-218, 175-182 y 330.
- LARMAN, Craig “UML y patrones” 1999, Prentice Hall Iberoamericana. Capítulos 13, 16, 17, 18, 19, 21, 34 y 35.
- Bruegge, B. Y Dutoit, A. “Ingeniería de Software Orientado a Objetos”. 2002. Prentice Hall – Pearson Educación. Capítulos 5 y 6. Páginas 146-149, 167-229.
- GAMMA, E.; HELM, R.; JOHNSON, R. y VLISSIDES, J. “Patrones de diseño”.2000. <http://www.vico.org/pages/PatronsDisseny.html>.
- UML y la Modelación de datos.pdf. Whitepaper de Rational Rose.
- AMBLER, S. “Mapping Object to Relational Databases”. October 21, 2000. <http://www.abyssoftw.com/mappingobject.html/mappingobjects.pdf>.

Capítulo 6 Sistemas en Tiempo Real

Un sistema es un conjunto o combinación de cosas y partes formando un todo complejo o unitario. Los sistemas tienen una frontera que establece los límites con el entorno. Cuando en el entorno se encuentran dispositivos de los que se obtiene información y sobre los que el sistema ejerce control, estamos posiblemente en presencia de un sistema en tiempo real.

En esta clase se aprenderá qué es un Sistema en Tiempo Real y se adecuarán los artefactos de UML a los elementos que caracterizan a estos sistemas. No es objetivo de este curso profundizar en los mecanismos de captación y actuación sobre el ambiente ni en los Sistemas Operativos en Tiempo Real sobre los que usualmente se montan este tipo de aplicaciones.

6.1- Características de los STR

Los sistemas en tiempo real son sistemas que se caracterizan por:

- Estar íntimamente ligados a otros sistemas o al mundo exterior, con los que intercambian datos y señales, y sobre los que realizan funciones de control. En el medio externo hay dispositivos como sensores, equipos, etc. Que brindan información o sobre los cuáles se ejerce una acción en respuesta a un evento que ha ocurrido (por ejemplo, abrir una válvula, apagar un calentador, etc.). Este control y captación de información se ejerce directamente por el sistema sin la intervención de los hombres.
- El programa no suele controlar completamente el flujo de ejecución, sino que éste viene marcado por eventos del entorno. El sistema recibe señales del entorno actúan como activadores de procesos. Estas señales son enviadas por los objetos físicos del medio con los que el sistema se relaciona. Supongamos una línea de ferrocarril que atraviesa una calle, se ha colocado un sensor que capta cuando un tren viene para bajar una taranquela que impida el paso de vehículos y pasajeros (figura1). En este ejemplo, la aplicación en tiempo real está en el mecanismo de subir / bajar la atráquela y el proceso de baja se “dispara” cuando la aplicación recibe una señal del sensor de que viene un tren.

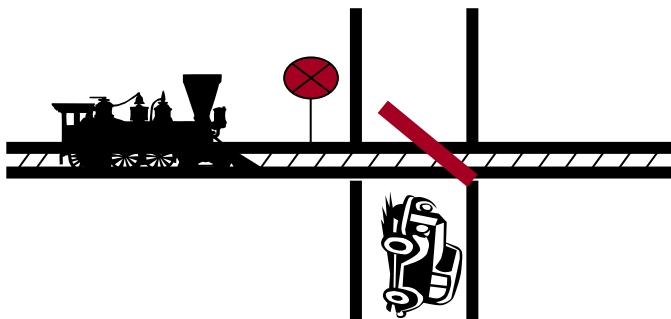


Figura 1 Línea de ferrocarril con taranquela para impedir paso.

- El programa debe realizar ciertas operaciones en un intervalo de tiempo estricto más rápido que los estándares. En estos sistemas es importante el tiempo en el que se produce la salida, no basta con que las acciones sean correctas, también es necesario que se ejecuten dentro del intervalo de tiempo especificado. En el ejemplo anterior, la taranquela no se puede demorar en bajar más tiempo de establecido porque se produce un accidente. En un sistema que controle la producción de determinados productos

químicos, cuando ha pasado el tiempo de calentamiento hay que apagar el calentador porque se pueden producir accidentes o no se logra el resultado deseado.

Para que se ejecute determinada tarea el sistema, tal vez, tendría que contemplar el instante en el que ocurre el evento que genera su ejecución ya que la tarea podría tener una restricción que diga que no puede ejecutarse antes de un tiempo dado o que cuando llegue un determinado tiempo ya debe estar completada.

Estas restricciones de tiempo deben quedar reflejadas en los artefactos que se construyan para que sean tomadas en cuenta cuando se construya la aplicación.

- Requieren procesamiento concurrente de múltiples tareas (procesamiento de dos o más entradas / salidas en el mismo intervalo de tiempo) ya que los elementos externos suelen actuar en paralelo. Supongamos el caso de un sistema de pronóstico del tiempo (Figura 2) en el que constantemente se están recibiendo datos de los sensores de temperatura, velocidad y dirección del viento y presión atmosférica; puede ocurrir que en un instante de tiempo se simultanee la entrada de datos, por lo que el sistema debe prever estos casos.

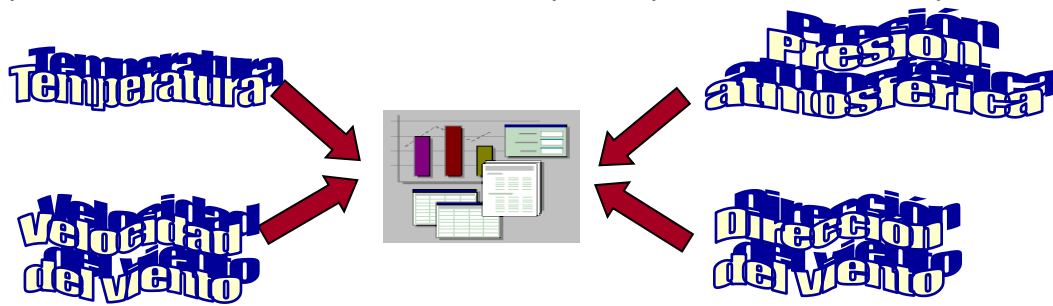


Figura 2 Ejemplo de un sistema de pronóstico del tiempo.

Hay sistemas, como el de pronóstico del tiempo, que se ejecutan en una máquina que tiene conectados, de alguna forma, los dispositivos del mundo exterior sobre los que ejerce control o toma información. Hay otras aplicaciones de tiempo real que están insertadas dentro del dispositivo que controlan. A estas aplicaciones se les conoce como aplicaciones en tiempo real empotradas (figura3) y en ellas se pueden identificar dispositivos que actúan como disparadores de tareas y dispositivos sobre los cuales recae el resultado de la ejecución de la tarea (un dispositivo puede jugar ambos roles). Un ejemplo de este tipo de aplicaciones es el de la taranquila, que tiene empotrado un sistema que controla la subida y bajada de acuerdo a la señal que envían los sensores que detectan que un tren se aproxima y que el tren paso por el paso vehicular.



Figura 3 Sistemas empotrados.

El hecho de que una característica importante de estos sistemas sea su relación con dispositivos externos, no implica que en un sistema en tiempo real no se produzca comunicación entre el programa que se ejecuta y un operador humano.

Un sistema en tiempo real no tiene por qué incluir todas las características anteriores, pero hay que tener cuidado con no confundirlos con sistemas interactivos, cuando no se trata de un sistema en tiempo real empujados. En ambos la respuesta en tiempo es un factor crítico, pero los sistemas en tiempo real trabajan por lo general con intervalos de tiempo mucho más pequeños y tiene una relación muy dependiente con dispositivos externos que actúan como sentidos y, a través de los cuales, el sistema puede modificar el mundo físico.

6.2- Proceso de desarrollo de software

Una aplicación en tiempo real, como cualquier otra aplicación sigue un proceso de desarrollo que parte del modelamiento del negocio hasta la implantación del producto final y el posterior control de cambios que se produzca, pero en su construcción hay que tener en cuenta sus características cuando se construyen los artefactos que permiten su modelación.

En particular vamos a analizar las bases para la identificación de clases, casos de uso, actores y requerimientos funcionales y no funcionales. Además, estudiaremos cómo se modelan dinámicamente a través de los diagramas de secuencia (como un de los tipos de diagramas de interacción, aunque se puede usar el de colaboración) y el de estado.

Todos los ejemplos estarán basados en el funcionamiento de una lavadora automática con las restricciones que se describen a continuación. En este caso se trata de un sistema empujado.

Caso de Estudio: "Funcionamiento de una lavadora"

Se desea desarrollar un sistema en tiempo real empujado en una lavadora automática, que rijan su funcionamiento una vez programada la opción lavar (Figura 4).

El funcionamiento de la lavadora que hay que modelar es el siguiente: Cuando una persona presiona el botón de iniciar el lavado, el sistema tiene que abrir la válvula de control de entrada de agua. La lavadora tiene un sensor de nivel que cuando el nivel de agua llega al punto en el que existe la cantidad de agua necesaria para lavar, envía una señal al sistema que cierra la válvula de control de entrada de agua y enciende el motor. El motor está trabajando durante un tiempo fijo que controla el sistema, cuando este tiempo transcurre, el sistema apaga la lavadora y abre la válvula de control de desagüe. Cuando el sensor de nivel detecta que el tanque de la lavadora está vacío, envía una señal al sistema para que cierre la válvula de control de desagüe y apague la lavadora desactivando el mecanismo de encendido.

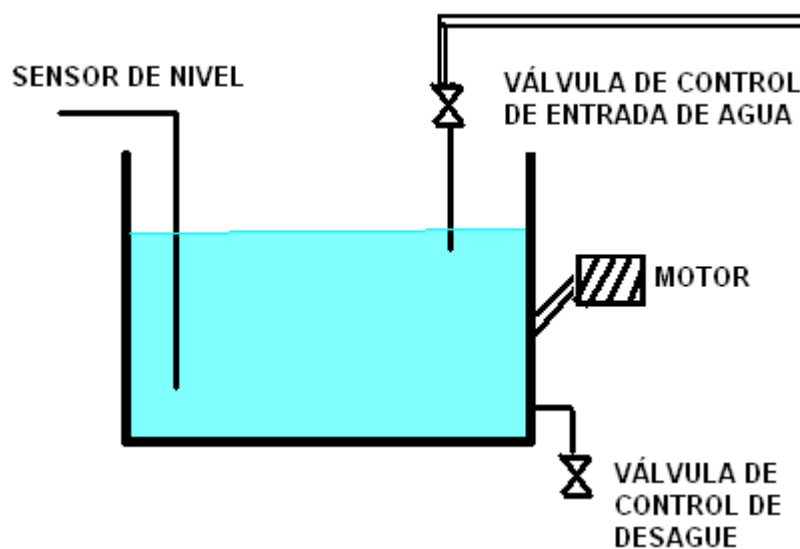


Figura 4 Lavadora automática.

Requerimientos:

Los requerimientos funcionales, vinculados al comportamiento en tiempo real de la aplicación, se obtienen a partir de los eventos ante los cuales se espera una respuesta del sistema. La idea es que se listen estos eventos y para cada uno de ellos se defina qué tiene que hacer el sistema en respuesta.

Para el caso de estudio que hemos tomado como referencia los eventos que se identifican y requerimientos que se obtienen a partir de esto son:

Eventos	Requerimientos funcionales
Una persona presiona el botón de inicio de lavado.	1. Abrir válvula de control de entrada de agua.
El sensor de nivel detecta que el agua llegó al nivel adecuado para lavar.	2. Cerrar válvula de control de entrada de agua. 3. Encender motor
Pasó el tiempo que debe estar lavándose la ropa.	4. Apagar motor. 5. Abrir válvula de control de desagüe.
El sensor de nivel detecta que el tanque está vacío.	6. Cerrar válvula de control de desagüe. 7. Desactivar mecanismo de encendido.

El resto de los requerimientos funcionales se obtienen de forma similar a cualquier otra aplicación que no es de tiempo real.

En el caso de los requerimientos no funcionales, hay tres tipos de requerimientos que usualmente hay que tener en cuenta por el impacto que tiene en la calidad del producto que se obtengan:

- Tiempo.
- Seguridad.
- Hardware y software.

En el caso de software es importante señalar que usualmente estas aplicaciones se ejecutan en sistemas operativos diseñados para aplicaciones en tiempo real y con lenguajes de programación que permiten modelar determinados aspectos que caracterizan a este tipo de aplicaciones. En el caso del hardware hay que tener en cuenta que la aplicación se comunica con dispositivos externos por lo que hay que analizar la interfaz con ellos.

Para el caso de la lavadora, se podrían tener en cuenta limitaciones en el tiempo como las siguientes:

- El tiempo máximo que debe transcurrir entre que se presiona el botón y se abre la válvula de control de entrada de agua no puede ser mayor a 1 segundo.
- El motor tiene que encenderse 5 segundos después de que se cierre la válvula de control de entrada de agua y apagarse 5 segundos antes de abrir la válvula de desagüe.
- El tiempo total entre el cierre de la válvula de control de desagüe y la desactivación del mecanismo de encendido no debe exceder de 1 segundo.

Actores:

Además de los actores del sistema que pueden ser identificados siguiendo los criterios conocidos, es importante recalcar que todos los dispositivos de los que se obtiene información de entrada o disparan la ejecución de una tarea y sobre los cuales se ejerce control, son actores del sistema.

Para el ejemplo, los actores que podrían identificarse son:

Actor	Descripción
Reloj	Monitoreo del tiempo de lavado.
Persona	Indicar que la lavadora tiene que empezar a lavar.
Sensor de nivel	Envía señales al sistema cuando el tanque de la lavadora llega al nivel que se puede lavar y cuando el tanque está vacío.
Válvula de control de entrada de agua	Dispositivo sobre el cual actúa el sistema (abriendo / cerrando) para que permita o no la entrada del agua al tanque de la lavadora, respectivamente.
Válvula de control de desagüe	Dispositivo sobre el cual actúa el sistema (abriendo / cerrando) para que permita o no la salida del agua del tanque de la lavadora, respectivamente.
Motor	Dispositivo sobre el cual actúa el sistema encendiéndolo para que lave y apagado cuando transcurre un tiempo determinado.
Mecanismo de encendido	Dispositivo sobre el cual actúa el sistema apagándolo cuando termine el proceso de lavado.

Fíjense que en este caso los actores son operadores humanos, actuadores o sensores. Ninguno de ellos brinda información y se ejerce control sobre él.

Casos de uso:

Una manera de identificar casos de uso del sistema es identificando escenarios. Los escenarios son instancias e casos de uso que contiene un conjunto de mensajes enviados en

una particular secuencia, por lo que están asociados con un conjunto de requerimientos funcionales.

Una definición más formal de un escenario podría ser: *“Modelan el orden y la dependencia entre una secuencia de mensajes enviados por objetos que colaboran para producir un comportamiento del sistema”*¹.

El analista del sistema debe identificar para cada caso de uso:

- Los roles de los objetos externos y el papel que juega el sistema.
- Flujo (interacciones) necesario para completar el escenario.
- La secuencia de eventos y datos para realizar el escenario.
- Variaciones del escenario que son posibles.

En la figura 5 se muestra el diagrama de casos de uso que resulta de los requerimientos funcionales identificados para el ejemplo de la lavadora automática.

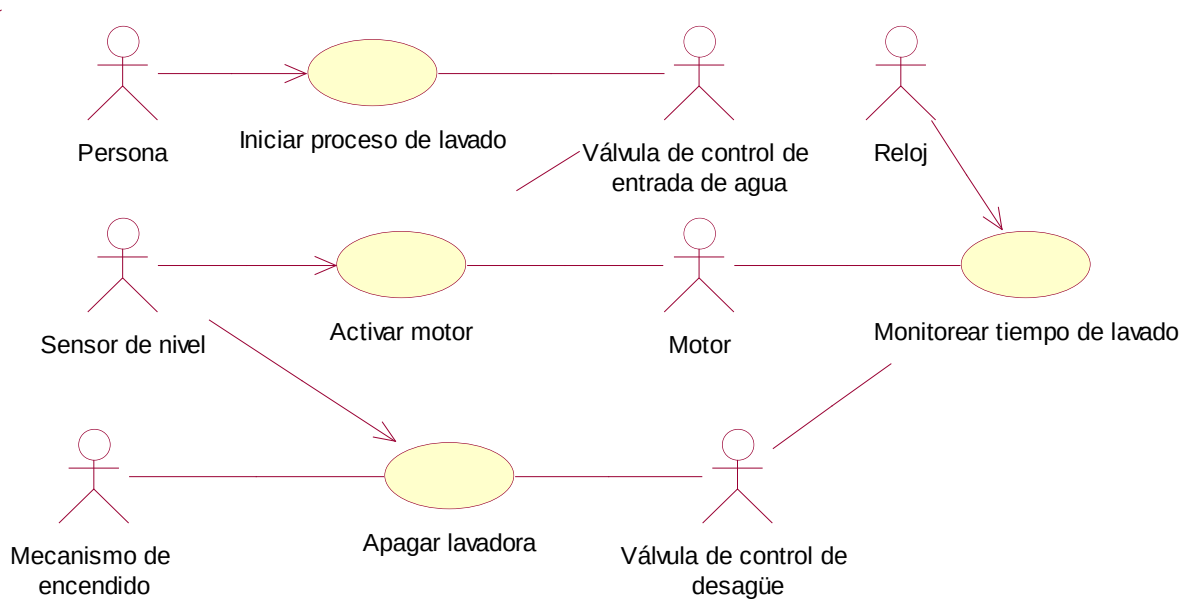


Figura 5 Diagrama de casos de uso para el caso de estudio de la lavadora automática.

La realización de los casos de uso se describe utilizando los artefactos ya conocidos (descripción textual, diagrama de clases y diagrama de interacción), pero para estos tipos de aplicaciones los diagramas de estado tiene una capacidad semántica superior que permite representar mejor la dinámica de interacciones entre los objetos externos y el sistema.

Descripción textual de los casos de uso:

La descripción de los casos de uso sigue el formato conocido con anterior, solo es importante recalcar que en el caso de los requerimientos funcionales hay que puntualizar las restricciones propias que se puedan generar por el tiempo y por razones de seguridad.

Caso de uso: <Nombre>

Actores: <Nombre de los actores>

¹ “Real-Time UML. Developing Efficient Objects for Embedded Systems”. Bruce Powell Douglas. Página 75.

Descripción: <Frases que describan las acciones indicando los actores involucrados, debe quedar claro cómo se inicia y termina el proceso y de que forma intervienen los actores>

Referencias: <Listado de requerimientos y casos de uso asociados, indicando tipo de asociación (include o extend)>

Precondiciones: <Cosas que tienen que cumplirse en el sistema para que se ejecute el CU>

Poscondiciones: <Condiciones en las que queda el sistema cuando termina la ejecución del CU>

Requerimientos especiales: <Precisar de qué manera restricciones de tiempo de respuesta, seguridad, velocidad, disponibilidad, exactitud o uso de memoria afectan al caso de uso>

Para el caso de la lavadora la descripción de los casos de uso es la siguiente:

Caso de uso:	Iniciar proceso de lavado
Actores:	Persona (Inicia), Válvula de control de entrada de agua
Propósito:	Comenzar a llenar de agua la lavadora de agua.
Resumen:	El caso de uso se inicia cuando una Persona presiona el botón para iniciar el lavado, el sistema entonces abre la válvula de control de entrada de agua para llenar de agua el tanque de la lavadora hasta el nivel que requiere de llenado para lavar, finalizando la ejecución del caso de uso.
Referencias:	R1
Precondiciones:	-
Poscondiciones:	El estado de la lavadora pasa a llenando tanque y el de la válvula de control de entrada de agua a abierta.
Requerimientos especiales:	El tiempo máximo que debe transcurrir entre que se presiona el botón y se abre la válvula de control de entrada de agua no puede ser mayor a 1 segundo.

Caso de uso:	Activar motor
Actores:	Sensor de nivel (Inicia), Válvula de control de entrada de agua, Motor
Propósito:	Poner a funcionar el motor de la lavadora para lavar
Resumen:	El caso de uso se inicia cuando el sensor de nivel envía una señal al sistema indicando que el tanque tiene la cantidad de agua necesaria para lavar. El sistema cierra la válvula de control de entrada de agua. El caso de uso finaliza encendiendo el motor para empezar a lavar.
Referencias:	R2, R3
Precondiciones:	El estado de la lavadora es llenando tanque y la válvula de control de entrada de agua abierta.
Poscondiciones:	El estado de la lavadora pasa a lavando, el de la válvula de control de entrada de agua a cerrada y el motor a encendido.

Requerimientos especiales:	El motor tiene que encenderse 5 segundos después de que se cierre la válvula de control de entrada de agua.
----------------------------	---

Caso de uso:	Monitorear tiempo de lavado.
Actores:	Reloj (Inicia), Válvula de control de desagüe, Motor
Propósito:	Culminar el proceso de lavado y comenzar a vaciar de agua la lavadora
Resumen:	El caso de uso se inicia cuando el Reloj detecta que ha transcurrido el tiempo de lavado, entonces el sistema apaga el motor. El caso de uso finaliza cuando el sistema abre la válvula de control de desagüe para comenzar a vaciar de agua la lavadora.
Referencias:	R4, R5
Precondiciones:	El estado de la lavadora es lavando y el del motor encendido.
Poscondiciones:	El estado de la lavadora pasa a vaciando tanque, el del motor a apagado y el de la válvula de control de desagüe a abierta.
Requerimientos especiales:	La válvula de control de desagüe tiene que abrirse 5 segundos después de que se apague el motor.

Caso de uso:	Apagar lavadora
Actores:	Sensor de nivel (Inicia), Válvula de control de desagüe, Mecanismo de encendido
Propósito:	Desactivar el mecanismo de encendido de la lavadora.
Resumen:	El caso de uso se inicia cuando el sensor de nivel detecta que el tanque de la lavadora está vacío y envía una señal al sistema, entonces el sistema cierra la válvula de control de desagüe. El caso de uso finaliza cuando el sistema desactiva el mecanismo de encendido.
Referencias:	R6, r7
Precondiciones:	El estado de la lavadora es vaciando tanque y el de la válvula de control de desagüe abierta.
Poscondiciones:	El estado de la lavadora pasa a apagada.
Requerimientos especiales:	El tiempo total entre el cierre de la válvula de control de desagüe y la desactivación del mecanismo de encendido no debe exceder de 1 segundo.

Diagrama de estado:

Los diagramas de estado se usan para mostrar la historia de la vida de un objeto de una clase o de una instancia de un CU, los eventos que causan una transición de un estado a otro y las acciones que resultan de un cambio de estado.

En el caso de los diagramas de estado que se construyen para los casos de uso permiten precisar el orden en que ocurren los eventos, las condiciones y los efectos que provocan; por lo tanto se recomienda construirlo para los casos de uso que describan procesos de control.

Un diagrama de estado está compuesto por: estado regulares, estados agregados, estados finales, estados iniciales y transiciones.

Los estados regulares contienen restricciones de integridad, que son reglas que especifican restricciones y requerimientos que deben afectar a la estructura y al comportamiento de los objetos y deben ser tratados en los métodos de la clase. Para el caso de los casos de uso representan una de las posibles situaciones en las que puede encontrarse la realización de un caso de uso.



Existen dos estados especiales que son:

- Estado inicial: indica la creación de la realización de un caso de uso o de una instancia de una clase, por lo que en un diagrama existe solo uno. Apunta a un estado regular.



- Estado final: Indica el fin de la vida de una realización de un CU o de un objeto, por lo que se corresponde con su destrucción. Pueden existir varios. Es el estado siguiente a un estado regular



Una transición es una relación entre dos estados que indica que cuando el evento (hecho interno, externo o temporal) ocurra pasa del estado anterior al siguiente.

Las transiciones tienen asociadas acciones (que son operaciones que se realizan instantáneamente), eventos (es la lista de eventos que pueden provocar –o disparar- la transición, relacionados entre sí por los operadores lógicos OR y AND, y pueden estar precedidos del operador lógico NOT) y condiciones (es una lista de condiciones relacionadas entre sí de forma similar a los eventos, que serán evaluadas en el momento en que se produzcan los eventos que dan lugar a la transición y esta sólo será disparada si el resultado de dicha evaluación es verdadero, de lo contrario se esperará a que ocurran nuevamente los eventos). Para especificar una transición se sigue el siguiente formato:

[<clase que genera evento>]:<evento>[[condición]][/acción].

La descomposición en varios niveles del DTE, utilizando los estados agregados, es un tópico poco definido en cuanto a cuáles criterios utilizar para agrupar. Se recomiendan los estudios de George Miller que indican la cantidad de 7 ± 2 elementos por nivel y que lo general esté en los niveles más altos y lo particular en los más bajos. Otros criterios para agrupar podrían ser: un criterio está relacionado con los atributos, de manera que si existen varios eventos que afectan a un mismo atributo ubicando al objeto en estados diferentes, con todos estos estados podría definirse uno agregado, otro criterio sería agrupar a estados que están fuertemente acoplados.

Los diferentes niveles de diagramas deben estar balanceados en cuanto a las transiciones de entrada y salida cumpliéndose las siguientes reglas:

- Para el caso de las transiciones de entrada es necesario que cada uno de los eventos en OR, de la lista de eventos de cada una de las transiciones de entrada al estado agregado, esté formando parte de la lista de eventos de al menos una de las transiciones que parten del estado inicial hacia los subestados.
- Para el caso de las transiciones de salida debe cumplirse: cada subestado tendrá una transición al subestado final y cada una de estas, contendrá entre su lista de eventos las listas de eventos de todas las transiciones de salida del estado agregado relacionadas entre sí por el operador lógico OR.

En la figura 6 se muestra el diagrama resultante para el proceso Activar motor. Fíjese que dentro de las actividades que se ejecutan en un determinado estado queda claro de qué clase es responsabilidad la ejecución de ese comportamiento.

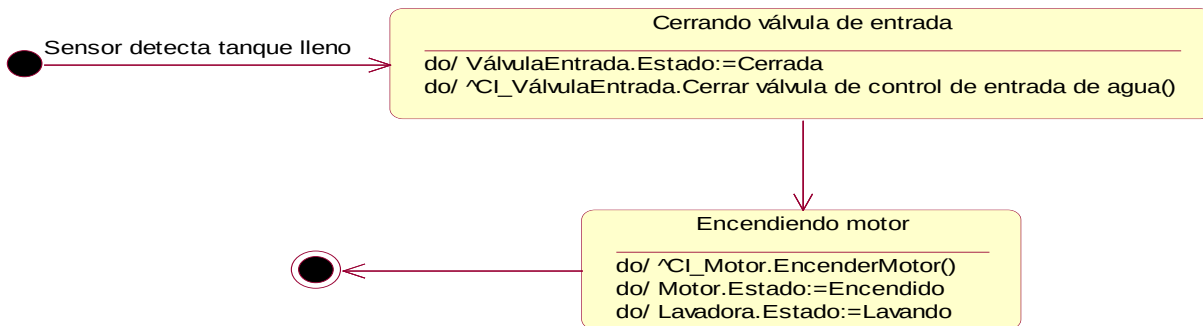


Figura 6 Diagrama de estado del Caso de uso Activar motor para el caso de estudio de la lavadora automática.

Diagrama de clases del análisis:

Gran parte de las clases que se identifican para los sistemas en tiempo real se obtiene de las mismas fuentes que otros tipos de sistemas, dígame:

- sustantivos,
- elementos del mundo real que no son dispositivos electrónicos,
- información persistente,
- elementos visuales,
- transacciones,

pero en estos sistemas hay otras fuentes de obtención de clases en:

- Fuente de acciones, eventos y mensajes, incluyendo la coordinación de acciones (objetos activos).
- Destino de acciones, eventos y mensajes que pasivamente brindan servicios (no inician acciones).
- Dispositivos físicos (sensores y actuadores) que realizan funciones de monitoreo y control.
- Elementos de control del comportamiento del sistema.

En este proceso de identificación juega un rol importante la clasificación de clases que ya hemos visto:

- Entidad: Modelan información que posee una vida larga y que es a menudo persistente, es decir, la información y el comportamiento asociado de algún fenómeno o concepto, como una persona, un objeto del mundo real o un suceso del mundo real (Por ejemplo, Lavadora).
- Control: Coordinan la realización de uno o unos pocos CU, coordinando las actividades de los objetos que implementan la funcionalidad del CU. Para el caso de estos sistemas, se debe definir una clase controladora por proceso de control (por ejemplo, CC_ControlProceso).
- Interfaz: Modelan la interacción entre el sistema y los actores (por ejemplo, CI_SensorNivel y CI_Motor que representa la relación del sistema con el sensor de nivel de agua y el motor de la lavadora, respectivamente).

Entre estas clases de un mismo tipo se establecen las relaciones ya conocidas (agregación / composición, generalización / especialización, asociación), pero entre clases de diferentes tipo sobre todos son asociaciones.

En la figura 6 se presentan en un diagrama de clases las clases identificados en el caso de estudio, aunque su especificación está incompleta pues faltan, por ejemplo, los métodos

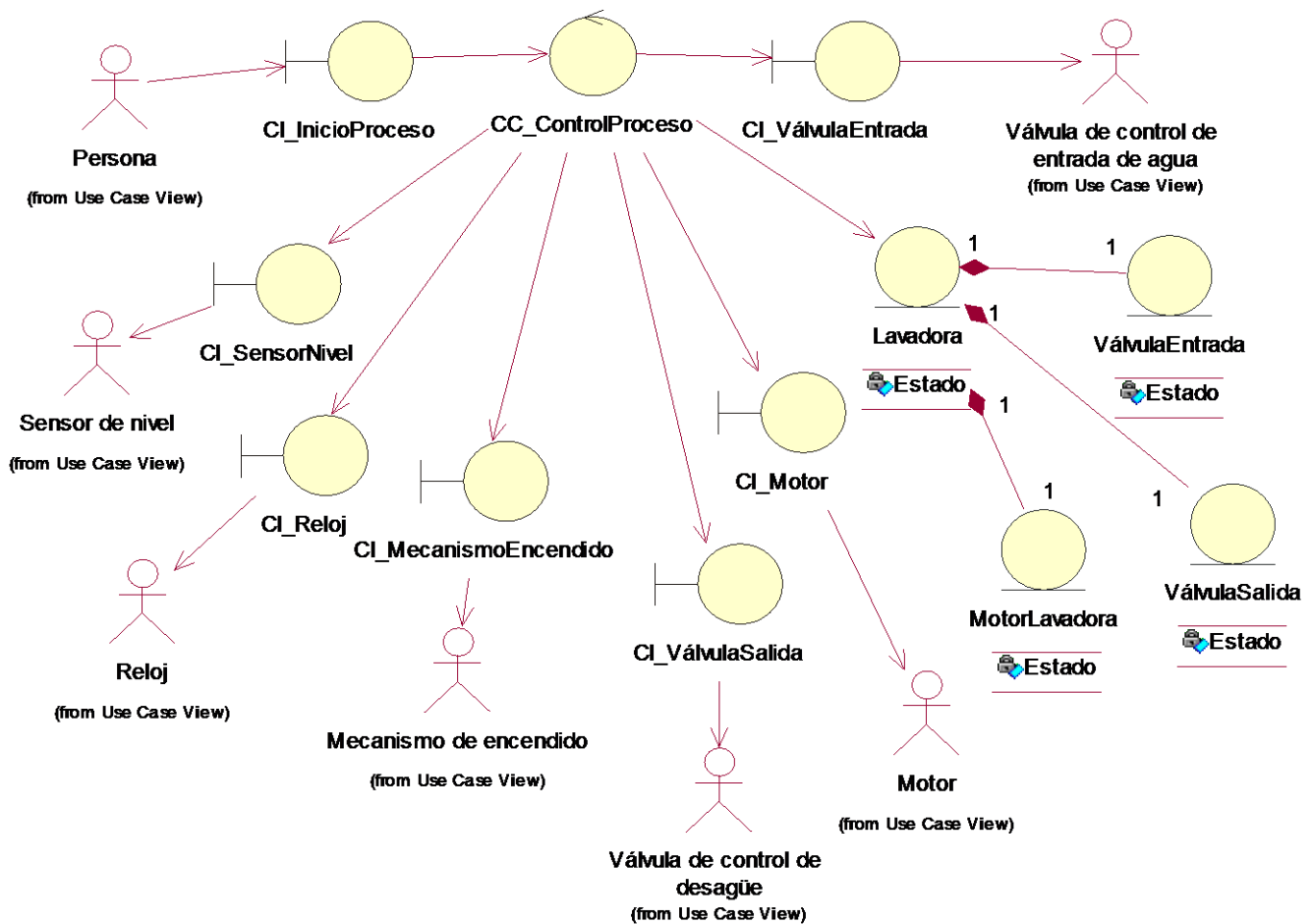


Figura 6 Diagrama de clases para el caso de estudio de la lavadora automática.

En la figura 7 se presenta el diagrama de estado resultante de la clase Lavadora. Fíjense que hay información que puede ser similar a la obtenida por el diagrama de estado del caso de uso, pero la construcción de ambos diagramas ayuda a describir mejor la dinámica del proceso y por lo tanto a identificar con mayor precisión las clases y su comportamiento. No todas las clases tienen que tener un diagrama de estado, para las clases que aporta su construcción es para aquellas que cambian de estado como consecuencia de los eventos externos que afectan a la aplicación.

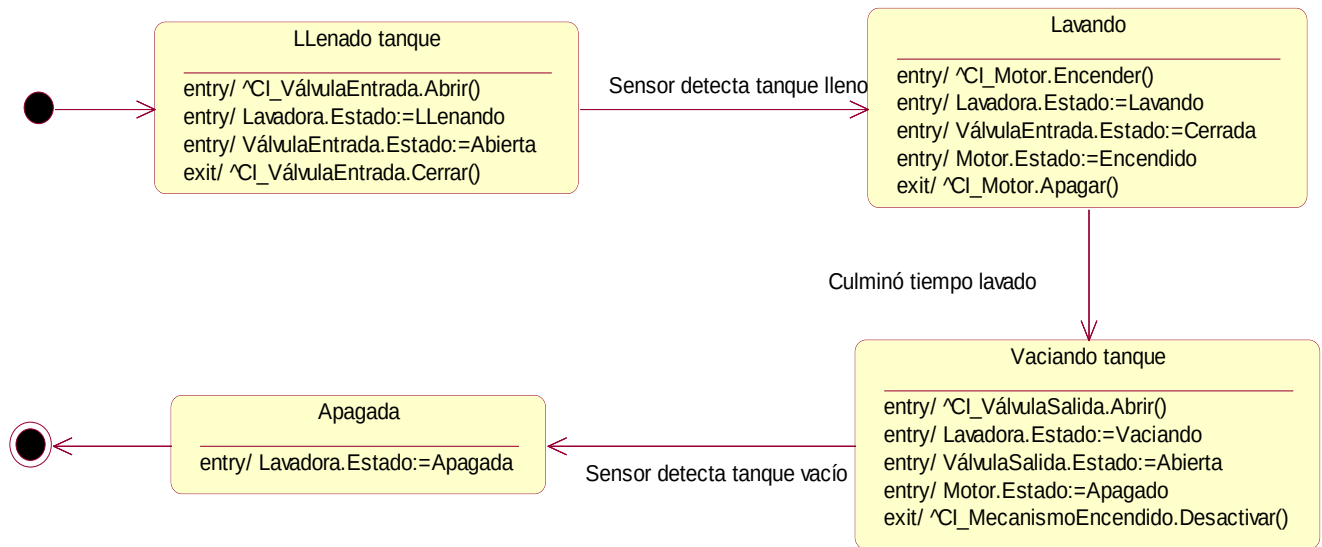


Figura 7 Diagrama de estado de la Clase Lavadora para el caso de estudio de la lavadora automática.

Diagrama de secuencia:

Podría pensarse que si en el diagrama de estado se incluyen las clases y los métodos que se ejecutan cuando se produce una transición o se está en un estado; no es necesario construir un diagrama de interacción para los casos de uso ya que se tiene los métodos y las clases a implementar definidas. Hay que dejar claro que estos diagramas brindan información dinámica sobre la interacción entre los objetos y esta capacidad expresiva no puede obtenerse del diagrama de estado.

Podría pensarse que no hay diferencias con respecto al diagrama de secuencia que se construye para cualquier otro tipo de aplicación, pero dado el rol que juega el tiempo en este tipo de mensajes, se recomienda poner las restricciones que en este sentido existan. Para ello se recomienda que se enumeren los mensajes para hacer referencias a ellos siguiendo este índice.

De esta manera podrían definirse restricciones como las siguientes (5, 6, a y b son referencias a mensajes):

- Tiempo que debe ocurrir entre dos eventos. Ejemplo: {6-5<2.5 segundos}
- Período entre el inicio de uno y el fin de otro. Ejemplo: {Período(ab)=30 segundos}
- Promedio del tiempo de ocurrencia entre dos eventos. Ejemplo: {AVE(ab)<=3 segundos}

En la figura 8 se representa el diagrama de secuencia del proceso Activar motor. Fíjense que el requerimiento especial que aparece en la descripción del caso de uso (El motor tiene

que encenderse 5 segundos después de que se cierre la válvula de control de entrada de agua) ha quedado representada como una nota que indica:

{10-9=5 segundos}

, siendo 10 y 9 los índices de los mensajes: Encender motor() y Cerrar , respectivamente.

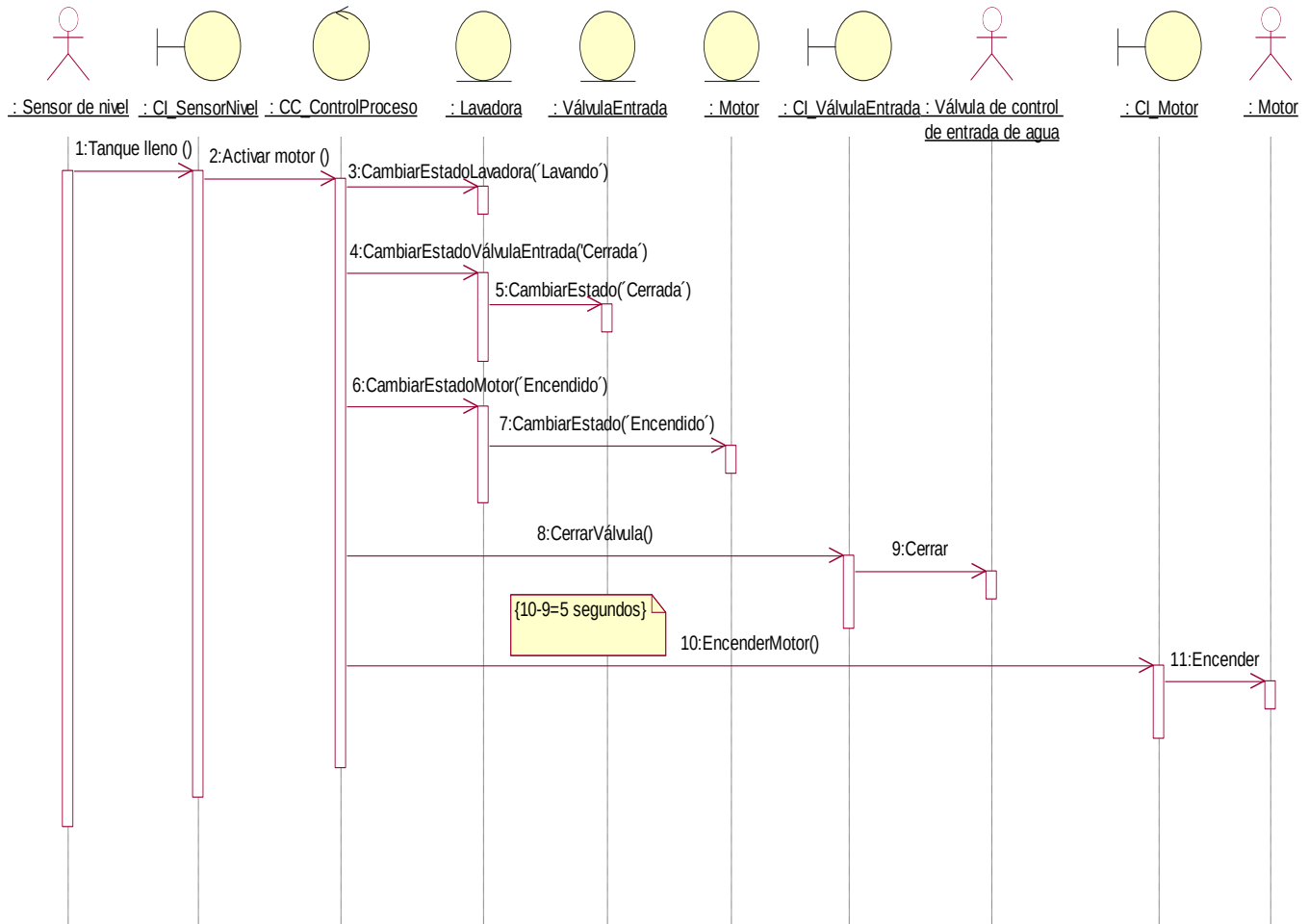


Figura 8 Diagrama de secuencia del Caso de uso Activar motor para el caso de estudio de la lavadora automática.

6.3 Casos de estudio: Semáforo

En la intersección de las calles 100 y 51 (Figura 9) se han colocado 5 semáforos por la necesidad de controlar el tráfico vehicular. Por ser este un punto en el que existe una gran afluencia de vehículos casi todo el día, es difícil determinar cuánto tiempo debe durar la luz hacia cada bocacalle. Se desea realizar un sistema en tiempo real que monitoree constantemente (cada cierta cantidad de tiempo que fija un Inspector de tráfico) los 5 sensores colocados en las sendas por las que llegan vehículos a los semáforos y a que a partir de esta información determine los cambios de luces que permitan disminuir la estancia de los vehículos.

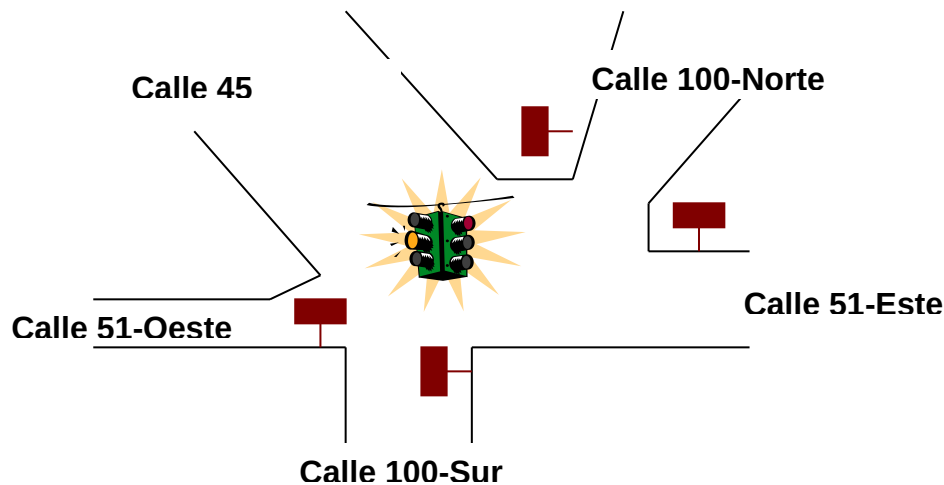


Figura 9 Semáforo de 100 y 51.

El sistema cambiaría las luces de los 5 semáforos (100-Sur, 100-Norte, 51-Este, 51-Oeste, 45). Cada sensor es capaz de determinar la cantidad de objetos diferentes con un peso superior a uno dado que está en los 30 metros siguientes al lugar donde está colocado; y, a partir de un peso promedio, determina cuántos vehículos están esperando por la luz verde. El peso mínimo para detectar un vehículo y el peso promedio para determinar cuántos vehículos hay, son configurados por el Inspector de Tráfico. Por lo tanto, en el sensor hay que hacer un pequeño sistema para configurarlo porque el vendedor da valores fijos y cobra muy caro por el sistema de configuración. El sistema deja de funcionar si se va la luz.

Requerimientos funcionales

Módulo semáforo:

Evento	Requerimiento funcional
Pasó el tiempo definido por el Inspector de tráfico	1. Obtener de sensores cantidad de vehículos en su senda. 2. Determinar cambio de luces. 3. Modificar luces de los semáforos.

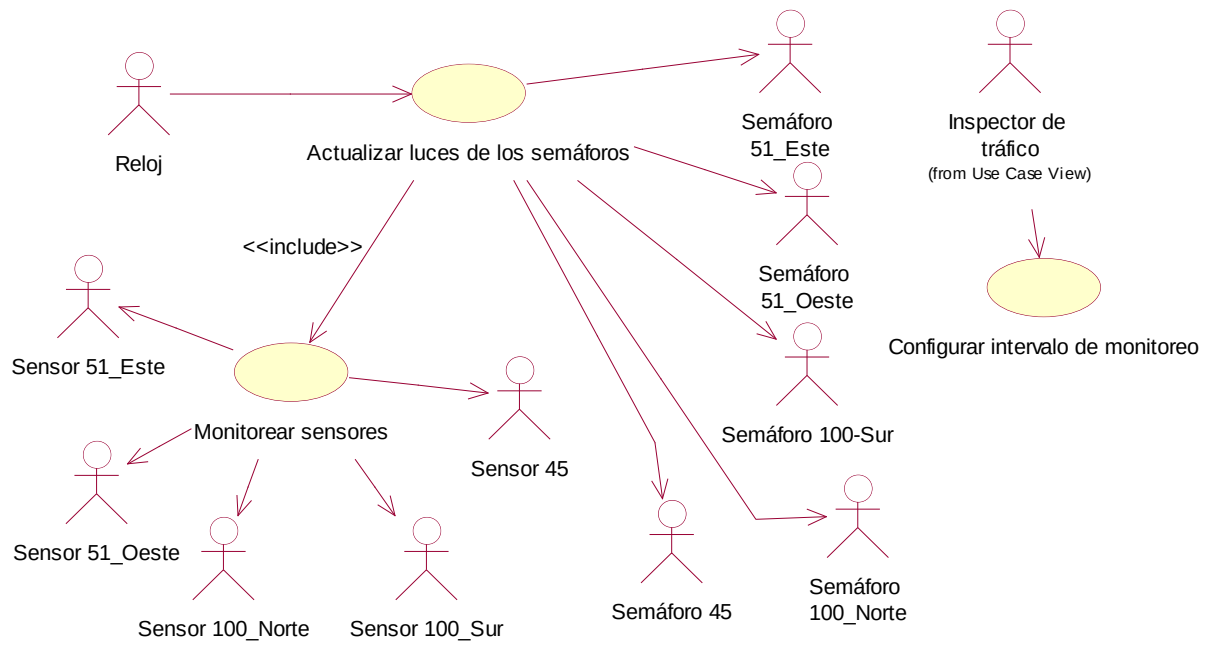
4- Configurar intervalo de tiempo entre monitoreo de monitores.

Módulo del sensor:

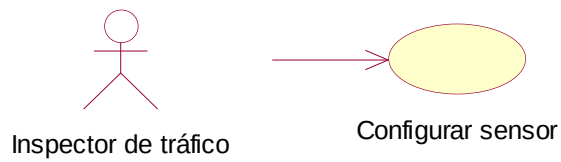
1. Configurar peso mínimo para detectar vehículo.
2. Configurar peso promedio para determinar cantidad de vehículos.

Diagrama de casos de uso (Figura 10)

Paquete Semáforo:



Paquete sensor:



Relación entre paquetes:



Figura 10 Diagrama de casos de uso agrupado en paquetes y dependencias entre ellos.

Diagrama de clases del análisis (Figura 11)

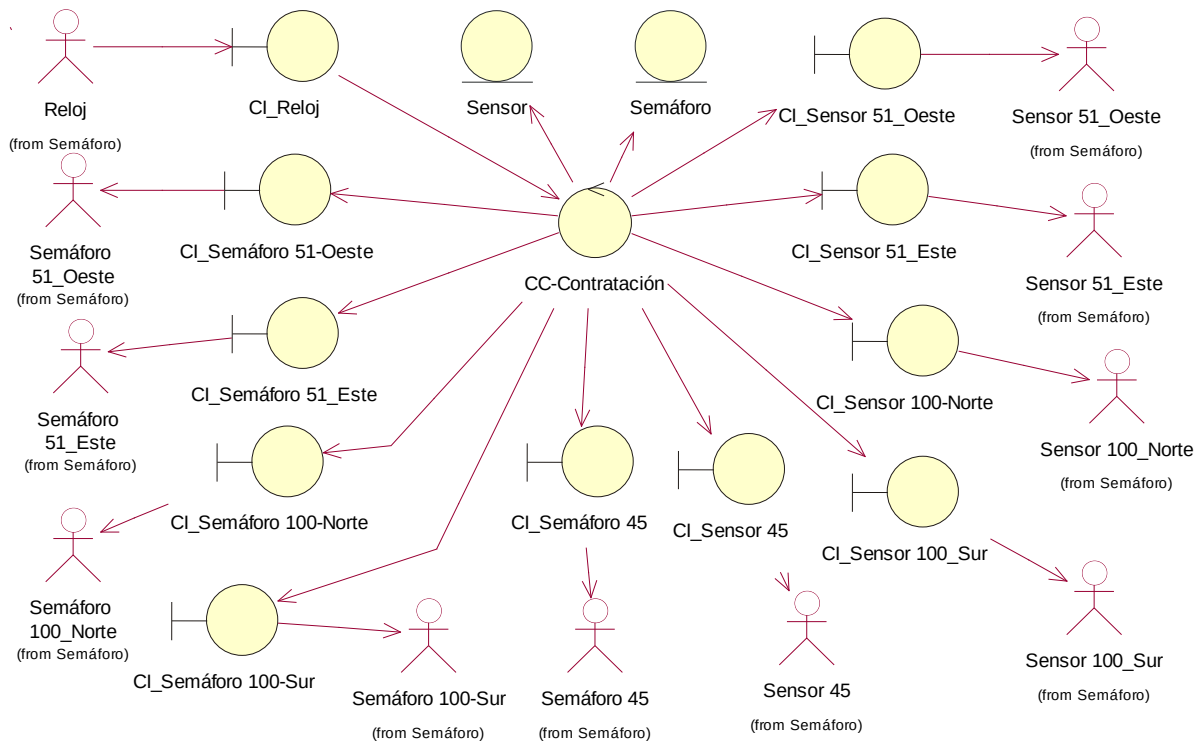


Figura 11 Diagrama de clases del análisis

Diagrama de estado (Figura 13)

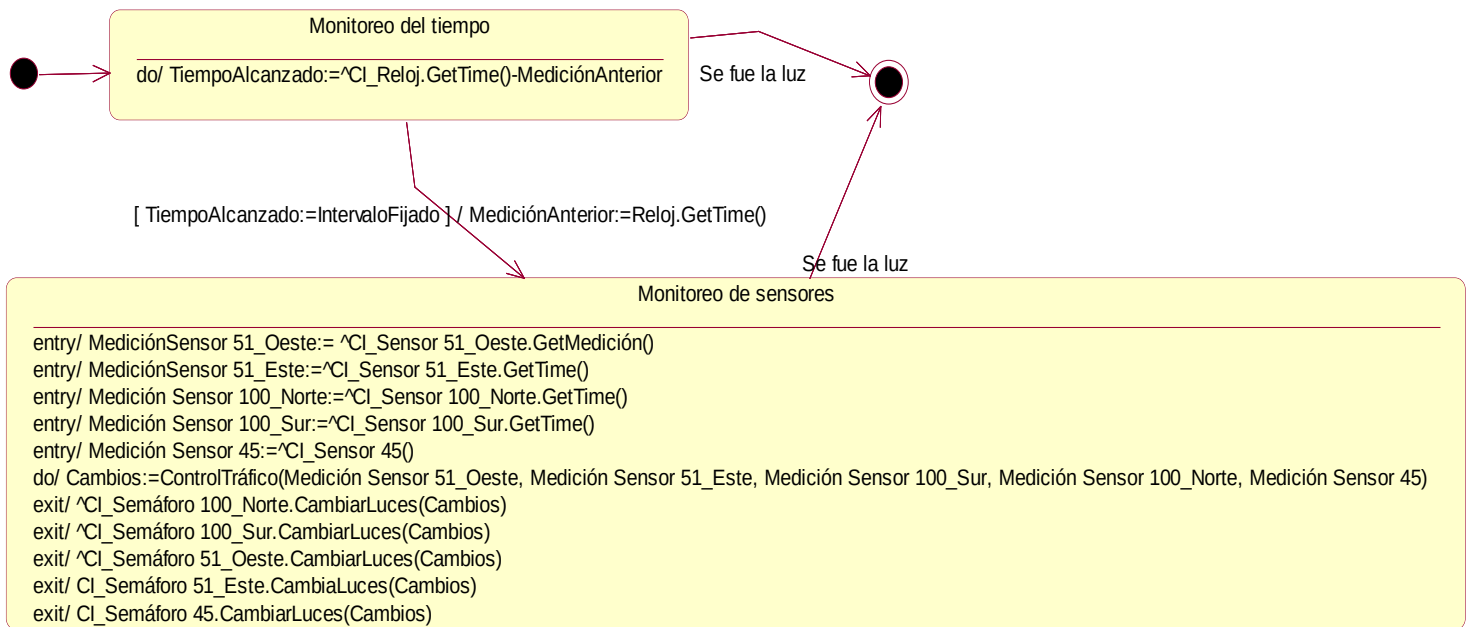


Figura 13 Diagrama de estado que modela la dinámica del proceso de control de tráfico.

6.4 Conclusiones

Lo que distingue a los Sistemas en Tiempo Real de otros es que las acciones de control deberán efectuarse dentro de unos intervalos de tiempo bien definidos ya que la propia dinámica del sistema controlado obliga a que las acciones del sistema de control sucedan antes de que se llegue a situaciones peligrosas.

El proceso de desarrollo que se sigue para su construcción es similar al de cualquier otro sistema, aunque hay algunas particularidades en algunos de los artefactos que se construyen, siendo el diagrama de estado un medio adecuado para describir el comportamiento dinámico del sistema en respuesta a los eventos que ocurren. Por lo tanto, al igual que en cualquier otra aplicación, hay un fuerte acoplamiento entre la descripción del caso de uso, las clases que se identifica que se requieren para su realización y la interacción entre los objetos que se producen.

6.5 Referencias Bibliográficas

- “Real_Time UML. Developing Eficiente Objects for Embedded Systems”. Bruce Powel Douglas. Addison Wesley-Longman. 1998.

Capítulo 7 Modelo de implementación

La arquitectura de un sistema puede representarse a través de 5 vistas:

- Vista de casos de uso: Comprende los casos de uso que describen el comportamiento del sistema tal y como son percibidos por los usuarios finales, los analistas y los que prueban.
- Vista lógica: Clases, interfaces y colaboraciones que forman el vocabulario del problema y su solución.
- Vista de procesos: Hilos y procesos que forman los mecanismos de sincronización y concurrencia del sistema.
- Vista de implementación: Incluye a los componentes y archivos que se utilizan para ensamblar y hacer disponible el sistema físico.
- Vista de despliegue: Contiene los nodos que forman la topología hardware sobre la que se ejecuta el sistema y la distribución de las partes del sistema en ellos.

De todas estas vistas las correspondientes a los casos de uso, lógica (cuando hay hilos concurrentes) y de procesos se obtienen como resultado de los flujos de trabajo vistos anteriormente, pero las vistas de implementación y despliegue son el resultado de los flujos de trabajo de implementación y del diseño, respectivamente.

El flujo de trabajo de diseño se propone crear un plano del modelo de implementación, por lo que sus últimas actividades están vinculadas a la creación del modelo de despliegue. El flujo de trabajo de implementación describe cómo los elementos del modelo del diseño se implementan en términos de componentes y cómo estos se organizan de acuerdo a los nodos específicos en el modelo de despliegue.

Los diagramas de despliegue y componentes conforman lo que se conoce como un modelo de implementación al describir los componentes a construir y su organización y dependencia entre nodos físicos en los que funcionará a aplicación.

7.1 - Flujo de trabajo de Diseño

Cuando se presentaron en clases anteriores a los trabajadores que participaban en el flujo de trabajo de diseño, se comentaba que el Arquitecto era el responsable del diseño de la arquitectura. Para ello no basta con definir las clases del diseño y los subsistemas; hay que estudiarla configuración física de la red en la que funcionará la aplicación por la influencia que tiene en la arquitectura del software.

Además, en el caso de las aplicaciones cliente – servidor las capas que se implementen condicionan o influyen en la determinación de los nodos que se definan en la arquitectura.

Para otros tipos de sistemas es igualmente importante definir cómo se funcionará la aplicación y si tiene relación con otros dispositivos o aplicaciones que funcionen en un nodo diferente al previsto para la aplicación.

Todas estas decisiones arquitectónicas quedan plasmadas en el diagrama de despliegue.

7.1.1 - Implementación de aplicaciones en capas

La implementación de una aplicación en capas se basa en el envío de mensaje y representa una estructura modular que mejora la usabilidad, flexibilidad, interoperabilidad y la escalabilidad.

Independientemente de las capas que se implementen, la esencia de esta forma de construir aplicaciones es definir un cliente que solicita servicios y un servidor como proveedor de servicios.

Un modelo cliente-servidor simple define dos capas: capa cliente (ambiente de trabajo del usuario por lo que tiene la interfaz de la aplicación) y capa servidora (contiene la base de datos). El procesamiento se divide entre estos dos ambientes por lo que son usados en exceso los procedimientos almacenados y los disparadores para implementar la lógica del negocio (figura1a).

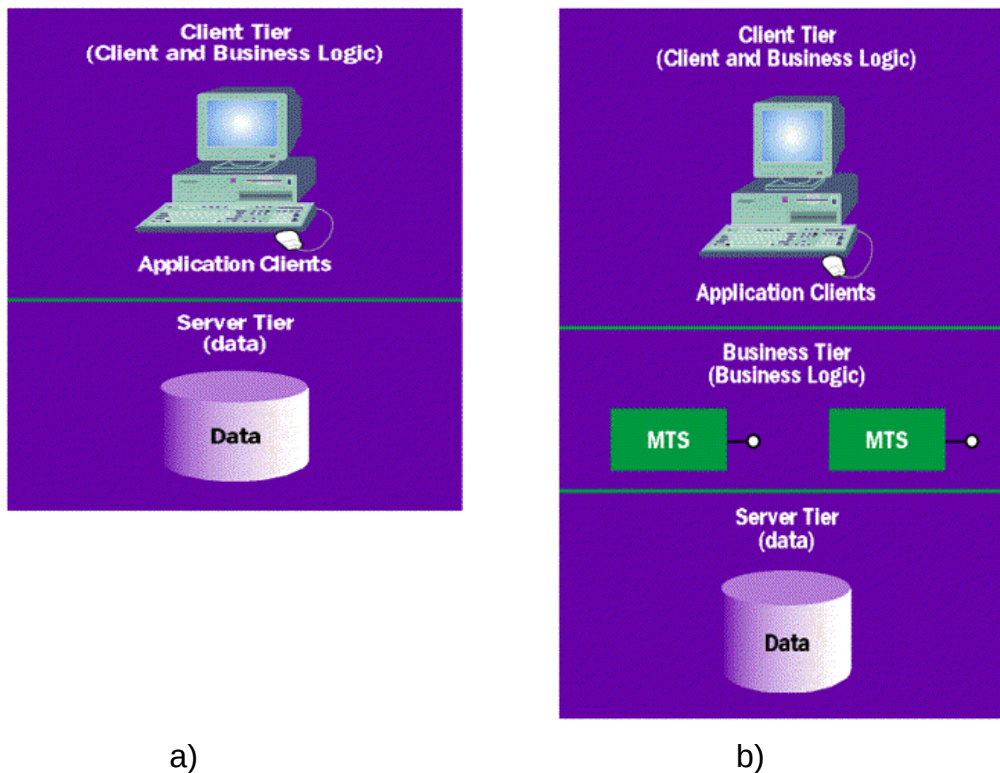


Figura 1 Arquitectura de dos capas (a) y tres capas (b).

Este modelo de dos capas presenta limitaciones cuando el número de usuarios excede de 100. Además, cuando se implementan los servicios usando procedimientos propietarios de la base de datos (procedimientos almacenados y disparadores) se restringe la flexibilidad y la elección del Sistema de Gestión de Base de Datos (SGBD) con el que se construye la aplicación.

Estos problemas se resolvieron creando una tercera capa que implementa la lógica del negocio y que proporciona un ambiente donde miles de usuarios pueden estar conectados simultáneamente, pues el SGBD no tiene que resolver él solo la comunicación con los clientes (figura 1b).

De manera general las aplicaciones de n-capas ($n > 2$) son fragmentadas en partes que pueden o no ejecutarse en computadoras diferentes, pero en las que pueden identificarse:

- Aplicación cliente que proporciona la interfaz con el usuario por lo que transfiere sus solicitudes.
- Servidor de aplicaciones a través del cual se actualiza y consulta la información porque tiene una relación directa con el servidor de datos.
- Servidor de datos como repositorio de la información.

Para definir los nodos físicos en que se ejecutará la aplicación, hay que tomar en consideración la arquitectura que se va a utilizar.

7.1.2 - Diagrama de despliegue

Un diagrama de despliegue es un diagrama que muestra la configuración de los nodos que participan en la ejecución y de los componentes que residen en ellos. Gráficamente, un diagrama de despliegue es una colección de nodos y arcos (Booch, página 362).

Nodo

- Es un elemento físico que existe en tiempo de ejecución.
- Representa un recurso computacional que, por lo general tiene memoria y capacidad de almacenamiento.
- Se usan para modelar la topología del hardware sobre el que se ejecuta el sistema y la distribución física del sistema.
- Representa a un procesador o un dispositivo sobre el cual se despliegan los componentes (los componentes modelan los elementos físicos que pueden hallarse en un nodo, tales como ejecutables, bibliotecas, BD, archivos, etc.). Representan las instancias específicas (MiPC) o una clase de computadoras (ServidorWEB).
- Entre los nodos pueden darse relaciones de: dependencia, generalización / especialización, agregación / composición y asociación. En el caso de las asociaciones, la que se establece entre dos nodos describe que entre ellos hay una conexión física. Usualmente se etiqueta con el protocolo de comunicación que se utiliza para que se dé esta conexión.
- Entre los nodos y componentes se dan relaciones de contenido.

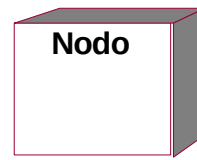
La relación entre nodos y componentes se describe en la tabla 1, de la que se puede concluir que un nodo contiene componentes.

Tabla 1 Relación entre nodos y componentes.

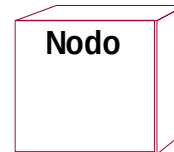
Componente	Nodo
Participan en la ejecución del sistema.	Lugar donde se ejecutan los componentes.
Empaquetamiento físico de los elementos lógicos.	Despliegue físico de componentes.

Se pueden identificar dos tipos de nodos:

- Procesador (Processor): Nodo con capacidad de procesamiento por lo que puede ejecutar un componente.



- Dispositivo (Device): Nodo sin capacidad de procesamiento, por lo general representa algo que interactúa con el mundo real.



Para todas las aplicaciones no tiene sentido construir un diagrama de despliegue. Si su aplicación se ejecuta en una sola máquina y todos los dispositivos con lo que se relaciona son los estándares (teclado, mouse, impresora, teclado); además, si hay relación con otro sistema y este se ejecuta y tiene la información en la misma máquina en la que funcionará la aplicación, entonces no es necesario construir este tipo de diagrama.

Como contraparte podrían considerarse el modelamiento de sistemas empujados, cliente / servidor o con información completamente distribuida.

Para el caso de los sistemas empujados hay que identificar los dispositivos que control el sistema y aquellos de los que se reciben estímulos externos. Cada uno de estos dispositivos es un nodo que puede ser considerado como "procesador" o "dispositivo" de acuerdo al rol que juegan para el sistema; no obstante se recomienda representar cada tipo de dispositivo utilizando estereotipos diferentes (figura 2 y 3).

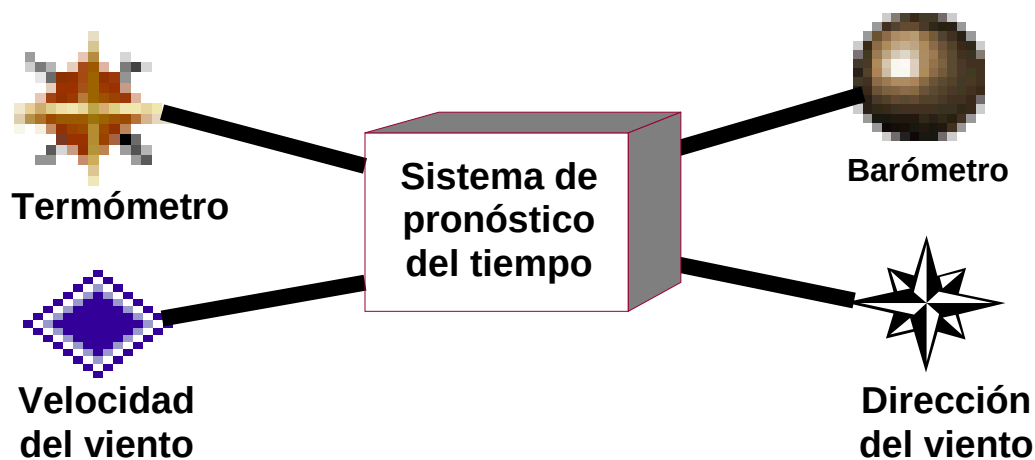


Figura 2 Sistema de pronóstico del tiempo.

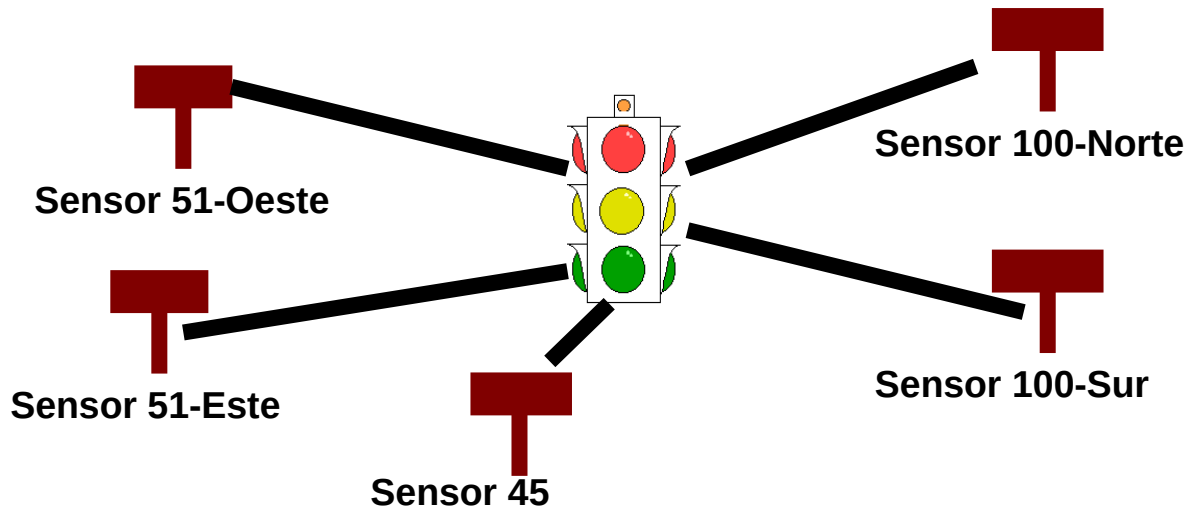


Figura 3 Semáforo de 100 y 51.

Para el caso de sistemas cliente / servidor hay que identificar como nodos a los procesadores cliente / servidor de acuerdo a las capas que se van a implementar. Además, como para cualquier otro sistema hay que identificar otros dispositivos de entrada / salida de información (que no sean lo usuales) si son importantes desde el punto de vista de la arquitectura.

En los sistemas en los que la información es distribuida completamente hay que seguir las mismas recomendaciones que en el caso de los sistemas cliente / servidor; con la salvedad de que la información no está en un único nodo servidor de datos sino que hay varios nodos de este tipo; además, hay que considerar que la información puede replicarse, en cuyo caso se deben definir estereotipos que indique cuándo la información de un nodo es réplica o copia de otro nodo.

Para algunos sistemas se contempla tener una BD espejo por lo que se está en otro nodo debe ser representada en el diagrama de despliegue.

Para el ejemplo de la Galería de arte, en la figura 4 se presenta el diagrama de despliegue correspondiente. Esta representación responde al hecho de que se pretende construir una aplicación que se ejecuta en la misma máquina en la que se encuentra la BD y se puede acceder por los usuarios solo en esa máquina. Esta política también se siguió cuando se construyó la aplicación que responde al sistema de área de Recursos Humanos.



Figura 4 Diagrama de despliegue del caso de estudio de la Galería de arte.

7.1.3 - Mapeo del diseño a la implementación.

Cada clase en diseño es implementada a través de su codificación en un lenguaje de programación y/o mediante el uso de componentes pre-existentes. La correspondencia exacta entre una clase de diseño y su implementación correspondiente dependerá del lenguaje de programación. Por ejemplo, en lenguajes orientados a objetos, como pudiera ser el caso de C++ o Java, una clase de diseño puede corresponder a una o varias clases. En VisualBasic, una clase de diseño corresponde a un componente de un tipo específico; por ejemplo, un módulo de clase, un formulario, o un control.

El modelo de diseño podrá ser más o menos cercano al modelo de implementación dependiendo de cómo se mapeen sus clases a clases o construcciones similares en el lenguaje de implementación. Por ejemplo, se le puede dar a las clases de implementación una relación 1:1 respecto a las clases de diseño. Ello conducirá a un modelo de diseño con rastreabilidad transparente al modelo de implementación, el cual es apropiado en ambientes de “ingeniería de viaje redondo” (ingeniería e ingeniería inversa).

Otra alternativa es permitir que una clase de diseño sea una abstracción de diversas clases de implementación. Cuando se usa esta perspectiva, es recomendable mapear cada clase de diseño con una clase “principal” (*head class*) que, a su vez, haga uso de distintas clases “auxiliares” (*help classes*) para realizar sus servicios. Se pueden emplear clases auxiliares para implementar atributos complejos o para construir estructuras de datos que se requieran para implementar una operación. En el diseño no se necesitará modelar las clases auxiliares, sino modelar algunos de los atributos, relaciones y operaciones definidas por la clase principal, en la medida en que el modelo cumpla con su propósito y sea una buena abstracción del código fuente. El cómo se relacionarán las clases de diseño con las de implementación se deberá decidir antes de que comience el diseño y deberá ser documentado en las *Guías de Diseño*.

Cada lenguaje de programación ofrece un número distinto de características que pueden ser difíciles de “ajustar” en un proceso para un marco de trabajo general. Por ejemplo, en C++ conceptos como clases bases virtuales, métodos y atributos constantes, métodos abstractos, funciones amigas, u otros, no tiene una equivalencia directa en otros lenguajes de programación. Inversamente, los lenguajes de programación pueden tener algunas limitaciones que pueden ser fácilmente representadas. La disciplina de implementación tiene como propósito albergar este marco de trabajo general para cualquier lenguaje orientado a objetos. Sin embargo, es necesario especializarse en la disciplina de implementación para explotar al máximo las posibilidades y manipular las limitaciones de cada lenguaje de programación.

7.2 - Flujo de trabajo de implementación

El flujo de trabajo de implementación se ejecuta esencialmente durante la fase de Construcción, aunque desde el Inicio se comienza a trabajar en el prototipo de la aplicación y todavía en la Transición se están corrigiendo defectos.

Los artefactos que se construyen en este flujo son:

- Modelo de implementación: Describe los componentes y la organización de acuerdo a los nodos; así como las dependencias de compilación entre ellos.
- Componente.

- Subsistema de implementación: Organización de los artefactos utilizando un mecanismo de empaquetamiento.
- Interfaz: Especifica las operaciones implementadas en componentes y subsistemas que son visibles a otros componentes y subsistemas. Cuando se definen interfaces quedan ocultos los detalles internos por lo que, por ejemplo, una clase de un componente no tendría una asociación de algún tipo con una clase de otro componente.
- Descripción de arquitectura (vista del modelo de implementación): Componentes, subsistemas e interfaces y sus relaciones y ubicación en nodos.
- Plan de integración de construcciones: Procedimientos a seguir para integrar las versiones (release) del producto que se obtiene en cada iteración de construcción.

En la construcción de estos artefactos intervienen :

- Arquitecto: Responsable de la integridad, corrección, completitud y legibilidad del modelo de implementación de acuerdo a lo descrito en el modelo de diseño.
- Ingeniero de componentes: Define y mantiene uno o varios componentes y subsistemas.
- Integrador de sistemas: Planifica las iteraciones de construcción definiendo el plan de integración de construcciones.

En la figura 5 se describen las actividades que se ejecutan como parte del flujo de trabajo de implementación.

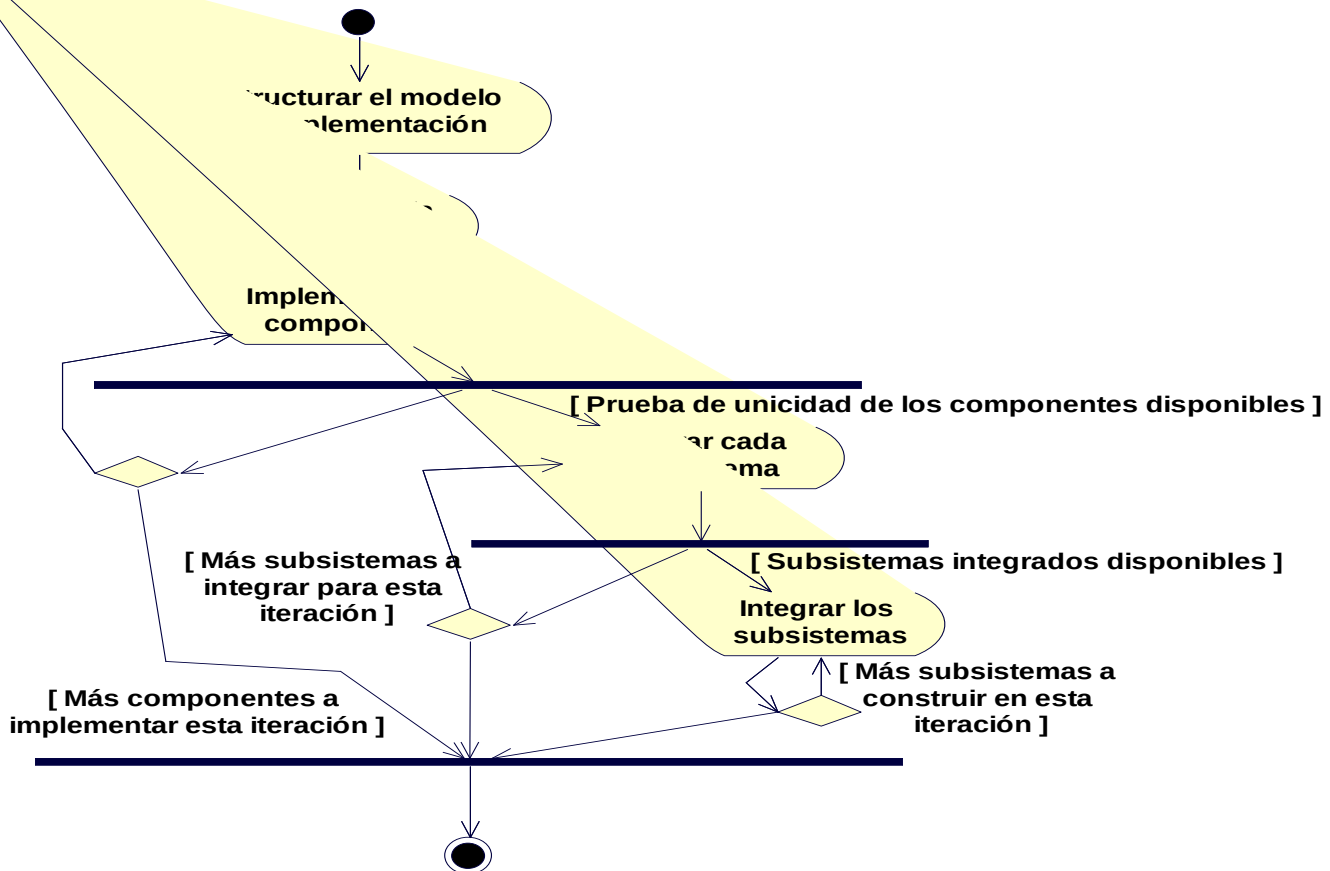


Figura 5 Flujo de trabajo de implementación.

7.2.1 - Diagrama de componentes

Un diagrama de componentes muestra un conjunto de componentes y sus relaciones. Gráficamente es una colección de nodos y arcos que contiene componentes, interfaces y relaciones de dependencia, generalización / especialización, asociación, agregación / composición y realización.

Un componente es una parte física y reemplazable de un sistema que se conforma con un conjunto de interfaces y proporciona la realización de dicho conjunto. Se usan para modelar los elementos físicos que pueden hallarse en un nodo por lo que empaquetan elementos como clases, colaboraciones e interfaces.

En RUP se definen un conjunto de tipos de componentes usando estereotipos. Estos estereotipos son:

- **Ejecutable:** Especifica un componente que se puede ejecutar en un nodo.
- **Biblioteca:** Especifica una biblioteca de objetos estática o dinámica.
- **Tabla:** Especifica un componente que representa una tabla de una BD.
- **Archivo:** Especifica un componente que representa un documento que contiene código fuente o datos.
- **Documento:** Especifica un componente que representa un documento.
- **Página Web:** Especifica una página que se obtiene de la ejecución del sistema.

Rational Rose (Component View) define un subconjunto diferente de estereotipos, entre los que se encuentran los que contiene la figura 6. No obstante, se recomienda modelar sus propios estereotipos de acuerdo a los tipos de componentes que se van a construir.

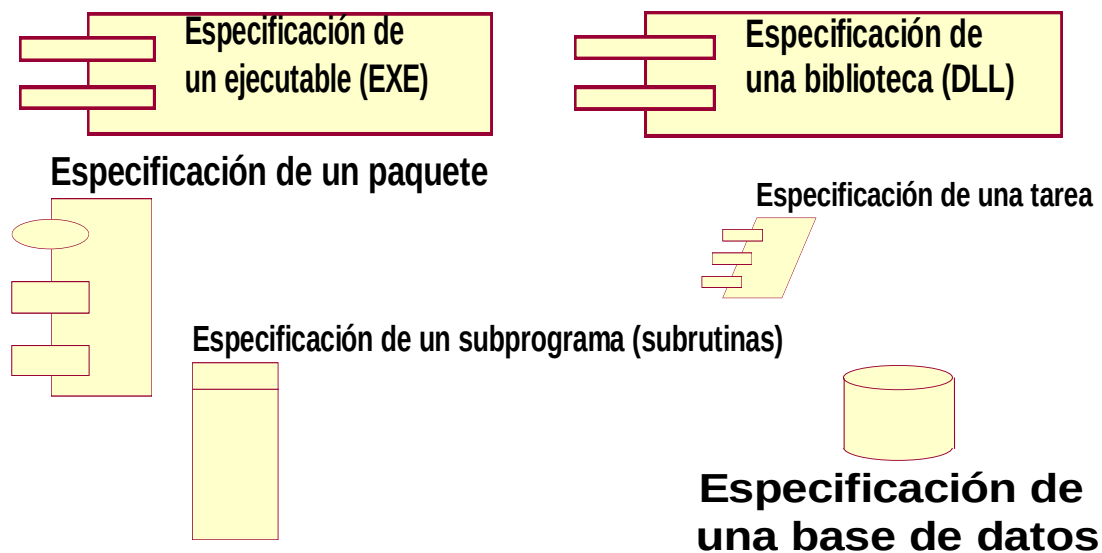


Figura 6 Algunos componentes definidos en Rational Rose.

Cuando se definen componentes hay tener en cuenta la seguridad.

Algunas consideraciones que se deben tener en cuenta cuando se construye el Diagrama de componentes (DC) son:

- Si el sistema consta de un único archivo (el ejecutable), no es necesario hacer el DC.

- Si se complica el diagrama se pueden agrupar los componentes en paquetes.
- Entre los componentes son válidas, relaciones de agregación /composición , asociación y generalización /especialización.
- Si es importante llevar el control de versiones, asociar al componente la versión y el autor.
- Se pueden representar las interfaces que se utilizan para relacionar los componentes.
- Si los estereotipos predefinidos por Rational no son suficientes, cree los suyos.

Para definir componentes se pueden considerar los siguientes elementos

Ejecutables, bibliotecas, subprogramas, tareas y paquetes:

- Componentes a reutilizar en o desde otros sistemas.
- Partes que pueden cambiar a lo largo del tiempo.
- Nodos en los que se utilizará el sistema.
- Todos los archivos de código fuente y las dependencias de compilación entre ellos.
- Estructuración en paquetes del sistema.

Base de datos

- Bases de datos que se construyen o consultan por el sistema.
- Modelar una tabla individual como componente cuando sea externa al sistema.

En la figura 7 se muestra un ejemplo de cómo podría modelarse componentes que tendrán instancias en diferentes nodos y las acciones que hacen que un componente migre.

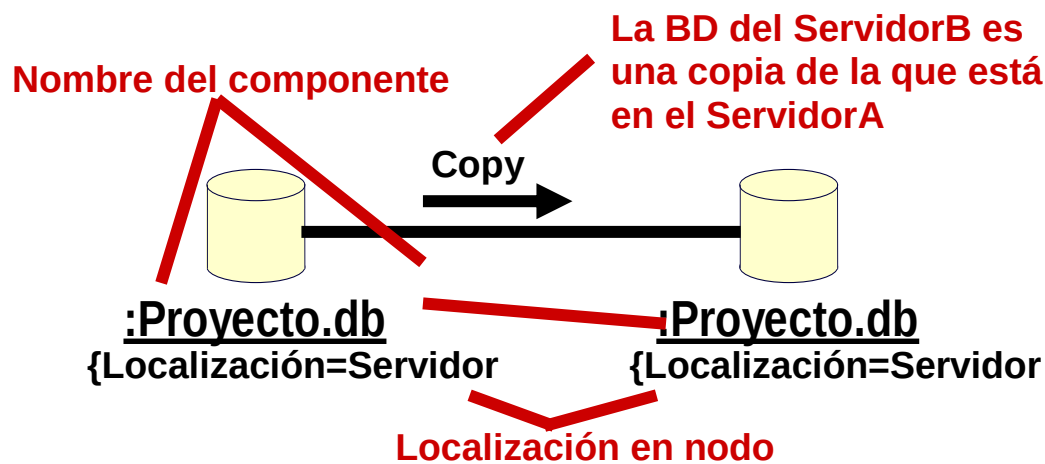


Figura 7 Ejemplo de relación ente componentes en un sistema distribuido.

En la figura 8 se muestran los componentes que serán construidos para la aplicación de la galería de arte. Por razones de entendimiento, se han omitido los componentes ya programados (suministrados por el lenguaje o de otras fuentes) que serán utilizados, pero que hay que ponerlos cuando se construye este diagrama, y se han agrupado en paquetes los componentes fuertemente acoplados. Se ha utilizado el mismo criterio de agrupamiento usado en la representación de los diagramas de casos de uso y de clases. Para completar esta representación habría que definir un diagrama de componentes para cada paquete.

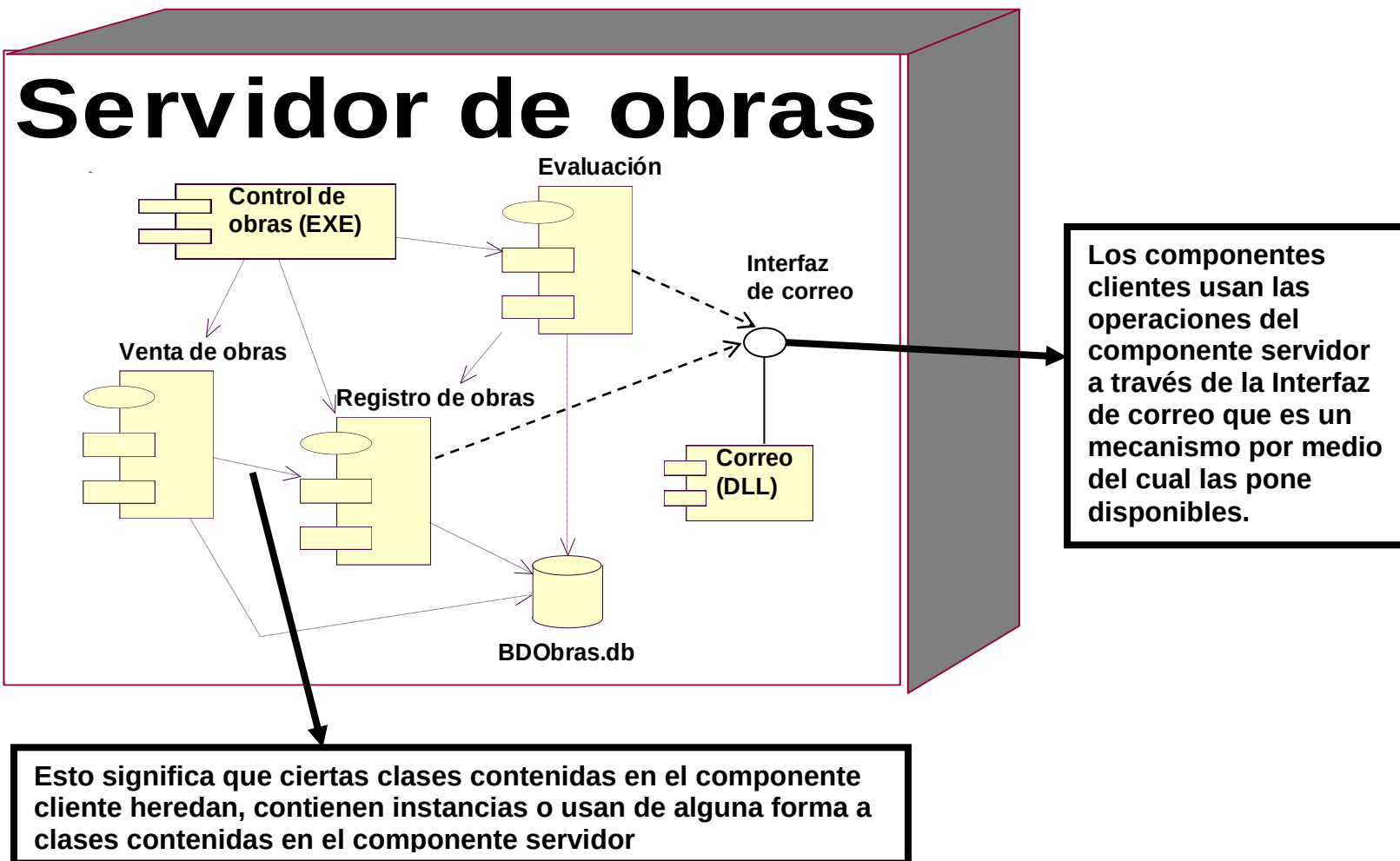


Figura 8 Diagrama de componentes del nodo "Servidor de obras".

Un diagrama de implementación sería un diagrama que contenga nodos y componentes por lo que en una sola vista se pueden apreciar los diagramas de componentes y de despliegue. Aunque se pueden construir por separado estos diagramas, en ellos no queda clara la relación directa que puede darse entre componentes ubicados en nodos diferentes. (figura 9).

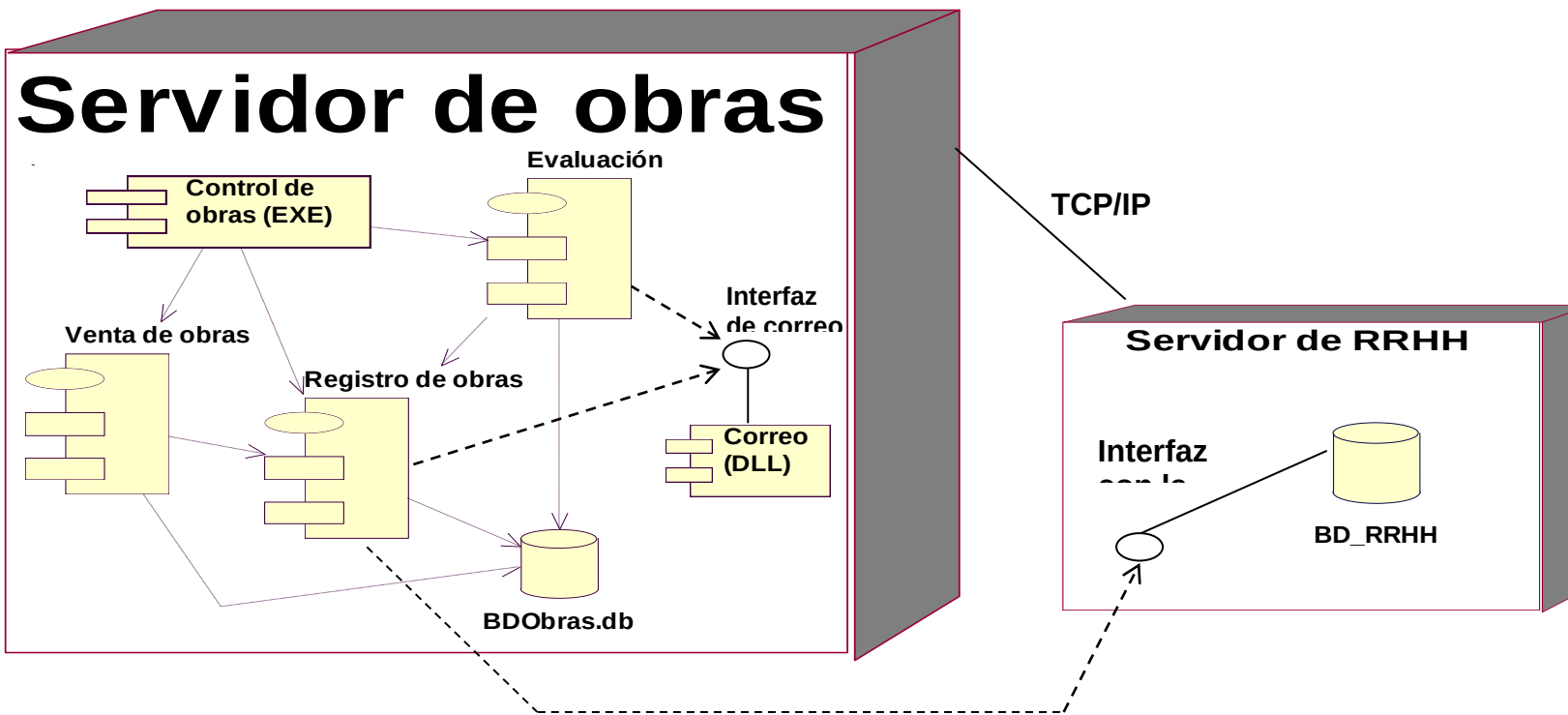


Figura 9 Diagrama de implementación del caso de estudio.

7.2.2 - Construcción del software

RUP propone la programación y prueba de todas las partes del sistema de forma ascendente a través de la definición de iteraciones en la fase de construcción. La idea por lo tanto es desarrollar el sistema siguiendo los siguientes pasos para cada iteración:

- **Integración y prueba de cada paquete:** Programar y probar cada clase individual y las relaciones entre ellas (Probar métodos y la verificación de las excepciones).
- **Integración de paquetes:** Comenzar por los paquetes de los cuales dependen otros (Probar las clases controladoras y las visibles a otros paquetes).
- **Prueba de validación del software:** Probar que cumpla con los requisitos funcionales y no funcionales.

7.3 - Conclusiones

El modelo de diseño podrá ser más cercano al modelo de implementación dependiendo de cómo se mapeen sus clases a clases o construcciones similares en el lenguaje de implementación.

Independientemente de que se desarrollen aplicaciones de 1, 2, 3 ó n capas; se puede realizar un aislamiento del servicio de interfaz del usuario, la implementación de las reglas de negocio y la BD a través de la definición de los componentes.

En Rational Rose el diagrama de componentes se construye en la Component View y el de despliegue en la Deployment View.

RUP recomienda que en la construcción de los diagramas de despliegue y componente se tengan en cuenta los siguientes criterios:

- Poner nombres que comuniquen el propósito.
- Usar elementos visuales (usando notaciones gráficas específicas a través de estereotipos) que comunique el significado de cada tipo de elemento.
- Minimizar los cruces y poner cerca de elementos relacionados.

Definir todos los elementos (componentes y nodos) que describen el modelo de implementación de la aplicación que se va a construir.

7.4 - Referencias Bibliográficas

- JACOBSON, Ivar; RUMBAUGH, James, BOOCH, Grady, "El lenguaje unificado de modelado".2000. Addison Wesley. Capítulo 9 y 10 página 217-218 y 255-279
- BOOCH, Grady, RUMBAUGH, James, JACOBSON, Ivar; "El lenguaje unificado de modelado".2000. Addison Wesley. Capítulo 2 Página 26-28 Capítulo 25 y 26 Página 305-325 Capítulo 29 y 30 Página 349-370
- BRUEGGE, Bernd, DUTOIT, Allen; "Ingeniería de software orientado a objetos".2002. Prentice Hall - Pearson Educación. Capítulos 6 Páginas 198-201

Capítulo 8 Modelo de prueba

El desarrollo del software implica una serie de actividades de producción en las que las posibilidades de que aparezca la falibilidad humana son comunes. Los errores pueden empezar a darse desde el primer momento del proceso en el que los objetivos pueden estar especificados de forma errónea e imperfecta; así en los posteriores pasos del diseño y desarrollo. Debido a la imposibilidad humana de trabajar y comunicarse de forma perfecta, el desarrollo del software ha de ir acompañado de una actividad que garantice la calidad.

La prueba de software es un elemento crítico para la garantía de la calidad del software y representa una revisión final de las especificaciones del diseño y de la codificación.

La creciente inclusión del software como un elemento más de muchos sistemas y la importancia de los costos asociados a un fallo del mismo, han motivado la creación de pruebas más minuciosas y bien planificadas. No es raro que una organización invierta el 40% del esfuerzo de un proyecto en la prueba. En casos extremos, por ejemplo en el control del tráfico aéreo y el control de reactores nucleares, pueden costar 3 ó 5 veces más que el resto de los pasos juntos.

Existe un mito que se basa en que si fuéramos buenos programando, nos concentráramos, empleáramos lenguajes apropiados, no habría errores que buscar. Es decir, existen errores porque somos malos en lo que hacemos. Por lo tanto, la prueba es un reconocimiento de nuestros fallos, lo que implica una buena dosis de culpabilidad y un castigo de nuestros errores.

Glen Myers establece una serie de reglas que sirven como objetivos de la prueba:

1. La prueba es un proceso de ejecución de un programa con la intención de descubrir errores.
2. Un buen caso de prueba es aquel que tiene una alta probabilidad de mostrar un error no descubierto hasta entonces.
3. Una prueba tiene éxito si descubre un error no detectado hasta entonces.

Estos objetivos suponen un cambio drástico de punto de vista al quitarnos la idea de que una prueba tiene éxito si no descubre errores. El objetivo es diseñar pruebas que sistemáticamente saquen a la luz diferentes clases de errores haciéndolo con la menor cantidad de tiempo y esfuerzo.

La prueba no puede asegurar la ausencia de defectos; solo pueden demostrar que existen defectos en el software.

RUP propone que en cada una de las fases las pruebas se comporten de la siguiente forma:

- **Inicio:** El desarrollo del prototipo exploratorio de demostración no requiere la elaboración de pruebas.
- **Elaboración:** Probar los componentes ejecutables que se han implementado y que deben corresponderse con la arquitectura básica de la aplicación.
- **Construcción:** Desarrollar los casos de prueba y procedimientos de prueba para hacerlos.

- **Transición:** El producto en su entorno de operación por lo que es probado por usuarios reales.

Como el centro del flujo de trabajo de prueba se desarrolla en la fase de construcción, en esta clase se hará hincapié en los artefactos que se construyen y actividades que se desarrollan en este período.

8.1 - Flujo de trabajo de Prueba

A menudo existen ciertos malentendidos que se pueden deducir equivocadamente:

- El que desarrolla el software no debe entrar en el proceso de prueba.
- El software debe ser “puesto a salvo” de extraños que puedan probarlo de forma despiadada.
- Los encargados de la prueba solo aparecen en el proyecto cuando comienzan las etapas de prueba.

Todo esto es incorrecto, el que desarrolle el software es responsable de probar los módulos o subsistemas asegurándose que cada una lleva a cabo la funcionalidad para el que fue diseñado. Una vez que la arquitectura del software esté completa, entra en juego un grupo independiente de prueba que elimina el conflicto de interés que para el desarrollador del tiene que probar sus propio producto.

Aunque las pruebas pueden ser realizadas por usuarios finales y otros trabajadores del equipo, RUP define que los trabajadores que participan en este flujo son.

- Diseñador de prueba: Planifican, diseñan y evalúan los resultados de la prueba.
- Ingeniero de componentes: Responsables de la elaboración de los componentes de prueba para los procedimientos que puedan ser automatizados.
- Ingeniero de pruebas de integración: Realizan las pruebas de integración.
- Ingeniero de pruebas de sistema: Realizan las pruebas del sistema.

En la fase de transición en la pruebas participa un equipo mixto de desarrolladores y clientes reales.

Una estrategia de prueba del software integra las técnicas de diseño de casos de prueba en una serie de pasos bien planificados que dan como resultado una correcta construcción del software.

Cualquier estrategia de prueba debe incorporar la planificación de la prueba, el diseño de casos de prueba, la ejecución de pruebas y la depuración y evaluación de los datos resultantes.

RUP propone tres grandes pasos para realizar las pruebas:

1. Planificar las pruebas de integración, tanto de integración para cada construcción dentro de la iteración como del sistema para cada iteración.
2. Diseñar e implementar las pruebas con la elaboración de casos de prueba, para especificar cómo realizar las pruebas. Además, se evalúa la utilización de algún software que permita automatizar las pruebas (construirlos o usar alguno que exista).
3. Realizar las pruebas y arreglar los defectos.

En la figura 1 se muestran las actividades que se realizan en este flujo de trabajo. Vale señalar que en la planificación hay que describir la estrategia de prueba y los recursos humanos necesarios para acometerlos.

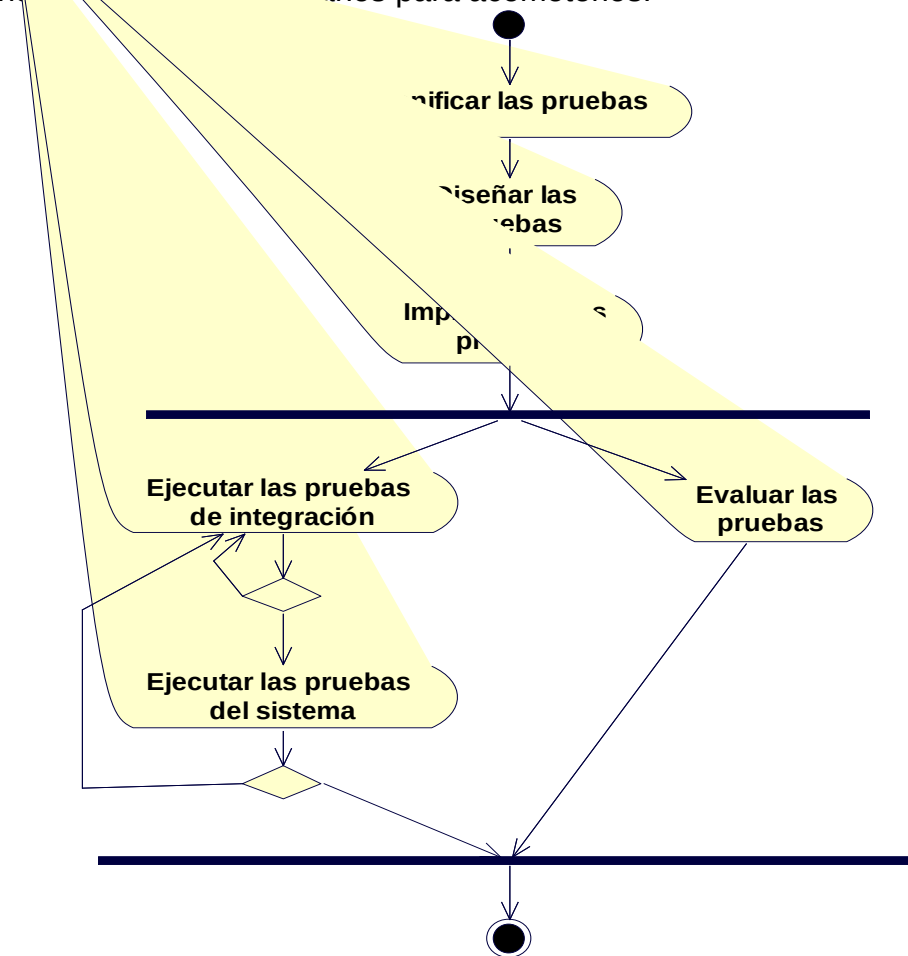


Figura 1 Flujo de trabajo de prueba.

Por lo tanto, el flujo de información para la prueba sigue el esquema de la figura 2.

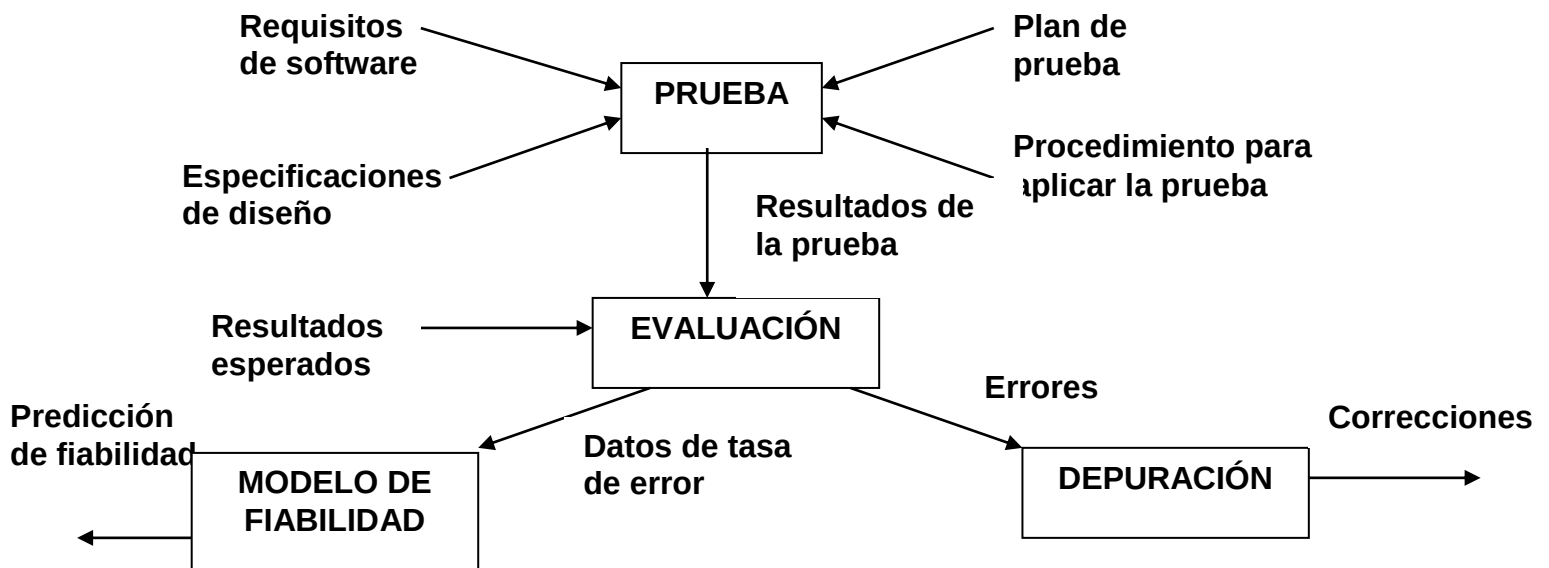


Figura 2 Flujo de información para la prueba.

Cada uno de estos procesos se encarga de:

- **Prueba:** Se ejecutan los procedimientos de prueba a partir de los casos de prueba especificados.
- **Evaluación:** Se comparan los resultados de la prueba con los esperados de acuerdo a los especificado en los casos de prueba.
- **Depuración:** Es el proceso que resulta en la eliminación del error. En él se tienen dos posibles resultados: se encuentra la causa, se corrige y se elimina o no se encuentra la causa en cuyo caso hay que sospechar la causa y diseñar casos de prueba que ayuden a validarla o rechazarla.
- **Modelo de fiabilidad:** Analiza la frecuencia y gravedad (en cuanto a modificaciones que se requieren realizar) de los errores que ocurren, lo que da una medida cualitativa de la calidad y de la fiabilidad del software.

Para el proceso de depuración se pueden seguir tres enfoques:

- **Fuerza bruta:** La computadora descubre el error. Para ello, por ejemplo, se corre el programa completo por trazas y se incluyen múltiples comentarios en el programa para llegar a la causa del error. Este enfoque es costoso en tiempo y esfuerzo.
- **Vuelta atrás:** Partiendo del lugar donde se descubre el error, se recorre hacia atrás manualmente el código fuente hasta que se llega a la posición del error. Es factible en programas pequeños.
- **Eliminación de causas:** Los datos relacionados con la ocurrencia del error se organizan para aislar las posibles causas. Si llega a una hipótesis de causa y se usan los datos para probar o revocar la hipótesis.

Como la corrección de un error puede introducir otros, Van Vleck sugiere tres preguntas antes de corregir:

1. ¿Se repite la causa del error en otra parte del programa?
2. ¿Cuál es el siguiente error que se podría presentar a raíz de la corrección que voy a realizar?
3. ¿Qué podríamos haber hecho para prevenir este error por 1ra vez?

Como resultado de la realización de las actividades incluidas en este flujo de trabajo, los trabajadores construyen los siguientes artefactos:

- **Modelo de prueba:** describe cómo se prueban los componentes construidos y el cumplimiento de los requisitos funcionales y no funcionales especificados. Es una colección de casos de prueba, procedimientos de prueba y componentes de prueba.
- **Casos de prueba.**
- **Procedimientos de prueba.**
- **Componentes de prueba:** automatiza uno o varios procedimientos de prueba o partes de ellos.
- **Plan de prueba:** Estrategia, recursos y planificación de la prueba.
- **Defecto:** Anomalía del sistema que hay que controlar y resolver.

- **Evaluación de prueba:** Evalúa los resultados de los esfuerzos de prueba.

8.2 - Casos de prueba

Los casos de prueba especifican la forma de probar el sistema, incluyendo la entrada o resultado con la que se ha de probar y las condiciones bajo las que ha de probarse. Es un conjunto de entradas y resultados esperados que ejercitan a un componente con el propósito de causar fallas y detectar defectos. Entre los casos de prueba pueden existir todos los tipos de relaciones, pero las más importantes son las de precedencia.

Para obtener los casos de prueba hay que definir las condiciones de prueba (tabla 1). Las condiciones de prueba describen todas las situaciones que reflejen qué se quiere probar del sistema. Hay que priorizar las condiciones de prueba porque puede que no se puedan probar todas o que algunas sean más críticas que otras.

Tabla 1 Condiciones de prueba

¿De dónde se obtienen?	¿Cómo obtenerlas?
Casos de uso	Los diferentes cursos normales y alternativos.
Requerimientos funcionales	Debe hacer todo lo declarado.
Requerimientos no funcionales	Tiempos de respuesta esperados, volumen de información a manejar, configuración de hardware y software sobre los que debe ejecutarse, recuperación ante fallas, seguridad, ...
Especificaciones de diseño	Condiciones especiales para validar la integridad de módulos, implementaciones parciales si se va a hacer por etapas, ...

Cualquier producto de ingeniería puede ser probado de una de estas formas:

1. Conociendo la funcionalidad específica para la cual fue diseñado el producto, se pueden llevar a cabo pruebas que demuestren que cada función es completamente operativa.
2. Conociendo el funcionamiento del producto se pueden desarrollar pruebas que aseguren que “todas las piezas encajen”, o sea, que la operación interna se ajusta a las especificaciones y que todos los componentes internos se han comprobado de forma adecuada.

El 1er enfoque se denomina prueba de caja negra y el 2do prueba de caja blanca.

Prueba de caja negra: Se refiere a las pruebas que se llevan a cabo sobre la interfaz del software, por lo que los casos de prueba pretenden demostrar que las funciones del software son operativas, que la entrada se acepta de forma adecuada y que se produce una salida correcta, así como que la integridad de la información externa se mantiene. Esta prueba examina algunos aspectos del modelo fundamentalmente del sistema sin tener mucho en cuenta la estructura interna del software.

Prueba de caja blanca: Se basa en el minucioso examen de los detalles procedimentales. Se comprueban los caminos lógico del software proponiendo casos de prueba que examinen la correctitud de todas las condiciones y/o bucles

para determinar si el estado real coincide con el esperado o afirmado. Esto genera gran cantidad de caminos posibles por lo que hay que dedicar esfuerzos a la determinación de las condiciones de prueba que se van a verificar.

Para profundizar en los tipos de prueba de caja blanca y negra, consulte el libro de Roger Pressman.

Una estrategia de prueba propone movernos hacia afuera en una espiral, de manera que primero se prueban las unidades más pequeñas del diseño del software (clases o módulos) y después como se integran los componentes en los cuales están contenidas estas unidades.

RUP propone definir casos de prueba de integración para verificar que los componentes interaccionan entre sí de forma apropiada.

A partir de un caso de uso se pueden realizar pruebas de caja negra, obteniéndose varios casos de prueba que permiten:

- Verificar el resultado de la interacción entre los actores y el sistema.
- Comprobar que se satisfagan las precondiciones y poscondiciones del caso de uso.
- Comprobar que se siga la secuencia de acciones especificado por el caso de uso.

También se pueden realizar pruebas de caja blanca a partir de la realización de un caso de prueba que permiten obtener casos de prueba en los que se verifica la integración ante los componentes que implementan dicho caso de uso.

Como conclusión podríamos decir que se pueden definir múltiples casos de prueba de integración para cada caso de uso en dependencia de las condiciones de prueba que se tengan en cuenta. El formato para describirlos podría ser:

Variante 1

Caso de uso: <Nombre>

Caso de prueba: <Nombre>

Entrada: <Descripción textual de lo que ocurre en el mundo real que hace necesario ejecutar el caso de prueba, precisando la data de entrada y los comandos a dar por el actor. Descripción textual del estado de la información almacenada>

Resultado: <Descripción textual del estado en el que queda la información y las alertas que puedan generarse, una vez ejecutado el caso de uso con los valores y el estado especificado en la entrada>

Condiciones: <Condiciones que deben cumplirse mientras se ejecuta el caso de prueba>

Variante 2

Caso de uso: <Nombre>

Rango de valores de entrada	Resultados

Esta 2da variante se usa cuando hay varios casos de prueba que verifican diferentes escenarios del mismo caso de uso.

Las pruebas del sistema se usan para probar que el sistema funciona correctamente como un todo. Como parte de estas pruebas hay que:

- Probar la instalación del software en la plataforma del cliente.
- Verificar el funcionamiento del software en diferentes configuraciones.
- Realizar pruebas negativas que busquen que el sistema falle.
- Realizar pruebas de tensión o estrés cuando hay competencia por los recursos.

El siguiente es un ejemplo de un caso de prueba que se genera a partir del caso de uso “Reportar obras registrar que no han sido vendidas” en el que se tiene en cuenta la condición de prueba asociada al hecho de que no hay obras registradas para la fecha que indica el Gerente en la Galería.:

Caso de uso: Reportar obras registradas que no han sido vendidas.
Caso de prueba: Reportar obras registradas que no han sido vendidas para un fecha donde no hay información disponible.
Entrada: • El Gerente de la galería necesita conocer todas las obras registradas que no han sido vendidas a partir del 1 de Agosto del 2003. • Existen obras registradas, pero todas han sido vendidas.
Resultado: • Se muestra un mensaje al Gerente de la galería informándole que no hay resultados registrados.
Condiciones: • No se permite que se ejecuten otras instancias del caso de uso para el Gerente de la galería.

8.3 - Procedimientos de prueba

Los procedimientos de prueba especifican cómo realizar uno o varios casos de prueba o partes de estos, por lo general define los pasos a seguir por el equipo de prueba cuando tienen que verificar las condiciones de prueba incluidas en los casos de prueba.

Tienen que corresponderse con el flujo de eventos del caso uso, pero dando valores. Se encierran entre corchetes para indicar que ese valor cambia en dependencia de la entrada de los casos de prueba.

Para el caso de prueba descrito anteriormente podría definirse el siguiente procedimiento de prueba:

Caso de uso: Reportar obras registradas que no han sido vendidas

Precondiciones: -	
Acción del actor	Respuesta del sistema
	1 El sistema permite especificar la fecha base que servirá para construir el reporte.
2 El Gerente de la galería especifica la fecha [01/08/2003] que servirá de base para el reporte.	

3 El Gerente de la galería decide que va a ver el reporte	4 Si decide ver o imprimir el reporte, el sistema verifica si hay obras registradas pendientes de venta recibidas en fecha posterior al [01/08/2003].
---	---

Cursos alternos

"Curso normal principal": Línea 4

Como no hay obras registradas en fecha posterior al [01/08/2003] que estén pendientes de venta, se presenta un mensaje al Gerente de la galería y se devuelve el control a la línea 1 del curso normal principal.

Poscondiciones: No se visualizan las obras pendientes de venta con fecha posterior al [01/08/2003] porque no hay obras que cumplan la condición.



8.4 - Conclusiones

El objetivo de la prueba de software es descubrir errores. Para conseguir este objetivo se planifica y se ejecuta una serie de pasos que van revisando todos los elementos del software.

En todas las fases del desarrollo del proyecto hay que probar el software que se va construyendo, aunque como el grueso de la programación se realiza en la construcción, es en esa fase en la que se centran los mayores esfuerzos de este flujo.

La etapa de prueba es tan o más importante que todas las realizadas hasta el momento puesto que en ella se refleja la calidad con que ha sido llevada a cabo la proyección del sistema.

8.5 - Referencias Bibliográficas

- JACOBSON, Ivar; RUMBAUGH, James, BOOCH, Grady, “El lenguaje unificado de modelado”. 2000. Addison Wesley. Capítulo 11 Página 281-302 Capítulo 16 Página 381-394 Páginas 339-340, 361-363, 377-378
- BRUEGGE, Bernd, DUTOIT, Allen; “Ingeniería de software orientado a objetos”.2002. Prentice Hall - Pearson Educación. Capítulos 9 Páginas 326-369
- “Condiciones y casos de prueba”. <http://www.rmya.com.ar>
- “The Complete Guide to software testing”. <http://www.softwareqatest.com>
- PRESSMAN, Roger “Ingeniería del Software. Un enfoque práctico”.2002. McGraw-Hill/Interamericana de España.

Capítulo 9 Caso de Estudio: Galería de arte

9.1 – Caso de estudio

Marco problémico y problema que se desea resolver

En la Galería de arte “Arte Cubano” se exponen obras de arte (Pinturas) de artistas nacionales provenientes de todo el país. Tiene su sede en la Habana, por lo que los artistas de provincias necesitan viajar ana para presentar la solicitud en la que ponen a la galería en conocimiento de las obras que desean exponer en sus salas.

En la actualidad existen dificultades con el control sobre las obras de arte que están pendientes de evaluación para su posible exposición en la galería, que han sido rechazadas o que se han vendido; lo que dificulta las gestiones asociadas con el montaje de exposiciones y la venta de obras.

El personal de la galería (Anfitriones, Especialistas de arte, Vendedores, Especialistas en economía y el Gerente General) no puede realizar el trabajo con la eficiencia requerida, lo que conlleva a pérdidas económicas y no se satisface la misión social de la galería como promotor de la cultura nacional. Esto provoca que los artistas pierdan confianza en la galería y no sea visitada tanto como se desea.

Con 10 años de experiencia, la galería “Arte Cubano” ha preparado más de 100 exposiciones individuales y grupales, en las que ha logrado vender el 55% de las obras en exposición. Solo el 30% de las obras en exposición se corresponden con artistas de provincias, pero el 80% de ellas se han vendido; lo que evidencia la calidad de sus obras.

Luego de la observación e investigación del proceso actual de gestión de obras de arte, se han detectado los siguientes problemas:

- ❖ Toda la gestión de las obras se hace en la sede de la galería, sin embargo hay un potencial en provincia que no es suficientemente explotado por las limitaciones de transporte.
- ❖ El formato en que se recibe la información difiere según la información que tiene cada artista, por lo que información como las dimensiones exactas de las obras (importante para valorar si se puede o no exponer una obra en las salas de la galería), se desconoce cuando se presenta la solicitud lo que provoca que se rechace la solicitud antes de poner las obras a disposición de los especialistas de arte que las evalúan.
- ❖ No se tiene control del estado exacto en el que se encuentra cada solicitud, por lo que, existiendo obras ya valoradas, que no se pueden tener en cuenta cuando se planifica una exposición.
- ❖ Existe un sistema automatizado que ayuda en el montaje de las exposiciones, pero normalmente no tiene información actualizada porque es el Anfitrión el que tiene que

recopilarla manualmente para actualizar el sistema y esta persona, además, se encarga de la atención al público en general (artistas y visitantes).

- ❖ El cumplimiento de la regla de negocio que impone que solo el especialista de arte asignado a evaluar las obras de una solicitud puede entregar los resultados de la evaluación de esas obras, no es posible verificarla porque no existe control sobre qué obras contiene cada solicitud y un artista puede presentar varias solicitudes con diferentes obras que serán asignadas a especialistas de arte diferentes para su evaluación en dependencia de las técnicas que usó el artista en su confección. Se han presentado problemas ya que un especialista cuando ha acudido al estudio del artista a valorar sus obras, ha evaluado obras no asignadas a él, pero que estaban pendientes de evaluación.
- ❖ Existiendo especialista de arte en provincia, no ha sido fácil utilizarlos en la evaluación de las obras porque el anfitrión no posee información actualizada de esto; lo que retrasa el proceso de evaluación de las solicitudes.

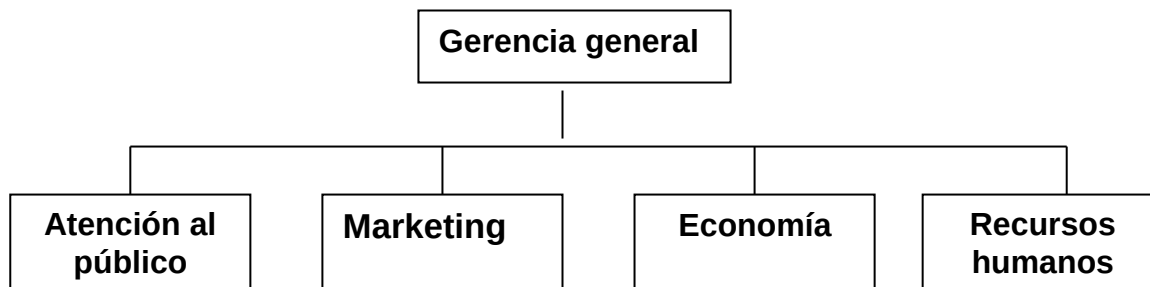
Objeto de Estudio

La galería de arte “Arte Cubano” es una prestigiosa galería que ha presentado durante 10 años muestras de artistas nacionales de gran prestigio nacional y de jóvenes figuras que ha promovido con sus exposiciones. Tiene una amplia experiencia y cuenta con la asesoría de especialistas de arte de un gran prestigio que, distribuidos en todo el país, evalúan las obras de los artistas para su posible exposición en las salas de la galería.

Su misión es lograr:

- ❖ Promover la cultura nacional a través de la presentación de obras de calidad.
- ❖ Generar utilidades para mantener su solidez financiera y poder continuar promoviendo el trabajo de nuevos artistas.

La estructura organizacional de la galería se muestra en la siguiente figura:



Esta galería se ha tomado como modelo para el levantamiento de información y generalización de la problemática sobre la gestión de obras de arte, que puede ser común a otras galerías

El proyecto propone, por tanto, desarrollar una aplicación que controle las solicitudes recibidas, la evaluación de las obras y la venta de obras y que pueda ser aplicada a otras galerías con estructura similar.

De manera general la gestión de las obras de arte en la galería “Arte cubano” puede describirse a grandes rasgos como sigue: se inicia cuando, como parte del proceso de *Atención a solicitudes*, un artista presenta una solicitud en la que pone a consideración de la galería un conjunto de obras de arte para su evaluación como obras aptas para exponer en la galería. Esta evaluación la realiza un especialista de arte. El área de marketing dentro de su labor de *promoción* de las obras disponibles para la venta, tiene que poner a sus vendedores en contacto con los artistas para definir el precio de venta de las obras y el % de la venta que le corresponde a la galería. Cuando el área de economía realiza la *venta* de una obra deberá actualizar su estado ya que esa obra queda fuera de próximas exposiciones y la venta se convierte en utilidades para la galería.

Campo de Acción

El proyecto abarca los siguientes procesos:

❖ Atención a solicitudes

- ✓ Definición de la plantilla de registro de solicitudes que será puesta a disposición de los artistas vía Internet para que puedan registrar directamente su solicitud sin que tengan que ir a la galería en la Habana y que será llenada por el Anfitrión cuando se persone en la sede.
- ✓ Asignación de especialistas de arte a solicitudes de acuerdo a las técnicas en las que se han especializado y el lugar de residencia del artista y del especialista.
- ✓ Evaluación de obras de acuerdo a los criterios de evaluación definidos, permitiendo que vía Internet los especialistas puedan registrar sus evaluaciones.

❖ Promoción de obras para la venta.

- ✓ Definición del precio de venta de las obras de los artistas y del % de la galería de acuerdo a la evaluación de las obras que emitió el especialista.

❖ Venta de obras

- ✓ Control de las obras vendidas.
- ✓ Obtención de indicadores sobre el estado de las obras que se han puesto en consideración de la galería y que no han sido vendidas.

El proyecto planteará un esquema de comunicación vía correo electrónico entre la Galería y los artistas y especialistas de arte encargados de la evaluación de las obras, que garantice un adecuado flujo de información entre estos. Además, tendrá dos páginas en Internet que podrán ser consultadas por los artistas y los especialistas para finalidades específicas.

Para la propuesta de solución se tomará en consideración que la galería cuenta con un sistema de información en el área de Recursos Humanos con el cual el sistema propuesto deberá relacionarse para tomar información de las técnicas y los especialistas de arte.

Objetivos

El objetivo general del presente proyecto es Automatizar para la galería “Arte Cubano” el proceso de control de las solicitudes recibidas, la evaluación y la venta de las obras de artistas nacionales.

Como objetivos específicos se plantean los siguientes:

- ❖ Controlar las solicitudes de arte registras así como su estado y el de las obras puestas a consideración de la galería.
- ❖ Retroalimentar a artistas y especialistas de arte con las gestiones realizadas.
- ❖ Generar una solución que se integre al sistema de información del área de Recursos Humanos.
- ❖ Generar reportes de información sobre el estado de las obras y las evaluaciones emitidas por los especialistas.
- ❖ Obtener una solución general aplicable a cualquier galería de arte que presente una problemática similar a la de la galería “Arte Cubano”.

Funcionalidades previstas

Para cumplir con los objetivos propuestos se prevé que el sistema tenga las siguientes funcionalidades:

- ❖ Registrar solicitudes de exposición de obras.
- ❖ Asignar especialistas de arte a las obras.
- ❖ Registrar evaluación de las obras.
- ❖ Registrar intención de venta de obras.
- ❖ Registrar la venta de obras.
- ❖ Obtener reportes de estado de las obras y de las evaluaciones de los especialistas.
- ❖ Consultar al sistema de Recursos Humanos sobre las técnicas y los especialistas de arte.
- ❖ Enviar correos a especialistas con las características de las obras que tienen que evaluar.
- ❖ Enviar correos a artistas sobre venta de obras.
- ❖ Gestionar usuarios del sistema de acuerdo a los niveles de acceso.

9.2 – Modelo del negocio

Reglas de negocio

Cuando un artista se acerca a la galería para utilizar sus salones en una exposición, es atendido por un Anfitrión de la galería que registra la solicitud. Al registrarse,

automáticamente se le asocia un código y el Anfitrión le asigna un especialista de arte para que evalúe las obras. Cuando se asigna el especialista, se le envía un correo electrónico al especialista con la información que contiene la solicitud.

El registro de una solicitud se puede hacer vía Internet, en cuyo caso el proceso de asignación de especialistas se realiza posteriormente por el anfitrión. Hasta que esto no se hace, no se considera como registrada la solicitud. Solo se permite acceder a esta posibilidad a los artistas que ya han sido registrados.

Una solicitud contiene los datos personales del artista, las características de las obras y la fecha en que se registra la solicitud (no se permite registrar solicitudes de días anteriores o posteriores a la fecha actual).

De una obra se almacena su nombre, fecha de realización, técnica (s) utilizada (s) para su realización y dimensiones.

Cuando se procede al registro de una solicitud, se verifica que los datos del artista estén registrados; en caso de que no sea así, el Anfitrión lo hace en ese momento. De un artista se almacena su nombre, provincia de residencia, dirección, edad, sexo, e-mail y técnicas que usa.

Aún cuando un artista no manifieste su interés en exponer, se ha decidido registrar información sobre aquellos que pueda interesar a la galería que expongan en sus salas. Sobre estos artistas se registra la misma información. Esta actualización la hace el Anfitrión. Por esta vía se mantiene actualizada la información del artista. No se permite la eliminación de un artista asociado a una obra.

Existe un sistema automatizado en Recursos Humanos (RRHH) que, entre otras cosas, mantiene un registro de especialistas de arte. Esta información será consultada por el Anfitrión para la asignación. Si no hay especialistas disponibles que dominen todas las técnicas de las obras de una solicitud, se rechaza la solicitud. En este proceso el Anfitrión toma en cuenta el lugar de residencia del especialista y del artista, aunque no es una limitación si no hay especialistas disponibles que vivan en la provincia del artista.

Cuando el Anfitrión realiza la asignación de especialistas a la solicitud, se asocia el especialista registrándose de él: nombre, provincia, e-mail y las técnicas en las que se especializa el especialista. Una solicitud es evaluada por un especialista y un especialista puede evaluar varias solicitudes.

Las solicitudes registradas se pueden eliminar y modificar las obras que contiene (no las características de las obras), siempre y cuando no se haya evaluado alguna de las obras. En cualquier caso se le comunica al Especialista de arte, por la vía del correo, las características de la solicitud. Una vez asignado un especialista, no puede ser cambiado.

Los Especialistas de arte tendrán acceso al sistema para introducir los resultados de las evaluaciones a las obras de las solicitudes a él asignadas. Cada obra de una solicitud se puede evaluar por varios criterios por lo que el Especialista de arte debe registrar la evaluación que da a cada obra en cada uno de los criterios. Al final debe decidir si la obra es aceptada o no por la galería. En caso de que la obra sea aceptada, se registra la fecha de aceptación (tiene que ser posterior a la fecha de recepción de la solicitud). Una vez registrada la evaluación, no puede cambiarla. Cuando todas las obras de una solicitud son evaluadas, se elimina la solicitud.

Un Vendedor consultará en el sistema las obras aceptadas que no tienen fijado precio de venta aún y obtiene información sobre el artista que la hizo y la evaluación dada. Con esta información se pone en contacto con el artista y coordina un precio de venta para aquellas obras que el artista desea vender. Los resultados de su gestión lo registra en el sistema, para lo cual indica, de cada obra que desea vender, el precio de venta acordado y el porcentaje de la galería. Una obra que no quiso vender un artista en un momento puede querer hacerlo en otro momento.

El Especialista en economía debe actualizar al sistema con las obras que se han vendido y la fecha de venta. Automáticamente se enviará un correo electrónico informándole de este hecho al artista.

El Gerente de la galería solicita información sobre todas las obras registradas cuya fecha de presentación es anterior a una fecha dada y que no han sido vendidas. En esta consulta se muestran las características de las obras y el artista que las hizo. Hay que especificar de cada obra su estado (pendiente de evaluación, aceptada para la venta y exposición y aceptada para exponer).

El Gerente de la galería define los criterios que pueden usarse para evaluar obras de arte. De cada criterio se tiene su nombre y una descripción de los aspectos que deben tenerse en cuenta. Solo se contempla la inclusión de nuevos criterios.

El Sistema automatizado de RRHH se encarga del mantenimiento de las técnicas que puede utilizar un artista en una obra y en las que se especializa un especialista. De una técnica solo se replica si nombre y de los especialistas su nombre, provincia, las técnicas que usa y su e-mail. Cuando se asigna el especialista a una solicitud, se registran las técnicas en las que trabaja un artista y se registra las técnicas usadas en una obra, se consulta la información que mantiene este sistema y se replica solo la necesaria en cada caso.

Siguiendo la política de la Galería, se ha decidido que para el envío de correos se usará una aplicación externa.

Stakeholder

- Artistas.
- Anfitrión.
- Gerente general.
- Público en general que visita la galería o compra las obras que se exponen.
- Especialistas de arte.
- Especialista en economía.
- Vendedores.

Actores del negocio

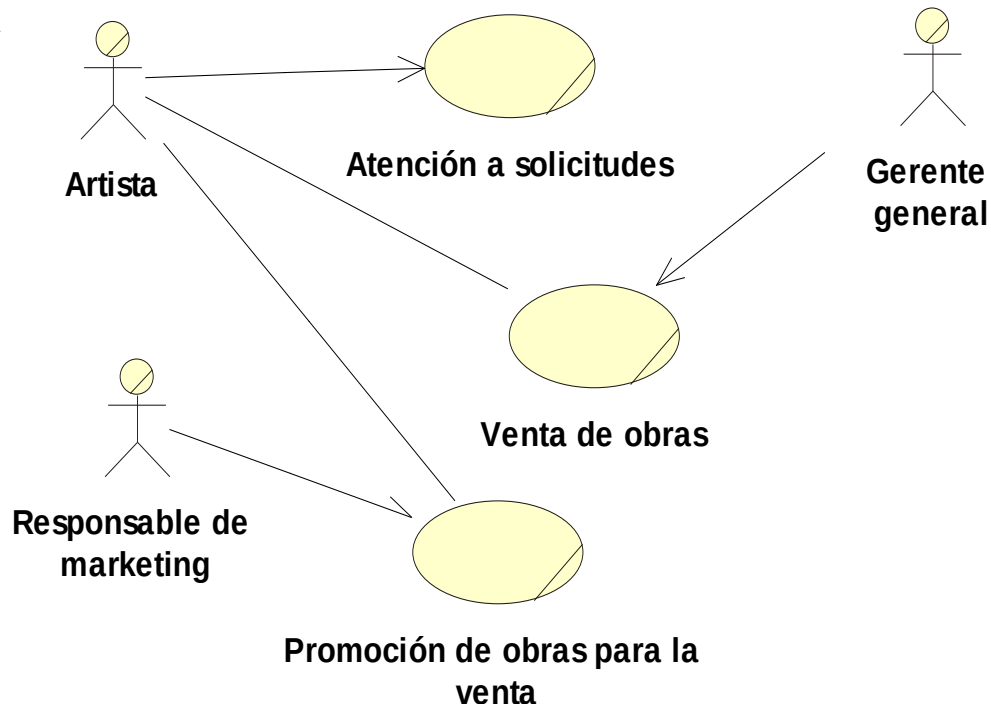
Artista	Interesado en que se expongan sus obras en la galería y posiblemente que sean vendidas.
Responsable de marketing	Es el Jefe del área de marketing por lo que está interesado en que se divulguen las obras de los artistas que estos desean vender para obtener las ganancias.

Gerente general	Interesado en que la galería promueva y venda sus obras por lo que exige que se tenga un control de las obras que se han vendido y las que no.
-----------------	--

Trabajadores del negocio

Anfitrión	Atiende a los artistas que desean exponer sus obras y les asigna un especialista de arte para que evalúe sus obras.
Sistema automatizado del área de Recursos Humanos (SA-RRHH)	Contiene información sobre las técnicas con las que trabaja la galería y los especialistas de arte que evalúan las obras, información que es utilizada en el proceso de atención a la solicitud.
Especialista de arte	Evalúa las obras de arte.
Vendedor	Divulga las obras aceptadas para vender que no han sido vendidas. Se pone en contacto con los artistas para fijar el precio de venta y el % de la galería.
Especialista en economía	Registra las ventas de obras que se producen.

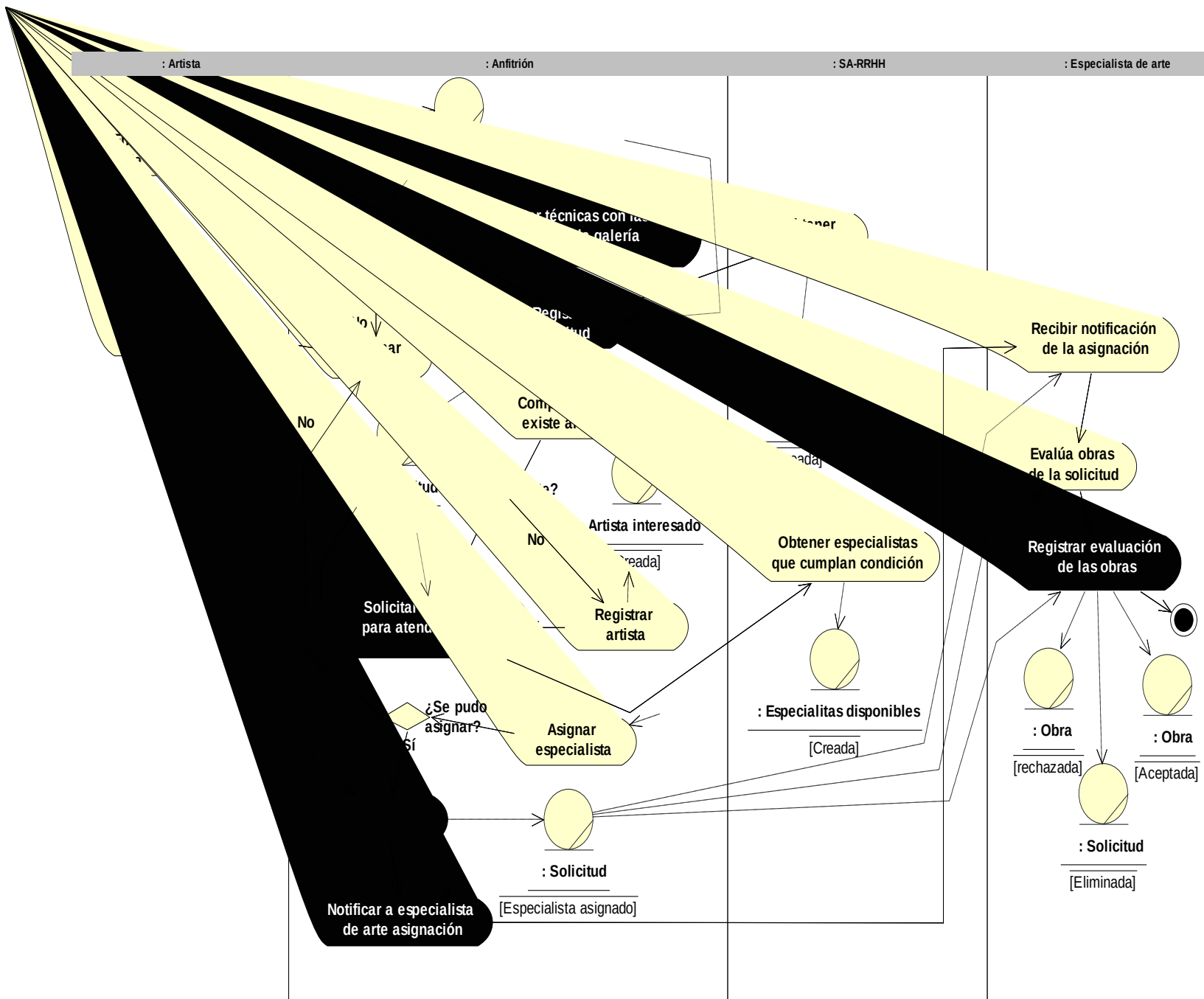
Diagrama de casos de uso del negocio

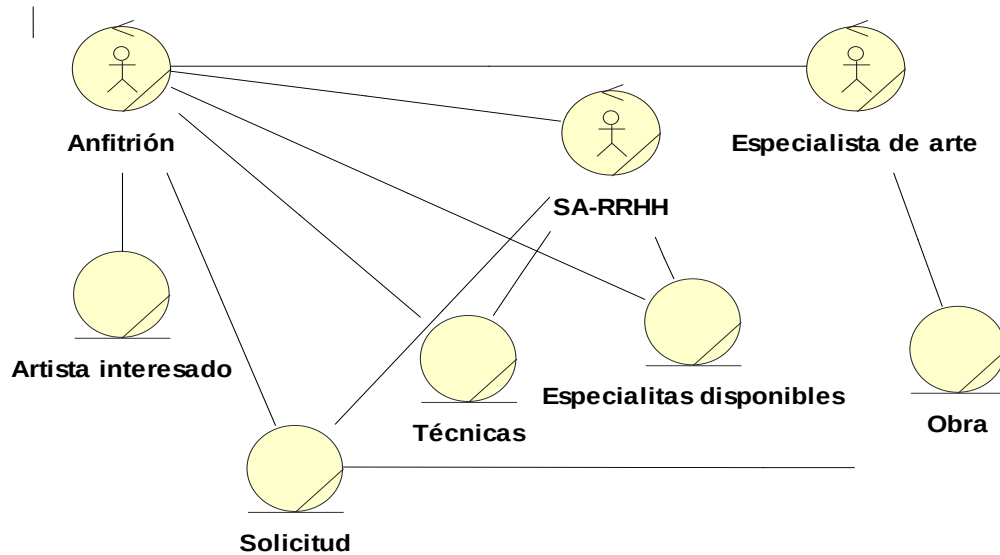


Realización de los casos de uso del negocio

CASO DE USO DEL NEGOCIO		ATENCIÓN A SOLICITUDES	
Actores	Artista (Inicia)		
Propósito	Registrar las solicitudes de los artistas asignándoles un especialistas que las evalúe, quién indicará si las obras que contienen son aceptadas o no por la galería para exponer.		
RESUMEN: El caso de uso se inicia cuando un artista se presenta en la galería con un conjunto de obras de arte que desea se tomen en cuenta para exponer en sus salas. El Anfitrión recepciona lo solicitud y revisa que contenga toda la información requerida para su valoración. En caso de que esté incompleta, la rechaza. En el caso contrario, se registran los datos del artista en caso de no existir y se le asigna un especialista de arte que de preferencia sea de la provincia de residencia y que domine las técnicas en las que se hicieron las obras. En este proceso de asignación se auxilia del sistema automatizado que existe en el área de Recursos Humanos. El caso de uso finaliza cuando el Especialista de arte indica si las obras que contiene la solicitud son aceptadas o rechazadas.			
ACCIÓN DEL ACTOR		RESPUESTA DEL PROCESO DE NEGOCIO	
1	El Artista entrega al Anfitrión una solicitud de exposición que contiene un conjunto de obras para que sean valoradas por la galería.	2	El Anfitrión revisa la solicitud buscando que esté completa. En caso de que esté completa, pasa al paso 4. En caso contrario, informa al Artista que su solicitud ha sido rechazada.
3	El Artista recibe el rechazo de la solicitud.	4	El Anfitrión solicita al SA-RRHH las técnicas con las que trabaja la galería.
		5	El SA-RRHH entrega la información sobre las técnicas con las que trabaja la galería.
		6	El Anfitrión registra la solicitud y comprueba que el artista exista. En caso negativo, lo registra.
		7	El Anfitrión solicita al SA-RRHH todos los especialistas de arte que se especializan en todas las técnicas que se usaron en la confección de las obras.
		8	El SA-RRHH entregan los especialistas de arte que cumplen la condición.
		9	El Anfitrión asigna el especialista de arte tratando de que sea de la misma provincia que el artista. Si no fue posible asignar al especialista, se informa al artista que la solicitud ha sido rechazada. Si fue posible asignar se registra la asignación y se pasa al paso 11.
10	El Artista recibe el rechazo de la solicitud.		

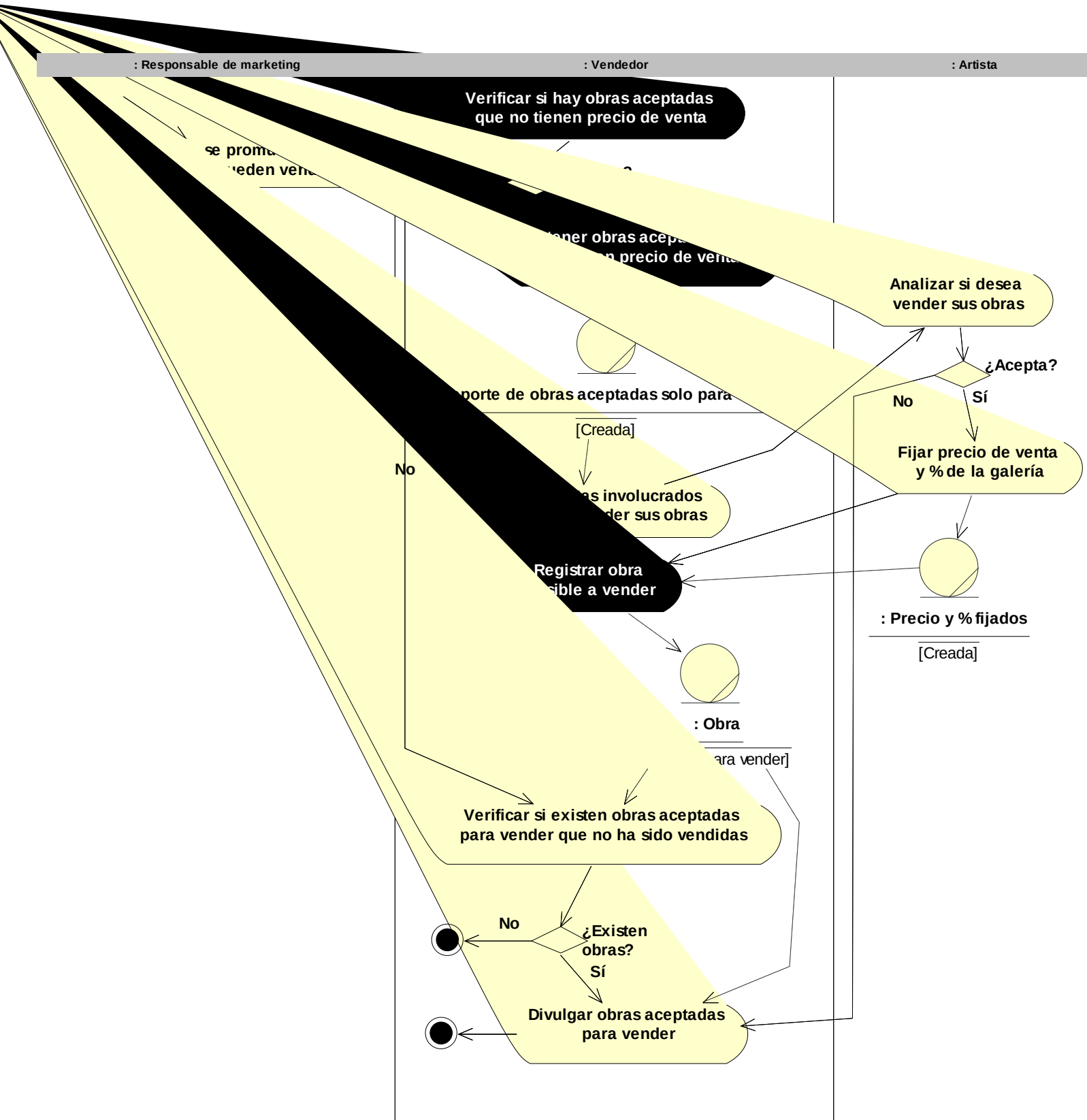
	11 El Anfitrión notifica al Especialista de arte que le han asignado una solicitud. 12 El Especialista de arte recibe la notificación de la asignación.
Prioridad	Responde al principal objetivo de automatización al resolver gran parte de los problemas actuales.
Mejoras	• Las notificaciones se harán vía correo electrónico. • El artista podrá registrar personalmente su solicitud por lo que el sitio en Internet debe obligarlo a indicar todos los datos para que se le acepte. Hay que permitir para estos casos que el anfitrión asigne el especialista de arte en otro momento. • El registro de las evaluaciones a las obras se hará vía Internet.
Otras secciones	-

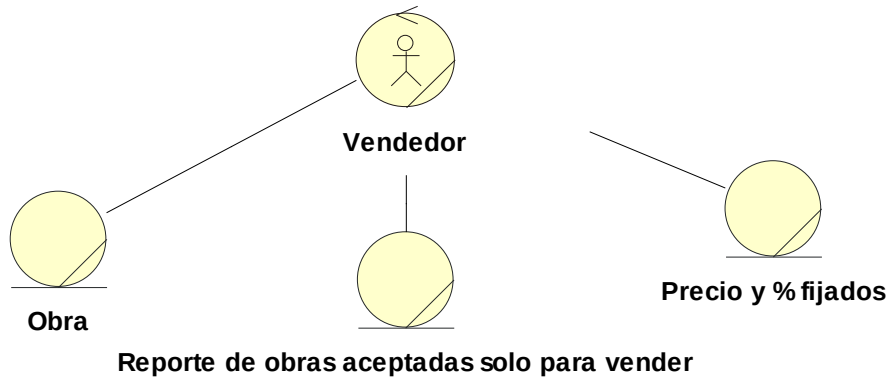




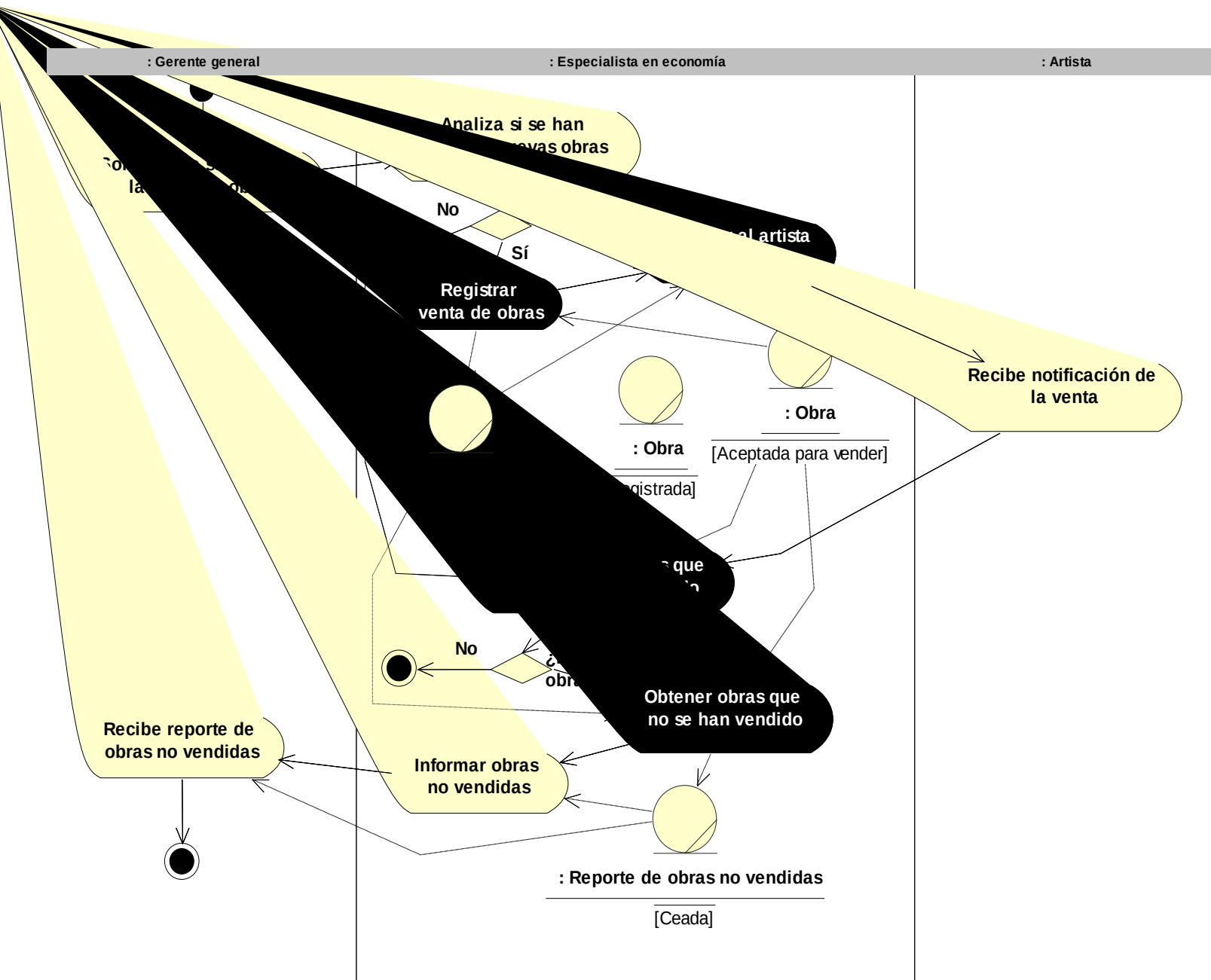
Caso de Uso del Negocio		PROMOCIÓN DE OBRAS PARA LA VENTA	
Actores	Responsable de marketing (Inicia), Artista		
Propósito	Divulgar las obras que los artistas aceptan vender.		
RESUMEN: El caso de uso se inicia cuando el Responsable de venta le indica al Vendedor que debe promover las obras que se pueden vender. En caso de existir obras aceptadas para exponer, pero que no están aceptadas para vender, el Vendedor consulta a los artistas que las hicieron para fijar el precio de venta y el % que le corresponde a la galería. El caso de uso finaliza divulgando las obras aceptadas para vender.			
ACCIÓN DEL ACTOR		RESPUESTA DEL PROCESO DE NEGOCIO	
1 El Responsable de marketing solicita al Vendedor que promueva las obras que se pueden vender.	4 El Artista fija el precio de venta y el % de la venta que le corresponde a la galería si decide vender.	2	El Vendedor verifica si hay obras aceptadas que no tienen precio de venta. En caso de que no existan, ir al paso 6. En caso contrario, obtener todas las obras que cumplan la condición.
		3	El Vendedor consulta a los artistas involucrados sobre si desean vender sus obras.
		5	En caso de que el artista desea vender sus obras, el Vendedor registra el precio de venta y el % de la galería. Y pasa al paso 7.
		6	El Vendedor verifica si existen obras aceptadas para vender que no han sido vendidas. En caso de no existir, finaliza el caso de uso.
		7	El Vendedor divulga las obras aceptadas para la venta.

Prioridad	-
Mejoras	-
Otras secciones	-





CASO DE USO DEL NEGOCIO		VENTA DE OBRAS	
Actores	Gerente general (Inicia), Artista		
Propósito	Controlas las obras que se han vendido.		
RESUMEN: El caso se uso se inicia cuando el Gerente de la galería solicita al Especialista en economía que controle la venta de las obras. El Especialista en economía registra las nuevas obras vendidas. El caso de uso finaliza cuando el Especialista en economía le entregar al Gerente de la galería un listado con las obras no vendidas indicando en qué estado está cada una.			
ACCIÓN DEL ACTOR		RESPUESTA DEL PROCESO DE NEGOCIO	
1	El Gerente general solicita que se controle la venta de las obras.	2	El Especialista en economía analiza si se han vendido nuevas obras. En caso positivo las registra. En caso negativo, pasa al paso 5.
4	El Artista se entera que una obra suya ha sido vendida.	3	El Especialista en economía notifica al artista de la venta.
		5	El Especialita en economía verifica si hay obras que no se han vendido. En caso negativo, finaliza el proceso.
7	El Gerente general recibe el reporte con todas las obras que no se han vendido.	6	El Especialista en economía obtiene todas las obras que no se han vendido y se lo informa al Gerente de la galería.
Prioridad	Es importante para la planificación del montaje de exposiciones que se tengan actualizadas las obras que se han vendido y las que no.		
Mejoras	Las notificaciones se harán vía correo electrónico.		
Otras secciones	-		



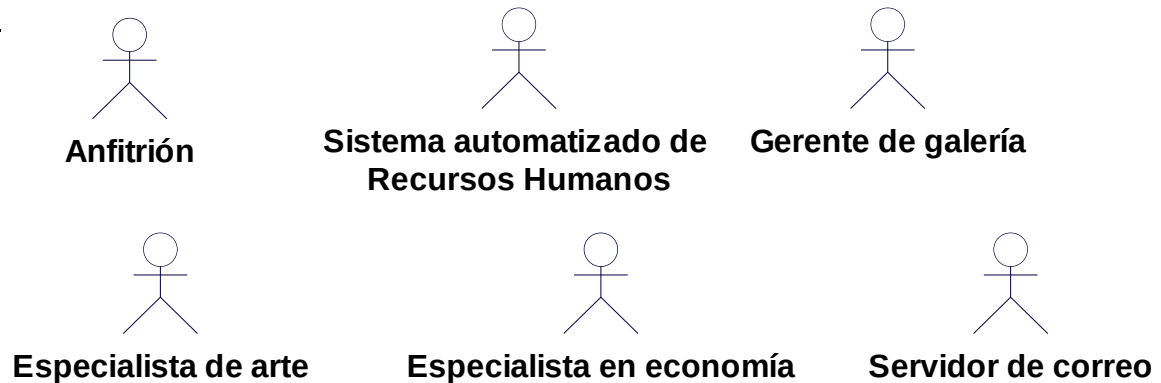
9.3 Requisitos

Requerimientos funcionales:

1. Registrar solicitud de exposición.
2. Asignar automáticamente código a una solicitud.
3. Asignar solicitud a un especialista de arte.
4. Enviar correo a especialista de arte sobre solicitud asignada.
5. Registrar información de artista.
6. Consultar en el sistema automatizado de RRHH los especialistas de arte.

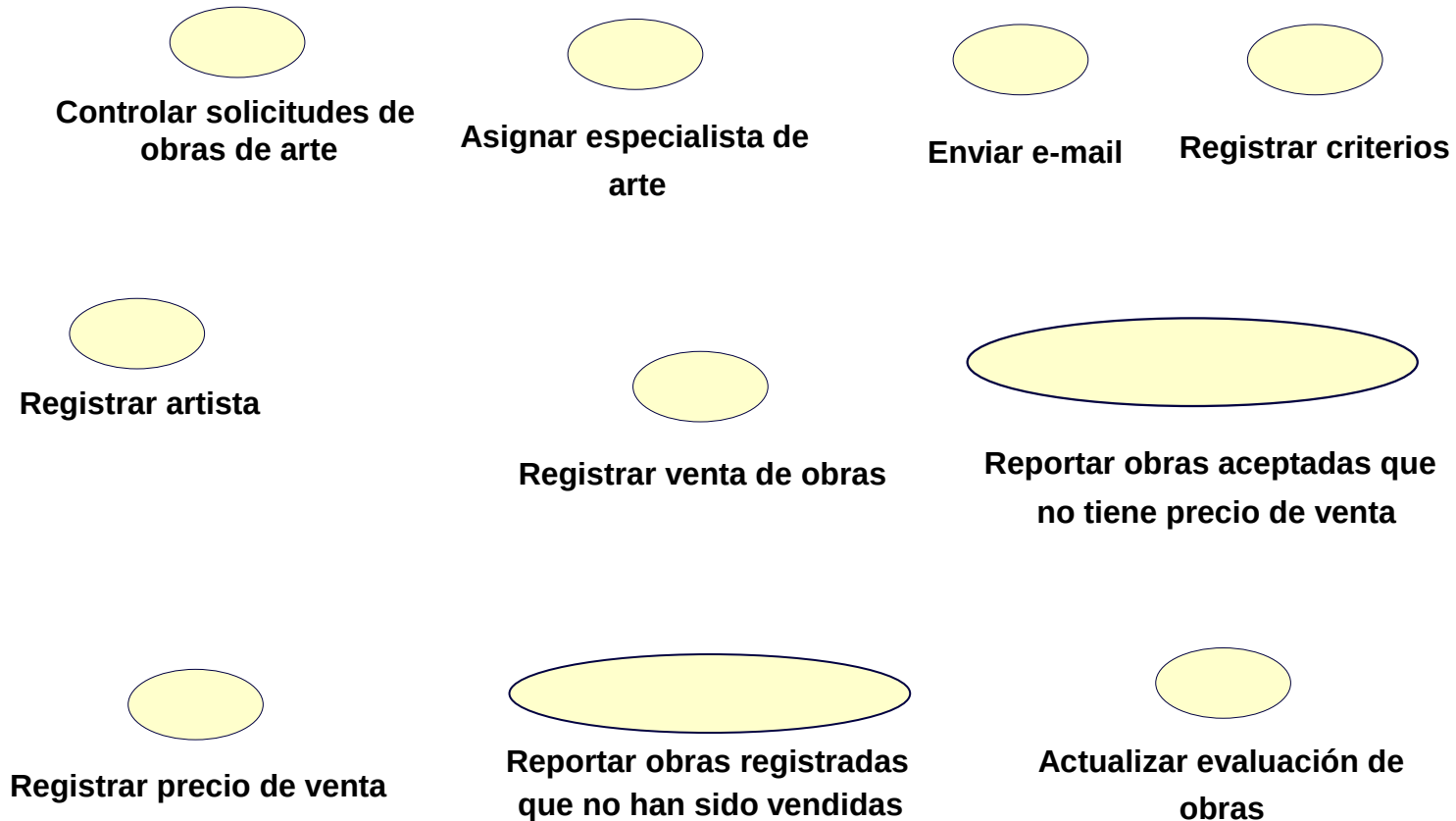
7. Registrar resultados de las evaluaciones a las obras de una solicitud.
8. Consultar obras aceptadas que no tienen precio.
9. Registrar precio de venta de obras aceptadas.
10. Registrar venta de obras.
11. Enviar correo a artista sobre la venta de sus obras.
12. Consultar información sobre obras registradas que no han sido vendidas.
13. Registrar criterios de evaluación

Actores:



Actores	Justificación
Anfitrión	Atiende a los artistas registrando en el sistema sus solicitudes de obras y asignándoles un especialista de arte. Registra información sobre los artistas que desean exponer en la galería o que a la galería le interesa que expongan.
Sistema automatizado de Recursos Humanos	Suministra los especialistas de arte y las técnicas en las que se especializan y usan los artistas.
Especialistas en economía	Consulta en el sistema las obras aceptadas que no tienen precio fijado y registra el precio de venta que pacta con los artistas en caso de que deseen venderlas. Actualiza el sistema cuando se efectúa una venta.
Especialista de arte	Registra los resultados de las evaluaciones que ha realizado a las obras contenidas en solicitudes a él asignadas, decidiendo si son aceptadas o no por la galería.
Gerente de la galería	Consulta en el sistema información sobre las obras registradas que no han sido vendidas.
Servidor de correo	Aplicación externa que se usa para el envío de correos.

Casos de uso:



Descripción de los casos de uso en formato de alto nivel:

Caso de uso:	Controlar solicitudes de obras de arte
Actores:	Anfitrión (Inicia)
Descripción:	El caso de uso se inicia cuando un artista va a una galería y presenta una solicitud con las obras que desea se expongan en la galería. Un Anfitrión registra la solicitud y le asigna un especialista de arte para que evalúe las obras. El caso de uso finaliza cuando el sistema, de forma automática, le envía un correo electrónico al especialista asignado con la información de la solicitud. En caso de que no estén registrados los datos del artista, se hace en ese momento. El Anfitrión puede eliminar una solicitud o modificar las obras que contiene, informando de ello al especialista asignado
Referencias:	R1, R2, R3, R4, R5 y R6 Casos de uso asociados: • Enviar e-mail (Extend). • Registrar artista (Extend). • Asignar especialista de arte (Include).
Precondiciones:	El sistema automatizado de RRHH ha replicado la información sobre las técnicas y los especialistas. Si no se ha replicado, el sistema presenta un mensaje al Anfitrión.
Poscondiciones:	Se registra(n) la(s) nueva(s) solicitud(es) o el(los) cambio(s) producidos informando al (los) Especialista(s) de arte

	involucrado(s). En el caso de las nuevas, se registra la solicitud y se envía el e-mail, solo si fue posible asignar especialistas.
Requerimientos especiales	-

Controlar solicitudes de obras

Solicitudes

Código	Fecha de registro	Artista	¿Obras evaluadas?
35439	20/05/2003	Roberto González	☐
28748	05/12/1999	René Raclift	☒

Agregar solicitud

Código: 35439
Fecha de registro: 25/05/2003

Artista

Nombre:

Dirección:

Edad: Técnicas que usa:

Sexo:

Obras

	Nombre de la obra	Fecha de venta	Dimensiones

Modificar solicitud

Código: 35439
 Fecha de registro: 20/05/2003

Artista

Nombre: Roberto González
 Dirección: Independencia 1487
 Edad: 45 Técnicas que usa:
 Sexo: Masculino Grabado

Obras

	Nombre de la obra	Fecha de venta	Dimensiones
1	Pasaje con río	23/08/1985	3.40x1.50
2	Vicuñas	17/02/1990	2.00x2.00

Σ Aregar Modificar Eliminar

✓ Aceptar ✗ Cancelar

Eliminar solicitud

¿Está seguro que desea eliminar la solicitud 354439,
 presentada por el artista Roberto González
 el 05/12/1999?

Sí No

Agregar obra

Nombre:

Fecha de realización:

Dimensiones:

Técnicas

Posibles a utilizar::

Relieve	☐
Collage	☐
Grabado	☐

Utiliza:

☒ Aceptar ☒ Cancelar

Modificar obra

Nombre: Vicuñas

Fecha de realización:

Dimensiones:

Técnicas

Posibles a utilizar::

Grabado	☐
Collage	☐

Utiliza:

☒ Aceptar ☒ Cancelar

Eliminar obra

¿Está seguro que desea eliminar la obra Vicuñas,
realizada el 17/02/1990 por el artista
Roberto González?

Caso de uso:	Enviar e-mail
Actores:	Servidor de correo
Descripción:	El caso de uso se inicia cuando algún proceso necesita enviar un correo electrónico, recibíéndose el mensaje a enviar y la dirección electrónica del destinatario. El sistema procede a conformar el mensaje y a través del Servidor de correo le envía el mensaje.
Referencias:	R4 y R11
Precondiciones:	Otro proceso requiere notificar vía correo electrónico sobre algo que ocurrió.
Poscondiciones:	-
Requerimientos especiales:	Si no fue posible enviar el correo por problemas de comunicación, hay que informar al usuario para que tome medidas alternativas.

Caso de uso:	Registrar artista
Actores:	Anfitrión (Inicia)
Descripción:	El caso de uso se inicia cuando al intentar registrar una nueva solicitud, se detecta que del artista que la presenta no están almacenados sus datos o cuando a la galería le interesa registrar información sobre artistas que desea expongan en la galería. En estos casos, el Anfitrión registra la información de un nuevo artista o los cambios que se produzcan en sus datos. El caso de uso finaliza con el registro de artistas actualizado. Si se va a eliminar a un artista, se verifica si está asociado a una obra, en cuyo caso no se realiza la eliminación; en cualquier otro caso se procede a la eliminación física.
Referencias:	R5
Precondiciones:	El sistema automatizado de RRHH ha replicado las técnicas. Si no se ha replicado, el sistema presenta un mensaje al Anfitrión.
Poscondiciones:	Se actualiza el registro de artistas almacenando los datos del nuevo artista, la modificación de los datos registrados o la eliminación física de una artista si es posible.
Requerimientos especiales	-

Registrar artista

Artistas

	Nombre	Dirección	e-mail	Edad	Sexo
1	Roberto González	Independencia 1487	rogo@yahoo.com	45	Masculino
2	René Raclift	Los Alamos 982374	rraclif@hotmail.com	33	Masculino

Agregar artista

Nombre:

Dirección:

e-mail:

Edad:

Sexo: ☒ Masculino ☐ Femenino

Técnicas

Posibles a utilizar::

Grabado	
Relieve	
Collage	

Utiliza:

 Aceptar  Cancelar

Modificar artista

Nombre: Roberto González

Dirección:

e-mail:

Edad: Sexo: Masculino

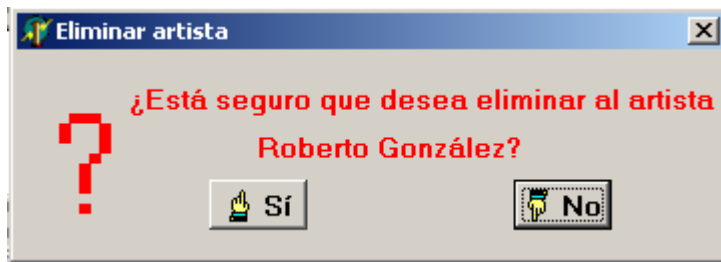
Técnicas

Posibles a utilizar::

Relieve	
Collage	

Utiliza:

 Aceptar  Cancelar



Caso de uso:	Asignar especialista de arte
Actores:	Anfitrión y Sistema Automatizado de RRHH
Descripción:	El caso de uso se inicia cuando se registra una nueva solicitud, automáticamente el sistema consulta al Sistema Automatizado de RRHH y extrae todos los especialistas de arte que se especializan en las mismas técnicas de las obras que contiene la solicitud. Si no hay especialistas que cumplan el requerimiento, finaliza el caso de uso. En caso contrario, el Anfitrión selecciona de los disponibles a un especialista, culminando la ejecución del caso de uso.
Referencias:	R3 y R6
Precondiciones:	Se registra una nueva solicitud.
Poscondiciones:	Se asocia un especialista de arte a una solicitud en caso de que existan especialistas en las técnicas de las obras de la solicitud.
Requerimientos especiales:	Si no fue posible relacionarse con el Sistema Automatizado de RRHH, hay que informar al usuario para que tome medidas.



Caso de uso:	Actualizar evaluación de obras
Actores:	Especialista de arte (Inicia)
Descripción:	El caso de uso se inicia cuando el especialista de arte va a actualizar la evaluación que ha dado a las obras de solicitudes a él asignadas. Al final consigna para las obras si serán o no aceptadas para su exposición en la galería. Si todas las obras de una solicitud han sido evaluadas, se elimina la solicitud y la asignación de especialistas.
Referencias:	R7
Precondiciones:	El especialista de arte tiene solicitudes asignadas con obras pendientes de su evaluación y estén registrados los criterios de evaluación. En caso de que no se cumpla alguna precondición, el sistema presenta un mensaje al Especialista de arte.
Postcondiciones:	Se ha registrado de las obras su evaluación, pasándolas a aceptadas para exponer o rechazadas para exponer en la galería. Se eliminan las solicitudes que tienen todas las obras evaluadas y la asignación de especialistas.
Requerimientos especiales:	Un especialista de arte solo tendrá acceso a las obras contenidas en solicitudes a él asignadas.

Actualizar evaluación de obras

Solicitudes pendientes

Código	Fecha de registro	Artista
35439	20/05/2003	Roberto González
28748	05/12/1999	René Raclift

Evaluar Salir

Evaluar

Obras

	Nombre de la obra	Dimensiones	Fecha de realización	¿Evaluada?
	Margaritas	3.45x4.50	23/09/1997	☐
	Comida campestre	1.50x2.56	01/01/1999	☒

Evaluación

Criterios

Tema

Colores

Criterio	Evaluación

Aceptar

Resultado final

☐ Aceptar obra

☐ Rechazar obra

Salir

Caso de uso:	Registrar precio de venta
Actores:	Especialista en economía (Inicia)
Descripción:	El caso de uso se inicia cuando el especialista en economía ha pactado un precio de venta con el artista y registra en el sistema este valor para la(s) obra(s) del autor y el % que le corresponde a la galería por la venta, cambiando el estado de la obra de aceptada para exponer a aceptada para exponer y vender.
Referencias:	R9
Precondiciones:	Hay artista que tienen obras aceptadas para exponer que no tienen precio de venta. Si no hay obras que cumplan la condición, el sistema presenta un mensaje al Especialista en economía.
Postcondiciones:	Se registra el precio de venta de las obras pasándose la obra de aceptada para exponer a aceptada para la venta y la exposición.
Requerimientos especiales:	-

Registrar precio de venta

Artistas:

Roberto González

René Raclift

Obras

	Nombre de la obra	Precio de venta	% de la galería
	Paisaje con río		
	Vicuñas		

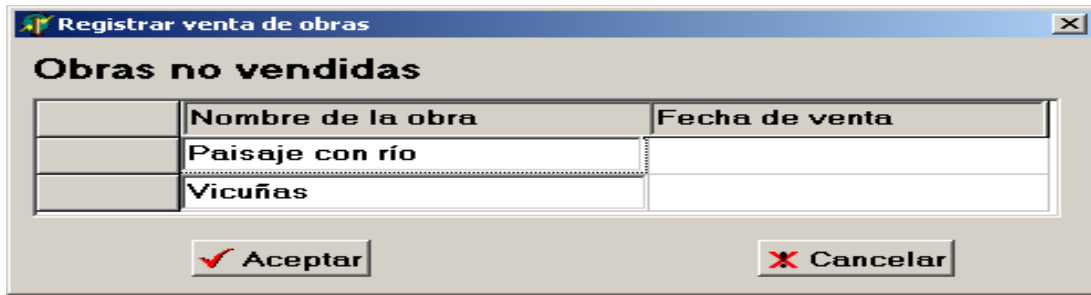
✓ Aceptar

✗ Cancelar

Fijar precio

Salir

Caso de uso:	Registrar venta de obras
Actores:	Especialista en economía (Inicia)
Descripción:	El caso de uso se inicia cuando se vende una obra y el especialista en economía actualiza esta información en el sistema. Solo es posible registrar la venta de obras que tiene fijado precio de venta y que no han sido vendidas. El caso de uso finaliza enviándole un correo al artista que hizo la obra.
Referencias:	R10 y R11 Casos de uso asociados • Enviar e-mail
Precondiciones:	Existen obras con precio de venta pactado que no han sido vendidas. Si no existen, el sistema presenta un mensaje al Especialista en economía.
Postcondiciones:	Se registra la fecha de venta de la obra vendida, pasándose de aceptada para exponer y vender a obra vendida.
Requerimientos especiales:	-



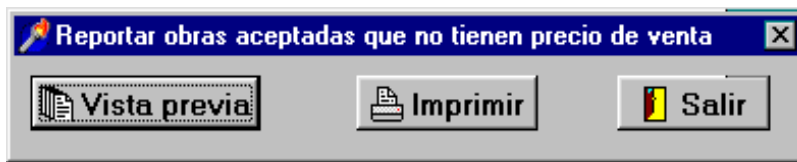
Registrar venta de obras

Obras no vendidas

Nombre de la obra	Fecha de venta
Paisaje con río	
Vicufias	

✓ Aceptar ✗ Cancelar

Caso de uso:	Reportar obras aceptadas que no tienen precio de venta
Actores:	Especialista en economía (Inicia)
Descripción:	El caso de uso se inicia cuando el especialista en economía necesita ponerse en contacto con los artistas que tiene obras aceptadas para exponer sin precio de venta fijado; para lo cual consulta al sistema y obtiene un reporte del artista con las obras.
Referencias:	R8
Precondiciones:	Hay obras aceptadas para exponer que no tiene precio de venta fijado. Si no obras que cumplan la condición, el sistema presenta un mensaje al Especialista en economía.
Postcondiciones:	-
Requerimientos especiales:	



ARTE CUBANO

GALERÍA DE ARTE

Reporte de obras aceptadas que no tienen precio de venta

Artista: _____
 Dirección: _____
 Sexo: Masculino ☐ Femenino ☐
 e-mail: _____
 Técnicas que usa: _____

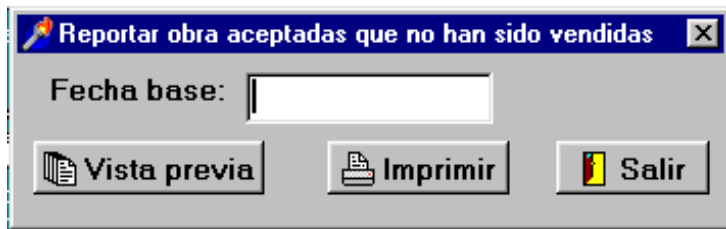
Obras:

Nombre:	
Fecha de realización:	Dimensiones:
Técnicas usadas:	
Evaluaciones	
Criterio	Resultado

Nombre:	
Fecha de realización:	Dimensiones:
Técnicas usadas:	
Evaluaciones	
Criterio	Resultado

Caso de uso:	Reportar obras registradas que no han sido vendidas
Actores:	Gerente de la galería (Inicia)

Descripción: El caso de uso se inicia cuando el Gerente de la galería necesita conocer las características; de las obras y del artista que las hizo; de las obras que están pendientes de evaluación, aceptadas para exponer y aceptadas para exponer y vender, cuya fecha de presentación es anterior a una fecha dada.	
Referencias:	R12
Precondiciones:	Existen obras registradas y no rechazadas que no han sido vendidas y presentadas a la galería en una fecha anterior a la especificada. Si no existen obras que cumplan la condición, el sistema presenta un mensaje al Gerente de la galería.
Postcondiciones:	-
Requerimientos especiales:	



ARTE CUBANO

GALERÍA DE ARTE

Reporte de obras registradas que no han sido vendidas

Fecha de referencia: _____

Obra	Artista	Fecha de recibo	Estado

Caso de uso:	Registrar criterio
Actores:	Gerente de la galería (Inicia)
Descripción: El caso de uso se inicia cuando el Gerente de la galería define un nuevo criterio que puede tenerse en cuenta cuando se evalúa la obra, terminando el caso de uso con el criterio registrado.	

Referencias:	R13
Precondiciones:	-
Postcondiciones:	Se registra el nuevo criterio.
Requerimientos especiales:	-

Registrar criterio

Criterios

	Nombre	Descripción
1	Tema	Correspondencia con la realidad nacional
2	Colores	Combinación

Agregar Salir

Agregar criterio

Nombre:

Descripción:

Aceptar Cancelar

9.4 Análisis

Descripción de los casos de uso en formato expandido:

Caso de uso:	Controlar solicitudes de obras de arte
Actores:	Anfitrión (Inicia)
Descripción:	El caso de uso se inicia cuando un artista va a una galería y presenta una solicitud con las obras que desea se expongan en la galería. Un Anfitrión registra la solicitud y le asigna un especialista de arte para que evalúe las obras. El caso de uso finaliza cuando el sistema, de forma automática, le envía un correo electrónico al especialista asignado con la información de la solicitud. En caso de que no estén registrados los datos del artista, se hace en ese momento. El Anfitrión puede eliminar una solicitud o modificar las obras que contiene, informando de ello al especialista asignado.
Referencias:	R1, R2, R3, R4, R5 y R6 Casos de uso asociados: • Enviar e-mail (Extend). • Asignar especialista de arte (Include). • Registrar artista (Extend).
Precondiciones:	El sistema automatizado de RRHH ha replicado la información sobre las técnicas y los especialistas. Si no se ha replicado, el sistema presenta un mensaje al Anfitrión.
Poscondiciones:	Se registra(n) la(s) nueva(s) solicitud(es) o el(los) cambio(s) producidos informando al (los) Especialista(s) de arte involucrado(s). En el caso de las nuevas, se registra la solicitud y se envía el e-mail, solo si fue posible asignar especialistas.

Requerimientos especiales		-
Curso normal de los eventos		
Acción del actor		Respuesta del sistema
1 El Anfitrión necesita registrar una nueva solicitud, eliminar una solicitud o modificar las obras que contiene una solicitud.	2 El sistema ejecuta alguna de las siguientes secciones: a) Si decide registrar una nueva solicitud, ir a la sección "Nueva." b) Si decide eliminar una solicitud, ir a la sección "Eliminar". c) Si decide modificar las obras contenidas en una solicitud, ir a la sección "Modificar".	
3 El Anfitrión termina el registro de solicitudes.	4 El sistema finaliza la ejecución del caso de uso.	
Sección "Nueva"		
Acción del actor		Respuesta del sistema
	1 El sistema genera un código para la solicitud.	
2 El Anfitrión especifica el artista que hizo la solicitud o decide registrar un nuevo artista.	3 El sistema ejecuta alguna de las siguientes acciones: a) Si selecciona un artista de los ya registrados, se extrae el resto de sus características y se le presentan al anfitrión. Pasar a la línea 3. b) Si decide registrar un nuevo artista, extender el caso de uso "Registrar artista", que actualiza los artistas. Pasar a la línea 1.	
4 El Anfitrión define si va a incluir una nueva obra en la solicitud, eliminar una obra ya incluida o modificar las características de una obra.	5 El sistema ejecuta alguna de las siguientes acciones: a) Si decide incluir una nueva obra en la solicitud, ir a la sección "Nueva obra". b) Si decide eliminar una obra de la solicitud, ir a la sección "Eliminar obra". c) Si decide modificar las características de una obra, ir a la sección "Modificar obra".	
6 El Anfitrión termina de registrar los datos de la nueva solicitud.	7 El sistema verifica si no hay otra solicitud registrada del artista para ese día. En caso negativo, incluye el caso de uso "Asignar especialistas de arte", para lo cual le envía las técnicas usadas en la confección de las obras de la solicitud y recibe el especialista asignado	
	8 Si fue posible asignar especialista a la solicitud, el sistema: 8.1 Busca el correo electrónico del especialista asignado. 8.2 Extiende el caso de uso "Enviar correo", para lo cual le da como información una dirección electrónica y la información contenida en la solicitud. 8.3 Registra la solicitud, consignando como fecha de registro la fecha actual.	
Sección "Modificar"		
Acción del actor		Respuesta del sistema
1 El Anfitrión selecciona la solicitud que desea modificarle las obras que contiene.	2 El sistema verifica que en la solicitud no existan obras ya evaluadas.	

3	El Anfitrión define si va a incluir una nueva obra en la solicitud, eliminar una obra ya incluida o modificar las características de una obra.	4	El sistema ejecuta alguna de las siguientes acciones: a) Si decide incluir una nueva obra en la solicitud, ir a la sección "Nueva obra". b) Si decide eliminar una obra de la solicitud, ir a la sección "Eliminar obra". c) Si decide modificar las características de una obra, ir a la sección "Modificar obra".
5	El Anfitrión termina de modificar las obras registradas en la solicitud.	6	El sistema busca el correo electrónico del especialista asignado y extiende el caso de uso "Enviar correo", para lo cual le da como información una dirección electrónica y la información que se ha modificado.
		7	Registrar los cambios a la solicitud.
Sección "Eliminar"			
Acción del actor		Respuesta del sistema	
1	El Anfitrión selecciona la solicitud que desea eliminar.	2	El Sistema verifica que en la solicitud no existan obras ya evaluadas.
3	El Anfitrión confirma la eliminación de la solicitud.	4	El Sistema busca el correo electrónico del especialista asignado y extiende el caso de uso "Enviar e-mail", para lo cual le da como información una dirección electrónica y la solicitud que se ha eliminado.
		5	El sistema elimina la solicitud y la asignación del especialista.
Sección "Nueva obra"			
Acción del actor		Respuesta del sistema	
1	El Anfitrión especifica las características de la nueva obra.	2	El sistema verifica que la obra no esté registrada dentro de una solicitud o aceptada por la galería. En caso de que no exista, se incluye la obra en la solicitud.
Sección "Modificar obra"			
Acción del actor		Respuesta del sistema	
1	El Anfitrión selecciona la obra que desea modificar sus características.	2	El sistema presenta las características registradas de la obra.
3	El Anfitrión especifica las características que cambian de la obra.	4	El sistema incluye los cambios a la obra en la solicitud.
Sección "Eliminar obra"			
Acción del actor		Respuesta del sistema	
1	El Anfitrión selecciona la obra que desea eliminar.	2	El sistema presenta las características registradas de la obra.
3	El Anfitrión confirma la eliminación de la obra.	4	El sistema elimina la obra de la solicitud.
Cursos alternos			

Sección "Nueva" : Línea 8

Si no fue posible asignar un especialista, finaliza la ejecución de la sección y se regresa a la línea 1 de la sección.

Sección "Nueva": Línea 8

Si estaba registrada otra solicitud del artista para ese día, el sistema presenta un mensaje al anfitrión y se regresa el control a la línea 2 de la sección.

Sección "Modificar" : Línea 2

Si la solicitud seleccionada tiene obras ya evaluadas, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección.

Sección "Eliminar" : Línea 2

Si la solicitud seleccionada tiene obras ya evaluadas, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección.

Sección "Nueva obra" : Línea 2

Si la obra ya esté registrada, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección.

Caso de uso:	Enviar e-mail		
Actores:	Servidor de correo		
Descripción: El caso de uso se inicia cuando algún proceso necesita enviar un correo electrónico, recibándose el mensaje a enviar y la dirección electrónica del destinatario. El sistema procede a conformar el mensaje y a través del Servidor de correo le envía el mensaje.			
Referencias:	R4 y R11		
Precondiciones:	Otro proceso requiere notificar vía correo electrónico sobre algo que ocurrió.		
Poscondiciones:	-		
Requerimientos especiales:		Si no fue posible enviar el correo por problemas de comunicación, hay que informar al usuario para que tome medidas alternativas.	
Curso normal de los eventos			
Acción del actor		Respuesta del sistema	
		1	El sistema conforma el mensaje con la dirección electrónica y el contenido a enviar y solicita a un Servidor de correo que envíe el mensaje.
2	El Servidor de correo envía el mensaje a la dirección indicada.	3	El sistema finaliza la ejecución del caso de uso y devuelve el control al caso de uso que lo invocó.

Caso de uso:	Registrar artista
Actores:	Anfitrión (Inicia)
Descripción: El caso de uso se inicia cuando al intentar registrar una nueva solicitud, se detecta que del artista que la presenta no están almacenados sus datos o cuando a la galería le interesa registrar información sobre artistas que desea expongan en la galería. En estos casos, el Anfitrión registra la información de un nuevo artista o los cambios que se produzcan en sus datos. El caso de uso finaliza con el registro de artistas actualizado. Si se va a eliminar a un artista, se verifica si está asociado a una obra, en cuyo caso no se realiza la eliminación; en	

cualquier otro caso se procede a la eliminación física.	
Referencias:	R5
Precondiciones:	El sistema automatizado de RRHH ha replicado las técnicas. Si no se ha replicado, el sistema presenta un mensaje al Anfitrión.
Poscondiciones:	Se actualiza el registro de artistas almacenando los datos del nuevo artista, la modificación de los datos registrados o la eliminación física de una artista si es posible.
Requerimientos especiales	-
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
1 El Anfitrión necesita registrar un nuevo artista, modificar las características de un artista o eliminar un artista.	2 El sistema ejecuta alguna de las siguientes acciones: j) Si decide registrar un nuevo artista, ir a la sección "Nuevo". k) Si decide modificar las características de un artista, ir a la sección "Modificar". l) Si decide eliminar un artista, ir a la sección "Eliminar".
3 El Anfitrión termina el registro de artistas.	4 El sistema finaliza la ejecución del caso de uso.
Sección "Nuevo"	
Acción del actor	Respuesta del sistema
1 El Anfitrión especifica las características de un nuevo artista.	2 El sistema verifica que el artista no esté registrado. En caso de que no exista, se registra el artista.
Sección "Modificar "	
Acción del actor	Respuesta del sistema
1 El Anfitrión selecciona al artista que desea modificar sus características.	2 El sistema presenta las características registradas del artista.
3 El Anfitrión especifica las características que cambian del artista.	4 El sistema incluye los cambios al artista.
Sección "Eliminar "	
Acción del actor	Respuesta del sistema
1 El Anfitrión selecciona el artista que desea eliminar.	2 El sistema verifica si el artista está asociado a una obra. En caso de que no esté asociado, el sistema presenta las características registradas del artista.
3 El Anfitrión confirma la eliminación del artista	4 El sistema elimina el artista.
Cursos alternos	
Sección "Nuevo" : Línea 2 Si el artista ya está registrado, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección.	
Sección "Eliminar" : Línea 2 Si el artista seleccionado tiene obras registradas, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección.	

Caso de uso:	Asignar especialista de arte
Actores:	Anfitrión y Sistema Automatizado de RRHH
Descripción: El caso de uso se inicia cuando se registra una nueva solicitud, automáticamente el sistema consulta al Sistema Automatizado de RRHH y extrae todos los especialistas de arte que se especializan en las mismas técnicas de las obras que contiene la solicitud. Si no hay especialistas que cumplan el requerimiento, finaliza el caso de uso. En caso contrario, el Anfitrión selecciona de los disponibles a un especialista, culminando la ejecución del caso de uso.	
Referencias:	R3 y R6
Precondiciones:	Se registra una nueva solicitud.
Poscondiciones:	Se asocia un especialista de arte a una solicitud en caso de que existan especialistas en las técnicas de las obras de la solicitud.
Requerimientos especiales:	Si no fue posible relacionarse con el Sistema Automatizado de RRHH, hay que informar al usuario para que tome medidas.
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
	1 El sistema solicita al Sistema Automatizado de RRHH todos los especialistas que se especializan en determinadas técnicas.
2 El Sistema Automatizado de RRHH selecciona todos los especialistas de arte que cumplen la condición.	3 En caso de que existan especialista que cumplan la condición, el sistema presenta la lista de especialistas.
4 El Anfitrión selecciona un especialista.	5 El sistema asocia el especialista a la solicitud, finaliza la ejecución del caso de uso y devuelve el control al caso de uso que lo invocó con el especialista asignado.
Cursos alternos	
Curso normal de los eventos : Línea 3 Si no se encontró especialistas que cumplieran la condición, el sistema presenta un mensaje al Anfitrión y finaliza el caso de uso devolviendo el control al caso de uso que lo invocó.	

Caso de uso:	Actualizar evaluación de obras
Actores:	Especialista de arte (Inicia)
Descripción: El caso de uso se inicia cuando el especialista de arte va a actualizar la evaluación que ha dado a las obras de solicitudes a él asignadas. Al final consigna para las obras si serán o no aceptadas para su exposición en la galería. Si todas las obras de una solicitud han sido evaluadas, se elimina la solicitud y la asignación de especialistas.	
Referencias:	R7
Precondiciones:	El especialista de arte tiene solicitudes asignadas con obras pendientes de su evaluación y estén registrados los criterios de evaluación. En caso de que no se cumpla alguna precondición, el sistema presenta un mensaje al Especialista de arte.

Poscondiciones:	Se ha registrado de las obras su evaluación, pasándolas a aceptadas para exponer o rechazadas para exponer en la galería. Se eliminan las solicitudes que tienen todas las obras evaluadas y la asignación de especialistas.		
Requerimientos especiales:	Un especialista de arte solo tendrá acceso a las obras contenidas en solicitudes a él asignadas.		
Curso normal de los eventos			
Acción del actor		Respuesta del sistema	
1	El especialista de arte elige una de las solicitudes a él asignadas para evaluar las obras que contiene.	2	El Sistema presenta todas las obras contenidas en la solicitud seleccionada que tienen obras pendientes de evaluación.
3	El especialista de arte decide evaluar una obra.	4	El sistema presenta todos los posibles criterios de evaluación.
5	El especialista de arte especifica el resultado de su evaluación para cada criterio que evalúo y si acepta o rechaza la obra.	6	Si se ha especificado si la obra fue aceptada o no, el sistema registra el resultado de la evaluación y pasa la obra de pendiente a aceptada o rechazada para exponer.
7	El especialista de arte finaliza el registro de evaluaciones.	8	El sistema revisa si existen solicitudes completamente evaluadas. En caso positivo, las elimina al igual que la asignación del especialista y finaliza la ejecución del caso de uso.
Cursos alternos			
"Curso normal de los eventos" : Línea 8 Si no hay solicitudes en las que todas sus obras han sido evaluadas, finaliza la ejecución de caso de uso.			
"Curso normal de los eventos": Línea 6 Si no se especificó si la obra es aceptada o no como parte de su evaluación, el sistema presenta un mensaje al Especialista de arte y se devuelve el control a la línea 5.			

Caso de uso:	Registrar criterio		
Actores:	Gerente de la galería (Inicia)		
Descripción:	El caso de uso se inicia cuando el Gerente de la galería define un nuevo criterio que puede tenerse en cuenta cuando se evalúa la obra, terminando el caso de uso con el criterio registrado.		
Referencias:	R13		
Precondiciones:	-		
Postcondiciones:	Se registra el nuevo criterio.		
Requerimientos especiales:	-		
Curso normal de los eventos			
Acción del actor		Respuesta del sistema	
1	El Gerente de la Galería necesita registrar un nuevo criterio y especifica sus características.	2	El sistema verifica que el criterio no exista. En caso de que no exista, se registra el criterio.
3	El Gerente de la Galería finaliza el registro de criterios.	4	El sistema finaliza la ejecución del caso de uso.

Cursos alternos
"Curso normal de los eventos" : Línea 2 Si el criterio ya está registrado, el sistema presenta un mensaje al Gerente de la galería y regresa a la línea 1 del curso normal de los eventos.

Caso de uso:	Registrar precio de venta
Actores:	Especialista en economía (Inicia)
Descripción:	El caso de uso se inicia cuando el especialista en economía ha pactado un precio de venta con el artista y registra en el sistema este valor para la(s) obra(s) del autor y el % que le corresponde a la galería por la venta, cambiando el estado de la obra de aceptada para exponer a aceptada para exponer y vender.
Referencias:	R9
Precondiciones:	Hay artista que tienen obras aceptadas para exponer que no tienen precio de venta. Si no hay obras que cumplan la condición, el sistema presenta un mensaje al Especialista en economía.
Postcondiciones:	Se registra el precio de venta de las obras pasándose la obra de aceptada para exponer a aceptada para la venta y la exposición.
Requerimientos especiales:	-

Curso normal de los eventos	
Acción del actor	Respuesta del sistema
1 El Especialista en economía selecciona al artista con el que contactó para poner precio a sus obras.	2 El sistema presenta las obras del artista que están aceptadas para exponer, pero no tienen fijado precio de venta.
3 El Especialista en economía especifica, para las obras que aceptó vender el artista, el precio de venta y el % de la galería.	4 El sistema pasa las obras indicadas de aceptadas para exponer a aceptadas para exponer y vender, registrando el precio de venta y el % para la galería de cada una de ellas.
5 El Especialista en economía termina el registro de precios de venta.	6 El sistema finaliza el caso de uso.
Cursos alternos	
-	

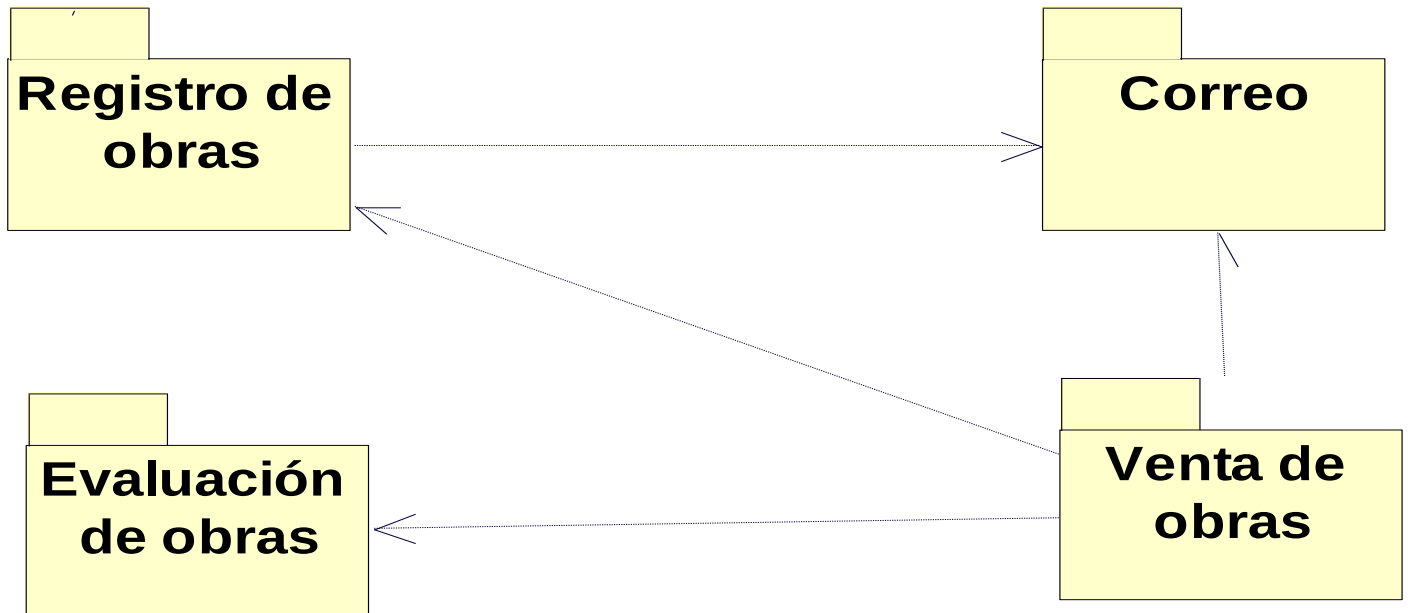
Caso de uso:	Registrar venta de obras
Actores:	Especialista en economía (Inicia)
Descripción:	El caso de uso se inicia cuando se vende una obra y el especialista en economía actualiza esta información en el sistema. Solo es posible registrar la venta de obras que tiene fijado precio de venta y que no han sido vendidas. El caso de uso finaliza enviándole un correo al artista que hizo la obra.
Referencias:	R10 y R11 Casos de uso asociados • Enviar e-mail
Precondiciones:	Existen obras con precio de venta pactado que no han sido vendidas. Si no existen, el sistema presenta un mensaje al Especialista en economía.

Postcondiciones:	Se registra la fecha de venta de la obra vendida, pasándose de aceptada para exponer y vender a obra vendida.
Requerimientos especiales:	-
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
1 El Especialista en economía especifica, de entre las obras aceptadas para vender y exponer, las obras que han sido vendidas y la fecha de venta de cada una.	2 El sistema registra, para las obras indicadas, su fecha de venta y cambia el estado de aceptadas para exponer y vender a obras vendidas. Para cada artista involucrado en la venta, extiende el caso de uso "Enviar correo", para lo cual le da como información la dirección electrónica de artista e información de la (s) obra (s) vendida(s).
3 El Especialista en economía termina el registro de obras vendidas.	4 El sistema finaliza el caso de uso.
Cursos alternos	
-	

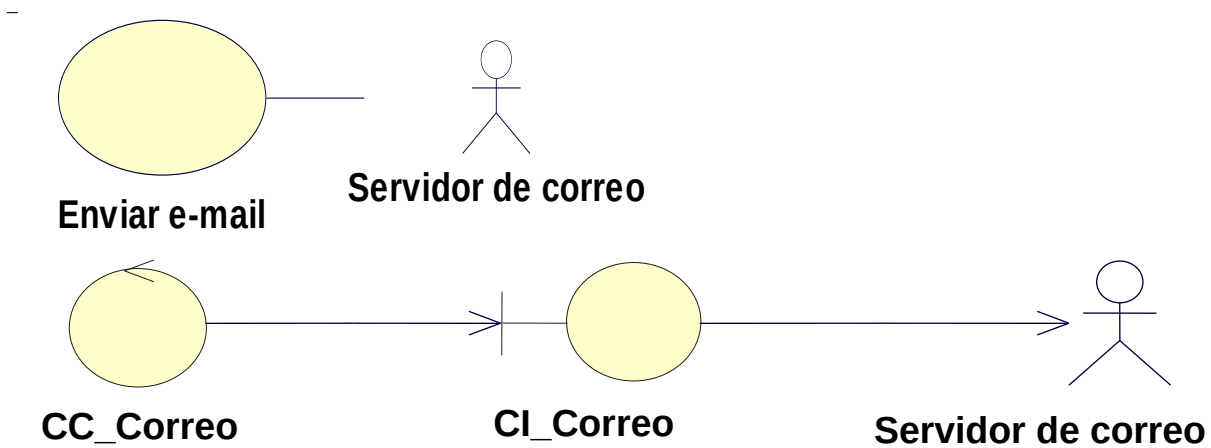
Caso de uso:	Reportar obras aceptadas que no tienen precio de venta
Actores:	Especialista en economía (Inicia)
Descripción: El caso de uso se inicia cuando el especialista en economía necesita ponerse en contacto con los artistas que tiene obras aceptadas para exponer sin precio de venta fijado; para lo cual consulta al sistema y obtiene un reporte del artista con las obras.	
Referencias:	R8
Precondiciones:	Hay obras aceptadas para exponer que no tiene precio de venta fijado. Si no obras que cumplan la condición, el sistema presenta un mensaje al Especialista en economía.
Postcondiciones:	-
Requerimientos especiales:	
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
	1 El sistema construye un reporte con las obras aceptadas para exponer que no tienen precio de venta pactado. El reporte lo organiza por artista indicando sus características y la de cada una de las obras que tienen este estado.
	3 El sistema presenta el reporte al Especialista en economía.
4 El Especialista en economía decide imprimir el reporte.	5 El sistema imprime el reporte.
6 El Especialista en economía decide terminar la ejecución del caso de uso.	7 El sistema finaliza la ejecución del caso de uso.
Cursos alternos	
-	

Caso de uso:	Reportar obras registradas que no han sido vendidas		
Actores:	Gerente de la galería (Inicia)		
Descripción: El caso de uso se inicia cuando el Gerente de la galería necesita conocer las características; de las obras y del artista que las hizo; de las obras que están pendientes de evaluación, aceptadas para exponer y aceptadas para exponer y vender cuya fecha de presentación es anterior a una fecha dad.			
Referencias:	R12		
Precondiciones:	Existen obras registradas y no rechazadas que no han sido vendidas y presentadas a la galería en una fecha anterior a la especificada. Si no existen obras que cumplan la condición, el sistema presenta un mensaje al Gerente de la galería.		
Postcondiciones:	-		
Requerimientos especiales:			
Curso normal de los eventos			
Acción del actor		Respuesta del sistema	
1	El Gerente de la galería especifica la fecha que servirá de base para el reporte.	2	El sistema verifica si hay obras registradas pendientes de venta recibidas en fecha posterior a la especificada. Si hay obras que cumplan la condición, construye un reporte en el que refleja las obras registradas que no han sido vendidas y fueron presentadas en fecha anterior a la especificada. La obras se organizan de acuerdo a su estado (pendientes, aceptadas para exponer y aceptadas para exponer y vender). De cada obra además de su estado, se incluye cuándo se recibió la solicitud de exposición en la galería y el artista que las hizo.
		3	El sistema presenta el reporte al Gerente de la galería.
4	El Gerente de la galería decide imprimir el reporte.	5	El sistema imprime el reporte.
6	El Gerente de la galería decide terminar la ejecución del caso de uso.	7	El sistema finaliza la ejecución del caso de uso.
Cursos alternos			
"Curso normal principal": Línea 2 Si no hay obras registradas en fecha posterior a la indicada que estén pendientes de venta, se presenta un mensaje al Gerente de la galería y se devuelve el control a la línea 1 del curso normal principal.			

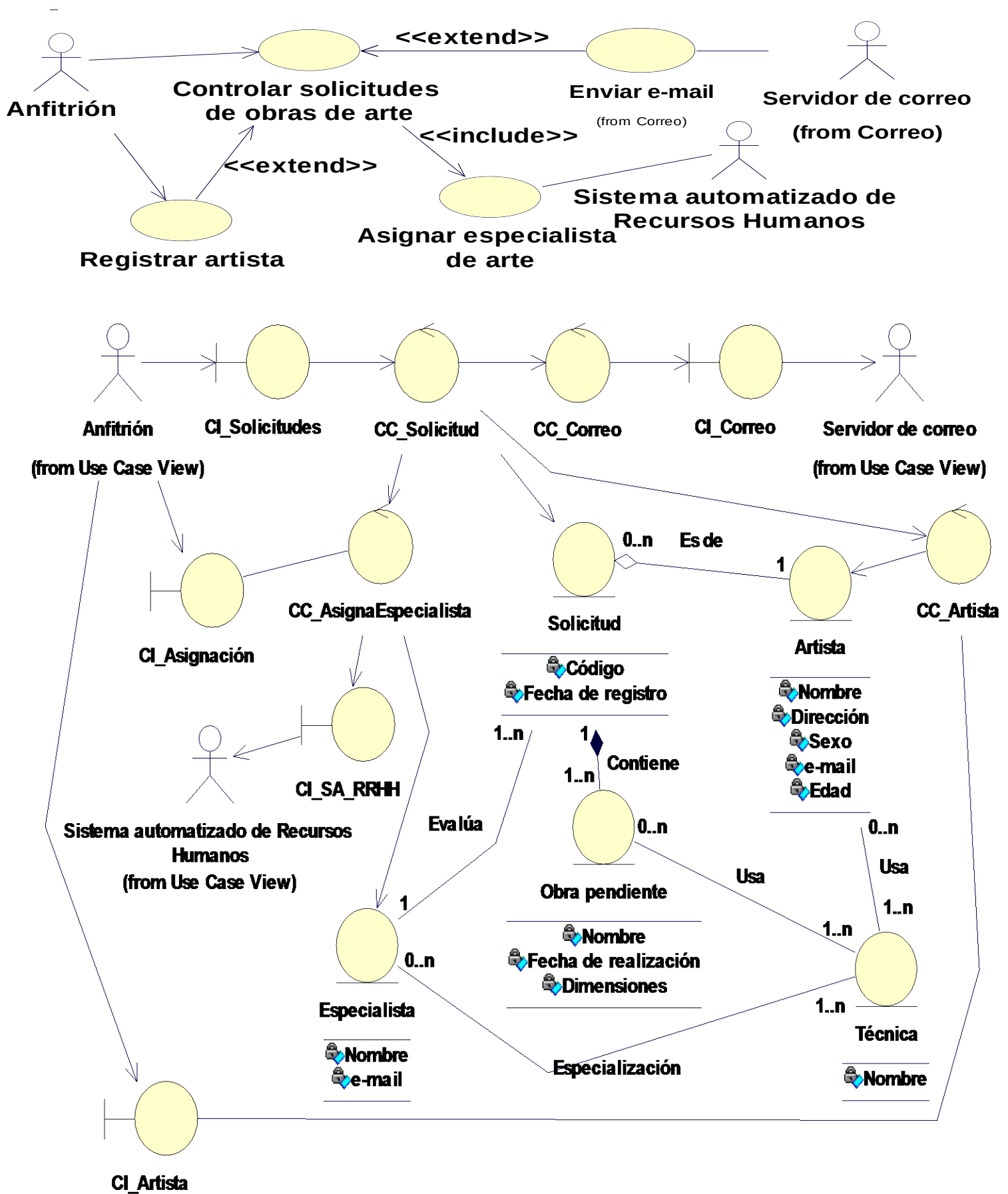
Organización en paquetes:



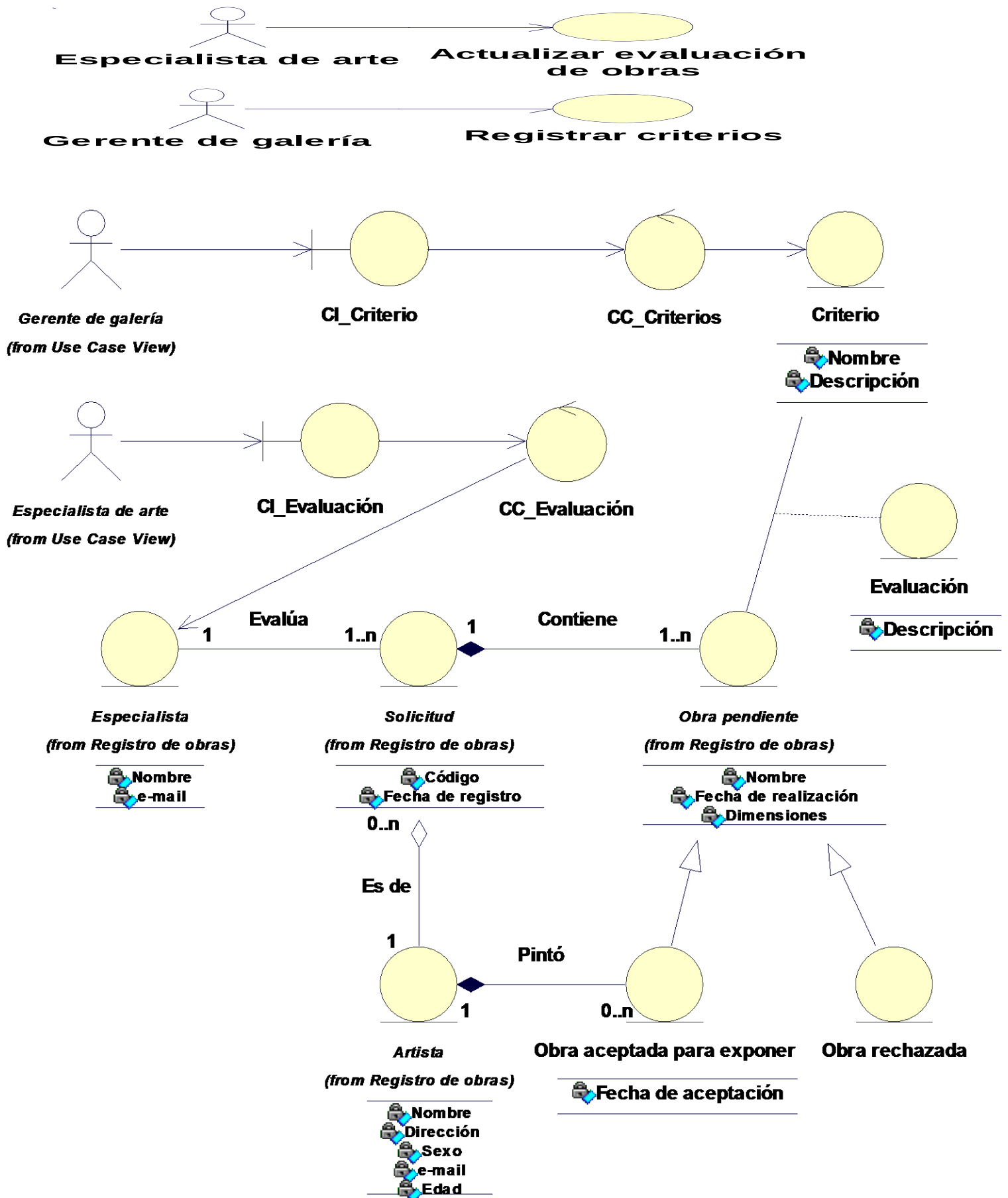
Paquete Correo



Paquete Registro de obras



Paquete Evaluación de obras



9.5 – Diseño

Refinamiento de la descripción de los casos de uso en formato expandido y diagrama de secuencia:

Caso de uso:	Controlar solicitudes de obras de arte
Actores:	Anfitrión (Inicia)
Descripción:	El caso de uso se inicia cuando un artista va a una galería y presenta una solicitud con las obras que desea se expongan en la galería. Un Anfitrión registra la solicitud y le asigna un especialista de arte para que evalúe las obras. El caso de uso finaliza cuando el sistema, de forma automática, le envía un correo electrónico al especialista asignado con la información de la solicitud. En caso de que no estén registrados los datos del artista, se hace en ese momento. El Anfitrión puede eliminar una solicitud o modificar las obras que contiene, informando de ello al especialista asignado.
Referencias:	R1, R2, R3, R4, R5 y R6 Casos de uso asociados: • Enviar e-mail (Extend). • Asignar especialista de arte (Include). • Registrar artista (Extend).
Precondiciones:	El sistema automatizado de RRHH ha replicado la información sobre las técnicas y los especialistas. Si no se ha replicado, el sistema presenta un mensaje al Anfitrión.
Poscondiciones:	Se registra(n) la(s) nueva(s) solicitud(es) o el(los) cambio(s) producidos informando al (los) Especialista(s) de arte involucrado(s). En el caso de las nuevas, se registra la solicitud y se envía el e-mail, solo si fue posible asignar especialistas.
Requerimientos especiales	-
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
	1 El sistema presenta de todas las solicitudes registradas que tienen obras pendientes de evaluación: el código, la fecha de recibo, el artista y si alguna de las obras que contiene ya ha sido evaluada.
2 El Anfitrión necesita registrar una nueva solicitud, eliminar una solicitud o modificar las obras que contiene una solicitud. En el caso de la eliminación y la modificación, se selecciona la solicitud antes de decidir la acción a seguir.	3 El sistema ejecuta alguna de las siguientes secciones: d) Si decide registrar una nueva solicitud, ir a la sección "Nueva. e) Si decide eliminar una solicitud, ir a la sección "Eliminar". f) Si decide modificar las obras contenidas en una solicitud, ir a la sección "Modificar".
El Anfitrión puede decidir repetir el paso 2.	
4 El Anfitrión termina el registro de solicitudes.	5 El sistema finaliza la ejecución del caso de uso.

Sección "Nueva"	
Acción del actor	Respuesta del sistema
	1 El sistema genera un código para la solicitud y lo presenta junto con la fecha actual y los artistas ya registrados; para que se pueda especificar al artista que presenta la solicitud y las características de las obras que pone a su consideración.
2 El Anfitrión especifica el artista que hizo la solicitud o decide registrar un nuevo artista.	3 El sistema ejecuta alguna de las siguientes acciones: c) Si selecciona un artista de los ya registrados, se extrae el resto de sus características y se le presentan al anfitrión. Pasar a la línea 4. d) Si decide registrar un nuevo artista, extender el caso de uso "Registrar artista", que actualiza los artistas. Pasar a la línea 2.
4 El Anfitrión define si va a incluir una nueva obra en la solicitud, eliminar una obra ya incluida o modificar las características de una obra. En el caso de la eliminación y modificación, selecciona antes la obra.	5 El sistema ejecuta alguna de las siguientes acciones: d) Si decide incluir una nueva obra en la solicitud, ir a la sección "Nueva obra". e) Si decide eliminar una obra de la solicitud, ir a la sección "Eliminar obra". f) Si decide modificar las características de una obra, ir a la sección "Modificar obra".
El Anfitrión puede decidir repetir el paso 4.	
6 El Anfitrión termina de registrar los datos de la nueva solicitud o decide no realizar el registro.	7 Si termina el registro, el sistema verifica si no hay otra solicitud registrada del artista para ese día. En caso negativo, el sistema incluye el caso de uso "Asignar especialistas de arte", para lo cual le envía las técnicas usadas en la confección de las obras de la solicitud y recibe el especialista asignado
	8 Si fue posible asignar especialista a la solicitud, el sistema: 8.1 Busca el correo electrónico del especialista asignado. 8.2 Extiende el caso de uso "Enviar correo", para lo cual le da como información una dirección electrónica y la información contenida en la solicitud. 8.3 Registra la solicitud, consignando como fecha de registro la fecha actual.
Sección "Modificar"	
Acción del actor	Respuesta del sistema
	1 El sistema verifica que en la solicitud no existan obras ya evaluadas.
	2 El sistema presenta todas las características de la solicitud seleccionada (código, fecha de recibo, datos del artista que la registró y características de las obras que contiene), permitiendo modificar solo las obras

	contenidas.
3 El Anfitrión define si va a incluir una nueva obra en la solicitud, eliminar una obra ya incluida o modificar las características de una obra incluida. En el caso de la eliminación y la modificación, antes selecciona la obra.	4 El sistema ejecuta alguna de las siguientes acciones: d) Si decide incluir una nueva obra en la solicitud, ir a la sección "Nueva obra". e) Si decide eliminar una obra de la solicitud, ir a la sección "Eliminar obra". f) Si decide modificar las características de una obra, ir a la sección "Modificar obra".
El Anfitrión puede decir repetir el paso 3.	
5 El Anfitrión termina de modificar las obras registradas en la solicitud o decide no registrar cambios a la solicitud.	6 Si terminó de modificar la solicitud, el sistema busca el correo electrónico del especialista asignado y extiende el caso de uso "Enviar correo", para lo cual le da como información una dirección electrónica y la información que se ha modificado.
	7 Registrar los cambios a la solicitud.
Sección "Eliminar"	
Acción del actor	Respuesta del sistema
	1 El sistema solicita la confirmación de la eliminación.
2 El Anfitrión confirma la eliminación de la solicitud o decide que no va a eliminar la solicitud.	4 Si confirma la eliminación, el Sistema verifica que en la solicitud no existan obras ya evaluadas. Si no hay obras evaluadas, el sistema busca el correo electrónico del especialista asignado y extiende el caso de uso "Enviar e-mail", para lo cual le da como información una dirección electrónica y la solicitud que se ha eliminado.
	5 El sistema elimina la solicitud y la asignación del especialista.
Sección "Nueva obra"	
Acción del actor	Respuesta del sistema
	1 El sistema obtiene las técnicas que usa el artista que hizo la solicitud y en las que está especializado el especialista de arte en caso de que ya esté asignado y extrae las técnicas que son comunes en ambos porque son las únicas que se admitirá que use la obra. En caso de que el especialista de arte no esté asignado, se admiten todas las técnicas que usa el artista. El sistema presenta las técnicas que puede usar la nueva obra y permite especificar el nombre, fecha de realización, dimensiones y técnicas que usa la obra.
2 El Anfitrión especifica las características de la nueva obra (nombre, dimensiones, técnicas y	3 Si decide registrar la obra, el sistema verifica que la obra no esté registrada. En caso de que no exista, se incluye la obra en la solicitud.

fecha de realización) o decide no registrar la obra.	
Sección "Modificar obra"	
Acción del actor	Respuesta del sistema
	1 El sistema presenta las características de la obra seleccionada y las técnicas que podría usar. Para obtener las técnicas busca las que usa el artista y en las que está especializado el especialista de arte, en caso de que ya esté asignado, y que no estaban declaradas que usaba la obra y para extraer las técnicas que son comunes en ambos y que no se utilizaban en la obra, porque son las únicas que se admitirá que use la obra. En caso de que el especialista de arte no esté asignado, se admiten todas las técnicas que usa el artista que no se usaban en la obra. Solo se puede modificar la fecha de realización de la obra, dimensiones y técnicas usadas.
2 El Anfitrión especifica las características que cambian de la obra (fecha de realización, dimensiones o técnicas) o decide no registrar los cambios.	3 Si decide registrar los cambios, el sistema incluye los cambios a la obra en la solicitud.
Sección "Eliminar obra"	
Acción del actor	Respuesta del sistema
	2 El sistema solicita la confirmación de la eliminación.
2 El Anfitrión confirma la eliminación de la obra o decide que no va a registrar la obra.	3 Si confirma la eliminación, el sistema elimina la obra de la solicitud.
Cursos alternos	

Sección "Nueva" : Línea 8

Si no fue posible asignar un especialista, finaliza la ejecución de la sección y se regresa a la línea 2 de la sección.

Sección "Nueva": Línea 8

Si estaba registrada otra solicitud del artista para ese día, el sistema presenta un mensaje al anfitrión y se regresa el control a la línea 2 de la sección.

Sección "Nueva": Línea 8

Si el Anfitrión decide no realizar el registro de la solicitud, finaliza la ejecución de la sección y se regresa a la línea 2 del curso normal principal.

Sección "Modificar" : Línea 1

Si la solicitud seleccionada tiene obras ya evaluadas, el sistema presenta un mensaje al Anfitrión y regresa a la línea 2 del curso normal principal.

Sección "Modificar": Línea 6

Si el Anfitrión decide no realizar cambios a la solicitud, finaliza la ejecución de la sección y se regresa a la línea 2 del curso normal principal.

Sección "Eliminar" : Línea 1

Si la solicitud seleccionada tiene obras ya evaluadas, el sistema presenta un mensaje al Anfitrión y regresa a la línea 2 del curso normal principal.

Sección "Eliminar": Línea 2

Si el Anfitrión decide no eliminar la solicitud, finaliza la ejecución de la sección y se regresa a la línea 2 del curso normal principal.

Sección "Nueva obra" : Línea 3

Si la obra ya está registrada, el sistema presenta un mensaje al Anfitrión y regresa a la línea 42 de la sección.

Sección "Nueva obra": Línea 3

Si el Anfitrión decide no realizar el registro de la obra, finaliza la ejecución de la sección y se regresa a la sección Nueva o Modificar en la línea 4 ó 3, respectivamente, en dependencia del camino seguido para llegar a la sección.

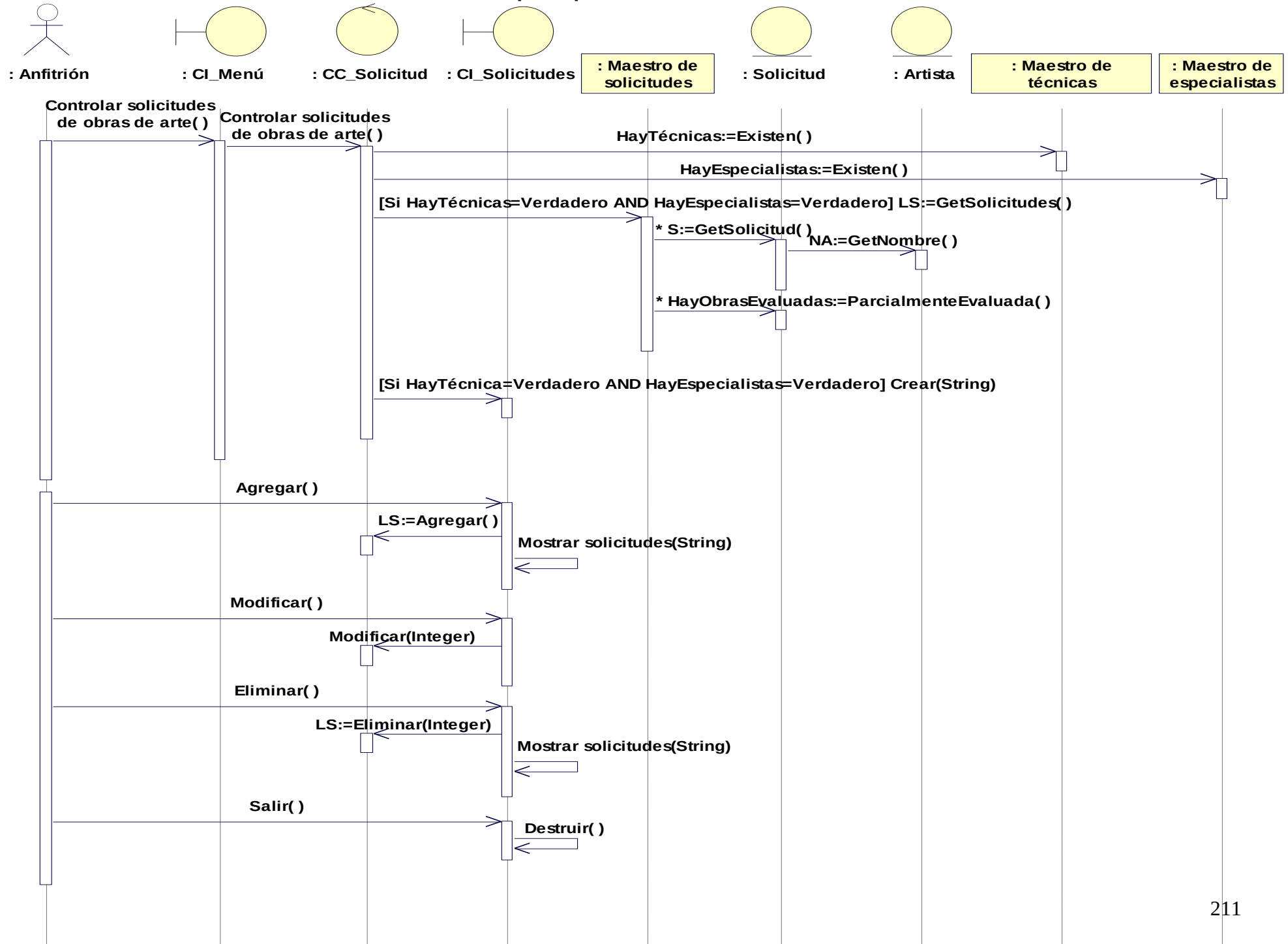
Sección "Modificar obra": Línea 3

Si el Anfitrión decide no registrar los cambios a la obra, finaliza la ejecución de la sección y se regresa a la sección Nueva o Modificar en la línea 4 ó 3, respectivamente, en dependencia del camino seguido para llegar a la sección.

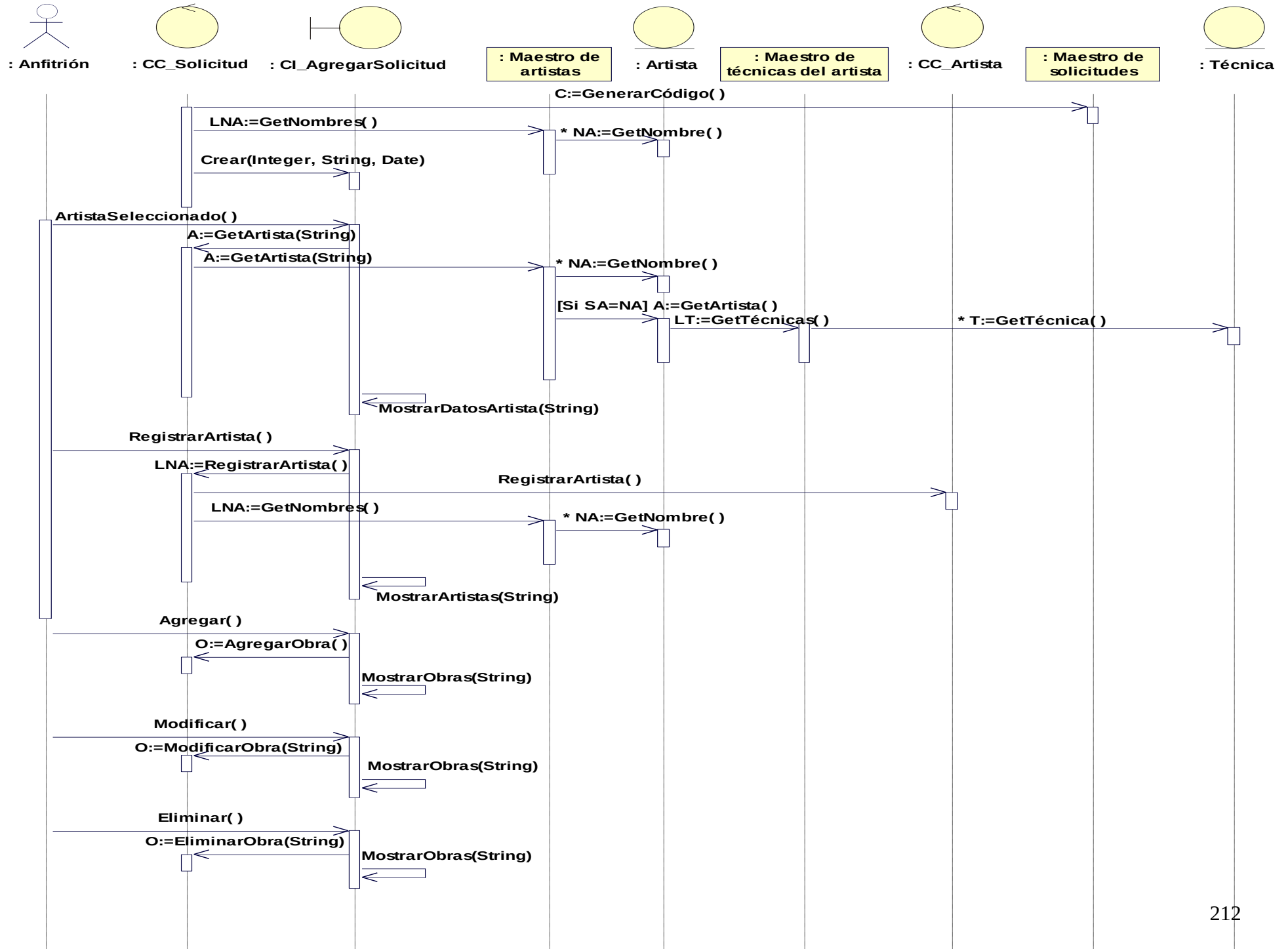
Sección "Eliminar obra": Línea 3

Si el Anfitrión decide no eliminar la obra, finaliza la ejecución de la sección y se regresa a la sección Nueva o Modificar en la línea 4 ó 3, respectivamente, en dependencia del camino seguido para llegar a la sección.

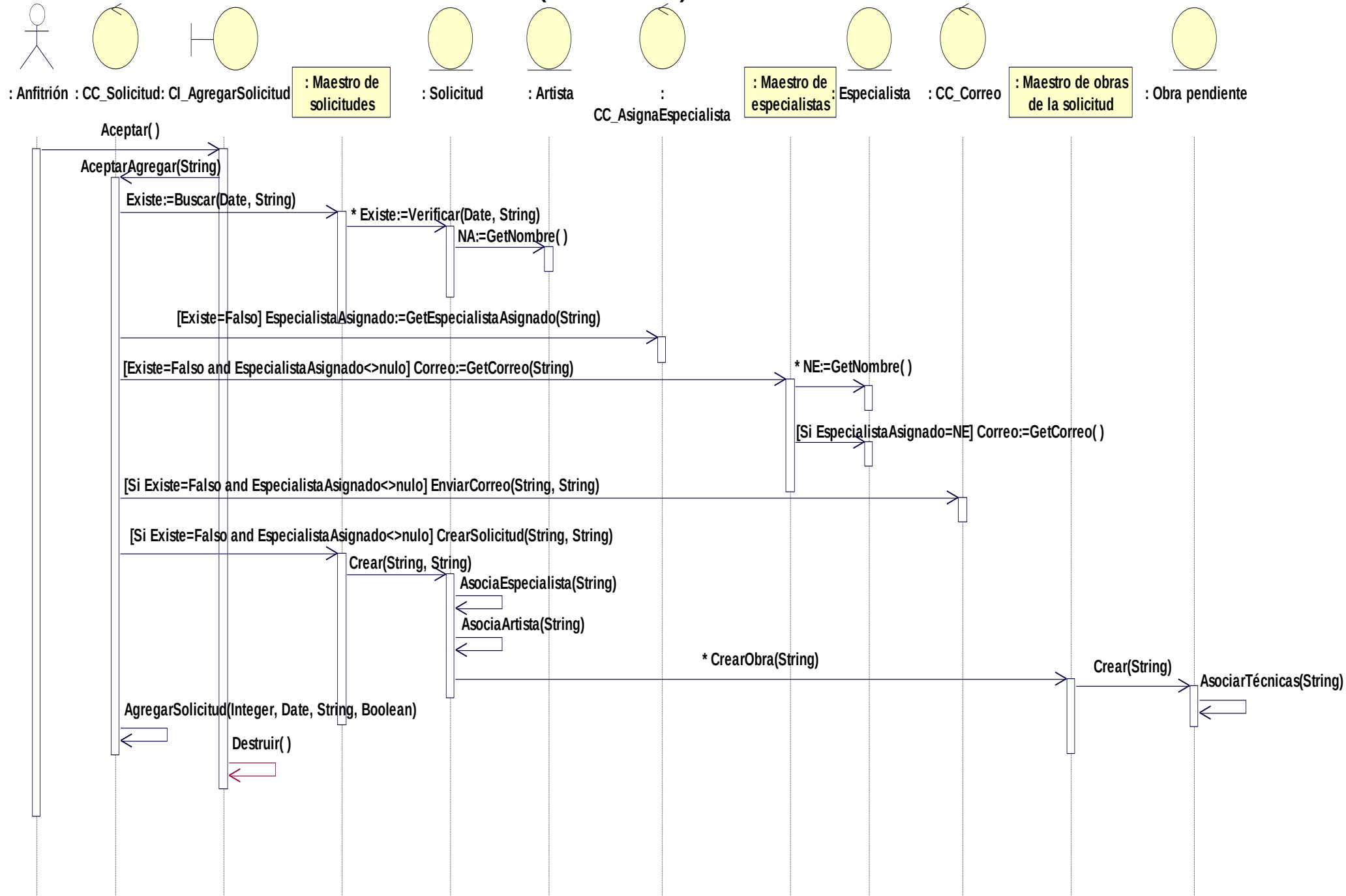
Controlar solicitudes de obras- Curso normal principal



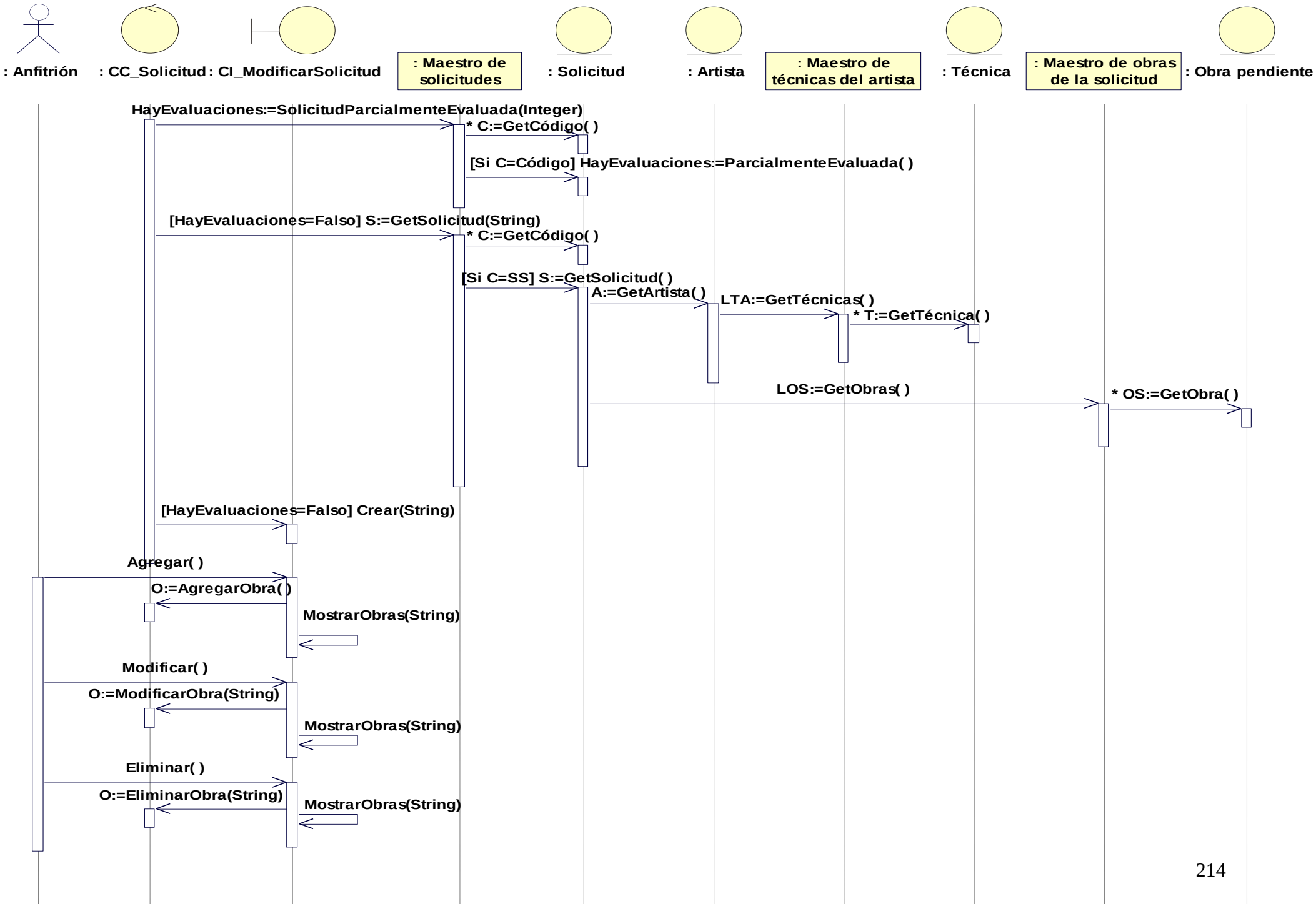
Controlar solicitudes de obras- Sección nueva



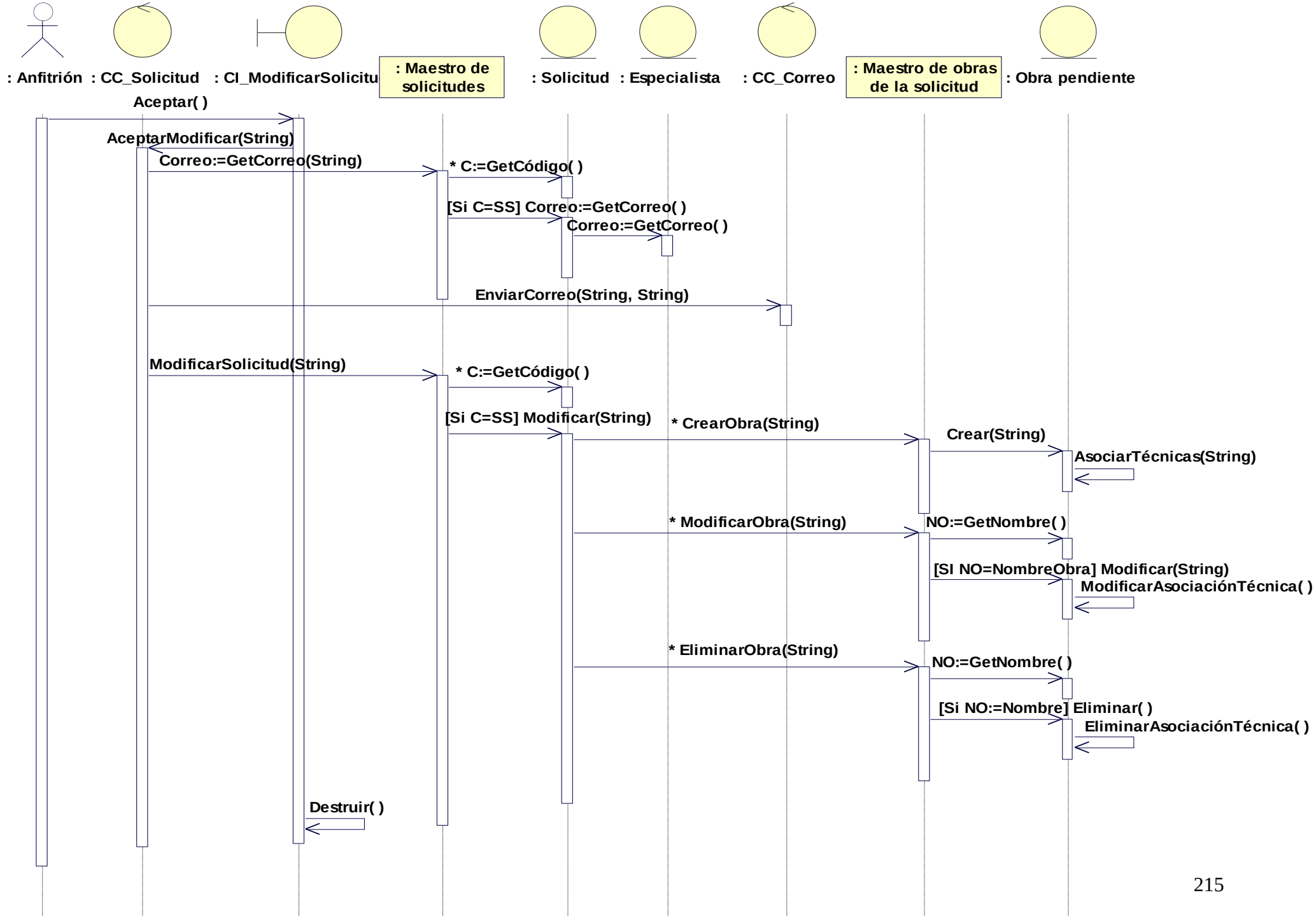
Controlar solicitudes de obras- Sección Nueva (Continuación)



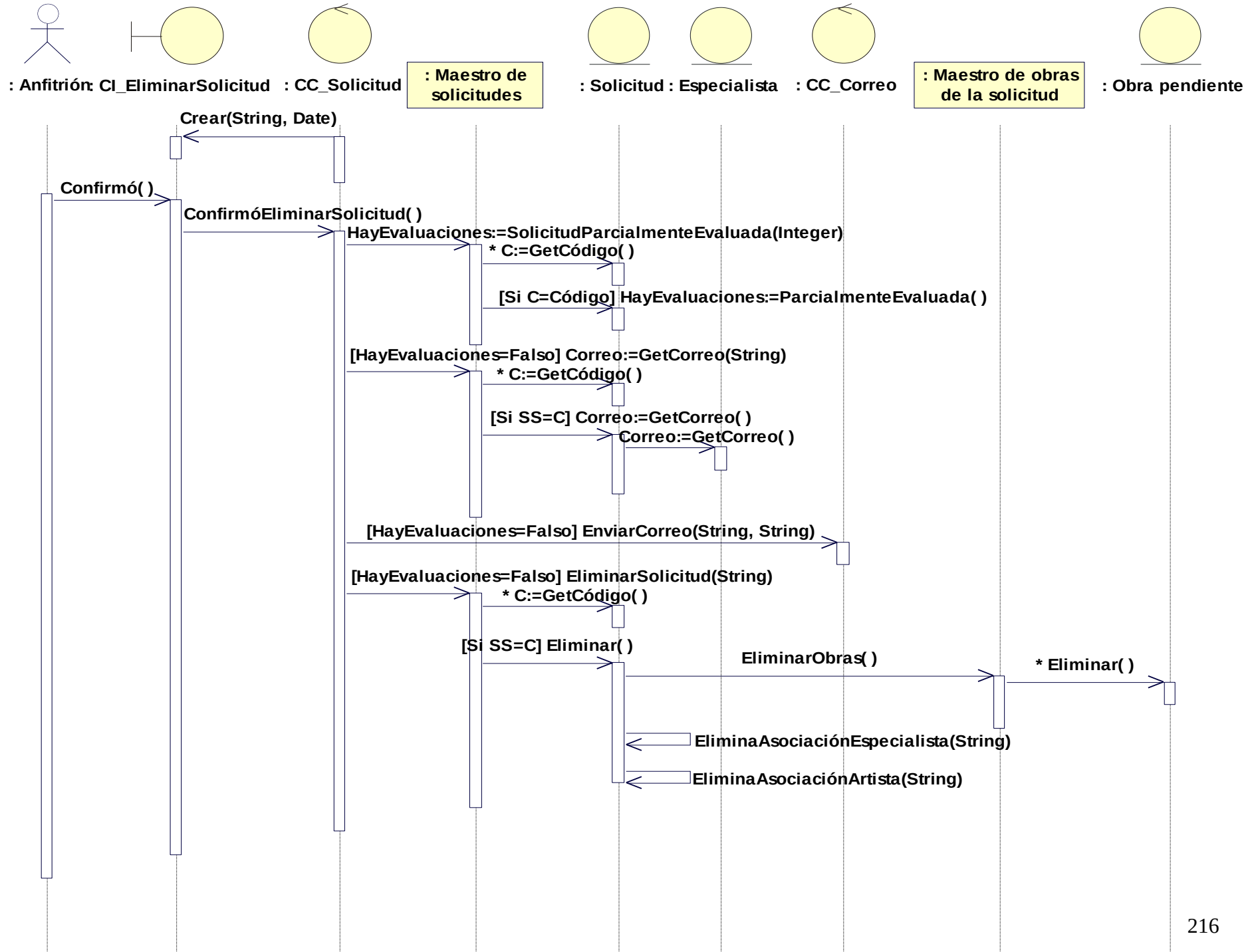
Controlar solicitudes de obras- Sección Modificar



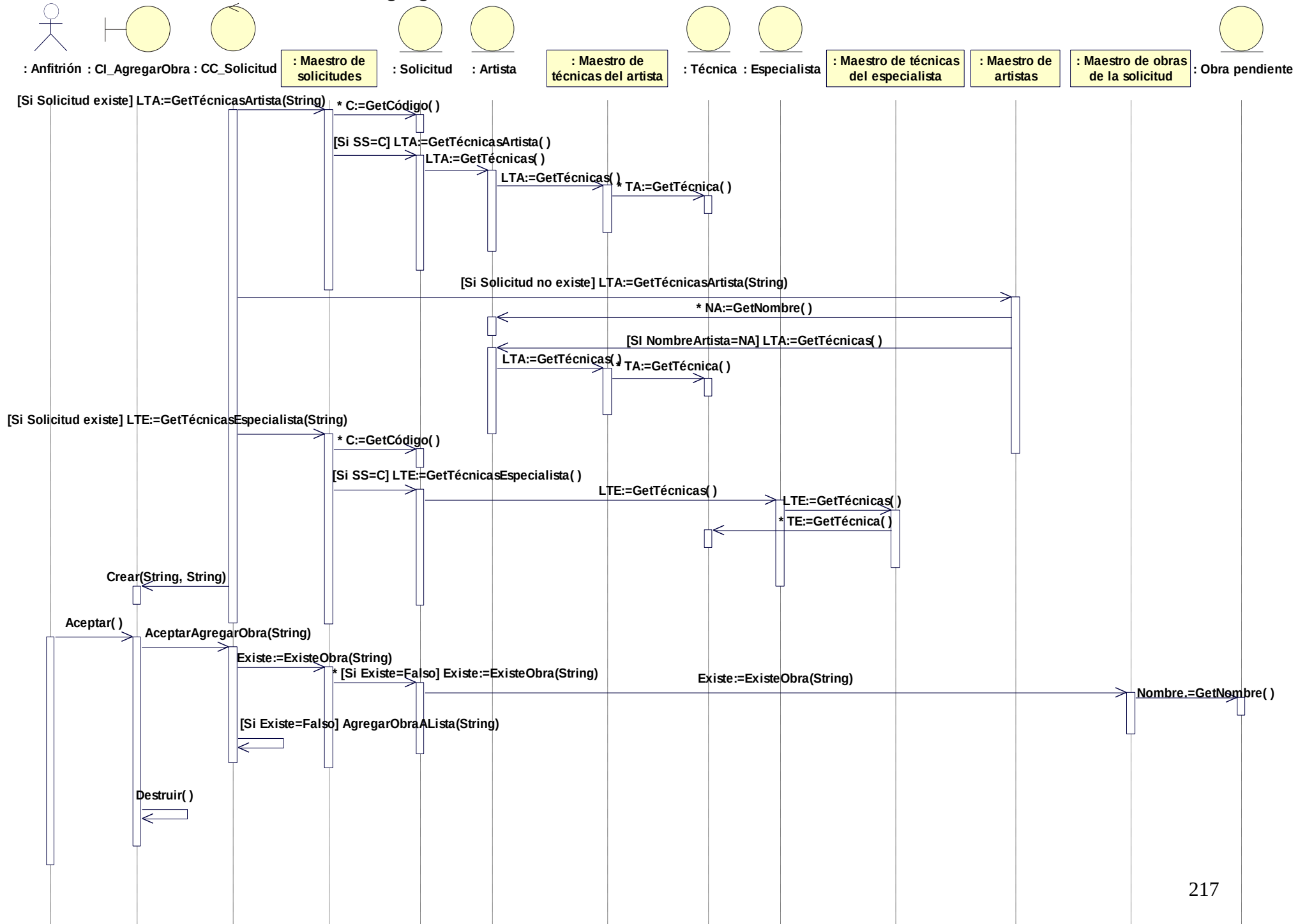
Controlar solicitudes de obras- Sección Modificar (Continuación)



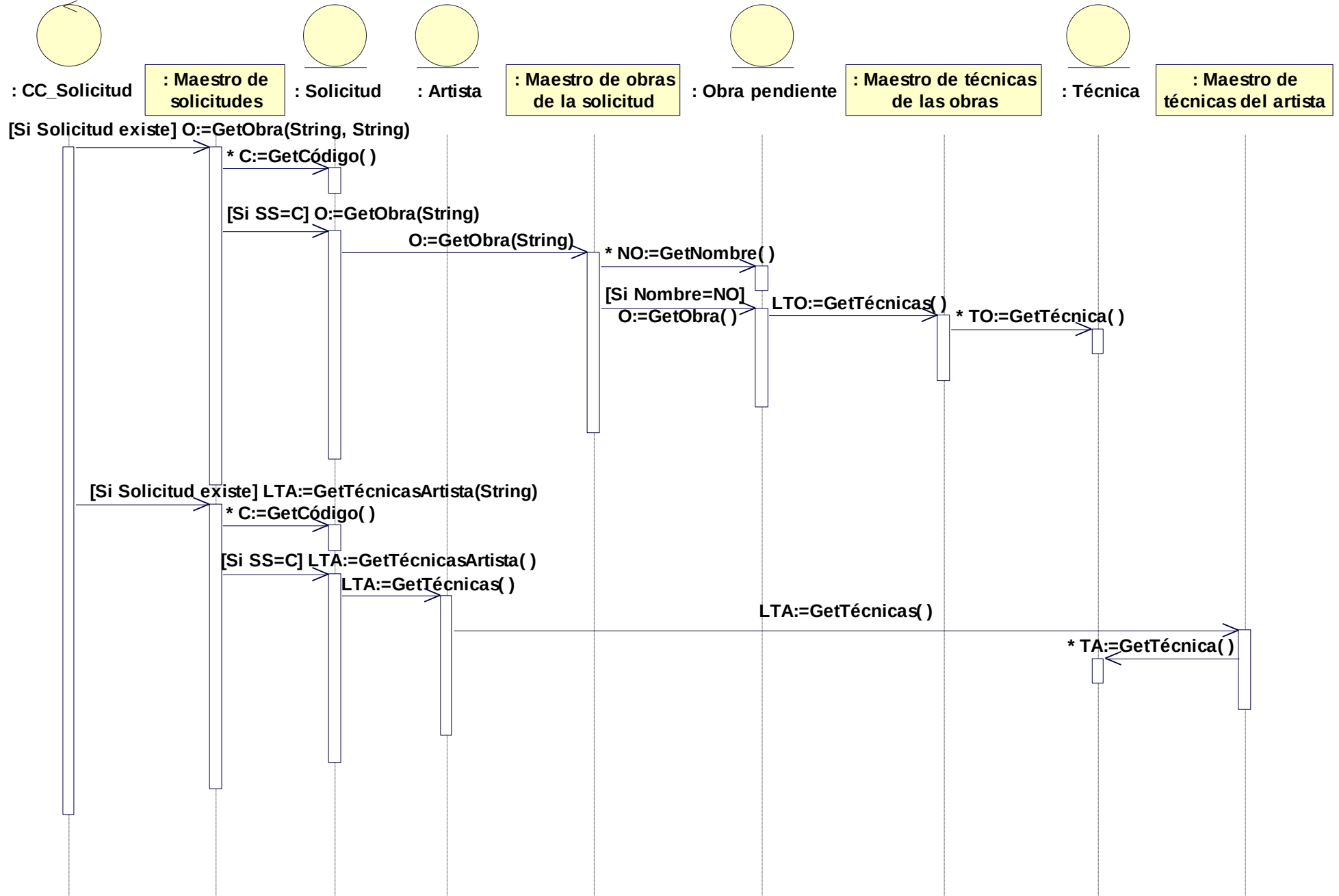
Controlar solicitudes de obras- Eliminar solicitud



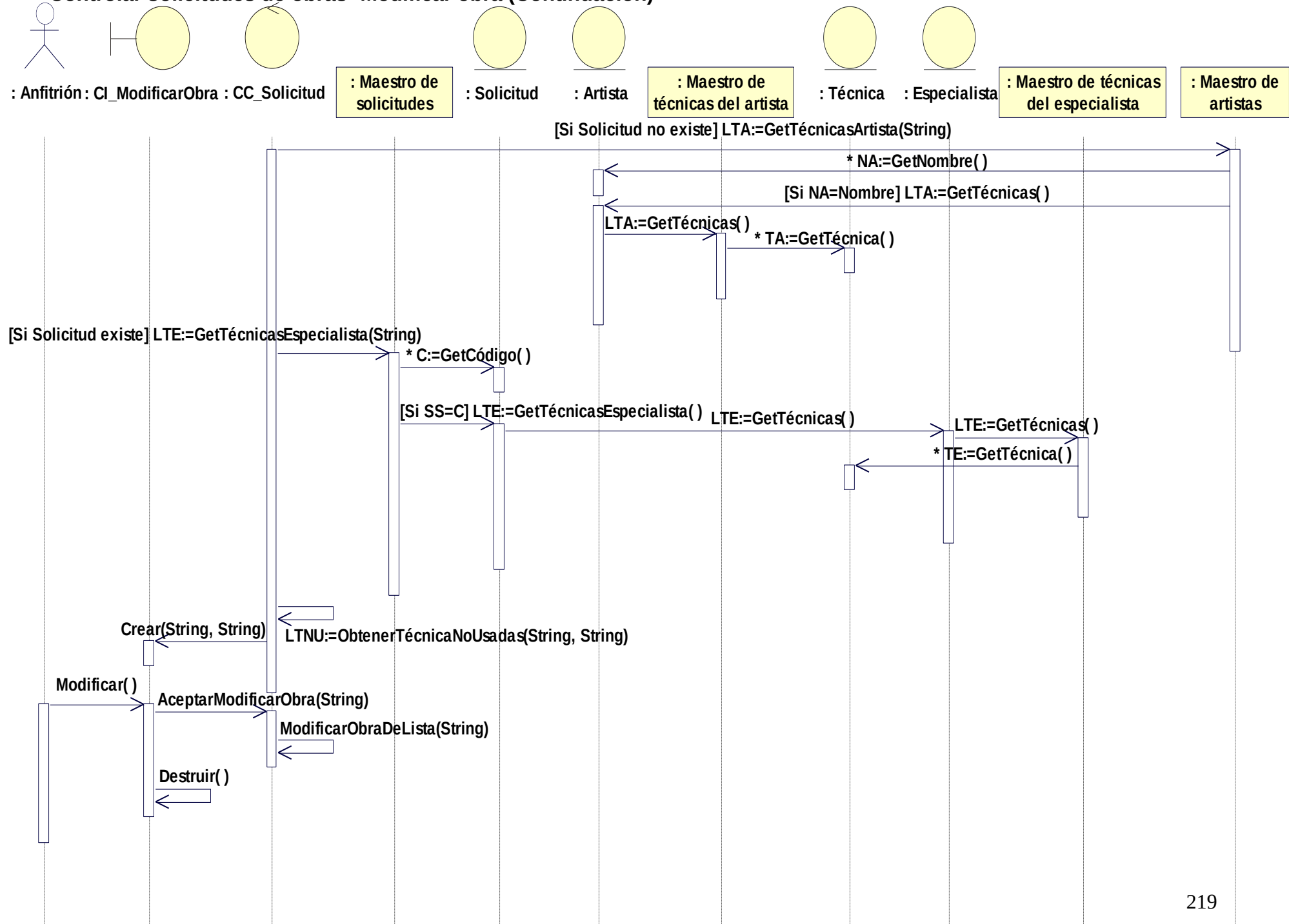
Controlar solicitudes de obras- Agregar obra



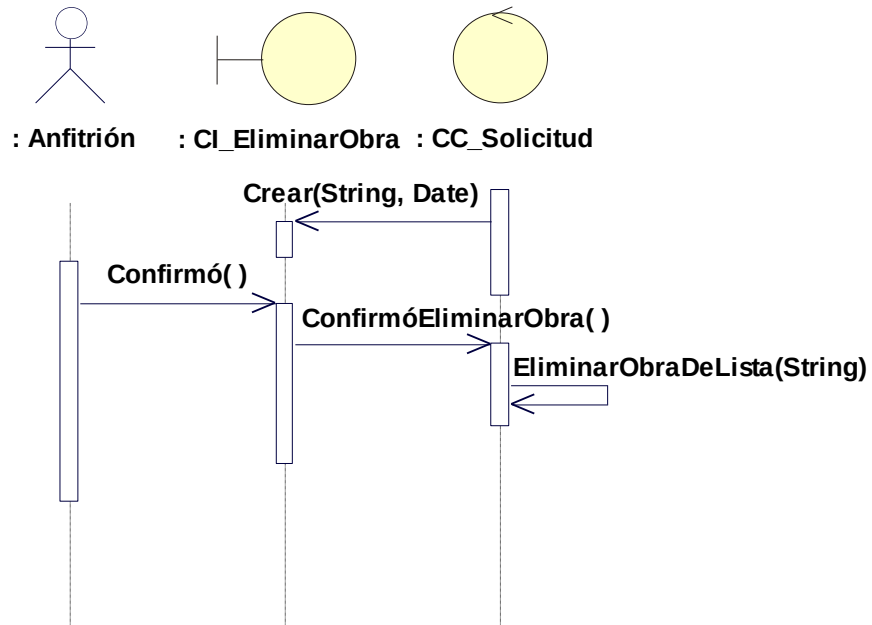
Controlar solicitudes de obras- Modificar obra



Controlar solicitudes de obras- Modificar obra (Continuación)

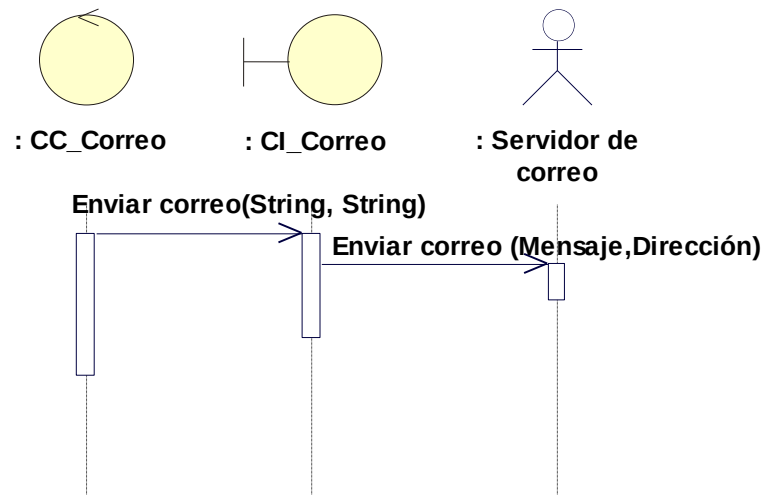


Controlar solicitudes de obras- Eliminar obra

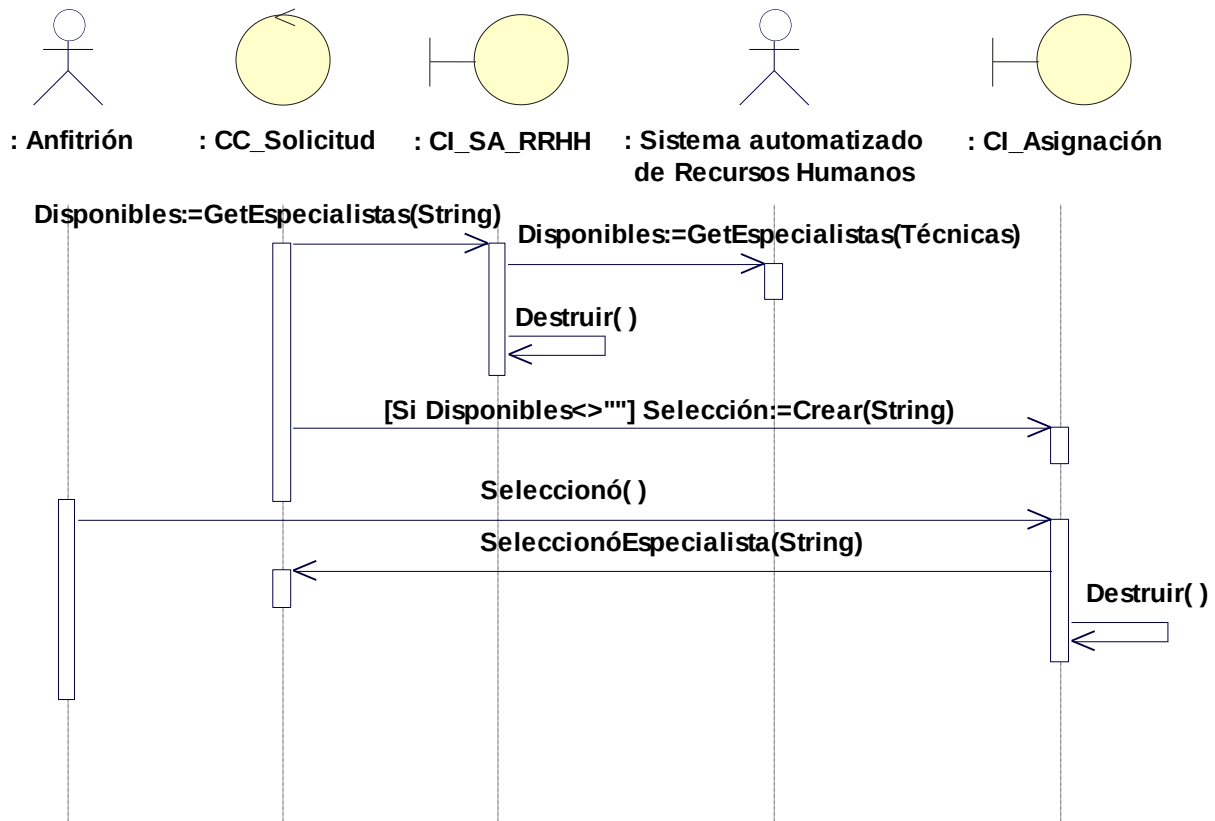


Caso de uso:	Enviar e-mail
Actores:	Servidor de correo
Descripción:	El caso de uso se inicia cuando algún proceso necesita enviar un correo electrónico, recibíéndose el mensaje a enviar y la dirección electrónica del destinatario. El sistema procede a conformar el mensaje y a través del Servidor de correo le envía el mensaje.
Referencias:	R4 y R11
Precondiciones:	Otro proceso requiere notificar vía correo electrónico sobre algo que ocurrió.
Poscondiciones:	-
Requerimientos especiales:	Si no fue posible enviar el correo por problemas de comunicación, hay que informar al usuario para que tome medidas alternativas.
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
	1 El sistema conforma el mensaje con la dirección electrónica y el contenido a enviar y solicita a un Servidor de correo que envíe el mensaje.
2 El Servidor de correo envía el mensaje a la dirección indicada.	3 El sistema finaliza la ejecución del caso de uso y devuelve el control al caso de uso que lo invocó.

Enviar correo



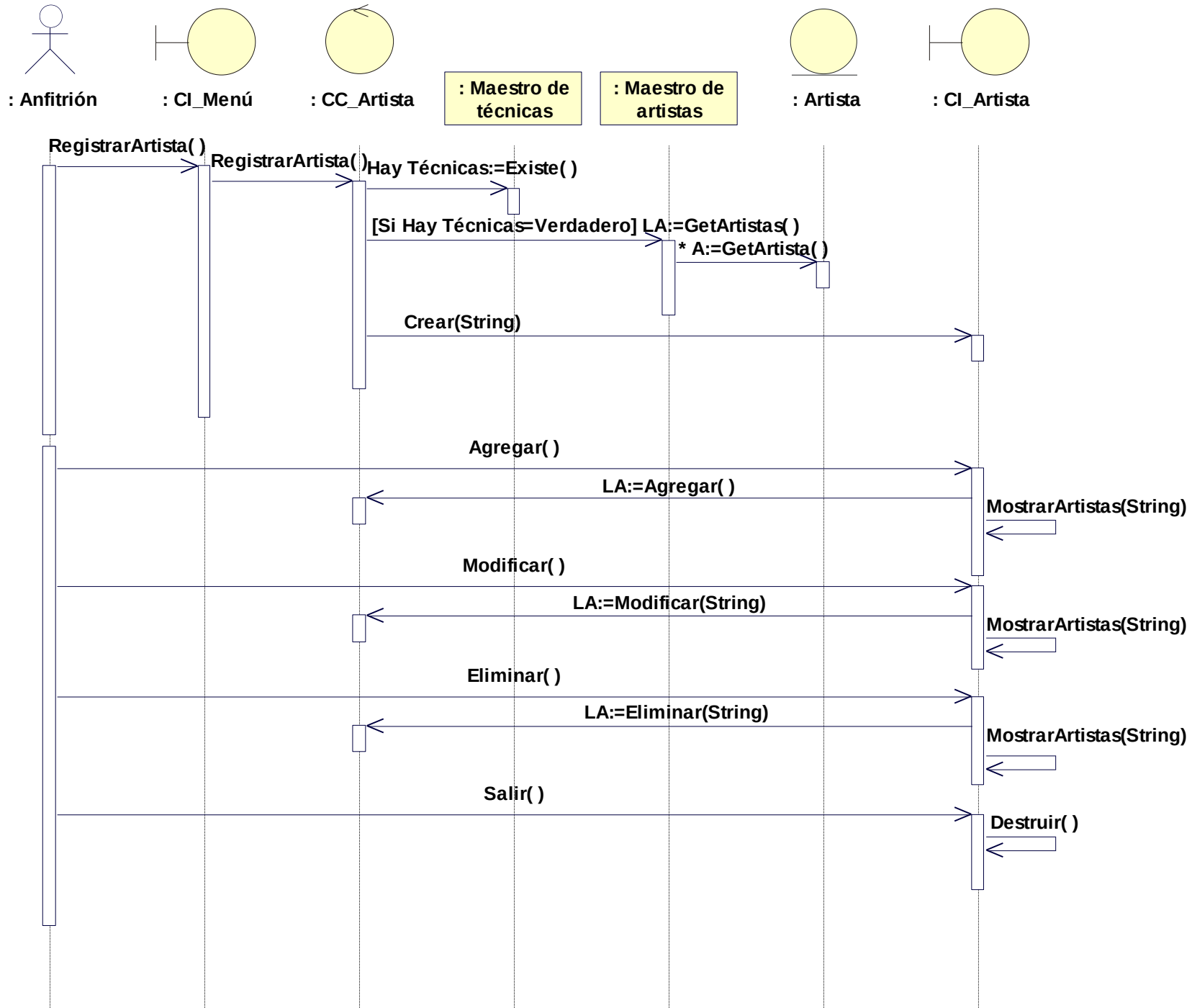
Caso de uso:	Asignar especialista de arte
Actores:	Anfitrión y Sistema Automatizado de RRHH
Descripción: El caso de uso se inicia cuando se registra una nueva solicitud, automáticamente el sistema consulta al Sistema Automatizado de RRHH y extrae todos los especialistas de arte que se especializan en las mismas técnicas de las obras que contiene la solicitud. Si no hay especialistas que cumplan el requerimiento, finaliza el caso de uso. En caso contrario, el Anfitrión selecciona de los disponibles a un especialista, culminando la ejecución del caso de uso.	
Referencias:	R3 y R6
Precondiciones:	Se registra una nueva solicitud.
Poscondiciones:	Se asocia un especialista de arte a una solicitud en caso de que existan especialistas en las técnicas de las obras de la solicitud.
Requerimientos especiales	Si no fue posible relacionarse con el Sistema Automatizado de RRHH, hay que informar al usuario para que tome medidas.
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
	1 El sistema solicita al Sistema Automatizado de RRHH el nombre de todos los especialistas que se especializan en determinadas técnicas.
2 El Sistema Automatizado de RRHH selecciona todos los especialistas de arte que cumplen la condición y entrega sus nombres.	3 En caso de que existan especialista que cumplan la condición, el sistema presenta los nombre de los especialistas de arte.
4 El Anfitrión selecciona un especialista o decide que no va a realizar la asignación.	5 Si decide asignar especialista, el sistema finaliza la ejecución del caso de uso y devuelve el control al caso de uso que lo invocó con el especialista seleccionado.
Cursos alternos	
Curso normal de los eventos : Línea 3 Si no se encontró especialistas que cumplieran la condición, el sistema presenta un mensaje al Anfitrión y finaliza el caso de uso devolviendo el control al caso de uso que lo invocó.	
Curso normal de los eventos : Línea 5 Si el Anfitrión decide no asignar especialista, finaliza la ejecución del caso de uso devolviendo el control al caso de uso que lo invocó.	



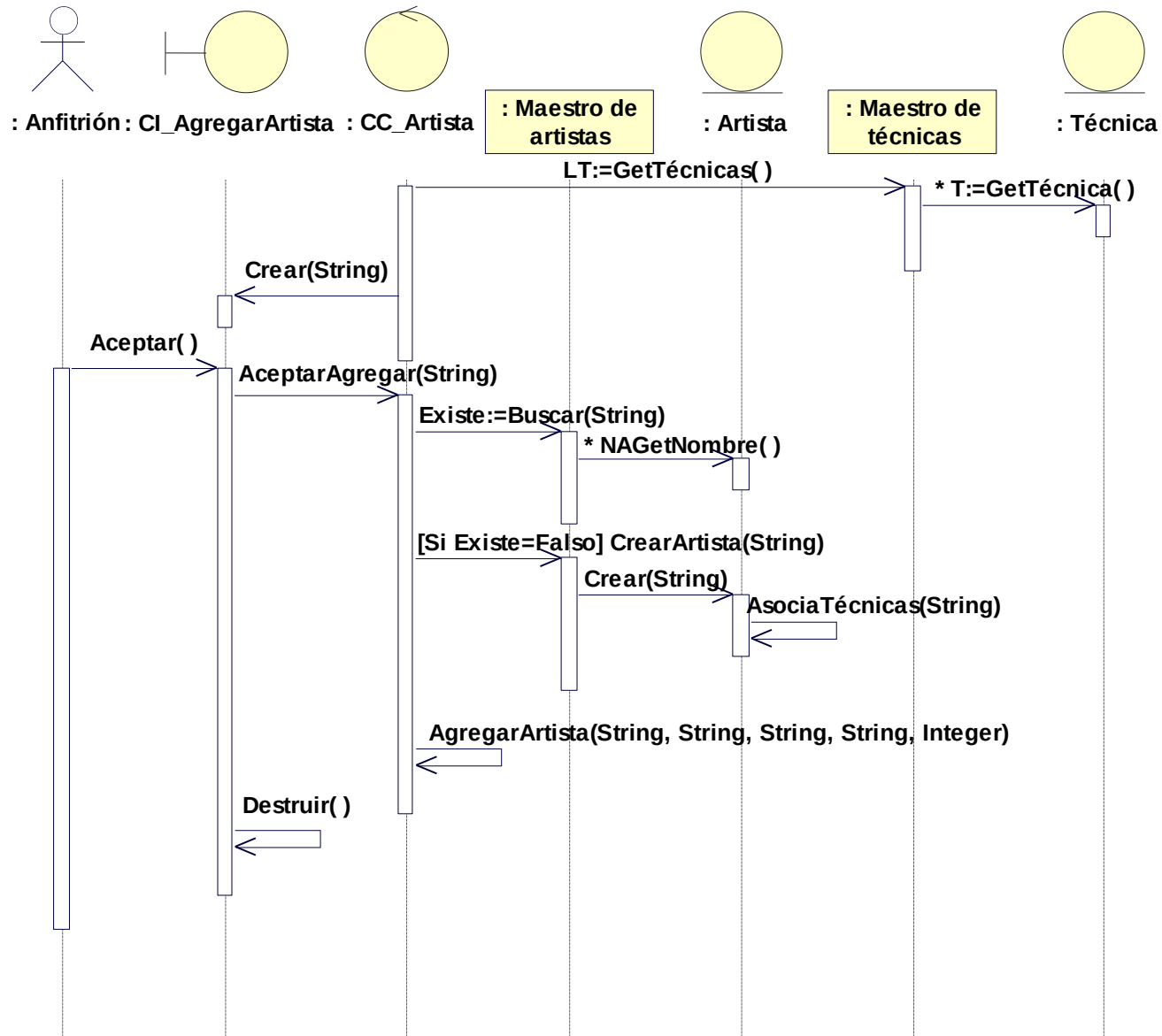
Caso de uso:	Registrar artista
Actores:	Anfitrión (Inicia)
Descripción: El caso de uso se inicia cuando al intentar registrar una nueva solicitud, se detecta que del artista que la presenta no están almacenados sus datos o cuando a la galería le interesa registrar información sobre artistas que desea expongan en la galería. En estos casos, el Anfitrión registra la información de un nuevo artista o los cambios que se produzcan en sus datos. El caso de uso finaliza con el registro de artistas actualizado. Si se va a eliminar a un artista, se verifica si está asociado a una obra, en cuyo caso no se realiza la eliminación; en cualquier otro caso se procede a la eliminación física.	
Referencias:	R5
Precondiciones:	El sistema automatizado de RRHH ha replicado las técnicas. Si no se ha replicado, el sistema presenta un mensaje al Anfitrión.
Poscondiciones:	Se actualiza el registro de artistas almacenando los datos del nuevo artista, la modificación de los datos registrados o la eliminación física de una artista si es posible.
Requerimientos especiales	-
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
	1 El sistema presenta de todos los artista registrados su nombre, edad, sexo, dirección y e-mail.
2 El Anfitrión necesita registrar un nuevo artista, modificar las características de un artista o eliminar un artista. En el caso de la eliminación o modificación, selecciona al artista antes.	3 El sistema ejecuta alguna de las siguientes acciones: m) Si decide registrar un nuevo artista, ir a la sección "Nuevo". n) Si decide modificar las características de un artista, ir a la sección "Modificar". o) Si decide eliminar un artista, ir a la sección "Eliminar".
El Anfitrión puede repetir el paso 2.	
4 El Anfitrión termina el registro de artistas.	5 El sistema finaliza la ejecución del caso de uso.
Sección "Nuevo"	
Acción del actor	Respuesta del sistema
	1 El sistema obtiene las posibles técnicas que puede usar un artista y las presenta para que se puede especificar el nombre, edad, sexo, dirección, e-mail y técnicas que usa el nuevo artista para confeccionar sus obras.
2 El Anfitrión especifica las características de un nuevo artista (nombre, edad, sexo, dirección, e-mail y técnicas que usa) o decide no registrar al artista.	3 Si decide registrar al artista, el sistema verifica que el artista no esté registrado. En caso de que no exista, se registra el artista.
Sección "Modificar "	

Acción del actor	Respuesta del sistema
	1 El sistema presenta las características registradas del artista (nombre, edad, sexo, dirección, e-mail y técnicas que usa) y las técnicas que puede usar y que no estaba usando. Solo se puede modificar la edad, dirección, e-mail y agregar nuevas técnicas.
2 El Anfitrión especifica las características que cambian del artista (edad, dirección, e-mail o las nuevas técnicas) o decide no registrar los cambios.	3 Si decide registrar los cambios, el sistema incluye los cambios al artista.
Sección "Eliminar "	
Acción del actor	Respuesta del sistema
	1 El sistema solicita la confirmación de la eliminación.
2 El Anfitrión confirma la eliminación del artista o decide que no va a eliminar al artista.	3 Si confirma la eliminación, el sistema verifica si el artista está asociado a una obra. En caso de que no esté asociado, elimina el artista.
Cursos alternos	
Sección "Nuevo" : Línea 2 Si el artista ya está registrado, el sistema presenta un mensaje al Anfitrión y regresa a la línea 1 de la sección. Sección "Nuevo" : Línea 3 Si el Anfitrión decide no registrar al artista, finaliza la ejecución de la sección y regresa a la línea 2 del curso normal principal. Sección "Modificar" : Línea 3 Si el Anfitrión decide no registrar los cambios al artista, finaliza la ejecución de la sección y regresa a la línea 2 del curso normal principal. Sección "Eliminar" : Línea 3 Si el artista seleccionado tiene obras registradas, el sistema presenta un mensaje al Anfitrión y regresa a la línea 2 del curso normal principal. Sección "Eliminar" : Línea 3 Si el Anfitrión decide no eliminar al artista, finaliza la ejecución de la sección y regresa a la línea 2 del curso normal principal.	

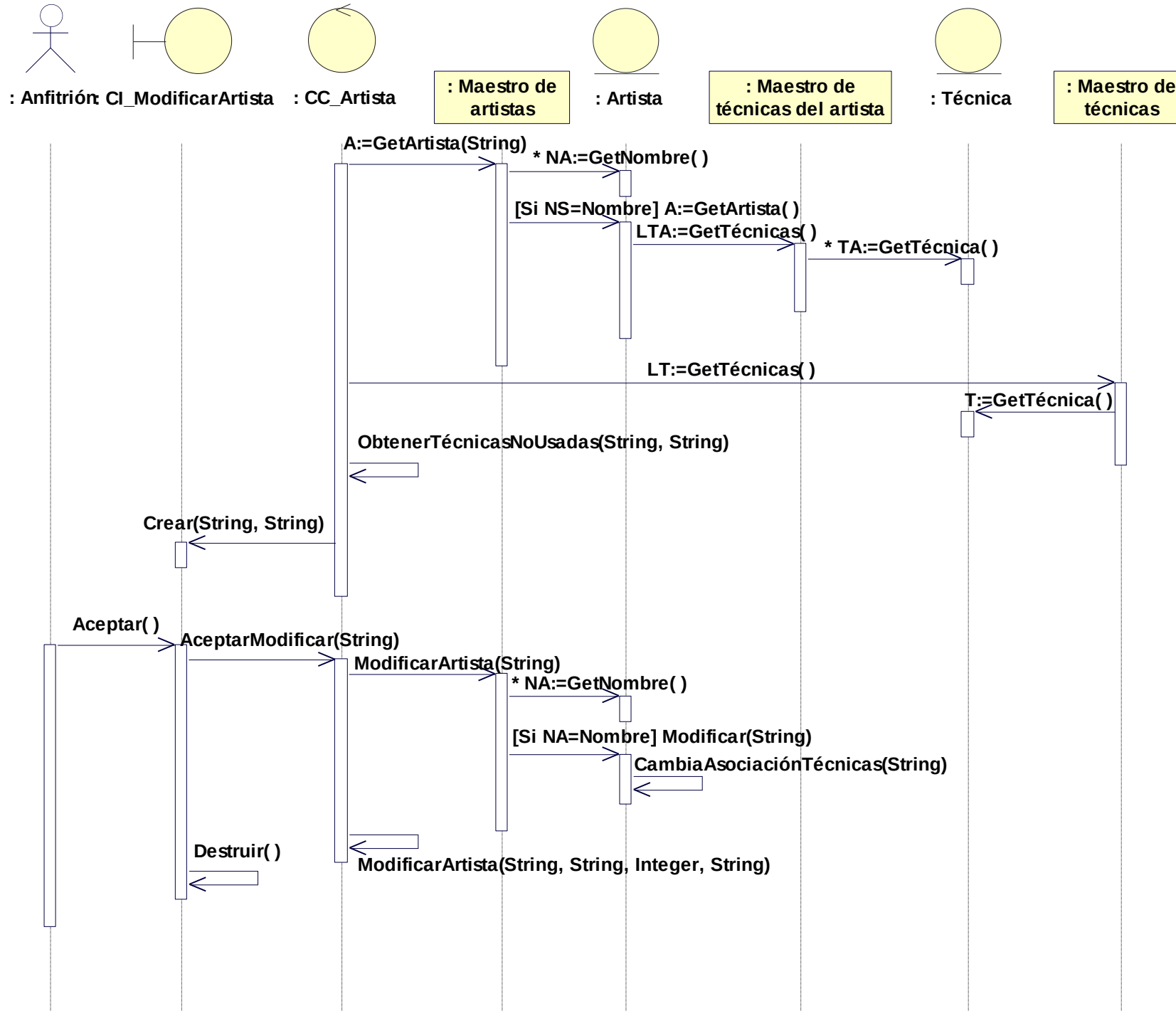
Registrar artista- Curso normal principal



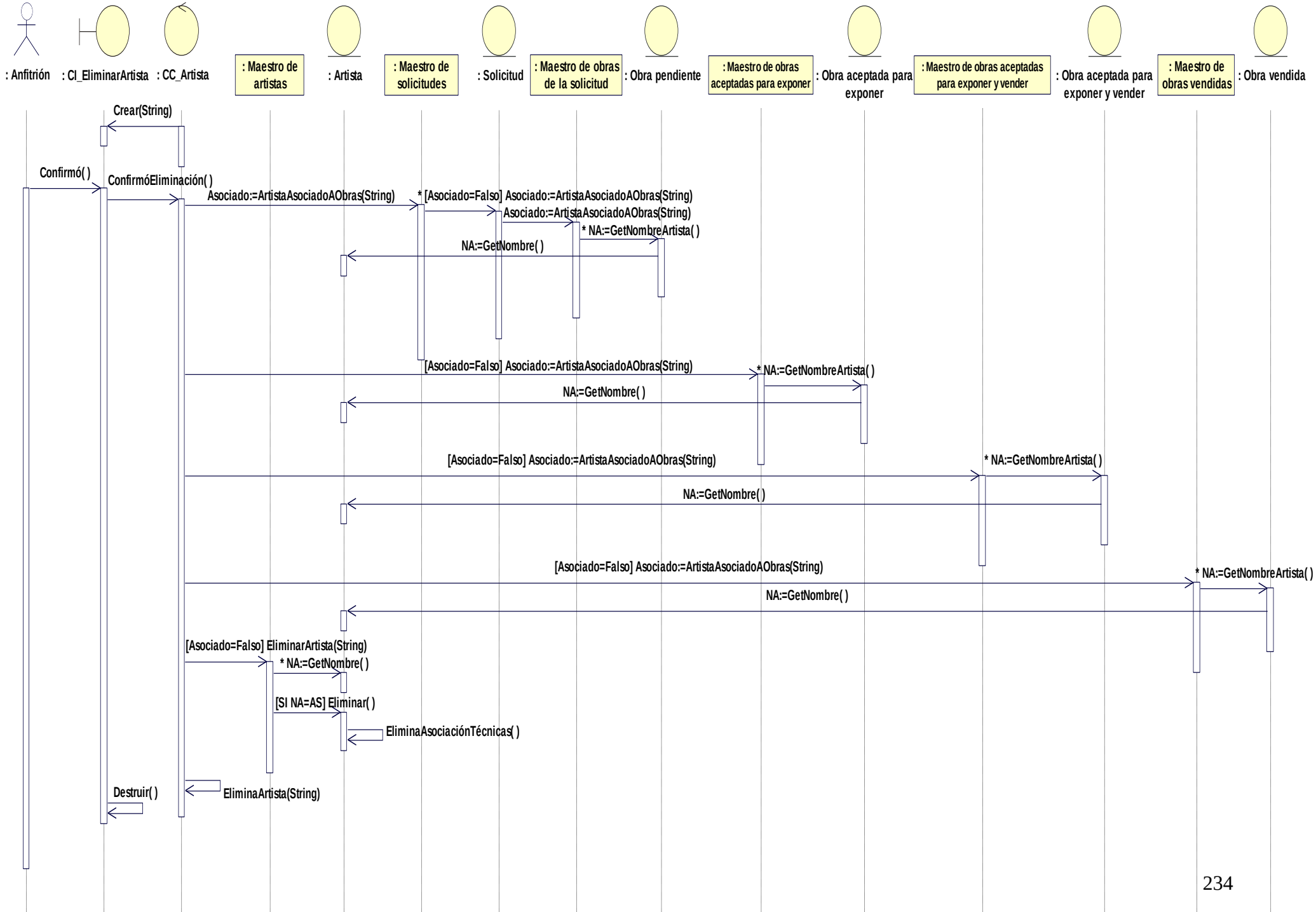
Registrar artista- Agregar artista



Registrar artista- Modificar artista



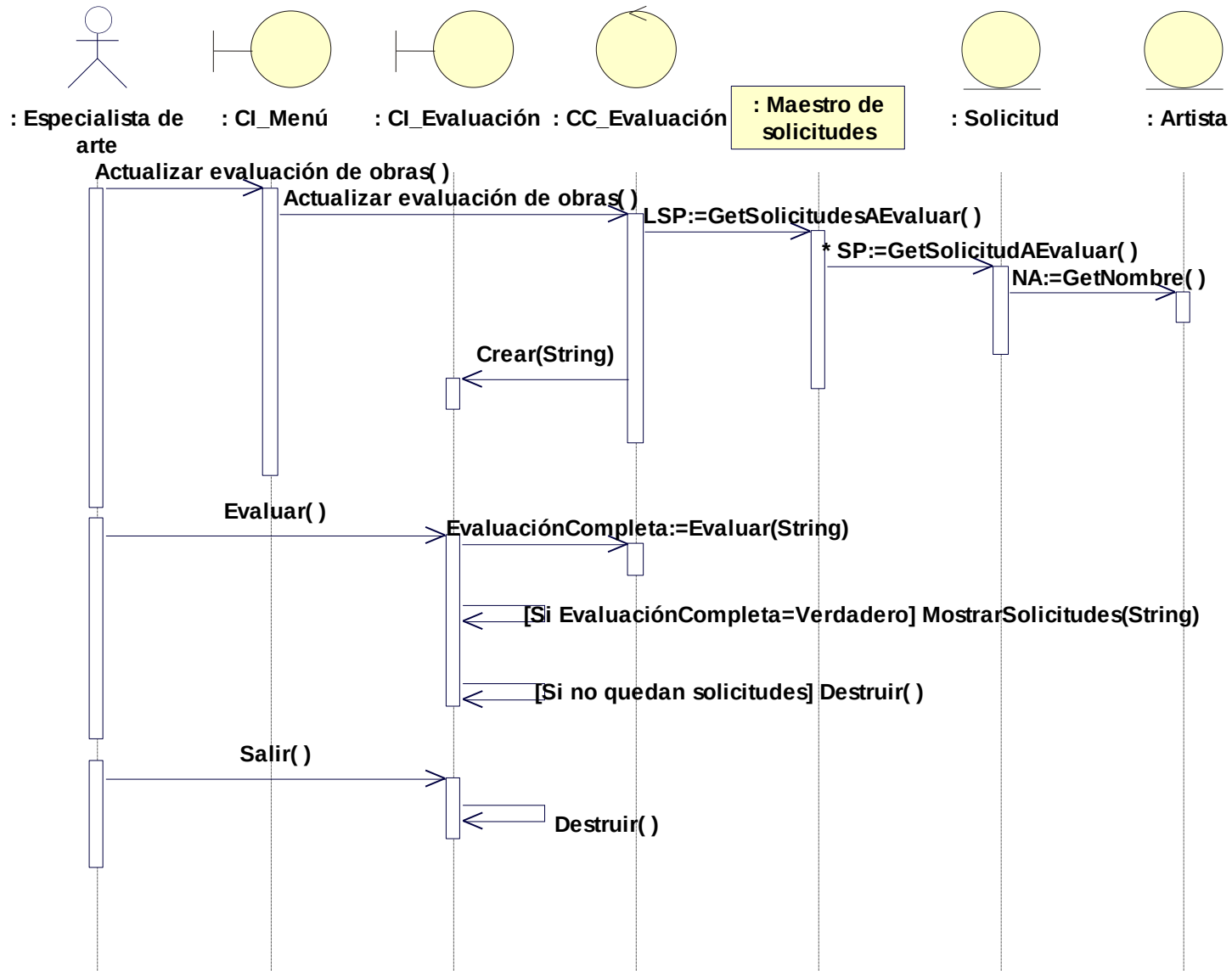
Registrar artista- Eliminar artista



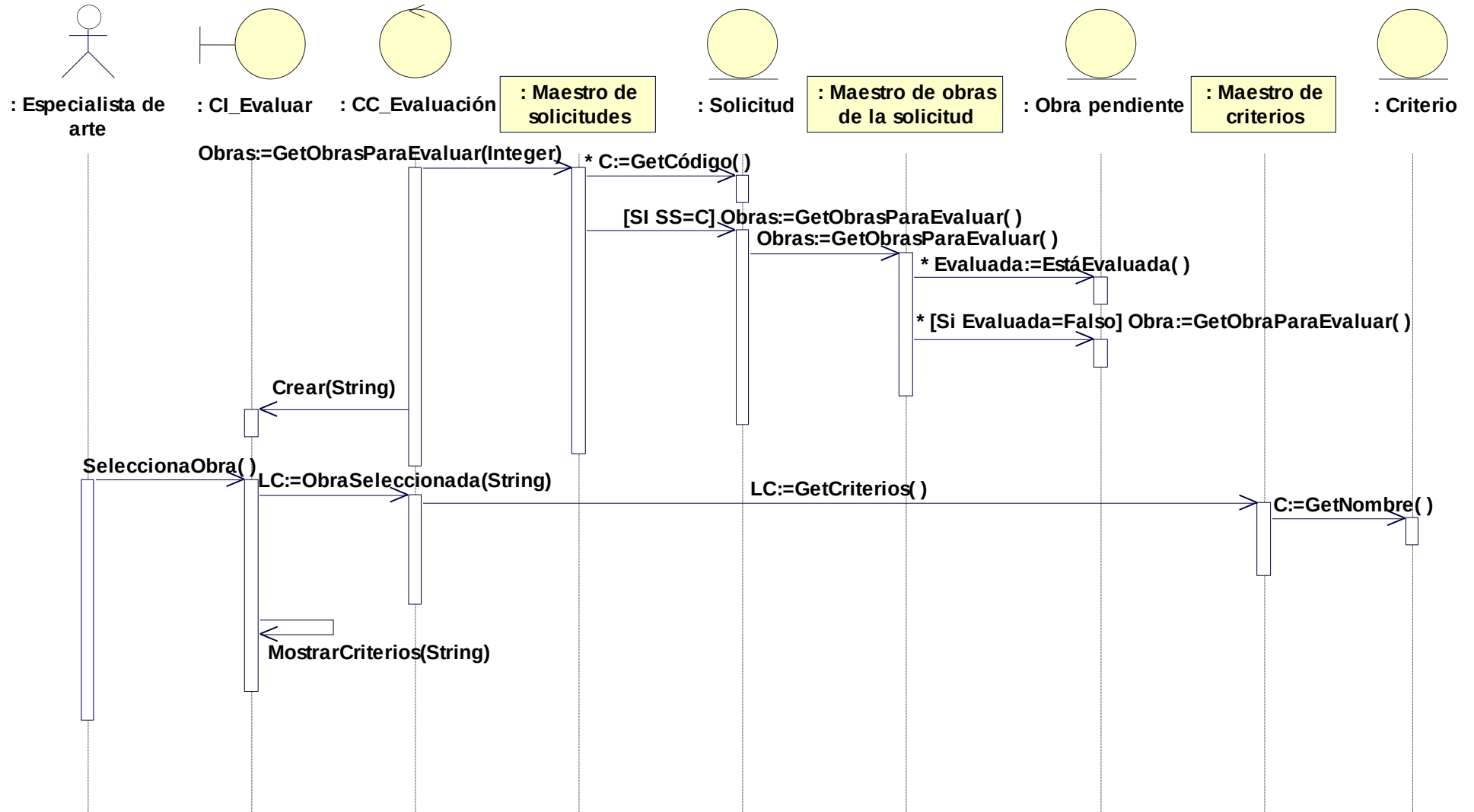
Caso de uso:	Actualizar evaluación de obras
Actores:	Especialista de arte (Inicia)
Descripción: El caso de uso se inicia cuando el especialista de arte va a actualizar la evaluación que ha dado a las obras de solicitudes a él asignadas. Al final consigna para las obras si serán o no aceptadas para su exposición en la galería. Si todas las obras de una solicitud han sido evaluadas, se elimina la solicitud y la asignación de especialistas.	
Referencias:	R7
Precondiciones:	El especialista de arte tiene solicitudes asignadas con obras pendientes de su evaluación y estén registrados los criterios de evaluación. En caso de que no se cumpla alguna precondición, el sistema presenta un mensaje al Especialista de arte.
Poscondiciones:	Se ha registrado de las obras su evaluación, pasándolas a aceptadas para exponer o rechazadas para exponer en la galería. Se eliminan las solicitudes que tienen todas las obras evaluadas y la asignación de especialistas.
Requerimientos especiales:	Un especialista de arte solo tendrá acceso a las obras contenidas en solicitudes a él asignadas.
Curso normal de los eventos	
Acción del actor	Respuesta del sistema
	1 El sistema presenta de todas las solicitudes, asignadas al especialista que inicia el caso de uso, que tienen obras pendientes de evaluación: el código, la fecha de registro y el artista.
2 El especialista de arte elige una de las solicitudes para evaluar las obras que contiene.	3 El Sistema ejecuta la sección Evaluar.
El Especialista de arte puede decidir repetir el paso 2.	
4 El especialista de arte finaliza el registro de evaluaciones.	5 El sistema finaliza la ejecución del caso de uso.
Sección "Evaluar"	
Acción del actor	Respuesta del sistema
	1 El sistema presenta de todas las obras contenidas en la solicitud el nombre, fecha de realización y dimensiones; marcando aquellas que ya están evaluadas ya que a esas obras no se le permite modificar la evaluación.
2 El Especialista de arte selecciona la obra que va a evaluar de las que no están evaluadas.	3 El sistema presenta los criterios de evaluación.
4 El Especialista de arte define los resultados de la evaluación en los criterios que analizó e indica si acepta o rechaza la obra.	

5 El Especialista de arte termina de registrar su evaluación para la obra o decide no evaluarla	6 Si decide evaluar la obra, el sistema verifica que se haya especificado si se rechaza o acepta la obra. En caso de que se haya especificado al aceptación o rechazo: 6.1 Registra los resultados de la evaluación, cambiando la obra del estado de pendiente al estado de aceptada para exponer o rechazada de acuerdo a la evaluación. En el caso de que sea aceptada registra los criterios evaluados. 6.2 Marca la obra como evaluada. 6.3 Si todas las obras de la solicitud han sido evaluadas, se presenta un mensaje al anfitrión, se elimina la solicitud y la asignación del especialista y se regresa a la línea 1 del curso normal principal.
El Especialista de arte puede repetir el paso 2.	
7 El especialista de arte finaliza el registro de evaluaciones para las obras de esa solicitud	8 El sistema finaliza la ejecución de la sección y devuelve el control a la línea 2 del curso normal principal.
Cursos alternos	
Sección "Evaluar" : Línea 6.3 Si hay obras en la solicitud que están pendientes de evaluación, se devuelve el control a la línea 2 de la sección Evaluar. Sección "Evaluar": Línea 6 Si el Especialista de arte decide no evaluar la obra, finaliza la ejecución de la sección y se regresa a la línea 2 del curso normal principal. Sección "Evaluar": Línea 6 Si no se especificó si la obra es aceptada o no como parte de su evaluación, el sistema presenta un mensaje al Especialista de arte y se devuelve el control a la línea 4 de la sección Evaluar. "Curso normal de los eventos": Línea 1 Si no quedan solicitudes con obras pendientes de evaluación, finaliza la ejecución del caso de uso.	

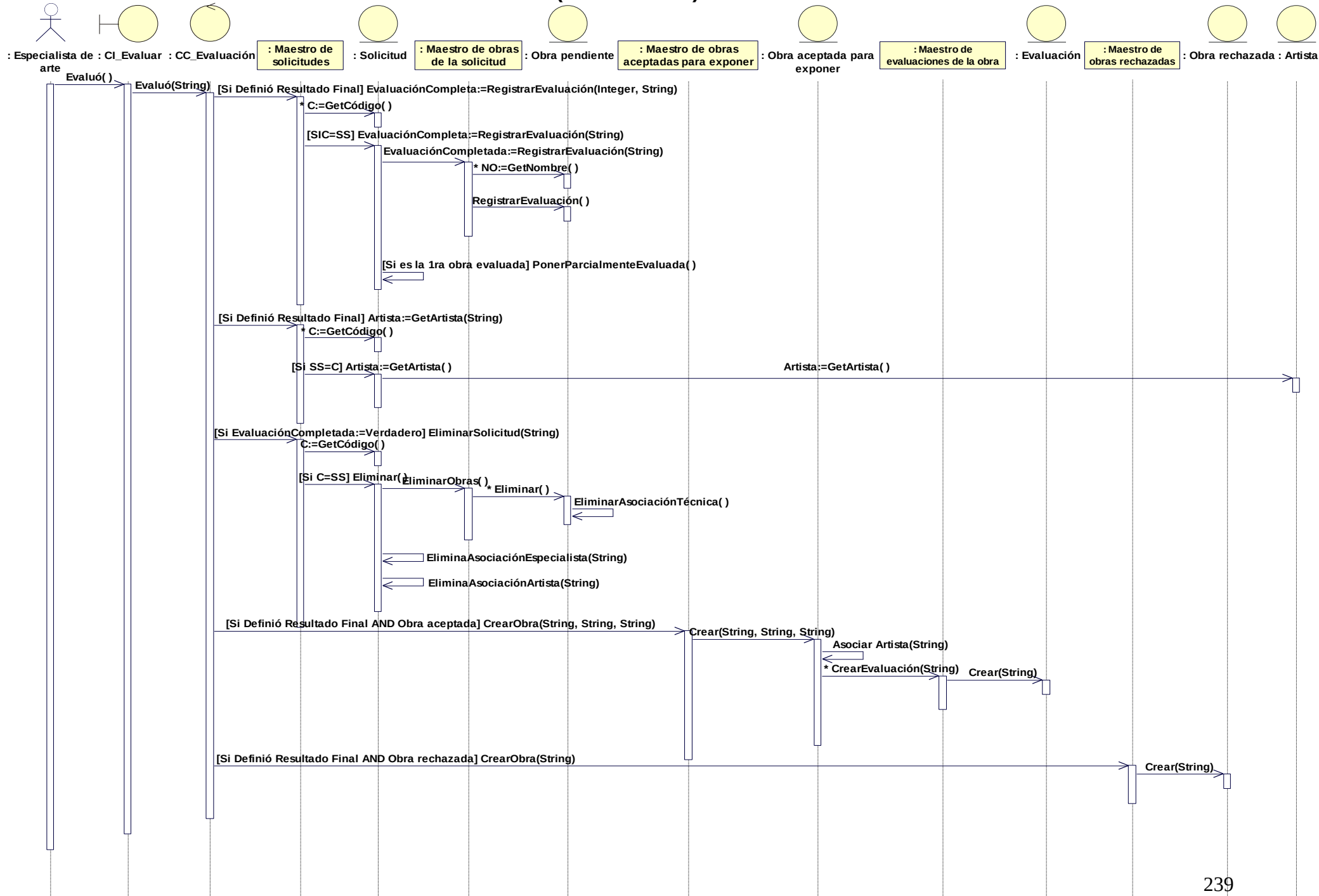
Actualiza evaluación de obras - Curso normal principal



Actualiza evaluación de obras - Sección Evaluar

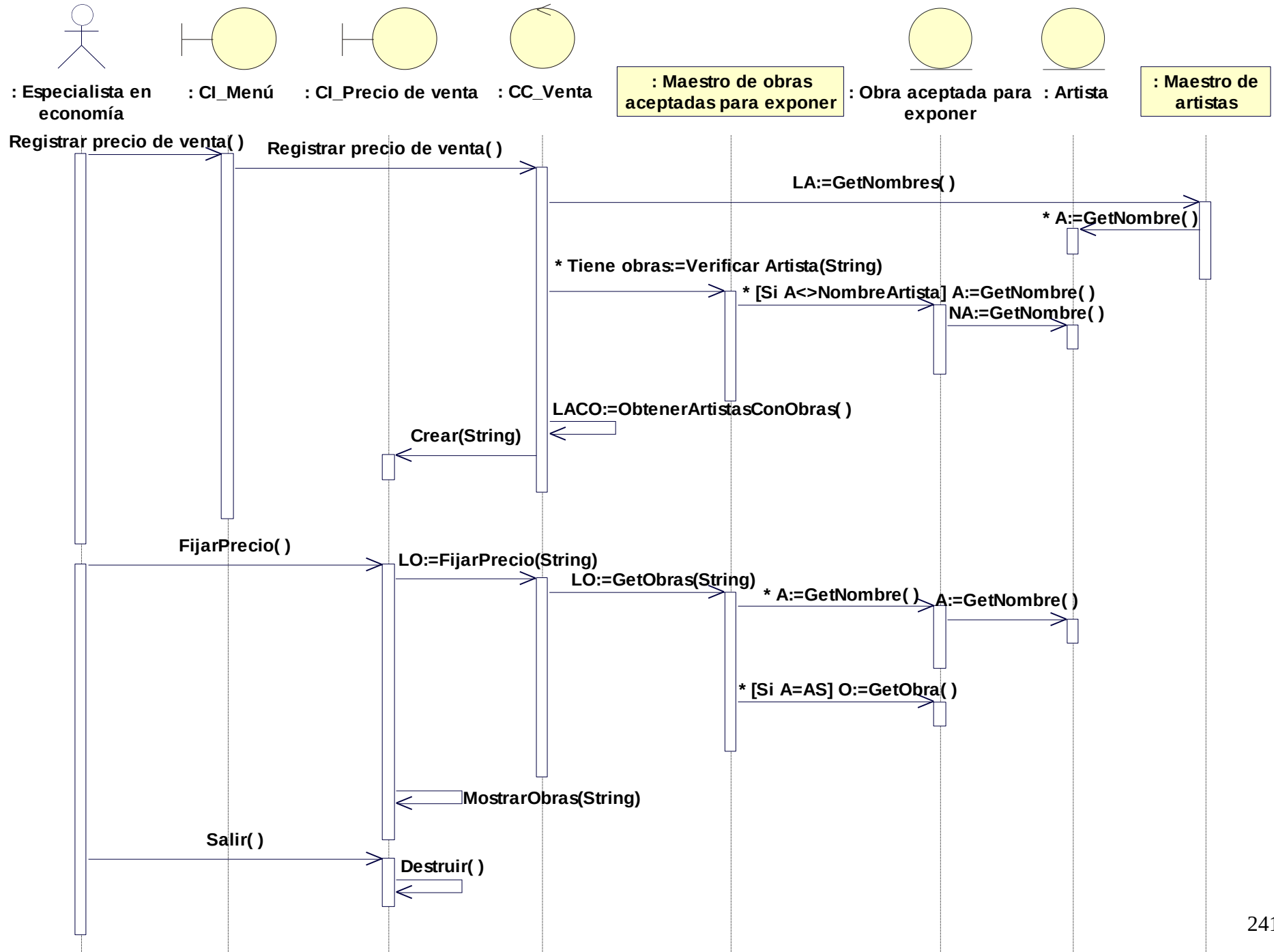


Actualiza evaluación de obras - Sección Evaluar (Continuación)

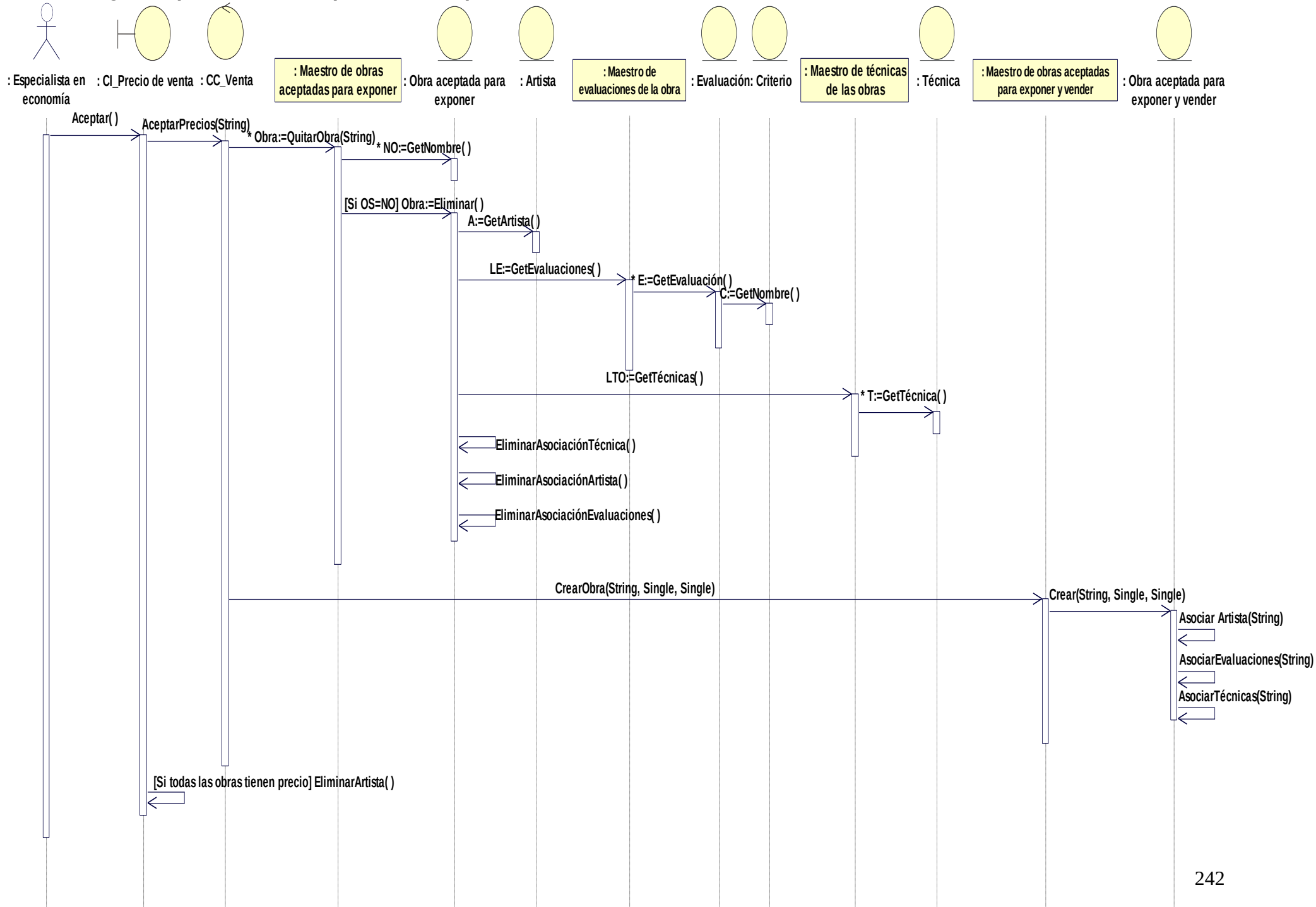


Caso de uso:	Registrar precio de venta		
Actores:	Especialista en economía (Inicia)		
Descripción:	El caso de uso se inicia cuando el especialista en economía ha pactado un precio de venta con el artista y registra en el sistema este valor para la(s) obra(s) del autor y el % que le corresponde a la galería por la venta, cambiando el estado de la obra de aceptada para exponer a aceptada para exponer y vender.		
Referencias:	R9		
Precondiciones:	Hay artista que tienen obras aceptadas para exponer que no tienen precio de venta. Si no hay obras que cumplan la condición, el sistema presenta un mensaje al Especialista en economía.		
Postcondiciones:	Se registra el precio de venta de las obras pasándose la obra de aceptada para exponer a aceptada para la venta y la exposición.		
Requerimientos especiales:	-		
Curso normal de los eventos			
Acción del actor		Respuesta del sistema	
		1 El sistema presenta los nombres de todos los artistas que tienen obras aceptadas para exponer, pero que no tienen fijado precio de venta.	
2 El Especialista en economía selecciona al artista con el que contactó para poner precio a sus obras y decide especificar el precio de venta pactado con el artista.	3 El sistema presenta los nombre de las obras del artista seleccionado que están aceptadas para exponer, pero no tienen fijado precio de venta y permite especificar el precio de venta y % de la galería.		
4 El Especialista en economía especifica, para las obras que aceptó vender el artista, el precio de venta y el % de la galería.			
5 El Especialista en economía decide registrar el precio de venta y el % de la galería o no registrar la actualización.	6 Si decide registrar la actualización, el sistema pasa las obras indicadas de aceptadas para exponer a aceptadas para exponer y vender, registrando el precio de venta y el % para la galería de cada una de ellas.		
El especialista el economía puede repetir el paso 2.			
7 El Especialista en economía termina el registro de precios de venta.	8 El sistema finaliza el caso de uso.		
Cursos alternos			
"Curso normal de los eventos": Línea 6 Si el especialista en economía decide registrar el precio de venta y el de la galería, regresa a la línea 2 del curso normal de los eventos.			

Registrar precio de venta

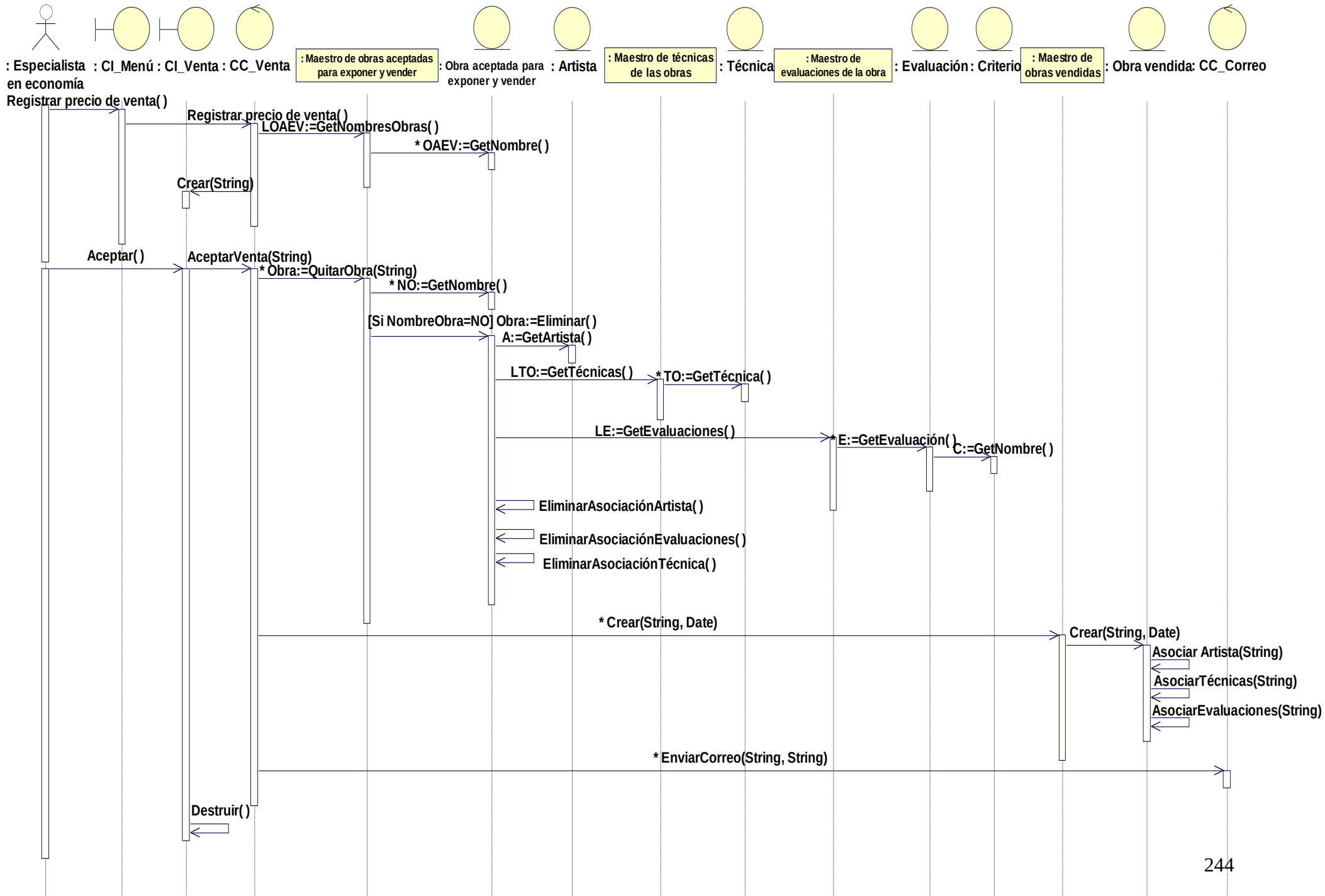


Registrar precio de venta (Continuación)



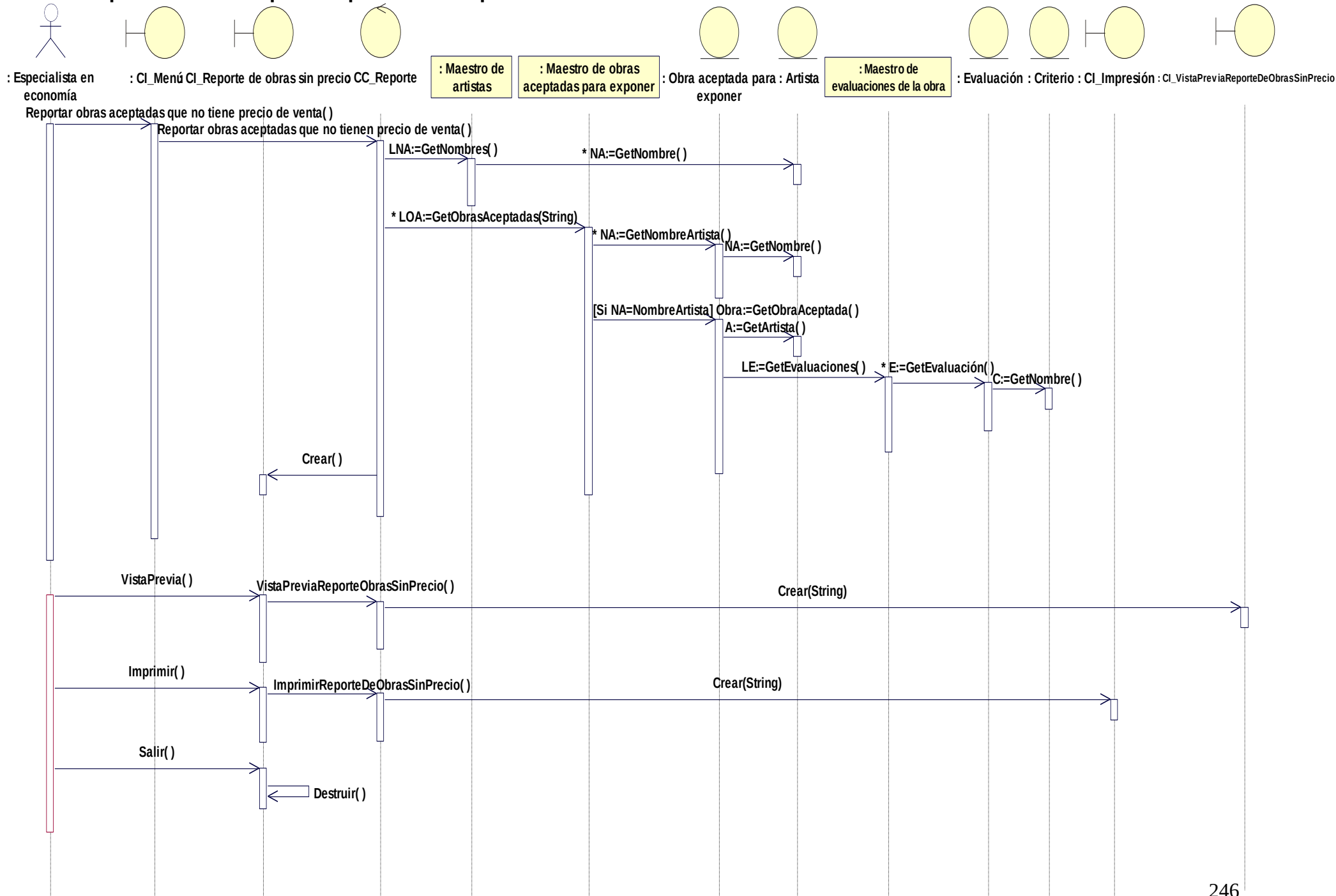
Caso de uso:	Registrar venta de obras		
Actores:	Especialista en economía (Inicia)		
Descripción: El caso de uso se inicia cuando se vende una obra y el especialista en economía actualiza esta información en el sistema. Solo es posible registrar la venta de obras que tiene fijado precio de venta y que no han sido vendidas. El caso de uso finaliza enviándole un correo al artista que hizo la obra.			
Referencias:	R10 y R11 Casos de uso asociados • Enviar e-mail		
Precondiciones:	Existen obras con precio de venta pactado que no han sido vendidas. Si no existen, el sistema presenta un mensaje al Especialista en economía.		
Postcondiciones:	Se registra la fecha de venta de la obra vendida, pasándose de aceptada para exponer y vender a obra vendida.		
Requerimientos especiales:	-		
Curso normal de los eventos			
Acción del actor		Respuesta del sistema	
		1 El sistema presenta los nombres de todas las obras con precio de venta fijado, que no han sido vendidas para que se pueda especificar la fecha de venta.	
2 El Especialista en economía especifica, de entre las obras aceptadas para vender y exponer, la fecha de venta de las obras que han sido vendidas.			
3 El Especialista en economía decide registrar las ventas o no registrarlas		4 Si decide registrar las ventas, el sistema: 4.1 Consigna, para las obras indicadas, su fecha de venta y cambia el estado de aceptadas para exponer y vender a obras vendidas. 4.2 Para cada artista que se le vendió alguna obra, busca su dirección de correo electrónico y extiende el caso de uso Enviar e-mail con la dirección de correo y el mensaje de cuáles obras se le vendieron. 4.3 Finaliza la ejecución del caso de uso	
Cursos alternos			
Curso normal de los eventos: Línea 4 Si el Especialista en economía decide no registrar las ventas, finaliza la ejecución del caso de uso.			

Registrar venta de obras



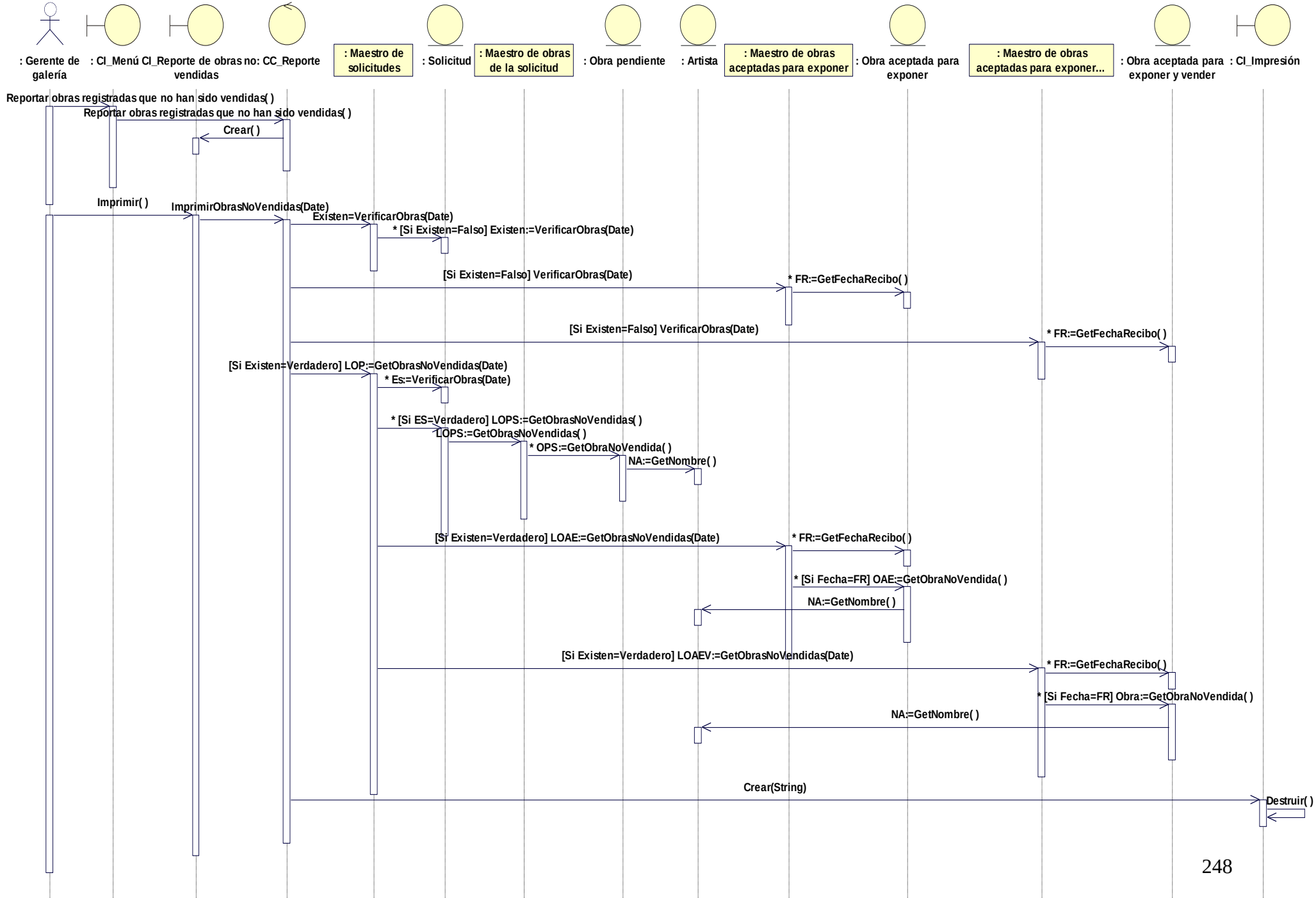
Caso de uso:	Reportar obras aceptadas que no tienen precio de venta	
Actores:	Especialista en economía (Inicia)	
Descripción:	El caso de uso se inicia cuando el especialista en economía necesita ponerse en contacto con los artistas que tiene obras aceptadas para exponer sin precio de venta fijado; para lo cual consulta al sistema y obtiene un reporte del artista con las obras.	
Referencias:	R8	
Precondiciones:	Hay obras aceptadas para exponer que no tiene precio de venta fijado. Si no obras que cumplan la condición, el sistema presenta un mensaje al Especialista en economía.	
Postcondiciones:	-	
Requerimientos especiales:		
Curso normal de los eventos		
Acción del actor	Respuesta del sistema	
	1 El sistema construye un reporte con las obras aceptadas para exponer que no tienen precio de venta pactado. El reporte lo organiza por artista indicando sus características (nombre, dirección, edad, sexo, e-mail y técnicas que usa) y la de cada una de las obras que tienen este estado (nombre, fecha de realización, dimensiones, técnicas que usa y los criterios usados en la evaluación con el resultado especificado.	
2 El Especialista en economía decide si va a ver el reporte, imprimirlo o terminar la ejecución del caso de uso.	3 El sistema ejecuta alguna de las siguientes acciones: a) Si decidió ver el reporte, presenta el reporte con el formato establecido y con la información solicitada, tal como se vería cuando lo imprima. b) Si decidió imprimir el reporte, lo imprime. c) Si decidió terminar la ejecución del caso de uso, finaliza la ejecución.	
El Especialista en economía puede repetir el paso 2.		
Cursos alternos		
-		

Reportar obras aceptadas que no tienen precio de venta

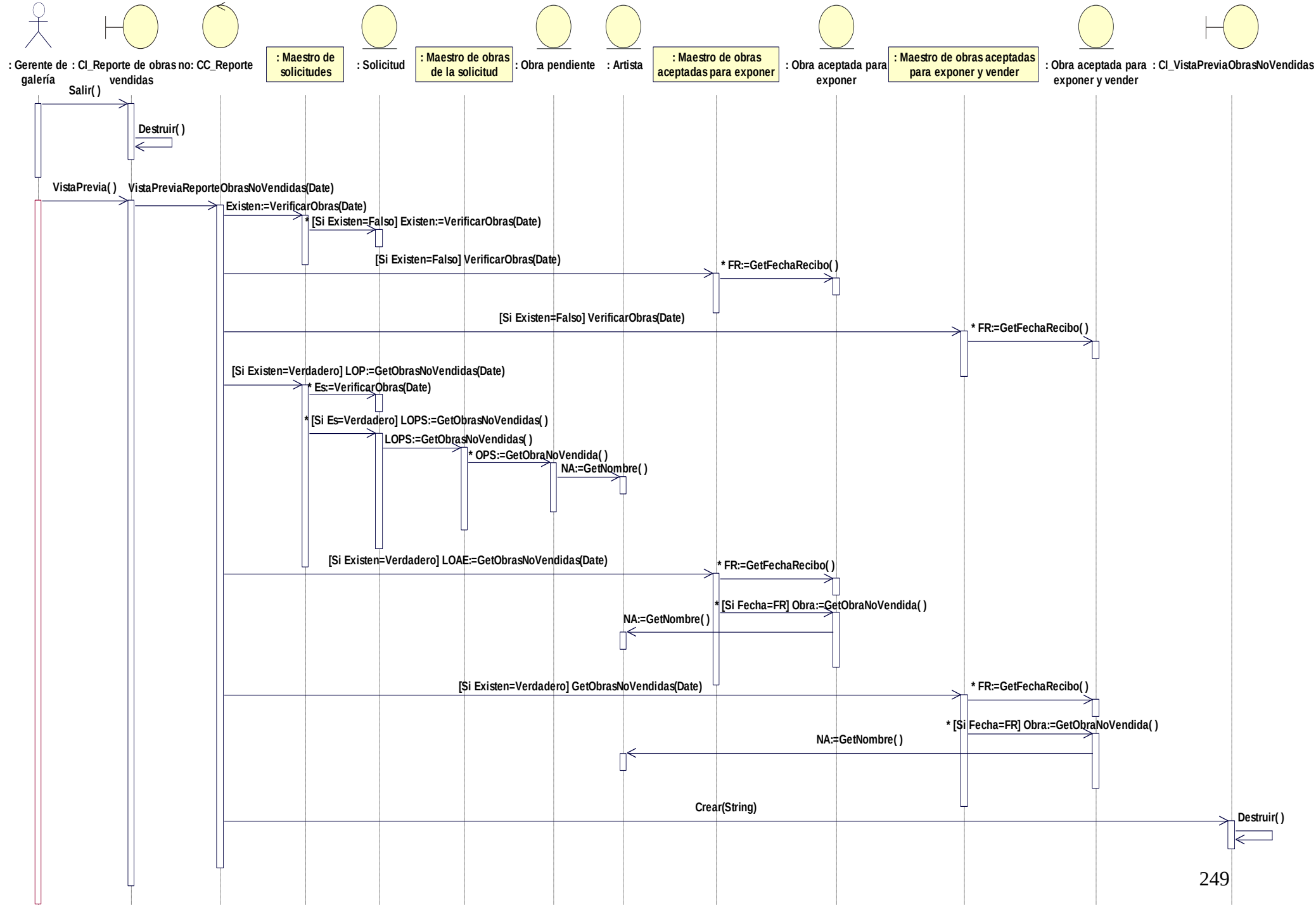


Caso de uso:	Reportar obras registradas que no han sido vendidas		
Actores:	Gerente de la galería (Inicia)		
Descripción:	El caso de uso se inicia cuando el Gerente de la galería necesita conocer las características; de las obras y del artista que las hizo; de las obras que están pendientes de evaluación, aceptadas para exponer y aceptadas para exponer y vender cuya fecha de presentación es anterior a una fecha dada.		
Referencias:	R12		
Precondiciones:	-		
Postcondiciones:	-		
Requerimientos especiales:			
Curso normal de los eventos			
Acción del actor	Respuesta del sistema		
	1 El sistema permite especificar la fecha base que servirá para construir el reporte.		
2 El Gerente de la galería especifica la fecha que servirá de base para el reporte.			
3 El Gerente de la galería decide si va a ver el reporte, imprimirlo o terminar la ejecución del caso de uso.	4 Si decide ver o imprimir el reporte, el sistema verifica si hay obras registradas pendientes de venta recibidas en fecha posterior a la especificada. Si hay obras que cumplan la condición, construye un reporte en el que refleja las obras registradas que no han sido vendidas y fueron presentadas en fecha anterior a la especificada. La obras se organizan de acuerdo a su estado (pendientes, aceptadas para exponer y aceptadas para exponer y vender). De cada obra además de su estado, se incluye cuándo se recibió la solicitud de exposición en la galería y el artista que las hizo. Además, en el reporte se indica la fecha utilizada como referencia. Si el Gerente de la galería decidió ver el reporte, se le presenta con el formato establecido y la información solicitada, tal como se vería cuando lo imprima. Si decide imprimir, lo imprime.		
El Gerente de la galería puede repetir el paso 1.			
Cursos alternos:			
"Curso normal principal": Línea 4			
Si no hay obras registradas en fecha posterior a la indicada que estén pendientes de venta, se presenta un mensaje al Gerente de la galería y se devuelve el control a la línea 1 del curso normal principal.			
"Curso normal principal": Línea 4			
Si decide terminar la ejecución del caso de uso, finaliza el caso de uso.			

Reportar obras registradas que no han sido vendidas



Reportar obras registradas que no han sido vendidas (Continuación)



Caso de uso:	Registrar criterio		
Actores:	Gerente de la galería (Inicia)		
Descripción: El caso de uso se inicia cuando el Gerente de la galería define un nuevo criterio que puede tenerse en cuenta cuando se evalúa la obra, terminando el caso de uso con el criterio registrado.			
Referencias:	R13		
Precondiciones:	-		
Postcondiciones:	Se registra el nuevo criterio.		
Requerimientos especiales:	-		
Curso normal de los eventos			
Acción del actor		Respuesta del sistema	
		1 El sistema presenta de todos los criterios registrados el nombre y dirección.	
2	El Gerente de la Galería elige definir un nuevo criterio o finalizar el registro de criterios.	3	Si decide finalizar el registro de criterios, el sistema finaliza la ejecución del caso de uso. En caso contrario, el sistema permite especificar las características del nuevo criterio.
4	El Gerente de la galería especifica las características del nuevo criterio (nombre y dirección).		
5	El Gerente de la galería decide registrar el nuevo criterio o no registrarlo.	6	Si decide registrar el nuevo criterio, el sistema verifica que el criterio no exista. En caso de que no exista, se registra el criterio. Se devuelve el control a la línea 2.
Cursos alternos			
"Curso normal de los eventos" : Línea 6 Si el criterio ya está registrado, el sistema presenta un mensaje al Gerente de la galería y regresa a la línea 1 del curso normal de los eventos.			
"Curso normal de los eventos" : Línea 6 Si el Gerente de la galería decide no registrar el criterio, se devuelve el control a la línea 2 del curso normal principal.			

Registrar criterio

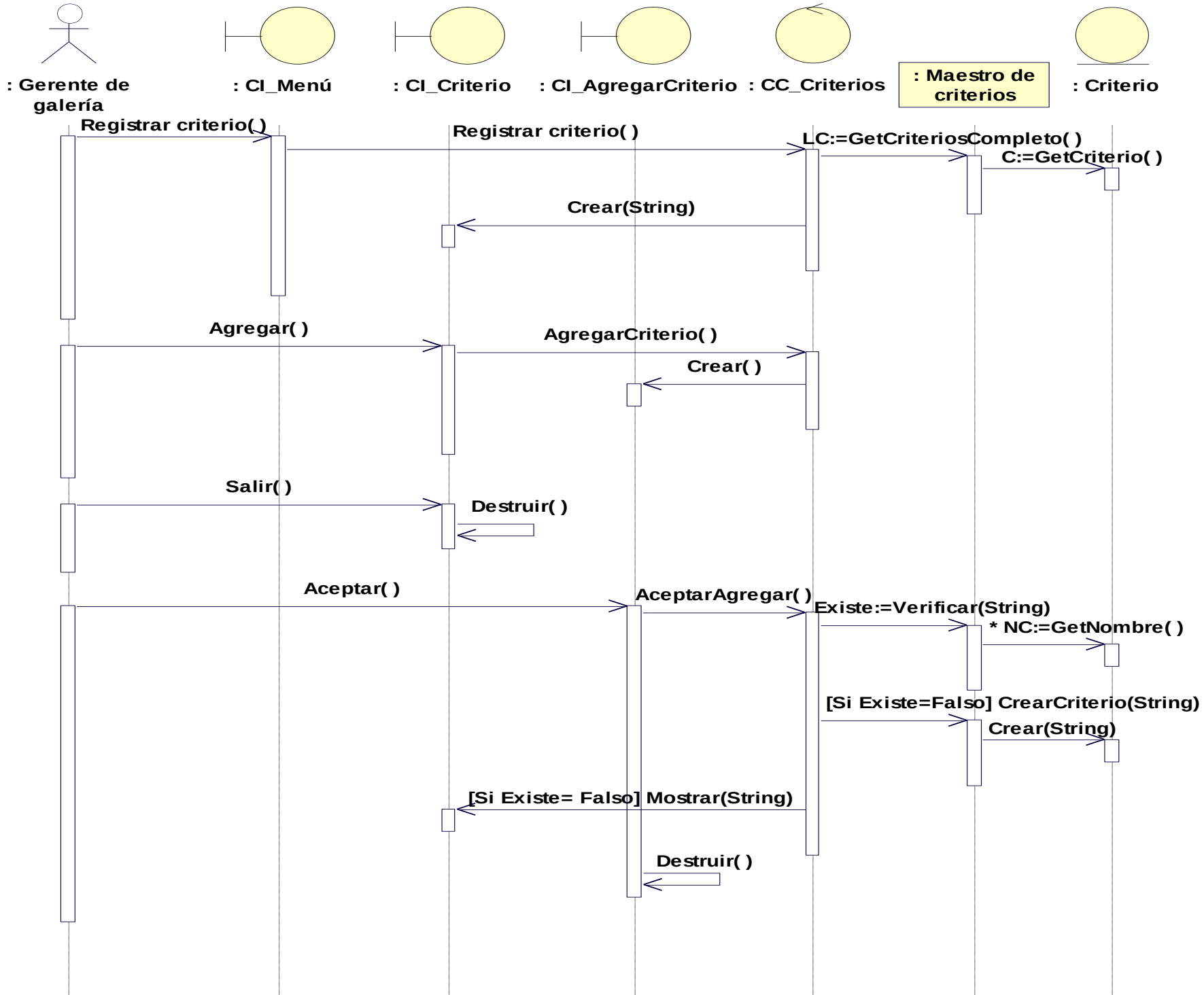


Diagrama de clases del diseño- Paquete Registro de obras

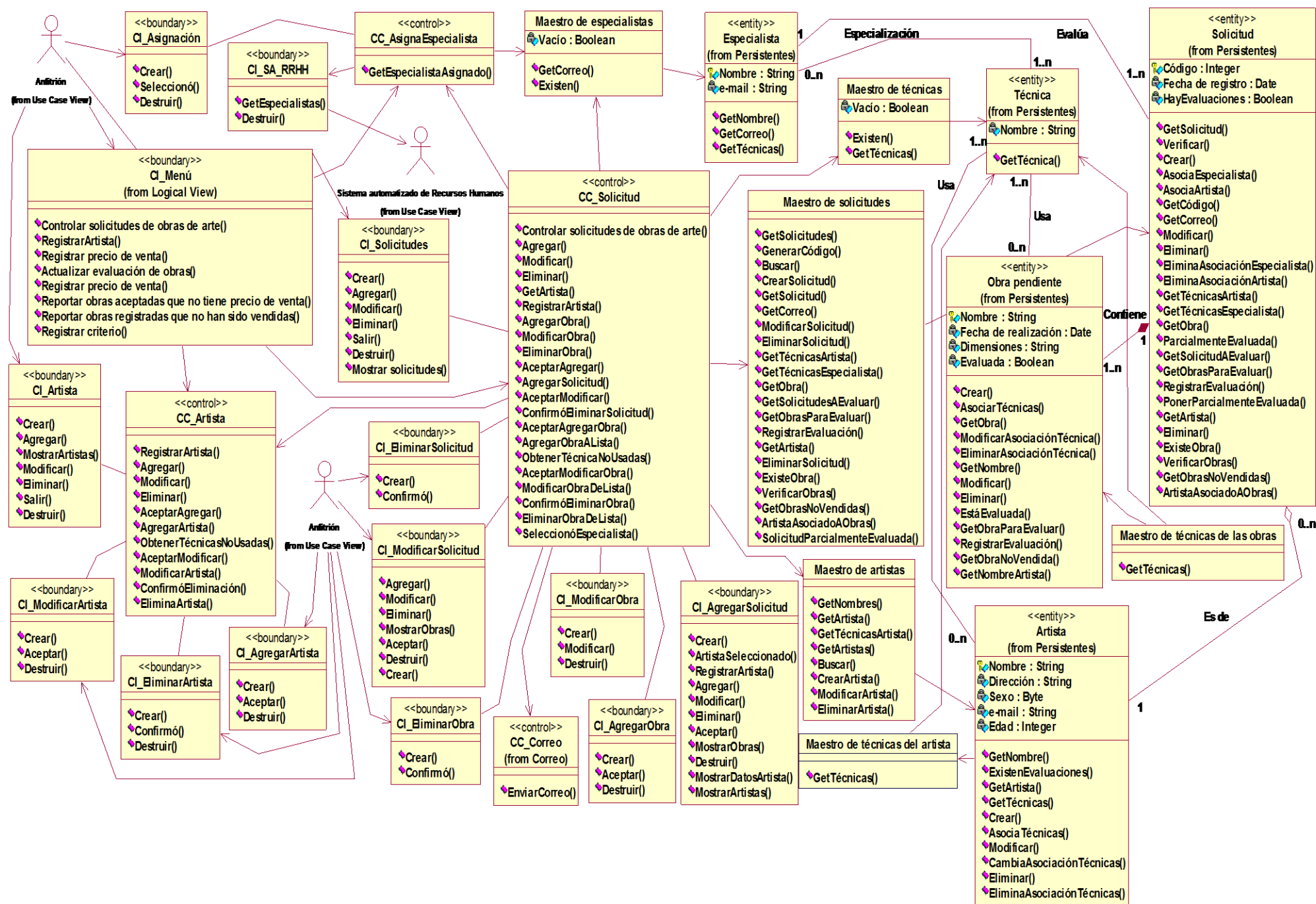


Diagrama de clases del diseño- Paquete Evaluación de obras

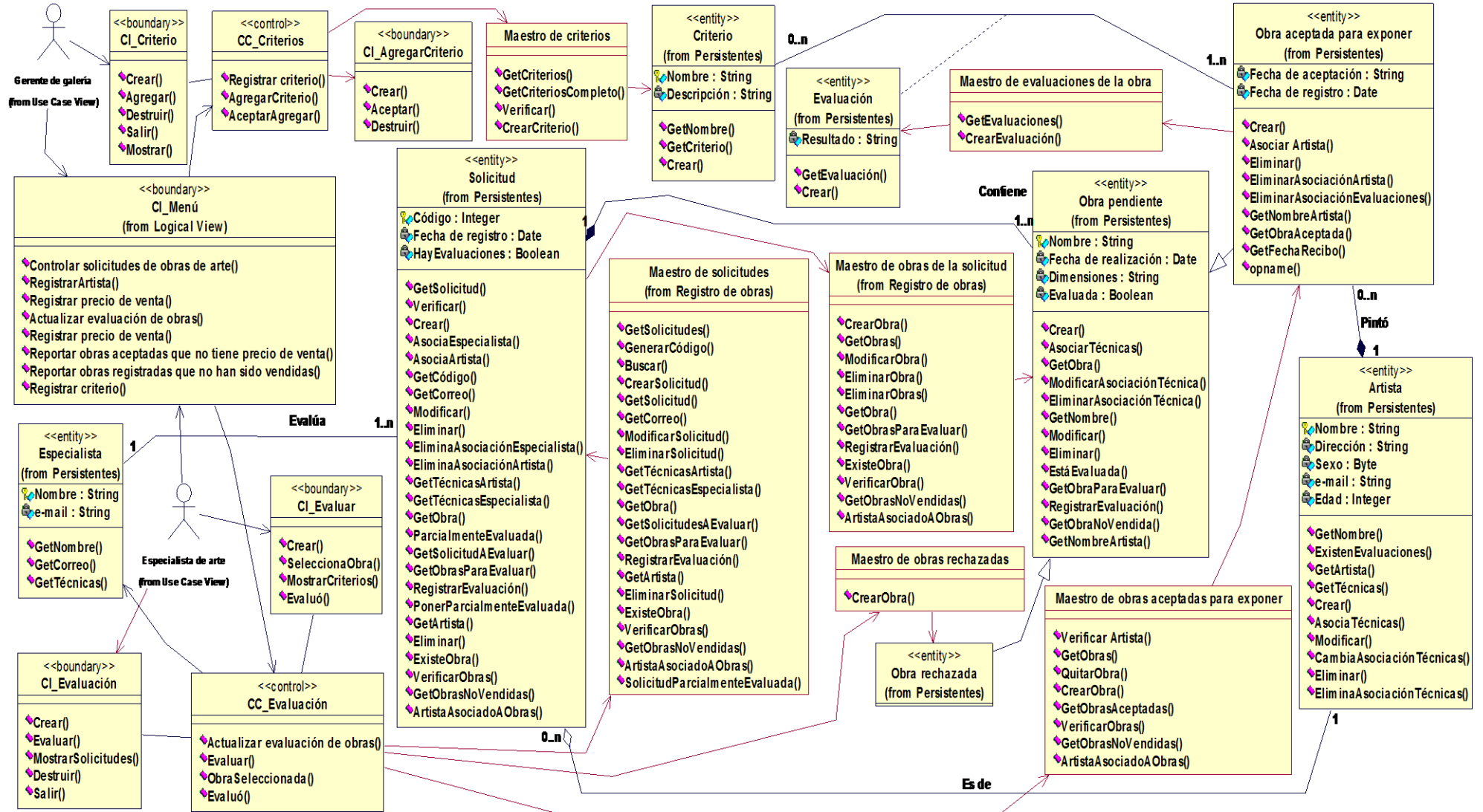
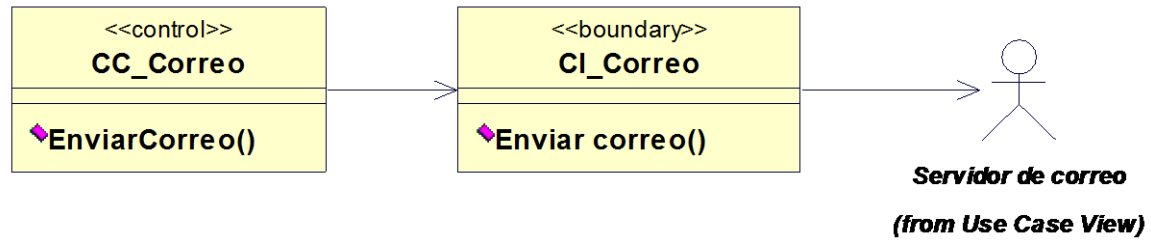
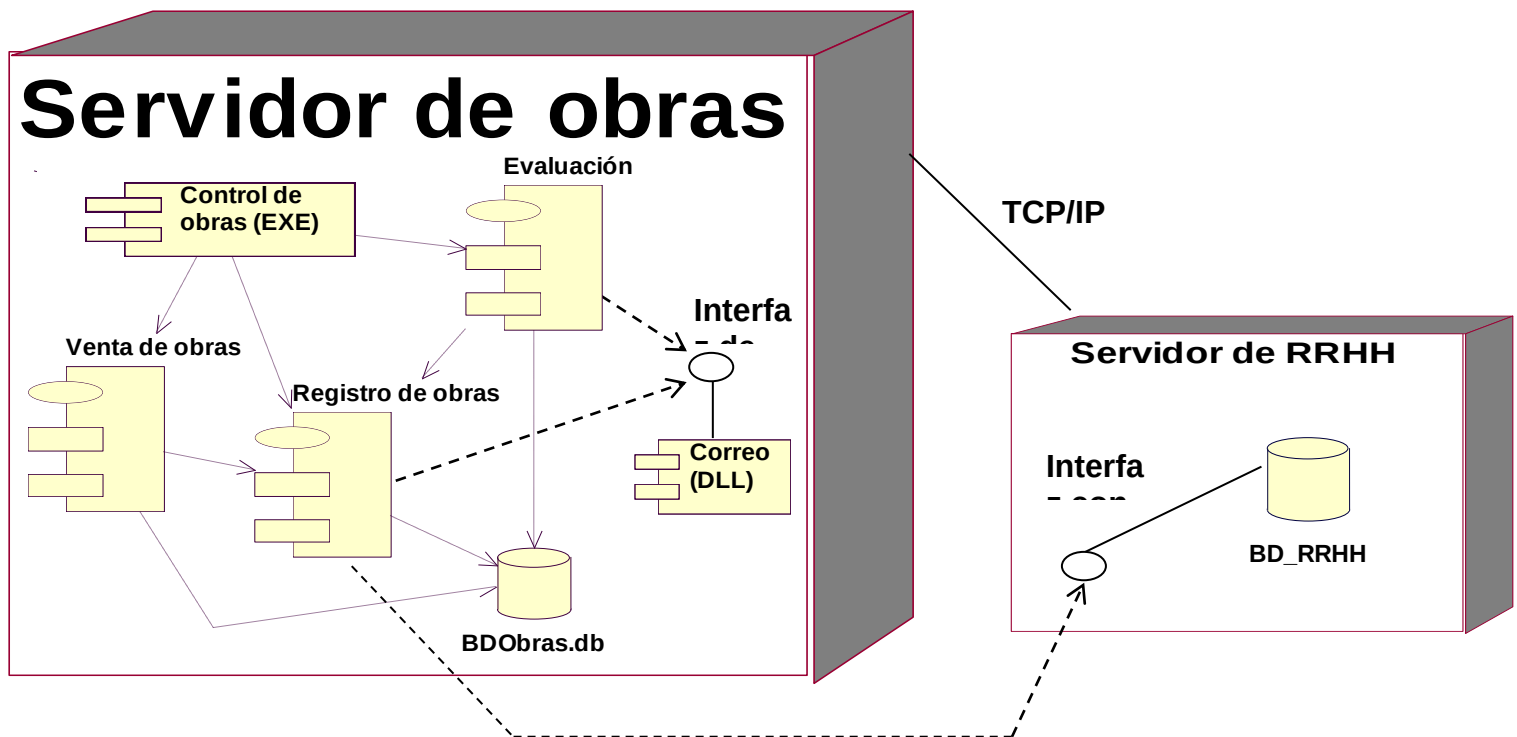


Diagrama de clases del diseño- Paquete Correo



9.6 – Implementación

Diagrama de implementación



9.7 – Prueba

Descripción de un caso de prueba:

Caso de uso: Reportar obras registradas que no han sido vendidas.

Caso de prueba: Reportar obras registradas que no han sido vendidas para un fecha donde no hay información disponible.

Entrada:

- El Gerente de la galería necesita conocer todas las obras registradas que no han sido vendidas a partir del 1 de Agosto del 2003.
- Existen obras registradas, pero todas han sido vendidas.

Resultado:

- Se muestra un mensaje al Gerente de la galería informándole que no hay resultados registrados.

Condiciones:

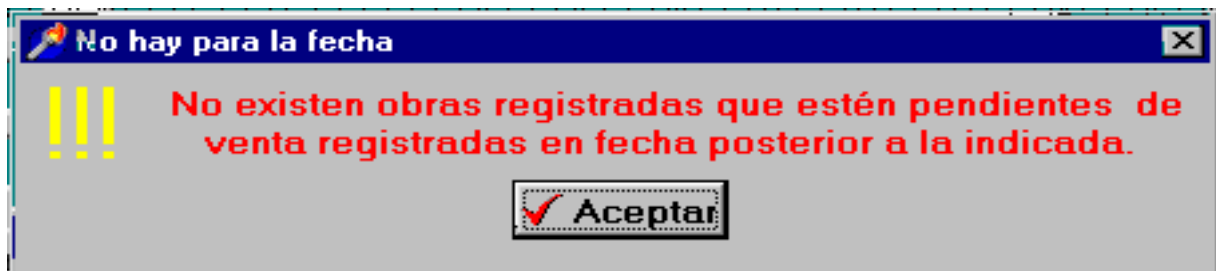
- No se permite que se ejecuten otras instancias del caso de uso para el Gerente de la galería.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

Procedimiento de prueba:

Caso de uso: Reportar obras registradas que no han sido vendidas

Precondiciones: -	
Acción del actor	Respuesta del sistema
	1 El sistema permite especificar la fecha base que servirá para construir el reporte.
2 El Gerente de la galería especifica la fecha [01/08/2003] que servirá de base para el reporte. 3 El Gerente de la galería decide que va a ver el reporte	4 Si decide ver o imprimir el reporte, el sistema verifica si hay obras registradas pendientes de venta recibidas en fecha posterior al [01/08/2003] .
Cursos alternos	
"Curso normal principal": Línea 4 Como no hay obras registradas en fecha posterior al [01/08/2003] que estén pendientes de venta, se presenta un mensaje al Gerente de la galería y se devuelve el control a la línea 1 del curso normal principal.	
Postcondiciones: No se visualizan las obras pendientes de venta con fecha posterior al [01/08/2003] porque no hay obras que cumplan la condición.	



Bibliografía

- Jacobson, I.; Booch, G. y Rumbaugh, J.; “El Proceso Unificado de Desarrollo de software”. 2000. Addison-Wesley.
- Booch, G.; Rumbaugh, J. y Jacobson, I.; “El Lenguaje Unificado de Modelado”. 2000. Addison-Wesley.
- Pressman, Roger; Ingeniería de software. Un enfoque práctico. 2002. McGraw.Hill/Interamericana de España.
- Rumbaugh, J.; Jacobson, I. y Booch, G.; “El Lenguaje Unificado de Modelado. Manual de referencia. 2000. Addison-Wesley.
- Von Hallen, B.; “Building a Business Rules”. <http://www.Kpiusa.com/ReadingRoom/ReadingRoom.htm>.
- Agualló, P.; “Desarrollo Cliente / servidor: ubicación de las reglas de negocio”. <http://www.ctv.es/USERS/pagullo/arti/esbr/esbr.htm>.
- Falção, H. y Fontes, J.; “¿En quién se pone el foco?. Identificando stakeholders para la formulación de la misión organizacional”. Revista del CLAD Reforma y Democracia”. No. 15 (Octubre 1999). Caracas.
- “Análisis de las partes interesadas”. <Http://wbIn0018.wordbank.org/caribbean/CaribbeanCMUOL.nsf/FILE/Db-an-pi.doc>.
- González, E.; “La empresa ante sus grupos de intereses: Una aproximación desde la literatura del análisis de los stakeholders”. Papeles de Ética y Dirección, No. 4, 1999.
- Leffingwell, Dean; “Features, Use Cases, requirements, Oh My!”. 2000. Rational Software. <Http://www.rational.com/media/whitepapers/featucregom.pdf>
- LARMAN, Craig “UML y patrones” 1999, Prentice Hall Iberoamericana.
- Bruegge, B. Y Dutoit, A. “Ingeniería de Software Orientado a Objetos”. 2002. Prentice Hall – Pearson Educación.
- GAMMA, E.; HELM, R.; JOHNSON, R. y VLISSIDES, J. “Patrones de diseño”. 2000. <http://www.vico.org/pages/PatronsDisseny.html>.
- UML y la Modelación de datos.pdf. Whitepaper de Rational Rose.
- AMBLER, S. “Mapping Object to Relational Databases”. October 21, 2000. <http://www.abyssoftw.com/mappingobject.html/mappingobjects.pdf>.
- “Real_Time UML. Developing Eficiente Objects for Embedded Systems”. Bruce Powel Douglas. Addison Wesley-Longman. 1998.

Aplicación del Proceso Unificado de Desarrollo a proyectos de software

“Condiciones y casos de prueba”. <http://www.rmya.com.ar>

- “The Complete Guide to software testing”. <http://www.softwareqatest.com>